

FEM Practices

Jan Tomek

August 6, 2026

Contents

1	Coordinate Transformations	17
1.1	Constructing Transformation Matrix	18
1.1.1	Rotation Matrix	18
1.1.2	Euler's angles of XYZ rotation	23
1.1.3	Euler's angles of ZYX rotation	25
1.1.4	Euler's angles of XZY rotation	26
1.1.5	Euler's angles of ZXY rotation	27
1.1.6	Euler's angles of YXZ rotation	29
1.1.7	Euler's angles of YZX rotation	30
1.1.8	Euler's angles of XYX rotation	31
1.1.9	Euler's angles of XZX rotation	32
1.1.10	Euler's angles of YXY rotation	33
1.1.11	Euler's angles of YZY rotation	34
1.1.12	Euler's angles of ZXZ rotation	34
1.1.13	Euler's angles of ZYZ rotation	35

1.2	Coordinate Transformation	36
1.3	Tensor Transformation	36
1.4	Moment of Inertia	54
1.5	Quaternions	57
1.5.1	Quaternion properties	57
1.5.2	Scalar and vector parts	59
1.5.3	Conjugation, the norm and reciprocal	60
1.5.4	Unit quaternion (or versor)	62
1.5.5	Quaternions and three-dimensional geometry	62
1.5.6	Matrix representations	64
1.5.7	Exponential, logarithm and power functions	66
1.5.8	Quaternions used for coordinate transformation	67
2	Boundary conditions not in GCS	73
2.1	Skew boundary conditions	73
2.2	Cartesian CSYS	75
2.3	Cylindrical CSYS	77
3	Multifreedom Constraints	83
3.1	Methods for Imposing MFCs	84
3.1.1	MFC Matrix Forms	85
3.1.2	The Example Structure	87
3.2	The Master-Slave Method	88

<i>CONTENTS</i>	5
3.2.1 A One-Constraint Example	88
3.2.2 Multiple Homogeneous MFCs	90
3.2.3 Nonhomogeneous MFCs	92
3.2.4 The General Case	93
3.2.5 Retaining the Original Freedoms	94
3.2.6 Model Reduction by Kinematic Constraints	95
3.2.7 Assesment of the Master-Slave Method	97
3.3 The Penalty Method	100
3.3.1 Choosing the Penalty Weight	101
3.3.2 The Square Root Rule	102
3.3.3 Penalty Elements for General MFCs	103
3.3.4 The Theory Behind the Penalty Method Coefficient Matrix	105
3.3.5 Assesment of the Penalty Method	106
3.4 Lagrange Multiplier Adjuion	107
3.4.1 Physical Interpretation	107
3.4.2 Lagrange Multipliers fro General MFCs	109
3.4.3 The Theory Behind	110
3.4.4 Assesment of the Lagrange Multiplier Method	110
3.4.5 The Augmented Lagrangian Method	111
3.5 Summary	112
4 Contact Formulations	113
4.1 A One-Dimensional Contact Example	114

4.6.5	Bonded	129
4.6.6	Comparison of the Definitions	130
4.6.7	Cohesive and Adhesive Interfaces	131
4.7	Contact Discretizations	131
4.7.1	Node-to-Node Contact	131
4.7.2	Node-to-Surface Contact	132
4.7.3	Surface-to-Surface Contact	132
4.7.4	Mortar Contact	133
4.7.5	One-Pass and Two-Pass Contact	133
4.8	Linearization and Nonlinear Solution	134
4.8.1	Contact Chattering	135
4.8.2	Initial Gap and Initial Penetration	135
4.8.3	Search Distance and Candidate Detection	135
4.9	Numerical Behaviour and Assessment	136
4.9.1	Quantities to Check	136
4.9.2	Pressure Singularities	137
4.9.3	Mesh Compatibility and Bias	137
4.9.4	Contact in Dynamic Problems	137
4.10	Relation to Multifreedom Constraints	138
4.11	Summary	139
5	Rigid Body Elements	141
5.1	RBE2	142

5.1.1	RBE2 Coefficients Example:	144
5.1.2	Editing the Master Stiffness Matrix	147
5.2	RBE3	152
6	Governing Equations	155
6.1	Material Matrices	155
7	Dynamics	157
7.1	Function properties	157
7.2	SDOF system	158
7.2.1	Definition	158
7.2.2	Equation of Motion for SDOF Systems	159
7.2.3	Time Solution for Unforced Undamped SDOF Systems . .	159
7.2.4	Time Solution for Unforced Damped SDOF Systems . . .	161
7.2.5	SDOF Systems under Harmonic Excitaion	168
7.3	Damping	174
7.3.1	Rayleigh Damping	174
7.4	Modal decomposition	180
7.4.1	Definition	180
7.4.2	Orthonormalised Modal Base	181
7.4.3	MDOF to SDOF	183
7.4.4	Initial Conditions	185

<i>CONTENTS</i>	9
8 Kinematics	187
8.1 Rigid Body Motion	187
9 Iterative Methods	189
9.1 Root finding problems	189
9.1.1 Incremental Search Method	189
9.1.2 Bisection Method	191
9.1.3 Methods Based on Linear Interpolation	193
9.2 Minimizing function of Multiple variables	207
9.3 second order ODE's	213
9.4 Explicit	214
9.4.1 Euler's method	214
9.4.2 Runge-Kutta 4th order method	221
9.5 Implicit	227
9.5.1 Newton-Raphson	228
10 Model reduction	233
10.1 Guyan reduction	233
10.2 Craig-Bampton reduction	235
10.2.1 Constraint modes	235
10.2.2 Fixed-interface normal modes	236
10.2.3 Craig-Bampton transformation	237
10.3 Flexible body modal reduction	239

<i>CONTENTS</i>	11
11.6 TET4 Element	300
11.6.1 Element Formulation	300
11.6.2 Python Implementation	301
12 Numerics	309
12.1 Numerical Spaces	309
12.1.1 Hilbert Space	310
12.1.2 Sobolev Space	311
12.1.3 Banach Space	312
12.1.4 Krylov Space	313
12.2 Orthogonalisation	315
12.2.1 Gram-Schmidt process	315
12.2.2 Modified Gram-Schmidt process	318
12.3 Factorisation	320
12.3.1 Cholesky Factorisation	320
12.3.2 LU Factorisation	323
12.3.3 QR Factorisation	331
12.3.4 Conversion to Hessenberg form using Givens rotations . .	339
12.3.5 Conversion to Hessenberg form using Householder reflections	341
12.3.6 Jacobi similarity rotations	342
12.3.7 Jacobi rotations to diagonal matrix	344
12.3.8 Arnoldi algorithm	346

12.3.9	Lanczos algorithm	348
12.3.10	Schur algorithm	350
12.3.11	Singular Value Decomposition (SVD)	352
12.3.12	Positive semidefinite matrices	358
12.3.13	Applications	359
12.3.14	Matrix Inversion	360
12.4	Linear Regression	360
12.4.1	Derivation	360
12.4.2	Result	365
12.5	Interpolation and Mapping of Values	366
12.5.1	Interpolation, Approximation and Extrapolation	367
12.5.2	Nearest-Neighbour Mapping	368
12.5.3	Inverse-Distance Weighting	370
12.5.4	One-Dimensional Interpolation	372
12.5.5	Interpolation on Structured Grids	375
12.5.6	Interpolation on Finite Elements	376
12.5.7	Extrapolation Outside the Source Domain	383
12.5.8	Interpolation of Scattered Data	387
12.5.9	Conservative Field Transfer	393
12.5.10	Mapping Vector and Tensor Quantities	398
12.5.11	Accuracy, Robustness and Validation	399
12.6	Modal Assurance Criterion and Similarity Metrics	401

12.6.1	Complex Inner Product and Normalised Correlation . . .	401
12.6.2	Modal Assurance Criterion	402
12.6.3	Direction- and Magnitude-Sensitive Vector Measures . . .	407
12.6.4	AutoMAC and CrossMAC Matrices	413
12.6.5	Weighted MAC and Cross-Orthogonality	415
12.6.6	Modal Scale Factor and Scaled Residual	417
12.6.7	Coordinate Modal Assurance Criterion	419
12.6.8	Repeated Modes and Subspace Similarity	421
12.6.9	Natural-Frequency Agreement and Mode Pairing	424
12.6.10	Frequency Response Assurance Criterion	425
12.6.11	Direct Error Measures	427
12.6.12	Choosing and Reporting Similarity Measures	430
12.7	Fast Fourier Transform (FFT)	431
12.7.1	Discrete Fourier Transform	432
12.7.2	Frequencies of the DFT Bins	433
12.7.3	Radix-2 Cooley–Tukey Algorithm	435
12.7.4	Negative Frequencies and Real Signals	437
12.7.5	Amplitude and Phase Scaling	438
12.7.6	Sampling, Aliasing and Leakage	439
12.7.7	Python Implementation	440
12.7.8	Relation to Campbell Diagrams and Order Analysis . . .	444
12.8	Campbell Diagram and Order Analysis	445

<i>CONTENTS</i>	15
13 Miscellaneous	483
13.1 Bolt	483
13.1.1 Bolt pretension	483

Chapter 1

Coordinate Transformations

1.1 Constructing Transformation Matrix

1.1.1 Rotation Matrix

From: wolfram

1. 2-D (in $\mathbb{R}^2$):

Rotating an object to a new coordinate system:

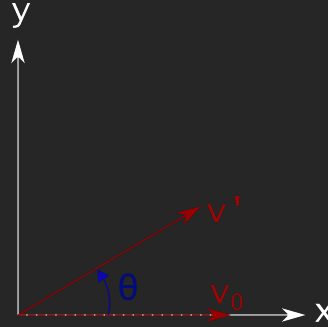


Figure 1.1: Rotating object

In $\mathbb{R}^2$, consider the matrix that rotates a given vector $\mathbf{v}_0$ by a counterclockwise angle θ in a fixed coordinate system. Then

$$\mathbf{T}_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (1.1)$$

so

$$\mathbf{v}' = \mathbf{T}_\theta \mathbf{v}_0. \quad (1.2)$$

On the other hand, consider the matrix that rotates the *coordinate system* through a counterclockwise angle θ . The coordinates of the fixed vector $\mathbf{v}$ in the rotated coordinate system are now given by a rotation matrix which is the *transpose* of the fixed-axis matrix, as can be seen on the second figure, is equivalent to rotating the vector by a counterclockwise angle $-\theta$ relative to a fixed set of axes, giving:

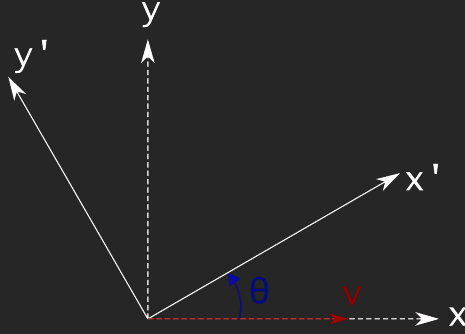


Figure 1.2: Rotating Axes

$$\mathbf{T}'_{\theta} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}. \quad (1.3)$$

This is the convention commonly used in textbooks.

2. 3-D ($\mathbb{R}^3$):

In $\mathbb{R}^3$, coordinate system rotations of the x -, y - and z -axis in a counter-clockwise direction when looking towards the origin give the matrices:

$$\mathbf{R}_x(\varphi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix}. \quad (1.4)$$

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix}. \quad (1.5)$$

$$\mathbf{R}_z(\psi) = \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (1.6)$$

It is best to think about a rotation around axis as about a **rotation in a plane**. Therefore rotation around $\mathbf{x}$ is rotation in $\mathbf{yz}$ plane, rotation around $\mathbf{y}$ is a rotation in $\mathbf{zx}$ plane and a rotation around $\mathbf{z}$ is a rotation in $\mathbf{xy}$ plane.

The defining the index of the axes such that $\mathbf{x} = 0$, $\mathbf{y} = 1$ and $\mathbf{z} = 2$ we can define the rotation matrix so that:

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (1.7)$$

then

$$\begin{aligned} \mathbf{R}_{i,i} &= \cos \theta \\ \mathbf{R}_{i,j} &= \sin \theta \\ \mathbf{R}_{j,i} &= -\sin \theta \\ \mathbf{R}_{j,j} &= \cos \theta \end{aligned} \quad (1.8)$$

where i is index of first plane axis and j is the index of the second plane axis.

The plane axes always cycle in the following manner: $xy, yz, zx, xy, \dots$

Any **rotation** can be given as a composition of rotations about three axes (**Euler's rotation theorem**), and thus be represented by a 3×3 matrix operating on a vector.

$$\begin{bmatrix} x_1' \\ x_2' \\ x_3' \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ t_{31} & t_{32} & t_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}. \quad (1.9)$$

An **arbitrary rotation** around a unit vector $\mathbf{U} = (U_x, U_y, U_z)$ by an angle θ is given by:

$$R_U(\theta) = \mathbf{I} + (\sin \theta)S + (1 - \cos \theta)S^2 \quad (1.10)$$

where $\mathbf{I}$ is an identity matrix,

$$\mathbf{S} = \begin{bmatrix} 0 & -U_x & U_y \\ U_z & 0 & -U_x \\ -U_y & U_x & 0 \end{bmatrix} \quad (1.11)$$

and $\theta > 0$ indicates a counterclockwise rotation in the plane normal to $\mathbf{U}$. The observer is positioned on the side of the plane to which the vector points and looking back at the origin.

3. Alternate way of constructing **Rotation matrix** without specifying angles:

The rotation matrix to a new coordinate system can be constructed from its axes, where the rotation matrix can be defined as:

$$\mathbf{T}_R = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix}, \quad (1.12)$$

where $\mathbf{x} = [x_1, x_2, x_3]^T$, $\mathbf{y} = [y_1, y_2, y_3]^T$ and $\mathbf{z} = [z_1, z_2, z_3]^T$ are unit vectors in the direction of x -, y - and z -axis of the new coordinate system.

The **transformation matrix** can be defined by any two vectors $\mathbf{u}$ and $\mathbf{v}$, where $\mathbf{u}$ defines the direction of *x-axis* and $\mathbf{v}$ defines the *xy-plane*. To construct the **rotation matrix** use the following steps:

- (a) create x-axis of unit length

$$\mathbf{x} = \frac{\mathbf{u}}{\|\mathbf{u}\|}. \quad (1.13)$$

- (b) create z-axis of unit length

$$\mathbf{z} = \frac{\mathbf{u} \times \mathbf{v}}{\|\mathbf{u} \times \mathbf{v}\|}. \quad (1.14)$$

- (c) create y-axis of unit length

$$\mathbf{y} = \frac{\mathbf{z} \times \mathbf{x}}{\|\mathbf{z} \times \mathbf{x}\|}. \quad (1.15)$$

- (d) compose **rotation matrix**

$$\mathbf{T}_R = \begin{bmatrix} \mathbf{x}^T \\ \mathbf{y}^T \\ \mathbf{z}^T \end{bmatrix}. \quad (1.16)$$

The creation of the **rotation matrix** by any two other vectors is analogic, as **orthogonal** base must be constructed first, then the matrix is composed of its vectors.

To test, whether the transformation matrix is correct, one can use the orthogonality criterion:

$$\mathbf{T}^T = \mathbf{T}^{-1} \quad (1.17)$$

$$\mathbf{T}\mathbf{T}^T = \mathbf{T}^T\mathbf{T} = \mathbf{I} \quad (1.18)$$

or

$$\det(\mathbf{T}) = 1 \quad (1.19)$$

1.1.2 Euler's angles of XYZ rotation

With the order of *xyz* rotation the rotation matrix is composed as:

$$\begin{aligned} \mathbf{T} &= \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\varphi) \\ &= \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \\ &= \begin{bmatrix} \cos \theta \cos \psi & \cos \psi \sin \theta \sin \varphi - \cos \varphi \sin \psi & \cos \varphi \cos \psi \sin \theta + \sin \varphi \sin \psi \\ \cos \theta \sin \psi & \cos \varphi \cos \psi + \sin \theta \sin \varphi \sin \psi & \cos \varphi \sin \theta \sin \psi - \cos \psi \sin \varphi \\ -\sin \theta & \cos \theta \sin \varphi & \cos \theta \cos \varphi \end{bmatrix} \end{aligned} \quad (1.20)$$

Then the three *Euler angles* for a *xyz* rotation can be obtained:

$$\varphi = \text{atan2}(t_{32}, t_{33}), \quad (1.21)$$

$$\theta = \text{atan2} \left(-t_{31}, \sqrt{t_{32}^2 + t_{33}^2} \right), \quad (1.22)$$

$$\psi = \text{atan2}(t_{21}, t_{11}), \quad (1.23)$$

where $\text{atan2}(y, x)$ definition is:

$$\text{atan2}(y, x) = \begin{cases} \arctan\left(\frac{y}{x}\right) & \text{if } (x > 0), \\ \arctan\left(\frac{y}{x}\right) + \pi & \text{if } (x < 0) \wedge (y \geq 0), \\ \arctan\left(\frac{y}{x}\right) - \pi & \text{if } (x < 0) \wedge (y < 0), \\ \frac{\pi}{2} & \text{if } (x = 0) \wedge (y > 0), \\ -\frac{\pi}{2} & \text{if } (x = 0) \wedge (y < 0), \\ \text{undefined} & \text{if } (x = 0) \wedge (y = 0) \end{cases}. \quad (1.24)$$

Note the angle limitation for values returned while doing an Euler angle decomposition of a **rotation** matrix:

$$\begin{aligned} \varphi &\in (-\pi, \pi) \\ \theta &\in \left(-\frac{\pi}{2}, \frac{\pi}{2}\right) \\ \psi &\in (-\pi, \pi) \end{aligned}. \quad (1.25)$$

Alternative way to obtain the angles:

$$\begin{cases} \varphi = \text{atan2}(-t_{21}, t_{22}) \\ \theta = \arcsin(t_{20}) \\ \psi = \text{atan2}(-t_{10}, t_{00}) \end{cases} \quad (1.26)$$

A python implementation with alternative components used to obtain the angles

```

1 import numpy as np
2
3 def EulerXYZ(R: np.ndarray) -> tuple:
4     if R[2,0] < 1.:
5         if R[2,0] > -1.:
6             b = np.arcsin(R[2,0])
7             a = np.arctan2(-R[2,1], R[2,2])
8             c = np.arctan2(-R[1,0], R[0,0])
9         else:
10            b = -np.pi / 2.
11            a = -np.arctan2(R[0,1], R[1,1])
12            c = 0.
```



```

13  else:
14      b = np.pi / 2.
15      a = np.arctan2(R[0,1], R[1,1])
16      c = 0.
17
18  return a, b, c

```

1.1.3 Euler's angles of ZYX rotation

For a rotation defined in *zyx* order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \varphi & -\cos \theta \sin \varphi & \sin \varphi \\ \cos \psi \sin \varphi + \cos \varphi \sin \theta \sin \psi & \cos \varphi \cos \psi - \sin \theta \sin \varphi \sin \psi & -\cos \theta \sin \psi \\ \sin \varphi \sin \psi - \cos \varphi \cos \psi \sin \theta & \cos \psi \sin \theta \sin \varphi + \cos \varphi \sin \psi & \cos \theta \cos \psi \end{bmatrix}. \tag{1.27}
 \end{aligned}$$

Note that the angles φ , θ and ψ defined here are different from the *XYZ* rotation defined above! The rotation is performed by first rotating around *z-axis* counterclockwise by an angle ψ , then around *y-axis* counterclockwise by an angle θ and finally around *x-axis*, counterclockwise, by an angle φ .

Then the *Euler's angles* can be obtained as:

$$\begin{cases} \varphi = \arctan\left(\frac{-t_{12}}{t_{11}}\right) \\ \theta = \arcsin(t_{13}) \\ \psi = \arctan\left(\frac{-t_{23}}{t_{33}}\right) \end{cases}. \tag{1.28}$$

Alternative way to obtain the angles:

$$\begin{cases} \varphi = \text{atan2}(t_{12}, t_{22}) \\ \theta = \arcsin(-t_{02}) \\ \psi = \text{atan2}(t_{01}, t_{00}) \end{cases} \tag{1.29}$$

A python implementation with alternative components used to obtain the angles

```

1 import numpy as np
2
3 def EulerZYX(R: np.ndarray) -> tuple:
4     if R[0,2] < 1.:
5         if R[0,2] > -1.:
6             b = np.arcsin(-R[0,2])
7             a = np.arctan2(R[0,1], R[0,0])
8             c = np.arctan2(R[1,2], R[2,2])
9         else:
10            b = np.pi / 2.
11            a = -np.arctan2(-R[2,1], R[1,1])
12            c = 0.
13    else:
14        b = -np.pi / 2.
15        a = np.arctan2(-R[2,1], R[1,1])
16        c = 0.
17
18    return a, b, c

```

1.1.4 Euler's angles of XZY rotation

For a rotation defined in xzy order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi)\mathbf{R}_z(\psi)\mathbf{R}_y(\theta) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \psi & \sin \varphi \sin \theta + \cos \varphi \cos \theta \sin \psi & -\cos \varphi \sin \theta + \cos \theta \sin \varphi \sin \psi \\ -\sin \psi & \cos \varphi \cos \psi & \cos \psi \sin \varphi \\ \cos \psi \sin \theta & -\cos \theta \sin \varphi + \cos \varphi \sin \theta \sin \psi & \cos \varphi \cos \theta + \sin \varphi \sin \theta \sin \psi \end{bmatrix}.
 \end{aligned} \tag{1.30}$$

The the angles are:

$$\begin{cases} \varphi = \text{atan2}(t_{12}, t_{11}) \\ \theta = \text{atan2}(t_{20}, t_{00}) \\ \psi = \arcsin(-t_{10}) \end{cases} \quad (1.31)$$

A python implementation used to obtain the angles

```

1 import numpy as np
2
3 def EulerXZY(R: np.ndarray) -> tuple:
4     if R[1,0] < 1.:
5         if R[1,0] > -1.:
6             b = np.arcsin(-R[1,0])
7             a = np.arctan2(R[1,2], R[1,1])
8             c = np.arctan2(R[2,0], R[0,0])
9         else:
10            b = np.pi / 2.
11            a = -np.arctan2(-R[0,2], R[2,2])
12            c = 0.
13     else:
14         b = -np.pi / 2.
15         a = np.arctan2(-R[0,2], R[2,2])
16         c = 0.
17
18     return a, b, c

```

1.1.5 Euler's angles of ZXY rotation

For a rotation defined in *zxy* order the rotation matrix is:

$$\begin{aligned}
\mathbf{T} &= \mathbf{R}_z(\psi)\mathbf{R}_x(\varphi)\mathbf{R}_y(\theta) \\
&= \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \\
&= \begin{bmatrix} \cos \theta \cos \psi - \sin \varphi \sin \theta \sin \psi & \cos \psi \sin \varphi \sin \theta + \cos \theta \sin \psi & -\cos \varphi \sin \theta \\ -\cos \varphi \sin \psi & \cos \varphi \cos \psi & \sin \varphi \\ \cos \psi \sin \theta + \cos \theta \sin \varphi \sin \psi & -\cos \theta \cos \psi \sin \theta + \sin \theta \sin \psi & \cos \varphi \cos \theta \end{bmatrix}.
\end{aligned} \tag{1.32}$$

The the angles are:

$$\begin{cases} \varphi = \text{atan2}(t_{12}, t_{11}) \\ \theta = \text{atan2}(t_{20}, t_{00}) \\ \psi = \arcsin(-t_{10}) \end{cases} \tag{1.33}$$

Python example of algorithm:

```

1 import numpy as np
2
3 def EulerZXY(R: np.ndarray) -> tuple:
4     if R[1,2] < 1.:
5         if R[1,2] > -1.:
6             b = np.arcsin(R[1,2])
7             a = np.arctan2(-R[1,0], R[1,1])
8             c = np.arctan2(-R[0,2], R[2,2])
9         else:
10            b = -np.pi / 2.
11            a = -np.arctan2(R[2,0], R[0,0])
12            c = 0.
13     else:
14         b = np.pi / 2.
15         a = np.arctan2(R[2,0], R[0,0])
16         c = 0.
17
18     return a, b, c

```

1.1.6 Euler's angles of YXZ rotation

For a rotation defined in yxz order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_y(\theta)\mathbf{R}_x(\varphi)\mathbf{R}_z(\psi) \\
 &= \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \psi + \sin \varphi \sin \theta \sin \psi & \cos \varphi \sin \psi & -\cos \psi \sin \theta + \cos \theta \sin \varphi \sin \psi \\ \cos \psi \sin \varphi \sin \theta - \cos \theta \sin \psi & \cos \varphi \cos \psi & \cos \theta \cos \psi \sin \varphi + \sin \theta \sin \psi \\ \cos \varphi \sin \theta & -\sin \varphi & \cos \varphi \cos \theta \end{bmatrix}.
 \end{aligned} \tag{1.34}$$

The the angles are:

$$\begin{cases} \varphi = \arcsin(-t_{21}) \\ \theta = \text{atan2}(t_{20}, t_{22}) \\ \psi = \text{atan2}(t_{01}, t_{11}) \end{cases} \tag{1.35}$$

Python example of algorithm:

```

1
2 import numpy as np
3
4 def EulerYXZ(R: np.ndarray) -> tuple:
5     if R[2,1] < 1.:
6         if R[2,1] > -1.:
7             b = np.arcsin(-R[2,1])
8             a = np.arctan2(R[2,0], R[2,2])
9             c = np.arctan2(R[0,1], R[1,1])
10        else:
11            b = np.pi / 2.
12            a = -np.arctan2(-R[1,0], R[0,0])
13            c = 0.
14    else:
15        b = -np.pi / 2.
16        a = np.arctan2(-R[1,0], R[0,0])

```

```

17         c = 0.
18
19     return a, b, c

```

1.1.7 Euler's angles of YZX rotation

For a rotation defined in yzx order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_y(\theta)\mathbf{R}_x(\varphi)\mathbf{R}_z(\psi) \\
 &= \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi & \sin \varphi \\ 0 & -\sin \varphi & \cos \varphi \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \psi & \sin \psi & -\cos \psi \sin \theta \\ \sin \varphi \sin \theta - \cos \varphi \cos \theta \sin \psi & \cos \varphi \cos \psi & \cos \theta \sin \varphi + \cos \varphi \sin \theta \sin \psi \\ \cos \varphi \sin \theta + \cos \theta \sin \varphi \sin \psi & -\cos \psi \sin \varphi & \cos \varphi \cos \theta - \sin \varphi \sin \theta \sin \psi \end{bmatrix}.
 \end{aligned} \tag{1.36}$$

The the angles are:

$$\begin{cases} \varphi = \text{atan2}(-t_{12}, t_{11}) \\ \theta = \text{atan2}(-t_{02}, t_{00}) \\ \psi = \arcsin(t_{01}) \end{cases} \tag{1.37}$$

Python example of algorithm:

```

1  import numpy as np
2
3  def EulerYZX(R: np.ndarray) -> tuple:
4      if R[0,1] < 1.:
5          if R[0,1] > -1.:
6              b = np.arcsin(R[0,1])
7              a = np.arctan2(-R[0,2], R[0,0])
8              c = np.arctan2(-R[2,1], R[1,1])
9          else:
10             b = -np.pi / 2.

```

```

11         a = -np.arctan2(R[1,2], R[2,2])
12         c = 0.
13     else:
14         b = np.pi / 2.
15         a = np.arctan2(R[1,2], R[2,2])
16         c = 0.
17
18     return a, b, c

```

1.1.8 Euler's angles of XYX rotation

For a rotation defined in xyx order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi_1) \mathbf{R}_y(\theta) \mathbf{R}_x(\varphi_0) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_1 & \sin \varphi_1 \\ 0 & -\sin \varphi_1 & \cos \varphi_1 \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_0 & \sin \varphi_0 \\ 0 & -\sin \varphi_0 & \cos \varphi_0 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \psi & \sin \psi \cos \varphi_1 & \sin \psi \sin \varphi_1 \\ -\sin \psi \cos \varphi_0 & \cos \psi \cos \varphi_0 \cos \varphi_1 - \sin \varphi_0 \sin \varphi_1 & \cos \psi \cos \varphi_0 \sin \varphi_1 + \cos \varphi_1 \sin \varphi_0 \\ \sin \psi \sin \varphi_0 & -\cos \varphi_0 \sin \varphi_1 - \cos \psi \cos \varphi_1 \sin \varphi_0 & \cos \varphi_0 \cos \varphi_1 - \cos \psi \sin \varphi_0 \sin \varphi_1 \end{bmatrix}.
 \end{aligned} \tag{1.38}$$

The the angles are:

$$\begin{cases} \varphi_0 = \text{atan2}(t_{02}, -t_{10}) \\ \psi = \arccos(t_{00}) \\ \varphi_1 = \text{atan2}(t_{02}, t_{01}) \end{cases} \tag{1.39}$$

Python example of algorithm:

```

1 import numpy as np
2
3 def EulerXYX(R: np.ndarray) -> tuple:
4     if R[0,0] < 1.:
5         if R[0,0] > -1.:

```

```

6         b = np.arccos(R[0,0])
7         a = np.arctan2(R[0,1], -R[0,2])
8         c = np.arctan2(R[1,0], R[2,0])
9     else:
10        b = np.pi
11        a = -np.arctan2(-R[2,1], R[1,1])
12        c = 0.
13    else:
14        b = 0.
15        a = np.arctan2(-R[2,1], R[1,1])
16        c = 0.
17
18    return a, b, c

```

1.1.9 Euler's angles of XZX rotation

For a rotation defined in xzx order the rotation matrix is:

$$\begin{aligned}
 \mathbf{T} &= \mathbf{R}_x(\varphi_1)\mathbf{R}_z(\psi)\mathbf{R}_x(\varphi_0) \\
 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_1 & \sin \varphi_1 \\ 0 & -\sin \varphi_1 & \cos \varphi_1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \varphi_0 & \sin \varphi_0 \\ 0 & -\sin \varphi_0 & \cos \varphi_0 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta & \sin \theta \sin \varphi_1 & -\sin \theta \cos \varphi_1 \\ \sin \theta \sin \varphi_0 & \cos \varphi_0 \cos \varphi_1 - \cos \theta \sin \varphi_0 \sin \varphi_1 & \cos \varphi_0 \sin \varphi_1 + \cos \theta \cos \varphi_1 \sin \varphi_0 \\ \sin \theta \cos \varphi_0 & -\cos \theta \cos \varphi_0 \sin \varphi_1 - \cos \varphi_1 \sin \varphi_0 & \cos \theta \cos \varphi_0 \cos \varphi_1 - \sin \varphi_0 \sin \varphi_1 \end{bmatrix}.
 \end{aligned} \tag{1.40}$$

The the angles are:

$$\begin{cases} \varphi_0 = \text{atan2}(t_{10}, t_{20}) \\ \theta = \arccos(t_{00}) \\ \varphi_1 = \text{atan2}(t_{01}, -t_{02}) \end{cases} \tag{1.41}$$

Python example of algorithm:

```

1 import numpy as np

```



```

2
3 def EulerXZX(R: np.ndarray) -> tuple:
4     if R[0,0] < 1.:
5         if R[0,0] > -1.:
6             b = np.arccos(R[0,0])
7             a = np.arctan2(R[0,2], R[0,1])
8             c = np.arctan2(R[2,0], -R[1,0])
9         else:
10            b = np.pi
11            a = -np.arctan2(R[1,2], R[2,2])
12            c = 0.
13     else:
14         b = 0.
15         a = np.arctan2(R[1,2], R[2,2])
16         c = 0.
17
18     return a, b, c

```

1.1.10 Euler's angles of YXY rotation

```

1 import numpy as np
2
3 def EulerYXY(R: np.ndarray) -> tuple:
4     if R[1,1] < 1.:
5         if R[1,1] > -1.:
6             b = np.arccos(R[1,1])
7             a = np.arctan2(R[1,0], R[1,2])
8             c = np.arctan2(R[0,1], -R[2,1])
9         else:
10            b = np.pi
11            a = -np.arctan2(R[2,0], R[0,0])
12            c = 0.
13     else:
14         b = 0.
15         a = np.arctan2(R[2,0], R[0,0])
16         c = 0.
17

```

```
18     return a, b, c
```

1.1.11 Euler's angles of YZY rotation

```
1  import numpy as np
2
3  def EulerYZY(R: np.ndarray) -> tuple:
4      if R[1,1] < 1.:
5          if R[1,1] > -1.:
6              b = np.arccos(R[1,1])
7              a = np.arctan2(R[1,2], -R[1,0])
8              c = np.arctan2(R[2,1], R[0,1])
9          else:
10             b = np.pi
11             a = -np.arctan2(-R[0,2], R[2,2])
12             c = 0.
13     else:
14         b = 0.
15         a = np.arctan2(-R[0,2], R[2,2])
16         c = 0.
17
18     return a, b, c
```

1.1.12 Euler's angles of ZXZ rotation

Note: One of the most used forms of the **Euler** angles.

```
1  import numpy as np
2
3  def EulerZXZ(R: np.ndarray) -> tuple:
4      if R[2,2] < 1.:
5          if R[2,2] > -1.:
6              b = np.arccos(R[2,2])
```

```

7         a = np.arctan2(R[2,0], -R[2,1])
8         c = np.arctan2(R[0,2], R[1,2])
9     else:
10         b = np.pi
11         a = -np.arctan2(-R[1,0], R[0,0])
12         c = 0.
13 else:
14     b = 0.
15     a = np.arctan2(-R[1,0], R[0,0])
16     c = 0.
17
18 return a, b, c

```

1.1.13 Euler's angles of ZYZ rotation

```

1 import numpy as np
2
3 def EulerZYZ(R: np.ndarray) -> tuple:
4     if R[2,2] < 1.:
5         if R[2,2] > -1.:
6             b = np.arccos(R[2,2])
7             a = np.arctan2(R[2,1], R[2,0])
8             c = np.arctan2(R[1,2], -R[0,2])
9         else:
10            b = np.pi
11            a = -np.arctan2(R[0,1], R[1,1])
12            c = 0.
13     else:
14         b = 0.
15         a = np.arctan2(R[0,1], R[1,1])
16         c = 0.
17
18 return a, b, c

```

1.2 Coordinate Transformation

- **Rotation:**

Let $\mathbf{A}^T = [a_x, a_y, a_z]$ be the original coordinates, $\mathbf{B}^T = [b_x, b_y, b_z]$ be the transformed coordinates, and $\mathbf{T}$ a tranformation matrix in the form:

$$\mathbf{T} = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \end{bmatrix}, \quad (1.42)$$

where $\mathbf{x} = [x_1, x_2, x_3]$, $\mathbf{y} = [y_1, y_2, y_3]$ and $\mathbf{z} = [z_1, z_2, z_3]$ are the *x-axis*, *y-axis* and *z-axis* **unit vectors**, respectively, of the new coordinate system defined in the original one, while sharing their origin.

The following applies:

$$\mathbf{T} \times \mathbf{A} = \mathbf{B}, \quad (1.43)$$

and

$$\mathbf{T}^T \times \mathbf{B} = \mathbf{A}. \quad (1.44)$$

Note: Perpendicular vector is created using *cross-product*.

1.3 Tensor Transformation

From: source 1, source 2, source 3

Let real symmetric matrix $\mathbf{A}$ be the original tensor, real symmetric matrix $\mathbf{B}$ the transformed tensor, and orthogonal matrix $\mathbf{T}$ a tranformation matrix in the form:

$$\mathbf{A} = \begin{bmatrix} \sigma_{11}^A & \sigma_{12}^A & \sigma_{13}^A \\ \sigma_{21}^A & \sigma_{22}^A & \sigma_{23}^A \\ \sigma_{31}^A & \sigma_{32}^A & \sigma_{33}^A \end{bmatrix}, \quad (1.45)$$

$$\mathbf{B} = \begin{bmatrix} \sigma_{11}^B & \sigma_{12}^B & \sigma_{13}^B \\ \sigma_{21}^B & \sigma_{22}^B & \sigma_{23}^B \\ \sigma_{31}^B & \sigma_{32}^B & \sigma_{33}^B \end{bmatrix}, \quad (1.46)$$

$$\mathbf{T} = \begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ z_1 & z_2 & z_3 \end{bmatrix} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{z} \end{bmatrix}, \quad (1.47)$$

where $\mathbf{x} = [x_1, x_2, x_3]$, $\mathbf{y} = [y_1, y_2, y_3]$ and $\mathbf{z} = [z_1, z_2, z_3]$ are the *x-axis*, *y-axis* and *z-axis unit vectors*, respectively, of the new coordinate system defined in the original one, while sharing their origin.

The transformation matrix $\mathbf{T}$ must satisfy $\mathbf{T}\mathbf{T}^T = \mathbf{T}^T\mathbf{T} = \mathbf{I}$, where $\mathbf{I}$ is an identity matrix. This means that transformation matrix must be orthogonal, therefore satisfying $\mathbf{T}^T = \mathbf{T}^{-1}$.

Then **tensor transformation** is performed as such:

$$\mathbf{T} \times \mathbf{A} \times \mathbf{T}^T = \mathbf{B}. \quad (1.48)$$

$$\mathbf{T}^T \times \mathbf{B} \times \mathbf{T} = \mathbf{A}. \quad (1.49)$$

- **Principal values:**

The principal values can be found as the **eigenvalues** of the tensor matrix.

Characteristic equation of tensor $\mathbf{\Sigma}$:

$$\mathbf{\Sigma}\mathbf{V} = \mathbf{\Lambda}\mathbf{V}, \quad (1.50)$$

therefore:

$$(\mathbf{\Sigma} - \mathbf{\Lambda}\mathbf{I})\mathbf{V} = 0, \quad (1.51)$$

and:

$$\det(\mathbf{\Sigma} - \mathbf{\Lambda}\mathbf{I}) = 0, \quad (1.52)$$

where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues (principal values) and $\mathbf{V}$ is the eigenvector matrix.

The characteristic cubic equation can be also written as:

$$\lambda^3 - I_1\lambda^2 + I_2\lambda - I_3 = 0, \quad (1.53)$$

giving three roots equal to tensor eigenvalues.

First compute **Invariants** $\mathbf{I}_n$:

$$\begin{aligned} I_1 &= \sigma_{11} + \sigma_{22} + \sigma_{33} \\ I_2 &= \sigma_{11}\sigma_{22} + \sigma_{22}\sigma_{33} + \sigma_{33}\sigma_{11} - |\sigma_{12}|^2 - |\sigma_{13}|^2 - |\sigma_{23}|^2 \\ I_3 &= -\sigma_{11}|\sigma_{23}|^2 - \sigma_{22}|\sigma_{13}|^2 - \sigma_{33}|\sigma_{12}|^2 + \sigma_{11}\sigma_{22}\sigma_{33} + 2\text{Re}(\sigma_{12}\sigma_{13}\sigma_{23}) \end{aligned} \quad (1.54)$$

where $\text{Re}()$ means **real part** in case some of the values are complex.

In matrix form:

$$\begin{aligned} I_1 &= \text{tr}[\mathbf{\Sigma}] \\ I_2 &= \begin{vmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{vmatrix} + \begin{vmatrix} \sigma_{11} & \sigma_{13} \\ \sigma_{13} & \sigma_{33} \end{vmatrix} + \begin{vmatrix} \sigma_{22} & \sigma_{23} \\ \sigma_{23} & \sigma_{33} \end{vmatrix} \\ I_3 &= \det(\mathbf{\Sigma}) \end{aligned} \quad (1.55)$$

Then compute *help values*:

$$Q = \frac{3I_2 - I_1^2}{9}, \quad (1.56)$$

$$R = \frac{2I_1^3 - 9I_1I_2 + 27I_3}{54}, \quad (1.57)$$

$$\theta = \cos^{-1} \left(\frac{R}{\sqrt{-Q^3}} \right). \quad (1.58)$$

Lastly to get **principal values**:

$$\bar{\sigma}_1 = 2\sqrt{-Q} \cos\left(\frac{\theta}{3}\right) + \frac{1}{3}I_1. \quad (1.59)$$

$$\bar{\sigma}_2 = 2\sqrt{-Q} \cos\left(\frac{\theta + 2\pi}{3}\right) + \frac{1}{3}I_1. \quad (1.60)$$

$$\bar{\sigma}_3 = 2\sqrt{-Q} \cos\left(\frac{\theta + 4\pi}{3}\right) + \frac{1}{3}I_1. \quad (1.61)$$

The **principal values are not sorted**. The principal tensor is then:

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}, \quad (1.62)$$

where $\bar{\sigma}_1, \bar{\sigma}_2$ and $\bar{\sigma}_3 \implies \sigma_1 > \sigma_2 > \sigma_3$.

Afterwards the **principal axes** are obtained as:

$$\begin{aligned} \boldsymbol{\Gamma}_1 &= \boldsymbol{\Sigma} - \sigma_1 \mathbf{I} \\ \boldsymbol{\Gamma}_2 &= \boldsymbol{\Sigma} - \sigma_2 \mathbf{I} , \\ \boldsymbol{\Gamma}_3 &= \boldsymbol{\Sigma} - \sigma_3 \mathbf{I} \end{aligned} \quad (1.63)$$

where $\boldsymbol{\Gamma}$ is a coordinate system corresponding to the principal value i and $\mathbf{I}$ is a 3×3 identity matrix.

The vectors corresponding to each **principal value** are obtained:

$$\boldsymbol{\Gamma}_1 = \begin{bmatrix} \bar{x}_{11} & \bar{x}_{12} & \bar{x}_{13} \\ \bar{y}_{11} & \bar{y}_{12} & \bar{y}_{13} \\ \bar{z}_{11} & \bar{z}_{12} & \bar{z}_{13} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_1 \\ \bar{\mathbf{y}}_1 \\ \bar{\mathbf{z}}_1 \end{bmatrix}, \quad (1.64)$$

$$\boldsymbol{\Gamma}_2 = \begin{bmatrix} \bar{x}_{21} & \bar{x}_{22} & \bar{x}_{23} \\ \bar{y}_{21} & \bar{y}_{22} & \bar{y}_{23} \\ \bar{z}_{21} & \bar{z}_{22} & \bar{z}_{23} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_2 \\ \bar{\mathbf{y}}_2 \\ \bar{\mathbf{z}}_2 \end{bmatrix}, \quad (1.65)$$

$$\boldsymbol{\Gamma}_3 = \begin{bmatrix} \bar{x}_{31} & \bar{x}_{32} & \bar{x}_{33} \\ \bar{y}_{31} & \bar{y}_{32} & \bar{y}_{33} \\ \bar{z}_{31} & \bar{z}_{32} & \bar{z}_{33} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_3 \\ \bar{\mathbf{y}}_3 \\ \bar{\mathbf{z}}_3 \end{bmatrix}, \quad (1.66)$$

where $\bar{\mathbf{x}}$, $\bar{\mathbf{y}}$ and $\bar{\mathbf{z}}$ are vectors of size 3.

$$\mathbf{x} = \frac{\bar{\mathbf{y}}_1 \times \bar{\mathbf{z}}_1}{|\bar{\mathbf{y}}_1 \times \bar{\mathbf{z}}_1|}, \quad (1.67)$$

$$\mathbf{y} = \frac{\bar{\mathbf{z}}_2 \times \bar{\mathbf{x}}_2}{|\bar{\mathbf{z}}_2 \times \bar{\mathbf{x}}_2|}, \quad (1.68)$$

$$\mathbf{z} = \frac{\bar{\mathbf{x}}_3 \times \bar{\mathbf{y}}_3}{|\bar{\mathbf{x}}_3 \times \bar{\mathbf{y}}_3|}. \quad (1.69)$$

The principal axes are also the **eigenvectors** of tensor Σ :

$$\mathbf{V} = [\mathbf{x} \quad \mathbf{y} \quad \mathbf{z}] = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix}. \quad (1.70)$$

- **Principal values, Cardano's formula derivation and python implementation:**

An efficient way to compute the **roots of the cubic equation** is the **Cardano's formula** or its variations. **Cardano's equation** works only for **depressed** 3rd degree polynomials.

Once again we start with a **real, symmetric** tensor Σ :

$$\Sigma = \begin{bmatrix} \sigma_{xx} & \tau_{xy} & \tau_{xz} \\ \tau_{yx} & \sigma_{yy} & \tau_{yz} \\ \tau_{zx} & \tau_{zy} & \sigma_{zz} \end{bmatrix}, \quad (1.71)$$

Then from the definition of eigenvalues and eigenvectors:

$$\Sigma \mathbf{V} = \Lambda \mathbf{V}, \quad (1.72)$$

therefore:

$$(\Sigma - \Lambda \mathbf{I}) \mathbf{V} = 0, \quad (1.73)$$

and:

$$\det(\Sigma - \Lambda \mathbf{I}) = 0, \quad (1.74)$$

where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues (principal values) and $\mathbf{V}$ is the eigenvector matrix.

$$\mathbf{\Lambda}_i = \begin{bmatrix} \lambda_i & 0 & 0 \\ 0 & \lambda_i & 0 \\ 0 & 0 & \lambda_i \end{bmatrix}, \quad (1.75)$$

$$\mathbf{V}_i = \begin{bmatrix} \bar{x}_{i1} & \bar{x}_{i2} & \bar{x}_{i3} \\ \bar{y}_{i1} & \bar{y}_{i2} & \bar{y}_{i3} \\ \bar{z}_{i1} & \bar{z}_{i2} & \bar{z}_{i3} \end{bmatrix} = \begin{bmatrix} \bar{\mathbf{x}}_i \\ \bar{\mathbf{y}}_i \\ \bar{\mathbf{z}}_i \end{bmatrix}. \quad (1.76)$$

It needs to be noted that each **principal value** of a **tensor matrix** corresponds to three vectors that define a coordinate system where first vector is perpendicular to the principal plane.

Rewriting the characteristic cubic equation:

$$\begin{aligned} 0 = & \lambda^3 \\ & + \lambda^2 (-\sigma_{xx} - \sigma_{yy} - \sigma_{zz}) \\ & + \lambda (\sigma_{xx}\sigma_{yy} + \sigma_{xx}\sigma_{zz} + \sigma_{yy}\sigma_{zz} - \tau_{xy}^2 - \tau_{xz}^2 - \tau_{yz}^2) \\ & - \sigma_{xx}\sigma_{yy}\sigma_{zz} + \sigma_{xx}\tau_{yz}^2 + \sigma_{yy}\tau_{xz}^2 + \sigma_{zz}\tau_{xy}^2 - 2\tau_{xy}\tau_{xz}\tau_{yz} \end{aligned} \quad (1.77)$$

and setting I_1, I_2, I_3 as invariants:

$$\begin{aligned} I_1 &= -\sigma_{xx} - \sigma_{yy} - \sigma_{zz} \\ I_2 &= \sigma_{xx}\sigma_{yy} + \sigma_{xx}\sigma_{zz} + \sigma_{yy}\sigma_{zz} - \tau_{xy}^2 - \tau_{xz}^2 - \tau_{yz}^2 \\ I_3 &= \sigma_{xx}\tau_{yz}^2 + \sigma_{yy}\tau_{xz}^2 + \sigma_{zz}\tau_{xy}^2 - \sigma_{xx}\sigma_{yy}\sigma_{zz} - 2Re(\tau_{xy}\tau_{xz}\tau_{yz}) \end{aligned} \quad (1.78)$$

where $Re()$ means **real part** (contrary to $Im()$ which would mean imaginary part).

in **matrix** form:

$$\begin{aligned}
I_1 &= -tr[\mathbf{\Sigma}] \\
I_2 &= \begin{vmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{12} & \sigma_{22} \end{vmatrix} + \begin{vmatrix} \sigma_{11} & \sigma_{13} \\ \sigma_{13} & \sigma_{33} \end{vmatrix} + \begin{vmatrix} \sigma_{22} & \sigma_{23} \\ \sigma_{23} & \sigma_{33} \end{vmatrix} \\
I_3 &= -det(\mathbf{\Sigma})
\end{aligned} \tag{1.79}$$

substituting 1.78 into 1.77 we get:

$$\lambda^3 + I_1\lambda^2 + I_2\lambda + I_3 = 0, \tag{1.80}$$

where the invariants are:

Cardano's method requires first transforming 1.77 to the form:

$$x^3 - 3x = t \tag{1.81}$$

by defining:

$$\begin{aligned}
p &= I_1^2 - 3I_2 \\
q &= -\frac{27}{2}I_3 - I_1^3 + \frac{9}{2}I_1I_2 \\
t &= 2p^{-3/2}q \\
x &= \frac{3}{\sqrt{p}} \left(\lambda + \frac{1}{3}I_1 \right)
\end{aligned} \tag{1.82}$$

the the solution to 1.81 is given by:

$$x = \frac{1}{u} + u \tag{1.83}$$

with

$$u = \sqrt[3]{\frac{t}{2} \pm \sqrt{\frac{t^2}{4} - 1}} \tag{1.84}$$

We can then write $u = e^{i\phi}$, with:

$$\begin{aligned}
\phi &= \frac{1}{3} \arctan \frac{\sqrt{|t^2/4 - 1|}}{t/2} \\
\phi &= \frac{1}{3} \arctan \frac{\sqrt{p^3 - q^2}}{q} \\
\phi &= \frac{1}{3} \arctan \frac{\sqrt{27 \left[\frac{1}{4} I_2^2 (p - I_2) + I_3 * \left(q + \frac{27}{4} I_3 \right) \right]}}{q}
\end{aligned} \tag{1.85}$$

The last step of 1.85 is necessary for numerical stability and precision as the term $p^3 - q^2$ is highly sensitive.

The 1.85 equation can be for better stability and to enforce the angle to the domain $\phi \in \langle -\pi, \pi \rangle$:

$$\phi = \frac{1}{3} \arctan2 \left(\sqrt{27 \left[\frac{1}{4} I_2^2 (p - I_2) + I_3 * \left(q + \frac{27}{4} I_3 \right) \right]}, q \right) \tag{1.86}$$

When evaluating either 1.85 or 1.86, care must be taken to correctly resolve the ambiguity in the arctan by taking account of the sign of q . For $q > 0$, the solution must lie in the first quadrant, for $q < 0$, it must be located in the second. In contrast to this, solutions differing by multiples of 2π are equivalent, so x can take three different values:

$$\begin{aligned}
x_1 &= 2 \cos \phi \\
x_2 &= 2 \cos \left(\phi + \frac{2\pi}{3} \right) = -\cos \phi - \sqrt{3} \sin \phi \\
x_3 &= 2 \cos \left(\phi - \frac{2\pi}{3} \right) = -\cos \phi + \sqrt{3} \sin \phi
\end{aligned} \tag{1.87}$$

These values correspond to the three eigenvalues of Σ :

$$\lambda_i = \frac{\sqrt{p}}{3} x_i - \frac{1}{3} I_1 \tag{1.88}$$

A python implementation:

```

1 import numpy as np
2
3 def principal_cardano_equation(flat_tensor_matrix: np.ndarray) ->
  ↳ np.ndarray:
4     """Eigenvalues of a Hermitian 3x3 matrix using Cardano equations
5
6     source: https://www.mpi-hd.mpg.de/personalhomes/globes/3x3/index.html
7            https://en.wikipedia.org/wiki/Cubic\_equation
8
9     [ sx, txy, txz]  [a, d, f]
10    [txy, sy, tyz] = [d, b, e]
11    [txz, tyz, sz]   [f, e, c]
12
13    Args:
14        flat_tensor_matrix (np.ndarray): a symmetric, positive definite
15        ↳ tensor components
16
17                                     [[sx, sy, sz, txy, tyz, txz],
18                                     [sx, sy, sz, txy, tyz, txz]
19                                     .
20                                     .
21                                     .
22                                     [sx, sy, sz, txy, tyz, txz]]
23
24    Returns:
25        np.ndarray: prinicpal stresses sorted from largest to smallest [s1,
26        ↳ s2, s3]
27
28    """
29    # store each component of the tensor into standalone vector
30    a, b, c, d, e, f = flat_tensor_matrix.T
31
32    # prepare values that are used multiple times
33    M_SQRT3 = np.sqrt(3)
34    de = d * e
35    dd = d * d
36    ee = e * e
37    ff = f * f
38
39    # tr(A)

```

```

36     m = a + b + c
37     # det(A23) + det(13) + det(12)
38     c1 = (a * b + a * c + b * c) - (dd + ee + ff)
39     # det(A)
40     c0 = c * dd + a * ee + b * ff - a * b * c - 2 * f * d * e
41
42     p = m * m - 3.0 * c1
43     q = m * (p - (3.0 / 2.0) * c1) - (27.0 / 2.0) * c0
44     sqrt_p = np.sqrt(np.abs(p))
45
46     phi = 27.0 * ( 0.25 * c1 * c1 * (p - c1) + c0 * (q + 27.0 / 4.0 * c0))
47     phi = (1.0 / 3.0) * np.arctan2(np.sqrt(np.abs(phi)), q)
48
49     c = sqrt_p * np.cos(phi)
50     s = (1.0 / M_SQRT3) * sqrt_p * np.sin(phi)
51
52     l2 = (1.0 / 3.0) * (m - c)
53     l3 = l2 + s
54     l1 = l2 + c
55     l2 -= s
56
57     # Sort the eigenvalues using a fast sorting network
58     l1, l2 = np.minimum(l1, l2), np.maximum(l1, l2)
59     l2, l3 = np.minimum(l2, l3), np.maximum(l2, l3)
60     l1, l2 = np.minimum(l1, l2), np.maximum(l1, l2)
61
62     return np.array([l3, l2, l1], dtype=float).T
63

```

- **Maximum Shear value:**

Maximum **shear stress** occurs at an angle of 45 degrees to principal axes. If principal stresses are aligned such that $\sigma_1 > \sigma_2 > \sigma_3$ and the stress tensor is:

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}, \quad (1.89)$$

then maximum shear stress τ_{max} can be obtained as follows:

$$\tau_{max} = \frac{\sigma_1 - \sigma_3}{2} \quad (1.90)$$

and is at 45 degrees angle in the 1-3 plane of the principal axes to the principal coordinate system.

More generally (in 2D):

$$\tau_{max} = \sqrt{\left(\frac{\sigma_x - \sigma_y}{2}\right)^2 + \tau_{xy}^2}. \quad (1.91)$$

When the principal tensor is rotated by 45 degrees to obtain **maximum shear stress**, the axial stress components corresponding to this shear stress are equal.

The value of maximal shear stress is obtained:

$$\tau_{max} = \frac{\sigma_{max} - \sigma_{min}}{2}. \quad (1.92)$$

The value of axial stresses corresponding to maximal shear stress is the average of maximal and minimal axial stresses:

$$\sigma_{\tau_{max}} = \frac{\sigma_{max} + \sigma_{min}}{2}. \quad (1.93)$$

- **Example:**

Let a stress tensor in principal coordinate system be:

$$\Sigma = \begin{bmatrix} 433 & 0 & 0 \\ 0 & 125 & 0 \\ 0 & 0 & 24 \end{bmatrix}, \quad (1.94)$$

then $\sigma_1 = \sigma_{max} = 433$ MPa and $\sigma_3 = \sigma_{min} = 24$ MPa.

Transformation matrix for rotating by 45 degrees in 3-1 plane is:

$$\mathbf{T} = \begin{bmatrix} \frac{\sqrt{2}}{2} & 0 & -\frac{\sqrt{2}}{2} \\ 0 & 1 & 0 \\ \frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{bmatrix}. \quad (1.95)$$

Rotating tensor Σ by transformation matrix $\mathbf{T}$ we get:

$$\Sigma_\tau = \mathbf{T}\Sigma\mathbf{T}^T = \begin{bmatrix} 228.5 & 0 & 204.5 \\ 0 & 125 & 0 \\ 204.5 & 0 & 228.5 \end{bmatrix}, \quad (1.96)$$

where:

$$\begin{aligned} \tau_{max} &= \sigma_{13} = \sigma_{31} = 204.5 \\ \sigma_{11} &= \sigma_{33} = 228.5 \\ \sigma_{22} &= \sigma_2 \end{aligned} \quad (1.97)$$

Check for correct values:

$$\tau_{max} = \frac{\sigma_1 - \sigma_3}{2} = \frac{433 - 24}{2} = \frac{409}{2} = 204.5 \text{ MPa} \implies \text{OK!}, \quad (1.98)$$

$$\sigma_{11} = \sigma_{33} = \frac{\sigma_1 + \sigma_3}{2} = \frac{433 + 24}{2} = 228.5 \text{ MPa} \implies \text{OK!}. \quad (1.99)$$

σ_2 has not changed as it is colinear with the axis of rotation.

The relationship between principal normal streses and maximum shear stresses can be better understood by examining a plot of the stresses as a function of the rotation angle.

Notice that there are multiple θ_p and $\theta_{\tau-max}$ angles because of the periodical nature of the equations. However, they will give the same absolute values.

At the principal stress angle, θ_p , the shear stress will always be zero, as shown on the diagram. And the maximum shear stress will occur when the two principal normal stresses, σ_1 and σ_2 , are equal.

When σ_x or σ_y are either max or min, the shear stress τ_{xy} is equal to 0. When shear stress τ_{xy} is max or min, the stresses σ_x and σ_y are equal to their average.

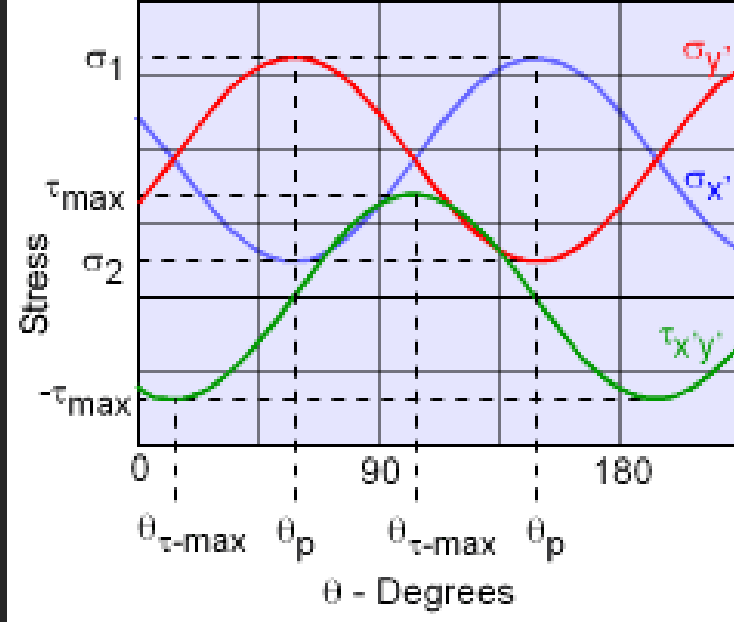


Figure 1.3: Stresses as a function of angle

- **Principal axes, obtaining principal axes with python implementation:**

The principal axes can be obtained from the following property

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \quad (1.100)$$

As the **principal axis** (eigenvector) of $\mathbf{A}$ is not changed by $\mathbf{A}$, only scaled, then it lies in the **nullspace** of the matrix $\mathbf{B}$ from the characteristic equation:

$$\mathbf{B} = (\mathbf{A} - \lambda\mathbf{I}) \quad (1.101)$$

The matrix $\mathbf{B}$ is **singular** with 2 independent rows and a third one being a linear combination of others. The vector that lies in the **nullspace** can be obtained by using a cross product of any two rows. This can be performed

for the first two principal values (eigenvalues λ_i and the third principal axis can be again computed using a cross product of the two previously extracted axes.

Python example:

```

1 import numpy as np
2
3 def gram_schmidt(vectors: np.ndarray) -> np.ndarray:
4     """Perform Gram-Schmidt orthogonalisation of vectors
5
6     Args:
7         vectors (np.ndarray) an array of row vectors to orthogonalise
8             ↪ against
9
10                each other, if the array is 3 dimensional then
11                preform orthogonalisation for each array of
12                row vectors separately
13
14     Returns:
15         (np.ndarray): an array of orthogonal vectors
16     """
17     ortho = np.copy(vectors)
18
19     if len(ortho.shape) == 3:
20         for i in range(1, ortho.shape[1]):
21             for j in range(i):
22                 # np.sum(A * B, axis=1) is equal to row by row dot product
23                 ↪ of two sets of vectors
24                 # written as a 2D matrix
25                 vec = ortho[:,j,:] / np.linalg.norm(ortho[:,j,:],
26                 ↪ axis=1)[:,None]
27                 ortho[:,i,:] -= vec * np.sum(ortho[:,i,:] * vec,
28                 ↪ axis=1)[:,None]
29
30     elif len(ortho.shape) == 2:
31         for i in range(1, ortho.shape[0]):
32             for j in range(i):
33                 # np.sum(A * B, axis=1) is equal to row by row dot product
34                 ↪ of two sets of vectors
35                 # written as a 2D matrix
36                 vec = ortho[j,:] / np.linalg.norm(ortho[j,:])
37                 ortho[i,:] -= vec * np.dot(ortho[i,:], vec)

```

```

30
31     return ortho
32
33
34 def normalise(vectors: np.ndarray) -> np.ndarray:
35     """Normalise supplied vectors to a length of 1"""
36     normalised = np.copy(vectors)
37     if len(normalised.shape) == 3:
38         for i in range(vectors.shape[1]):
39             normalised[:,i,:] /= np.linalg.norm(normalised[:,i,:],
39             ↪ axis=1)[:,None]
40
41     elif len(normalised.shape) == 2:
42         normalised /= np.linalg.norm(normalised, axis=1)[:,None]
43
44     elif len(normalised.shape) == 1:
45         normalised /= np.linalg.norm(normalised)
46
47     return normalised
48
49
50 def orthonormalise(vectors: np.ndarray) -> np.ndarray:
51     """Orthonormalise sets of supplied vectors using GramSchmidt method
52     first and then normalisation to a length of 1
53     """
54     return normalise(gram_schmidt(vectors))
55
56
57 def principal_axes(tensors: np.ndarray, principal_values: np.ndarray) ->
58 ↪ np.ndarray:
59     """Get the principal axes corresponding to the principal values by
60     getting the vector that is in the nullspace of the rank 2 homogeneous
61     equation:
62          $(A - \lambda I) = 0$ 
63     by setting  $\lambda$  to  $\lambda_1$ ,  $\lambda_2$  and  $\lambda_3$ , we can obtain three vectors that are
64     in the nullspace of the first, second and third eigenvalue by using
65     cross product of any two rows of the characteristic equation.
66
67     Args:
68         tensors (np.ndarray):          an array of tensors, one tensor
69                                     on each row

```

```

69         [[sx, sy, sz, txy, tyz, tzx], ... ]
70     principal_values (np.ndarray): an array of the same number of rows
71         as the tensors and 3 columns with
72         principal values
73
74     Returns:
75         (np.ndarray): the principal axes belonging to the supplied
76             principal values
77     """
78     principal = np.copy(principal_values)
79     if len(principal.shape) == 1 and principal.shape[0] == 3:
80         principal = principal.reshape(1, 3)
81
82     if len(principal.shape) != len(tensor.shape):
83         raise ValueError(f"A mismatch in tensor and principal values
84             ↳ dimensions " +
85             f"{str(tensor.shape):s} != {str(principal.shape):s}")
86     elif principal.shape[0] != tensor.shape[0]:
87         raise ValueError(f"The number of principal values does not match
88             ↳ the number of tensors " +
89             f"{str(tensor.shape):s} != {str(principal.shape):s}")
90     elif principal.shape[1] != 3:
91         raise ValueError(f"The number of principal values is not 3:
92             ↳ {str(principal.shape):s}")
93
94     # sort principal stresses by amount from largest to smallest
95     amount_idx = np.argsort(np.abs(principal), axis=1)[::-1]
96     principal = np.take_along_axis(principal, amount_idx, axis=1)
97
98     # prepare reverse sort from largest to smallest
99     amount_idx = np.argsort(principal, axis=1)[::-1]
100
101     a, b, c, d, e, f = tensor.T
102     l1, l2, l3 = principal.T
103
104     # prepare the array to store the 3x3 transformation matrix composed of
105     # rows of principal axes
106     T = np.zeros((principal.shape[0], 3, 3), dtype=tensor.dtype)
107
108     # prepare a mask where True is if the stress tensor is 0
109     mask_zero = np.isclose(a ** 2 + b ** 2 + c ** 2, 0.)

```

```

107
108     # prepare a mask where True is if the prinicpal stresses are equal
109     mask_equal = np.equal(l1, l2, l3)
110
111     # other values - a general case
112     mask_nonz = np.logical_not(mask_zero) & np.logical_not(mask_equal)
113
114     # a zero tensor
115     if np.any(mask_zero):
116         T[mask_zero] = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]],
117             ↪ dtype=tensor.dtype)
118
119     # triaxial tensrion/compression
120     if np.any(mask_equal):
121         T[mask_equal] = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0.,
122             ↪ 1.]], dtype=tensor.dtype)
123
124     # process each eigenvalue (principal stress) individually (only first
125     ↪ two needed)
126     for i, l in enumerate([l1, l2]): # , l3]:
127         # get rows of the tensor (homogeneous equation)
128         v1 = np.array([a - l, d, f], dtype=tensor.dtype).T
129         v2 = np.array([d, b - l, e], dtype=tensor.dtype).T
130         v3 = np.array([f, e, c - l], dtype=tensor.dtype).T
131
132         # get the vector in the tensor nullspace for all combinations of
133         ↪ rows
134         v0 = np.array([np.cross(v1, v2),
135             ↪ np.cross(v1, v3),
136             ↪ np.cross(v2, v3)],
137             ↪ dtype=tensor.dtype).transpose(1,0,2)
138
139         # get the vector length
140         len_v0 = np.array([
141             ↪ np.linalg.norm(v0[:,0], axis=1),
142             ↪ np.linalg.norm(v0[:,1], axis=1),
143             ↪ np.linalg.norm(v0[:,2], axis=1)], dtype=tensor.dtype).T
144
145         # find the index of largest column value for each row
146         idx_v0 = np.argmax(len_v0, axis=1)

```

```

143     # select the longest vector in the nullspace of the homogeneous
144     ↪ equation
145     v0 = np.take_along_axis(v0, idx_v0[:,None,None],
146     ↪ axis=1).reshape(-1,3)
147
148     # and its respective length
149     len_v0 = np.take_along_axis(len_v0, idx_v0[:,None],
150     ↪ axis=1).flatten()
151
152     # prepare an array masking vectors with 0 or close to 0 length
153     mask_zero_len = np.isclose(len_v0, 0.)
154     mask_some_len = np.logical_not(mask_zero_len)
155
156     # norm the vectors that have nonzero length
157     v0[mask_some_len] /= len_v0[mask_some_len,None]
158
159     # now to deal with vectors that have zero length
160     # 1. case where we are dealing with 1st eigenvalue
161     # should not happen due to the presorting of eigenvalues by
162     # magnitude from largest to smallest. If the largest (by magnitude)
163     # eigenvalue has close to 0 length, it is a degenerative case where
164     # there are infinite possibilities for axis direction, so we can
165     # choose one aligned with the global x axis
166     if i == 0:
167         # fallback axis value - global x axis because it does not
168         ↪ matter
169         v0[mask_zero_len] = np.array([1., 0., 0.], dtype=float)
170
171     # 2. case where we are dealing with 2nd eigenvalue
172     elif i == 1:
173         # get first principal axis
174         v = T[mask_zero_len,0,:]
175         # check whether it is the fallback axis
176         mask_occupied = np.all(np.isclose(v, np.array([1., 0., 0.])),
177         ↪ axis=1)
178         # where it is a fallback axis, use global z axis
179         v[mask_occupied] = np.array([0., 0., 1.])
180         # obtain an arbitrary axis that is perpendicular to 1st and
181         ↪ fallback
182         v = np.cross(v, np.array([1., 0., 0.]))
183         # norm it

```

```

178         v /= np.linalg.norm(v, axis=1)[:None]
179         # store it
180         v0[mask_zero_len] = v
181
182         # 3. case where we are dealing with 3rd eigenvalue
183         else:
184             # create an axis perpendicular to the first 2
185             v = np.cross(T[mask_zero_len,0,:], T[mask_zero_len,1,:])
186             # norm it
187             v /= np.linalg.norm(v, axis=1)[:None]
188             # store it
189             v0[mask_zero_len] = v
190
191         # store the axis
192         T[mask_nonz,i,:] = v0[mask_nonz,:]
193
194         # create the 3rd axis using a cross product of the two previous ones
195         T[:,2,:] = np.cross(T[:,0,:], T[:,1,:])
196         T[:,2,:] /= np.linalg.norm(T[:,2,:], axis=1)[:None]
197
198
199         # reorder the principal values and axes back to s1 > s2 > s3
200         principal = np.take_along_axis(principal, amount_idx, axis=1) # for
        ↪ checking
201         T = np.take_along_axis(T, amount_idx[:,None], axis=1)
202
203         return orthonormalise(T)

```

1.4 Moment of Inertia

From: Parallel axis theorem

Parallel axis theorem, also known as *Huygens-Steiner's theorem* or just *Steiner's theorem* can be used to determine the **moment of inertia** or **second moment of area** of a rigid body about any axis, given the body's moment of inertia about a parallel axis through the objects center of gravity and the perpendicular distance between the axes.

Moment of Inertia = **tensor**.

- Steiner's share:

$$\mathbf{I}_{ss} = m \times \begin{bmatrix} y^2 + z^2 & -xy & -xz \\ -yz & x^2 + z^2 & -yz \\ -zx & -zy & x^2 + y^2 \end{bmatrix}. \quad (1.102)$$

Identities for a skew-symmetric matrix

In order to compare formulations of the parallel axis theorem using skew-symmetric matrices and the tensor formulation, the following identities are useful.

Let $[\mathbf{R}]$ be the skew-symmetric matrix associated with the position vector $\mathbf{R} = (x, y, z)$, then the product in the inertia matrix becomes:

$$-[\mathbf{R}][\mathbf{R}] = \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}^2 = \begin{bmatrix} y^2 + z^2 & -xy & -xz \\ -yz & x^2 + z^2 & -yz \\ -zx & -zy & x^2 + y^2 \end{bmatrix}. \quad (1.103)$$

This product can be computed using the matrix formed by the outer product $[\mathbf{R}\mathbf{R}^T]$ using the identity

$$\begin{aligned} -[\mathbf{R}]^2 &= |\mathbf{R}|^2 |\mathbf{E}_3| - [\mathbf{R}\mathbf{R}^T] \\ &= \begin{bmatrix} x^2 + y^2 + z^2 & 0 & 0 \\ 0 & x^2 + y^2 + z^2 & 0 \\ 0 & 0 & x^2 + y^2 + z^2 \end{bmatrix} - \begin{bmatrix} x^2 & xy & xz \\ yx & y^2 & yz \\ zx & zy & z^2 \end{bmatrix}, \end{aligned} \quad (1.104)$$

where $[\mathbf{E}_3]$ is the $n \times n$ identity matrix.

Also notice, that

$$|\mathbf{R}|^2 = \mathbf{R} \cdot \mathbf{R} = \text{tr}[\mathbf{R}\mathbf{R}^T]. \quad (1.105)$$

where tr denotes the sum of the diagonal elements of the outer product matrix, known as its *trace*.

To obtain matrix of moments of inertia by rotating about any random point A while having the moment of inertia in the center of gravity of the object $\mathbf{I}_{COG}$ and the coordinate distance $d = [x, y, z]$ between the point A and **COG** we:

1. Compute the moment of inertia relative to point A

$$\mathbf{I}_A = \mathbf{I}_{COG} + m \times \mathbf{I}_{SS}^{A \rightarrow COG}. \quad (1.106)$$

2. Transform the tensor $\mathbf{I}_A$ to new rotated coordinate system using transformation matrix $\mathbf{T}$:

$$\mathbf{I}'_A = \mathbf{T} \times \mathbf{I}_A \times \mathbf{T}^T. \quad (1.107)$$

3. Subtract the **Steiner's share** relative to new **COG**

$$\mathbf{I}'_{COG} = \mathbf{I}'_A - \mathbf{I}_{SS}^{A' \rightarrow COG'}. \quad (1.108)$$

1.5 Quaternions

In mathematics, the quaternion number system extends the complex numbers. Quaternions were first described by the Irish mathematician William Rowan Hamilton in 1843 and applied to mechanics in three-dimensional space. The algebra of quaternions is often denoted by $\mathbf{H}$ or $\mathbb{H}$ (for Hamilton).

Although multiplication of quaternions is noncommutative, it gives a definition of the quotient of two vectors in a three-dimensional space. Quaternions are generally represented in the form:

$$a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} \quad (1.109)$$

where the coefficients a, b, c, d are *real numbers* and $\mathbf{1}, \mathbf{i}, \mathbf{j}, \mathbf{k}$ are *basis vectors* or *basis elements*.

The Quaternion multiplication table (left column shows pre-multiplier, top row shows post-multiplier):

$r \times c$	$\mathbf{1}$	$\mathbf{i}$	$\mathbf{j}$	$\mathbf{k}$
$\mathbf{1}$	$\mathbf{1}$	$\mathbf{i}$	$\mathbf{j}$	$\mathbf{k}$
$\mathbf{i}$	$\mathbf{i}$	$-\mathbf{1}$	$\mathbf{k}$	$-\mathbf{j}$
$\mathbf{j}$	$\mathbf{j}$	$-\mathbf{k}$	$-\mathbf{1}$	$\mathbf{i}$
$\mathbf{k}$	$\mathbf{k}$	$\mathbf{j}$	$-\mathbf{i}$	$-\mathbf{1}$

Table 1.1: Quaternion multiplication table

also, $a\mathbf{b} = \mathbf{b}a$ and $-\mathbf{b} = (-1)\mathbf{b}$ for $a \in \mathbb{R}$, $b = \mathbf{i}, \mathbf{j}, \mathbf{k}$.

1.5.1 Quaternion properties

The following properties apply:

1. Addition:

$$\begin{aligned}
& (a_1 + b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}) + (a_2 + b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}) = \\
& = (a_1 + a_2) + (b_1 + b_2)\mathbf{i} + (c_1 + c_2)\mathbf{j} + (d_1 + d_2)\mathbf{k}
\end{aligned} \tag{1.110}$$

2. Component-wise scalar multiplication:

$$\lambda(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}) = \lambda a + (\lambda b)\mathbf{i} + (\lambda c)\mathbf{j} + (\lambda d)\mathbf{k} \tag{1.111}$$

3. Multiplication of basis elements (the revelation of Hamilton in 1943):

$$\begin{aligned}
\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 &= -1 \\
\mathbf{ijk} &= -1
\end{aligned} \tag{1.112}$$

4. Multiplicative identity:

$$\begin{aligned}
\mathbf{i}1 &= 1\mathbf{i} = \mathbf{i} \\
\mathbf{j}1 &= 1\mathbf{j} = \mathbf{j} \\
\mathbf{k}1 &= 1\mathbf{k} = \mathbf{k}
\end{aligned} \tag{1.113}$$

5. Products of basis elements:

$$\begin{aligned}
\mathbf{ij} &= -\mathbf{ji} = \mathbf{k} \\
\mathbf{jk} &= -\mathbf{kj} = \mathbf{i} \\
\mathbf{ki} &= -\mathbf{ik} = \mathbf{j}
\end{aligned} \tag{1.114}$$

6. Inverse:

$$(a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k})^{-1} = \frac{1}{a^2 + b^2 + c^2 + d^2}(a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}) \tag{1.115}$$

7. Product of two **quaternions** (also called the **Hamilton product**) is determined by the products of the basis elements and the **distributive** law:

$$(a_1 + b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}) \times (a_2 + b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}) = \quad (1.116)$$

$$\begin{aligned} &= a_1a_2 & +a_1b_2\mathbf{i} & +a_1c_2\mathbf{j} & +a_1d_2\mathbf{k} \\ &+b_1a_2\mathbf{i} & +b_1b_2\mathbf{i}^2 & +b_1c_2\mathbf{ij} & +b_1d_2\mathbf{ik} \\ &+c_1a_2\mathbf{j} & +c_1b_2\mathbf{ij} & +c_1c_2\mathbf{j}^2 & +c_1d_2\mathbf{jk} \\ &+d_1a_2\mathbf{k} & +d_1b_2\mathbf{ik} & +d_1c_2\mathbf{jk} & +d_1d_2\mathbf{k}^2 \end{aligned} \quad (1.117)$$

$$\begin{aligned} &= a_1a_2 & -b_1b_2 & -c_1c_2 & -d_1d_2 \\ &+(a_1b_2 & +b_1a_2 & +c_1d_2 & -d_1c_2) \times \mathbf{i} \\ &+(a_1c_2 & -b_1d_2 & +c_1a_2 & +d_1b_2) \times \mathbf{j} \\ &+(a_1d_2 & +b_1c_2 & -c_1b_2 & +d_1a_2) \times \mathbf{k} \end{aligned} \quad (1.118)$$

1.5.2 Scalar and vector parts

A quaternion of the form $a + 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k}$, where a is a real number, is called **scalar**, and a quaternion of the form $0 + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ where b, c and d are real numbers, and at least one of b, c or d is nonzero, is called a **vector quaternion**. If $a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ is any quaternion, then a is called its **scalar part** and $b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ is called its **vector part**. Although every quaternion can be viewed as a vector in a four-dimensional vector space, it is common to refer to the **vector** part as vectors in three-dimensional space.

Hamilton also called vector quaternions **right quaternions** and real numbers (considered as quaternions with zero vector part) **scalar quaternions**.

If a quaternion is divided up into a scalar and a vector part, that is,

$$\mathbf{q} = (r, \vec{v}), \mathbf{q} \in \mathbb{H}, r \in \mathbb{R}, \vec{v} \in \mathbb{R}^3 \quad (1.119)$$

then the formulas for **addition**, **multiplication** and **multiplicative inverse** are:

$$(r_1, \vec{v}_1) + (r_2, \vec{v}_2) = (r_1 + r_2, \vec{v}_1 + \vec{v}_2) \quad (1.120)$$

$$(r_1, \vec{v}_1) \times (r_2, \vec{v}_2) = (r_1 r_2 - \vec{v}_1 \cdot \vec{v}_2, r_1 \vec{v}_2 + r_2 \vec{v}_1 + \vec{v}_1 \times \vec{v}_2) \quad (1.121)$$

$$(r, \vec{v})^{-1} = \left(\frac{r}{r^2 + \vec{v} \cdot \vec{v}}, \frac{-\vec{v}}{r^2 + \vec{v} \cdot \vec{v}} \right) \quad (1.122)$$

1.5.3 Conjugation, the norm and reciprocal

Conjugation of quaternions is analogous to conjugation of complex numbers. To define it, let $\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ be a quaternion. The **conjugate** of $\mathbf{q}$ is the quaternion $\mathbf{q}^* = a - b\mathbf{i} - c\mathbf{j} - d\mathbf{k}$. It is denoted by $\mathbf{q}^*$, $\mathbf{q}^t$ or $\bar{\mathbf{q}}$.

Conjugation is **involution**, meaning that it is its own inverse, so **conjugating** an element **twice** returns the original element. The **conjugate** of a **product** of two quaternions is the product of the conjugates in the **reverse order**. That is, if $\mathbf{p}$ and $\mathbf{q}$ are quaternions, then:

$$(\mathbf{pq})^* = \mathbf{q}^* \mathbf{p}^* \quad (1.123)$$

The conjugation of a quaternion, in stark contrast to the complex setting, can be expressed with multiplication and addition of quaternions:

$$\mathbf{q}^* = -\frac{1}{2}(\mathbf{q} + \mathbf{iqi} + \mathbf{jqj} + \mathbf{kqi}) \quad (1.124)$$

Conjugation can be used to extract the **scalar** and **vector** part of a quaternion. The **scalar** part of $\mathbf{q}$ is:

$$r_q = \frac{1}{2}(\mathbf{q} + \mathbf{q}^*) \quad (1.125)$$

The **vector** part of $\mathbf{q}$ is:

$$\vec{v}_q = \frac{1}{2}(\mathbf{q} - \mathbf{q}^*) \quad (1.126)$$

The **square root** of the product of a quaternion with its conjugate is called its **norm** and is denoted $\|q\|$ (Hamilton called this quantity the **tensor** of $\mathbf{q}$).

$$\|\mathbf{q}\| = \sqrt{\mathbf{q}^* \mathbf{q}} = \sqrt{\mathbf{q} \mathbf{q}^*} = \sqrt{a^2 + b^2 + c^2 + d^2} \in \mathbb{R} \geq 0 \quad (1.127)$$

This is always a non-negative, real number and is the same as the Euclidean norm on $\mathbb{H}$ considered as the vector space $\mathbb{R}^4$. Multiplying a quaternion by a real number scales its norm by the absolute value of the number. That is, if $\alpha \in \mathbb{R}$ then:

$$\|\alpha \mathbf{q}\| = |\alpha| \|\mathbf{q}\| \quad (1.128)$$

The norm is **multiplicative**, meaning that:

$$\|\mathbf{p} \mathbf{q}\| = \|\mathbf{p}\| \|\mathbf{q}\| \quad (1.129)$$

for any two quaternion $\mathbf{p}$ and $\mathbf{q}$. Multiplicativity is a consequence of the formula for the conjugate of a product. Alternatively it follows from the identity

$$\det \begin{pmatrix} a + ib & id + c \\ id - c & a - ib \end{pmatrix} = a^2 + b^2 + c^2 + d^2 \quad (1.130)$$

(where i denotes the usual imaginary unit) and hence from the multiplicative property of determinants of square matrices.

This norm makes it possible to define the **distance** $d(\mathbf{p}, \mathbf{q})$ between $\mathbf{p}$ and $\mathbf{q}$ as the norm of their difference:

$$d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\| \quad (1.131)$$

This makes $\mathbb{H}$ a **metric space**. Addition and multiplication are **continuous** in regard to the associated metric topology. This follows exactly the same proof as for the real numbers $\mathbb{R}$ from the fact that $\mathbb{H}$ is a **normed algebra**.

1.5.4 Unit quaternion (or versor)

A **unit quaternion** is a quaternion of norm one. Dividing a nonzero quaternion $\mathbf{q}$ by its norm produces a unit quaternion $\mathbf{Uq}$ called the **versor** of $\mathbf{q}$:

$$\mathbf{Uq} = \frac{\mathbf{q}}{\|\mathbf{q}\|} \quad (1.132)$$

Every nonzero quaternion has a unique **polar decomposition** $\mathbf{q} = \|\mathbf{q}\| \cdot \mathbf{Uq}$, while the zero quaternion can be formed from any unit quaternion.

Using conjugation and the norm makes it possible to define the **reciprocal** of a nonzero quaternion. The product of a quaternion with its reciprocal should be equal 1, and the considerations above imply that the product of $\mathbf{q}$ and $\mathbf{q}^*/\|\mathbf{q}\|^2$ is 1 (for any order of multiplication). So the **reciprocal** of $\mathbf{q}$ is defined to be:

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|^2} \quad (1.133)$$

Since the multiplication is non-commutative, the quotient quantities $\mathbf{pq}^{-1}$ or $\mathbf{q}^{-1}\mathbf{p}$ are different (except if $\mathbf{p}$ and $\mathbf{q}$ are scalar multiples of each other or if one is a scalar). The notation $\frac{\mathbf{p}}{\mathbf{q}}$ is ambiguous and **should not be used**.

1.5.5 Quaternions and three-dimensional geometry

The vector part of a quaternion can be interpreted as a coordinate vector in $\mathbb{R}^3$, therefore the algebraic operations for the quaternions reflect the geometry of $\mathbb{R}^3$. Operations such as the vector dot and cross products can be defined in terms of quaternions and this makes it possible to apply quaternion techniques wherever spatial vectors arise. A useful application of quaternions has been to interpolate the orientations of key-frames in computer graphics.

For the remainder of this section, $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ will denote both the three imaginary basis vectors of $\mathbb{H}$ and a basis for $\mathbb{R}^3$. Replacing $\mathbf{i}$ by $-\mathbf{i}$, $\mathbf{j}$ by $-\mathbf{j}$ and $\mathbf{k}$ by $-\mathbf{k}$ sends a vector to its **additive inverse**, so the additive inverse of a vector is the

same as its conjugate as a quaternion. For this reason, conjugation is sometimes called the **spatial inverse**.

For two vector quaternions $\mathbf{p} = b_1\mathbf{i} + c_1\mathbf{j} + d_1\mathbf{k}$ and $\mathbf{q} = b_2\mathbf{i} + c_2\mathbf{j} + d_2\mathbf{k}$ their **dot product**, by analogy to vectors in $\mathbb{R}^3$, is:

$$\mathbf{p} \cdot \mathbf{q} = b_1b_2 + c_1c_2 + d_1d_2 \quad (1.134)$$

It can also be expressed in a component-free manner as:

$$\mathbf{p} \cdot \mathbf{q} = \frac{1}{2}(\mathbf{p}^*\mathbf{q} + \mathbf{q}^*\mathbf{p}) = \frac{1}{2}(\mathbf{pq}^* + \mathbf{qp}^*) \quad (1.135)$$

This is equal to the scalar parts of the products $\mathbf{pq}^*$, $\mathbf{qp}^*$, $\mathbf{p}^*\mathbf{q}$ and $\mathbf{q}^*\mathbf{p}$. Note that their vector parts are different.

The **cross product** of $\mathbf{p}$ and $\mathbf{q}$ relative to the orientation determined by the ordered basis $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ is:

$$\mathbf{p} \times \mathbf{q} = (c_1d_2 - d_1c_2)\mathbf{i} + (d_1b_2 - b_1d_2)\mathbf{j} + (b_1c_2 - c_1b_2)\mathbf{k} \quad (1.136)$$

(The orientation is necessary to determine the sign.) This is equal to the vector part of the product $\mathbf{pq}$ (as quaternions), as well as the vector part of $-\mathbf{q}^*\mathbf{p}^*$. It also has the formula:

$$\mathbf{p} \times \mathbf{q} = \frac{1}{2}(\mathbf{pq} - \mathbf{qp}) \quad (1.137)$$

For the **commutator**, $[\mathbf{p}, \mathbf{q}] = \mathbf{pq} - \mathbf{qp}$ of two vector quaternions one obtains:

$$[\mathbf{p}, \mathbf{q}] = 2\mathbf{p} \times \mathbf{q} \quad (1.138)$$

In general, let $\mathbf{p}$ and $\mathbf{q}$ be quaternions and write:

$$\begin{aligned}\mathbf{p} &= \mathbf{p}_s + \mathbf{p}_v \\ \mathbf{q} &= \mathbf{q}_s + \mathbf{q}_v\end{aligned}\tag{1.139}$$

where $\mathbf{p}_s$ and $\mathbf{q}_s$ are the scalar parts, and $\mathbf{p}_v$ and $\mathbf{q}_v$. Then we have the formula:

$$\mathbf{pq} = (\mathbf{pq})_s + (\mathbf{pq})_v = (\mathbf{p}_s\mathbf{q}_s - \mathbf{p}_v \cdot \mathbf{q}_v) + (\mathbf{p}_s\mathbf{q}_v + \mathbf{q}_s\mathbf{p}_v + \mathbf{p}_v \times \mathbf{q}_v) \tag{1.140}$$

This shows that the noncommutativity of quaternion multiplication comes from the multiplication of vector quaternions. It also shows that two quaternions commute if and only if their vector parts are collinear. **Hamilton** showed that this product computes the third vertex of spherical triangle from two given vertices and their associated arc-lengths, which is also an algebra of points in **Elliptic geometry**.

Unit quaternions can be identified with rotations in $\mathbb{R}^3$ and were called **versors** by **Hamilton**.

1.5.6 Matrix representations

Just as complex numbers can be **represented as matrices**, so can be quaternions. There are *at least* two ways to represent quaternions as matrices in such a way that quaternion **addition** and **multiplication** correspond to the matrix addition and multiplication.

The quaternion $a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ can be represented:

1. as a 2×2 complex matrix:

$$\begin{bmatrix} a + bi & c + di \\ -c + di & a - bi \end{bmatrix} \tag{1.141}$$

This representation has the following properties:

- Constraining any two of b , c and d to zero produces a representation for complex numbers. For example, setting $c = d = 0$ produces a diagonal complex matrix representation of complex numbers, and setting $b = d = 0$ produces a real matrix representation.
- The **norm** of a quaternion (the square root of the product with its conjugate, as with complex numbers) is the square root of the **determinant** of the corresponding matrix.
- The **conjugate** of a quaternion corresponds to the **conjugate transpose** of the matrix.
- There is a strong relation between quaternion units and **Pauli matrices** (these are needed to describe the *spin in quantum mechanics*). Obtain the eight quaternion unit matrices by taking a , b , c and d , set three of them to zero and the fourth to 1 or -1 . Multiplying any two Pauli matrices always yields a quaternion unit matrix, all of them except for -1 . One obtains -1 via the last equality:

$$\begin{aligned} \mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} &= -1 \\ \mathbf{ijk} &= \sigma_1 \sigma_2 \sigma_3 \sigma_1 \sigma_2 \sigma_3 = -1 \end{aligned} \tag{1.142}$$

2. Using 4×4 real matrix the same quaternion can be written as:

$$\begin{aligned} \begin{bmatrix} a & -b & -c & -d \\ b & a & -d & c \\ c & d & a & -b \\ d & -c & b & a \end{bmatrix} &= a \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &+ b \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\ &+ c \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{bmatrix} \\ &+ d \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{aligned} \tag{1.143}$$

However the representation above is not unique. For example, the same quaternion can be also represented as:

$$\begin{aligned}
 \begin{bmatrix} a & d & -b & -c \\ -d & a & c & -b \\ b & -c & a & -d \\ c & b & d & a \end{bmatrix} &= a \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &+ b \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\
 &+ c \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \\
 &+ d \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix}
 \end{aligned} \tag{1.144}$$

There exist 48 distinct matrix representations of thei form in which one of the matrices represents the scalar part and the other three are all skew-symmetric.

1.5.7 Exponential, logarithm and power functions

Given a quaternion:

$$\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k} = r + \vec{\mathbf{v}} \tag{1.145}$$

1. the exponential is computed as

$$\exp(\mathbf{q}) = \sum_{n=0}^{\infty} \frac{\mathbf{q}^n}{n!} = e^a \left(\cos \|\vec{\mathbf{v}}\| + \frac{\vec{\mathbf{v}}}{\|\vec{\mathbf{v}}\|} \sin \|\vec{\mathbf{v}}\| \right) \tag{1.146}$$

2. the logarithm is:

$$\ln(\mathbf{q}) = \ln \|\mathbf{q}\| + \frac{\vec{\mathbf{v}}}{\|\vec{\mathbf{v}}\|} \arccos \frac{r}{\|\mathbf{q}\|} \quad (1.147)$$

3. it follows that the polar decomposition of a quaternion may be written

$$\mathbf{q} = \|\mathbf{q}\| e^{\hat{\mathbf{n}}\varphi} = \|\mathbf{q}\| (\cos(\varphi) + \hat{\mathbf{n}} \sin(\varphi)) \quad (1.148)$$

where the angle φ is defined as

$$r = \|\mathbf{q}\| \cos(\varphi) \quad (1.149)$$

and the unit vector $\hat{\mathbf{n}}$ is defined by

$$\vec{\mathbf{v}} = \hat{\mathbf{n}} \|\vec{\mathbf{v}}\| = \hat{\mathbf{n}} \|\mathbf{q}\| \sin(\varphi) \quad (1.150)$$

Any unit quaternion may be expressed in polar form as:

$$\mathbf{q} = \exp(\hat{\mathbf{n}}\varphi) \quad (1.151)$$

4. The power of a quaternion raised to an arbitrary (real) exponent x is given by:

$$\mathbf{q}^x = \|\mathbf{q}\|^x e^{\hat{\mathbf{n}}x\varphi} = \|\mathbf{q}\|^x (\cos(x\varphi) + \hat{\mathbf{n}} \sin(x\varphi)) \quad (1.152)$$

1.5.8 Quaternions used for coordinate transformation

One can think of **quaternions** as a stored information about coordinate mapping from one space to another. We can split arbitrary **quaternion**

$\mathbf{q} = a + b\mathbf{i} + c\mathbf{j} + d\mathbf{k}$ into **two** parts:

1. a **real** part $r = a$ and
2. a **vector** part $\mathbf{v} = [b, c, d]$.

Next, if we **impose** restrictions on such **quaternions** that:

1. the **real** part must satisfy: $r \in \langle -\pi; \pi \rangle$
2. the **vector** part must have **unit** length: $|\mathbf{v}| = \sqrt{b^2 + c^2 + d^2} = 0$

Then, if one thinks of the complex numbers **i**, **j** and **k** as of basis vectors of cartesian system **x**, **y** and **z**, where

$$\begin{aligned}\mathbf{x} &= [1.0, 0.0, 0.0] \\ \mathbf{y} &= [0.0, 1.0, 0.0] \\ \mathbf{z} &= [0.0, 0.0, 1.0]\end{aligned}\tag{1.153}$$

the quaternion multiplication table 1.1 makes sense in relation to **vector cross product**:

r × c	1	x	y	z
1	1	x	y	z
x	x	-1	z	-y
y	y	-z	-1	x
z	z	y	-x	-1

Table 1.2: Quaternion cartesian base vectors multiplication table

In 3-dimensional space, according to the **Euler's rotation theorem**, any rotation or a sequence of rotations of a rigid body or coordinate system about a fixed point is equivalent to a single rotation by a given angle θ about a fixed axis (called the *Euler axis*) that runs through the fixed point. The Euler axis is typically represented by a **unit vector** $\vec{\mathbf{u}}$. Therefore, any rotation in three dimensions can be represented as via a **vector** $\vec{\mathbf{u}}$ and and angle θ .

Quaternions give a simple way to encode this **axis-angle** representation using four real numbers and can be used to apply (calculate) the corresponding rotation to a **position vector** $[x, y, z]$, representing a point relative to the origin in $\mathbb{R}^3$.

Euclidean vectors such as $[a_x, a_y, a_z]$ can be rewritten as $a_x \mathbf{i} + a_y \mathbf{j} + a_z \mathbf{k}$, where $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ are unit vectors representing the three **Cartesian axes** (traditionally $\mathbf{x}$, $\mathbf{y}$ and $\mathbf{z}$) and also obey the **multiplication** rules of the fundamental quaternion units by interpreting the Euclidean vector $[a_x, a_y, a_z]$ as the vector part of the pure quaternion $(0, a_x, a_y, a_z)$.

A rotation angle θ around the axis defined by the unit vector

$$\vec{\mathbf{u}} = (u_x, u_y, u_z) = u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k} \quad (1.154)$$

can be represented by **conjugation** by a unit quaternion $\mathbf{q}$.

Since the quaternion product

$$(0 + u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k})(0 - u_x \mathbf{i} - u_y \mathbf{j} - u_z \mathbf{k}) = -1 \quad (1.155)$$

using **Taylor series** of the exponential function, the extension of the Euler's formula results in:

$$\begin{aligned} \mathbf{q} &= e^{\frac{\theta}{2}(u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k})} \\ &= \cos \frac{\theta}{2} + (u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k}) \sin \frac{\theta}{2} \\ &= \cos \frac{\theta}{2} + \vec{\mathbf{u}} \sin \frac{\theta}{2} \end{aligned} \quad (1.156)$$

It can be shown that the desired rotation can be applied to an ordinary vector

$$\mathbf{p} = (p_x, p_y, p_z) = p_x \mathbf{i} + p_y \mathbf{j} + p_z \mathbf{k} \quad (1.157)$$

in 3-dimensional space, considered as the vector part of the pure quaternion $\mathbf{p}'$, by evaluating the **conjugation** of $\mathbf{p}'$ by $\mathbf{q}$, given by:

$$\begin{aligned} L(\mathbf{p}') &:= \mathbf{q} \mathbf{p}' \mathbf{q}^{-1} \\ r &= \left(\cos^2 \frac{\theta}{2} - \|\mathbf{u}\|^2 \right) + 2(\mathbf{u} \cdot \mathbf{p})\mathbf{u} + 2 \cos \frac{\theta}{2} (\mathbf{u} \times \mathbf{p}) \end{aligned} \quad (1.158)$$

using the **Hamilton product** where the vector part of the pure quaternion $L(\mathbf{p}') = (0, r_x, r_y, r_z)$ is the new position vector after the rotation. In a programmatic implementation **the conjugation** is achieved by constructing a pure quaternion whose vector part is $\mathbf{p}$, and then performing the quaternion conjugation. The vector part of the resulting pure quaternion is the desired vector $\mathbf{r}$. Clearly L provides a linear transformation of the quaternion space to itself, also, since $\mathbf{q}$ is unitary, the transformation is an isometry. Also $L(\mathbf{q}) = \mathbf{q}$ and so L leaves vectors parallel to $\mathbf{q}$ invariant. So, by decomposing $\mathbf{p}$ as a vector parallel to the vector part $(u_x, u_y, u_z) \sin \frac{\theta}{2}$ of $\mathbf{q}$ and a vector normal to the vector part of $\mathbf{q}$ and showing that the application of L to the normal component of $\mathbf{p}$ rotates it, the claim is shown.

So let $\mathbf{n}$ be the component of $\mathbf{p}$ orthogonal to the vector part of $\mathbf{q}$ and let $\mathbf{n}_T = \mathbf{n} \times \mathbf{u}$. It turns out that the vector part of $L(0, \mathbf{n})$ is given by:

$$\left(\cos^2 \frac{\theta}{2} - \sin^2 \frac{\theta}{2} \right) \mathbf{n} + 2 \left(\cos^2 \frac{\theta}{2} \sin^2 \frac{\theta}{2} \right) \mathbf{n}_T = \cos \theta \mathbf{n} + \sin \theta \mathbf{n}_T \quad (1.159)$$

A geometric fact independent of quaternion is the existence of **two-to-one** mapping from physical rotations to rotational transformation matrices.

If $0 \leq \theta \leq 2\pi$, a physical rotation about $\bar{\mathbf{u}}$ by θ and a physical rotation about $-\bar{\mathbf{u}}$ by $2\pi - \theta$ both achieve the **same final orientation** by disjoint paths through intermediate orientations.

By inserting those vectors and angles into the formula for $\mathbf{q}$ above, one finds that if $\mathbf{q}$ represents the first rotation, $-\mathbf{q}$ represents the second rotation. This is a geometric proof that conjugation by $\mathbf{q}$ and by $-\mathbf{q}$ must produce the same rotational transformation matrix. That fact is confirmed algebraically by noting that the conjugation is quadratic in $\mathbf{q}$, so the sign of $\mathbf{q}$ cancels, and does not affect the result.

If both rotations are a half-turn ($\theta = \pi$), both $\mathbf{q}$ and $-\mathbf{q}$ will have real coordinate equal to zero. Otherwise, one will have a positive real part, representing a rotation by an angle less than π , and the other will have a negative real part, representing a rotation by an angle greater than π .

Mathematically, this operation carries the set of all "pure" quaternions $\mathbf{p}$ (those with real part equal to zero) - which constitute a 3-dimensional space among the

quaternions - into itself, by the desired rotation about the axis $\mathbf{u}$, by the angle θ .

The rotation is **clockwise** if our line of sight points in the same direction as $\bar{\mathbf{u}}$.

In the instance that $\mathbf{q}$ is a **unit quaternion** and

$$\mathbf{q}^{-1} = e^{-\frac{\theta}{2}(u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k})} = \cos \frac{\theta}{2} - (u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k}) \sin \frac{\theta}{2} \quad (1.160)$$

It follows that conjugation by the product of two quaternions is the composition of conjugations by these quaternions. If $\mathbf{p}$ and $\mathbf{q}$ are unit quaternions, then conjugation (**rotation**) by $\mathbf{pq}$ is:

$$\mathbf{pq}\bar{\mathbf{v}}(\mathbf{pq})^{-1} = \mathbf{pq}\bar{\mathbf{v}}\mathbf{q}^{-1}\mathbf{p}^{-1} = \mathbf{p}(\mathbf{q}\bar{\mathbf{v}}\mathbf{q}^{-1})\mathbf{p}^{-1} \quad (1.161)$$

which is **the same** as conjugating (**rotating**) by $\mathbf{q}$ and then by $\mathbf{p}$. The scalar component of the result is **necessarily zero**.

The quaternion **inverse** of a rotation is the opposite rotation, since

$$\mathbf{q}^{-1}(\mathbf{q}\bar{\mathbf{v}}\mathbf{q}^{-1})\mathbf{q} = \bar{\mathbf{v}} \quad (1.162)$$

The square of a quaternion rotation is a rotation by twice the angle around the same axis.

Chapter 2

Boundary conditions not in GCS

2.1 Skew boundary conditions

Skew boundary conditions, also known as **Rotated BCs** are implemented after assembling the global stiffness matrix $\mathbf{K}$.

Basic equation:

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{f} \quad (2.1)$$

First we need to identify all prescribed BCs that do not correspond to the defined assemblage DOFs and transform the finite equilibrium equations to correspond to the prescribed BCs. Thus we write:

$$\mathbf{u} = \mathbf{T}\bar{\mathbf{u}} \quad (2.2)$$

where $\bar{\mathbf{u}}$ is the vector of nodal point DOFs in the required directions. The transformation matrix $\mathbf{T}$ is an identity matrix that has been altered by the directional

sines and cosines of the components $\bar{\mathbf{u}}$ measured in the original displacement directions. Using (2.2), we obtain:

$$\bar{\mathbf{M}}\bar{\mathbf{u}} + \bar{\mathbf{C}}\dot{\bar{\mathbf{u}}} + \bar{\mathbf{K}}\bar{\mathbf{u}} = \bar{\mathbf{f}} \quad (2.3)$$

where:

$$\begin{aligned} \bar{\mathbf{M}} &= \mathbf{T}^T \mathbf{M} \mathbf{T} \\ \bar{\mathbf{C}} &= \mathbf{T}^T \mathbf{C} \mathbf{T} \\ \bar{\mathbf{K}} &= \mathbf{T}^T \mathbf{K} \mathbf{T} \\ \bar{\mathbf{f}} &= \mathbf{T}^T \mathbf{f} \end{aligned} \quad (2.4)$$

We should note that the matrix multiplications in (2.4) involve changes only in those columns and rows of $\mathbf{M}$, $\mathbf{C}$, $\mathbf{K}$ and $\mathbf{f}$ that are actually affected. In practice, the transformation is carried out effectively on the element level just prior to adding the element matrices to the global matrices.

2.2 Cartesian CSYS

When a node or an SPC is defined as a **rotated/skewed** or in a different type of Coordinate system (e.g. cylindrical) a transformation matrix needs to be applied to such nodes.

Cartesian Coordinate system definition by angle (2D):

$$\mathbf{u}_l = \mathbf{T}\mathbf{u}_g \quad (2.5)$$

Unpacked:

$$\begin{bmatrix} u_l \\ w_l \\ \psi_l \end{bmatrix} = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_g \\ w_g \\ \psi_g \end{bmatrix} \quad (2.6)$$

Cartesian Coordinate system definition by vectors (3D):

when a **CSYS** is defined by an **origin** and **two vectors** $\mathbf{x} = (x_1 \ x_2 \ x_3)$ and $\mathbf{y} = (y_1 \ y_2 \ y_3)$, then for a Cartesian CSYS it suffices to:

$$\begin{aligned} \bar{\mathbf{x}} &= \frac{\mathbf{x}}{||\mathbf{x}||} \\ \bar{\mathbf{y}} &= \frac{\mathbf{y}}{||\mathbf{y}||} \\ \bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\mathbf{y}} \end{aligned} \quad (2.7)$$

The transformation relation:

$$\mathbf{u}_l = \mathbf{T}\mathbf{u}_g \quad (2.8)$$

when unpacked:

$$\begin{bmatrix} u_l \\ v_l \\ w_l \\ \phi_l \\ \psi_l \\ \rho_l \end{bmatrix} = \begin{bmatrix} \mathbf{x} & \mathbf{0} \\ \mathbf{y} & \mathbf{0} \\ \mathbf{z} & \mathbf{0} \\ \mathbf{0} & \mathbf{x} \\ \mathbf{0} & \mathbf{y} \\ \mathbf{0} & \mathbf{z} \end{bmatrix} \begin{bmatrix} u_g \\ v_g \\ w_g \\ \phi_g \\ \psi_g \\ \rho_g \end{bmatrix} \quad (2.9)$$

Where $\mathbf{0}$ is a 3×1 zero vector.

For a **2D** case just **one vector** is enough, the $\bar{\mathbf{z}} = (0 \ 0 \ 1)$ and following relations apply:

$$\begin{aligned}\bar{\mathbf{x}} &= \bar{\mathbf{y}} \times \bar{\mathbf{z}} \\ \bar{\mathbf{y}} &= \bar{\mathbf{z}} \times \bar{\mathbf{x}} \\ \bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\mathbf{y}}\end{aligned}\tag{2.10}$$

This means that a **CSYS** defined by an **origin** point $\mathbf{O} = (x_O \ y_O \ z_O)$, point laying on **x axis** $\mathbf{P}_x = (x_{P_x} \ y_{P_x} \ z_{P_x})$ and a point laying in **xy plane** $\mathbf{P}_{xy} = (x_{P_{xy}} \ y_{P_{xy}} \ z_{P_{xy}})$ define a CSYS followingly:

$$\begin{aligned}\mathbf{x} &= (x_{P_x} - x_O \ y_{P_x} - y_O \ z_{P_x} - z_O) \\ \bar{\mathbf{x}} &= \frac{\mathbf{x}}{||\mathbf{x}||} \\ \hat{\mathbf{y}} &= (x_{P_{xy}} - x_O \ y_{P_{xy}} - y_O \ z_{P_{xy}} - z_O) \\ \bar{\hat{\mathbf{y}}} &= \frac{\hat{\mathbf{y}}}{||\hat{\mathbf{y}}||} \\ \bar{\mathbf{z}} &= \bar{\mathbf{x}} \times \bar{\hat{\mathbf{y}}} \\ \bar{\mathbf{y}} &= \bar{\mathbf{z}} \times \bar{\mathbf{x}}\end{aligned}\tag{2.11}$$

2.3 Cylindrical CSYS

The basic relation between a point in **cartesian** and **cylindrical** CSYS is (when the cartesian CSYS has the same origin as the cylindrical one and both CSYS z axes overlay):

$$\begin{aligned}x &= r \cos \varphi \\y &= r \sin \varphi \\z &= z\end{aligned}\tag{2.12}$$

in one direction and:

$$\begin{aligned}r &= \sqrt{x^2 + y^2} \\ \varphi &= \begin{cases} \text{indeterminate} & \text{if } x = 0 \text{ and } y = 0 \\ \arcsin \frac{y}{r} & \text{if } x \geq 0 \\ -\arcsin \frac{y}{r} + \pi & \text{if } x < 0 \text{ and } y \geq 0 \\ -\arcsin \frac{y}{r} + \pi & \text{if } x < 0 \text{ and } y < 0 \end{cases}\end{aligned}\tag{2.13}$$

in the other. One can also use the arctan function to compute φ :

$$\varphi = \begin{cases} \text{indeterminate} & \text{if } x = 0 \text{ and } y = 0 \\ \frac{\pi}{2} \frac{y}{|y|} & \text{if } x = 0 \text{ and } y \neq 0 \\ \arctan \frac{y}{x} & \text{if } x > 0 \\ \arctan \frac{y}{x} + \pi & \text{if } x < 0 \text{ and } y \geq 0 \\ \arctan \frac{y}{x} - \pi & \text{if } x < 0 \text{ and } y < 0 \end{cases}\tag{2.14}$$

This is also called the $\text{atan2}(y, x)$ function!

To transform node coordinates from **cartesian** to **cylindrical**, three transforms are in order:

1. translate the coordinates so that **origin** of the new CSYS conforms to the **origin** of the **cylindrical** CSYS.

$$\mathbf{x}_1 = \mathbf{T}_T \mathbf{x}_0\tag{2.15}$$

$$\begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & -x_O^c \\ 0 & 1 & 0 & -y_O^c \\ 0 & 0 & 1 & -z_O^c \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \\ z_0 \\ 1 \end{bmatrix} \quad (2.16)$$

where $\mathbf{x}_O^c$ denotes the **cylindrical** CSYS origin and $\mathbf{T}_T$ denotes the translation matrix.

2. **rotate** the node to a new **cartesian** CSYS so that the new CSYS **x-axis** conforms to the cylindrical CSYS **r-axis** and the new CSYS **z-axis** conforms to the cylindrical **z-axis**.

$$\mathbf{x}_2 = \mathbf{T}_R \mathbf{x}_1 \quad (2.17)$$

When the cylindrical CSYS is defined by three points - **origin** $\mathbf{x}_O^c = [x_O^c \ y_O^c \ z_O^c]^T$, point on **r-axis** $\mathbf{x}_R^c$ and a point in **rz-plane** $\mathbf{x}_{RZ}^c$, then:

$$\vec{\mathbf{r}} = \frac{\mathbf{x}_R^c - \mathbf{x}_O^c}{\|\mathbf{x}_R^c - \mathbf{x}_O^c\|} \quad (2.18)$$

$$\hat{\mathbf{z}} = \frac{\mathbf{x}_{RZ}^c - \mathbf{x}_O^c}{\|\mathbf{x}_{RZ}^c - \mathbf{x}_O^c\|} \quad (2.19)$$

$$\vec{\mathbf{y}} = \hat{\mathbf{z}} \times \vec{\mathbf{r}} \quad (2.20)$$

$$\vec{\mathbf{z}} = \vec{\mathbf{r}} \times \vec{\mathbf{y}} \quad (2.21)$$

then the transformation can be written:

$$\begin{bmatrix} x_2 \\ y_2 \\ z_2 \\ 1 \end{bmatrix} = \begin{bmatrix} \vec{\mathbf{r}}^T & 0 \\ \vec{\mathbf{y}}^T & 0 \\ \vec{\mathbf{z}}^T & 0 \\ \vec{\mathbf{0}}^T & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix} \quad (2.22)$$

where $\vec{\mathbf{0}} = [0 \ 0 \ 0]^T$

Combining $\mathbf{T}_R$ and $\mathbf{T}_T$ together:

$$\mathbf{T} = \mathbf{T}_R \mathbf{T}_T \quad (2.23)$$

A **cylindrical** coordinate system in FEM in the range of **small displacements** is basically a Cartesian one, just skewed for each node separately so that its **x** axis coincides with **r** axis.

Then the cylindrical to global coordinate transformation is such that:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{global} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} r \cos \varphi \\ r \sin \varphi \\ z \end{bmatrix}_{local} + \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{origin} \quad (2.27)$$

Therefore the Cartesian CSYS transformation matrix for definition of boundary conditions is:

1. first subtract the cylindrical coordinate origin from the global coordinates:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{local} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{global} - \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{origin} \quad (2.28)$$

2. then create a transformation matrix that maps the global csys to rotated one such that the **r** axis coincides with the **x** axis, **φ** axis coincides with the new **y** axis and **z** is the same:

$$\mathbf{r}' = \frac{\mathbf{x}_{local}}{\|\mathbf{x}_{local}\|} \quad (2.29)$$

$$\mathbf{z} = \frac{\mathbf{z}_{cyl}}{\|\mathbf{z}_{cyl}\|} \quad (2.30)$$

$$\boldsymbol{\varphi} = \mathbf{z} \times \mathbf{r}' \quad (2.31)$$

$$\mathbf{r} = \boldsymbol{\varphi} \times \mathbf{z} \quad (2.32)$$

the transformation matrix is then:

$$\mathbf{T} = \begin{bmatrix} r_1 & r_2 & r_3 \\ \varphi_1 & \varphi_2 & \varphi_3 \\ z_1 & z_2 & z_3 \end{bmatrix} \quad (2.33)$$

3. This transformation matrix **T** is finally applied and **BCs** are imposed.

$$\mathbf{T}^T \mathbf{K} \mathbf{T} \bar{\mathbf{u}} = \mathbf{T}^T \mathbf{f} \quad (2.34)$$

If there already is a transformation matrix for the cylindrical system prepared, such that:

$$\mathbf{x}_{local} = \mathbf{T}_{cyl} (\mathbf{x}_{global} - \mathbf{x}_{origin}) \quad (2.35)$$

or:

$$\mathbf{x}_{global} = \mathbf{T}_{cyl}^T \mathbf{x}_{local} + \mathbf{x}_{origin} \quad (2.36)$$

then:

$$r = \sqrt{x_{local}^2 + y_{local}^2} \quad (2.37)$$

$$\varphi = \text{atan2} \frac{y_{local}}{x_{local}} \quad (2.38)$$

$$z = z_{local} \quad (2.39)$$

because $\cos \varphi = x/r$ and $\sin \varphi = y/r$, then the new transformation matrix is composed:

$$\mathbf{T} = \begin{bmatrix} \cos & -\sin & 0 \\ \sin & \cos & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x/r & -y/r & 0 \\ y/r & x/r & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.40)$$

and finally displacements are transformed from global to local:

$$\mathbf{u}_{local} = \mathbf{T} \mathbf{T}_{cyl} \mathbf{u}_{global} \quad (2.41)$$

Chapter 3

Multifreedom Constraints

The *multifreedom equality constraints*, or *multifreedom constraints* for short (**MFCs**) are functional equations that connect *two or more* displacement components:

$$F(\text{nodal displacement components}) = \text{prescribed value} \quad (3.1)$$

where function F vanishes if all its nodal displacement arguments do. Equation (3.1) is called the *canonical form* of the constraint.

An **MFC** of this form is called a *multipoint* or *multinode* if it involves displacement components at different nodes. The constraint is called *linear* if all displacement components appear linearly on the **LHS**, and *nonlinear* otherwise.

The constraint is called *homogeneous* if, upon transferring all terms that depend on displacement components to the **LHS**, the **RHS** - the "prescribed value" in (3.1) is **zero**. It is called *nonhomogeneous* otherwise.

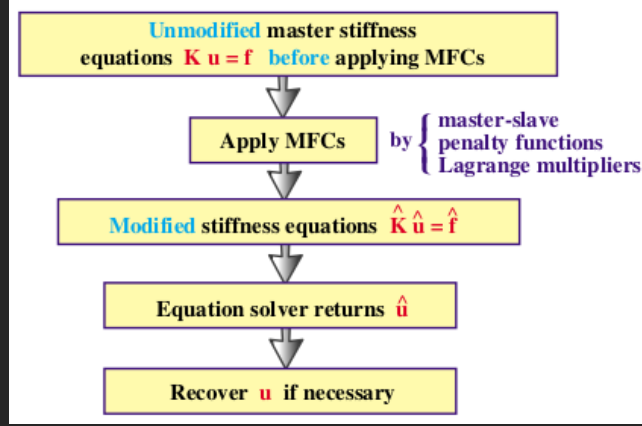


Figure 3.1: Schematics of MFC application.

3.1 Methods for Imposing MFCs

Accounting for MFCs is done, at least conceptually, by changing the assembled master stiffness equations to produce a *modified* system of equations:

$$\mathbf{K} \mathbf{u} = \mathbf{f} \xrightarrow{MFC} \hat{\mathbf{K}} \hat{\mathbf{u}} = \hat{\mathbf{f}} \quad (3.2)$$

The modification process (3.2) is also called *constraint application* or *constraint imposition*. The modified system is that submitted to the equation solver, which returns $\hat{\mathbf{u}}$.

Three methods for applying MFCs are written below:

1. *Master-Slave Elimination*

The DOFs involved in each MFC are separated into master and slave freedoms. The slave freedoms are then explicitly eliminated. The modified equations do not contain the slave freedoms.

2. *Penalty Augmentation*

Also called the *penalty function method*. Each **MFC** is viewed as the presence of a fictitious elastic structural element called the *penalty element* that enforces it approximately. This element is parametrized by a numerical *weight*. The exact constraint is recovered if the weight goes to **infinity**. The MFCs are imposed by augmenting the finite element model with the penalty elements.

3. *Lagrange Multiplier Adjunction*

For each MFC an additional unknown is adjoined to the master stiffness equations. Physically this set of unknowns represent *constraint forces* that would enforce the constraints exactly should they be applied to the unconstrained system.

The master stiffness equations are assembled ignoring all constraints. Then the MFCs are imposed by appropriate modification of those equations. There are, however, two important practical differences:

1. The modification process is not unique because there are alternative constraint imposition methods. These methods offer tradeoffs in generality, programming implementation complexity, computational effort, numerical accuracy and stability.
2. In the implementation of some of these methods - notably penalty augmentation - constraint imposition and assembly are carried out simultaneously. In that case the framework "first assemble, then modify", is not strictly respected in the actual implementation.

3.1.1 MFC Matrix Forms

Matrix forms of *linear* **MFCs** are often convenient for compact notation. An individual constraint may be written:

$$\begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \begin{bmatrix} u_{x2} \\ u_{x4} \\ u_{x6} \end{bmatrix} = 0.25 \quad (3.3)$$

In direct matrix notation:

$$\bar{\mathbf{a}}_i \bar{\mathbf{u}}_i = g_i, \quad (\text{no sum on } i) \quad (3.4)$$

in which index i ($i = 1, 2, \dots$) identifies constraint, $\bar{\mathbf{a}}_i$ is a row vector, $\bar{\mathbf{u}}_i$ collects the set of **DOFs** that participate in the constraint, and g_i is the **RHS** scalar. The bars over $\mathbf{a}$ and $\mathbf{u}$ distinguishes (3.4) from the expanded form below.

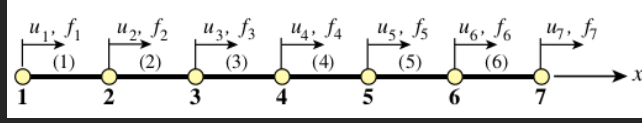


Figure 3.2: 1D problem discretized with six bar finite elements.

For method description and general proof it is often convenient to *expand* matrix forms so that they embody *all* DOFs. For example, if (3.3) is a part of a two-dimensional FE model with 12 DOFs:

$u_{x1}, u_{y1}, \dots, u_{y6}$, the LHS row vector may be expanded with 9 zeros as follows:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & -2 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} u_{x1} \\ u_{y1} \\ u_{x2} \\ \vdots \\ u_{y6} \end{bmatrix} = 0.25 \quad (3.5)$$

In which case the matrix notation:

$$\mathbf{a}_i \mathbf{u}_i = g_i \quad (3.6)$$

is used. Finally, all **MFCs** expressed as (3.6) may be collected into a single matrix relation:

$$\mathbf{A} \mathbf{u} = \mathbf{g} \quad (3.7)$$

in which the rectangular matrix $\mathbf{A}$ is formed by stacking the $\mathbf{a}_i$'s as rows and column vector $\mathbf{g}$ is formed by stacking the g_i s as entries. If there are 12 DOFs in $\mathbf{u}$ and 5 MFCs, then $\mathbf{A}$ will be 5×12 .

3.1.2 The Example Structure

The one-dimensional FE discretization shown in Figure 3.2 will be used to illustrate the three MFC application methods. The structure consists of six bar elements connected by seven nodes that can only displace in the x direction.

Before imposing various **multifreedom** constraints discussed below, the master stiffness equations for this problem are assumed to be:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.8)$$

or

$$\mathbf{K}\mathbf{u} = \mathbf{f} \quad (3.9)$$

The nonzero stiffness coefficients K_{ij} in (3.8) depend on the bar rigidity properties. For example, if $E^e A^e / L^e = 100$ for each element $e = 1, \dots, 6$, then $K_{11} = K_{77} = 100$, $K_{22} = \dots = K_{66} = 200$, $K_{12} = K_{23} = \dots = K_{67} = -100$. However, for the purposes of the following treatment the coefficient may be kept arbitrary. The component index x in the nodal displacements u and nodal forces f has been omitted for brevity.

Now let us specify a MFC that states that nodes **2** and **6** must move by the same amount:

$$u_2 = u_6 \quad (3.10)$$

Passing all node displacements to the RHS gives the canonical form:

$$u_2 - u_6 = 0 \tag{3.11}$$

Constraint conditions of this type are sometimes called **rigid links** because they can be mechanically interpreted as forcing node points 2 and 6 to move together as if they were tied together by a rigid member.

We now study the imposition of constraint (3.11) on the master equations (3.8) by the methods mentioned above. First the master-slave method is treated

3.2 The Master-Slave Method

To apply this method by *hand*, the MFCs are taken one at a time. For each constraint a **slave** DOF is chosen. The freedoms remaining in that constraint are labeled **master**. A new set of DOFs $\hat{\mathbf{u}}$ is established by removing all slave freedoms from $\mathbf{u}$. This new vector contains master freedoms as well as those that do not appear in the MFCs. A matrix transformation equation that relates $\mathbf{u}$ to $\hat{\mathbf{u}}$ is generated. This equation is used to apply a congruent transformation to the master stiffness equations. This procedure yields a set of modified stiffness equations that are expressed in terms of the new freedom set $\hat{\mathbf{u}}$. Because the modified system does not contain the slave freedoms, these have been effectively eliminated.

3.2.1 A One-Constraint Example

The mechanics of the process is best seen by going through an example. To impose (3.11) pick u_6 as slave and u_2 as master. Relate the original unknowns $u_1, \dots, u_7$ to the new set in which u_6 is missing:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_7 \end{bmatrix} \quad (3.12)$$

This is the required transformation relation. In compact form:

$$\mathbf{u} = \mathbf{T}\hat{\mathbf{u}} \quad (3.13)$$

Replacing (3.12) into (3.9) and premultiplying by $\mathbf{T}^T$ yields the modified system:

$$\hat{\mathbf{K}}\hat{\mathbf{u}} = \hat{\mathbf{f}}, \quad \text{in which} \quad \hat{\mathbf{K}} = \mathbf{T}^T \mathbf{K} \mathbf{T}, \quad \hat{\mathbf{f}} = \mathbf{T}^T \mathbf{f}. \quad (3.14)$$

The form of modified system (3.14) can be remembered by a simple mnemonic rule: premultiply both sides of $\mathbf{T}\hat{\mathbf{u}} = \mathbf{u}$ by $\mathbf{T}^T \mathbf{K}$ and replace $\mathbf{K}\mathbf{u}$ by $\mathbf{f}$ on the right hand side.

Carrying out the indicated matrix multiplication yields:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} + K_{66} & K_{23} & 0 & K_{56} & K_{67} \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 \\ 0 & K_{56} & 0 & K_{45} & K_{55} & 0 \\ 0 & K_{67} & 0 & 0 & 0 & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 + f_6 \\ f_3 \\ f_4 \\ f_5 \\ f_7 \end{bmatrix} \quad (3.15)$$

Equation (3.15) is a new linear system containing 6 equations in the remaining 6 unknowns u_1, u_2, u_3, u_4, u_5 and u_7 . Upon solving it, u_6 is recovered from the constraint (3.10).

For a simple freedom constraint such as $u_4 = 0$ the only possible choice of slave is of course u_4 and there is no master. The congruent transformation is then nothing more than the elimination of u_4 by striking out rows and columns from the master stiffness equations.

3.2.2 Multiple Homogeneous MFCs

The matrix equation (3.14) in fact holds for the general case of multiple homogeneous linear constraints. Direct establishment of the transformation equation, however, is more complicated if slave freedoms in one constraint appear as masters in another. To illustrate this point, suppose that for the example system we have three homogeneous MFCs:

$$\begin{aligned} u_2 - u_6 &= 0 \\ u_1 + 4u_4 &= 0 \\ 2u_3 + u_4 + u_5 &= 0 \end{aligned} \tag{3.16}$$

Picking as slave freedoms u_6 , u_4 and u_3 from the first, second and third constraint, respectively, we can solve for them as:

$$\begin{aligned} u_6 &= u_2 \\ u_4 &= -\frac{1}{4}u_1 \\ u_3 &= -\frac{1}{2}(u_4 + u_5) = \frac{1}{8}u_1 - \frac{1}{2}u_5 \end{aligned} \tag{3.17}$$

Observe that solving for u_3 from the third constraint brings u_4 to the RHS. But because u_4 is also slave freedom (it was chosen as such for the second constraint) it is replaced in favor of u_1 using $u_4 = -\frac{1}{4}u_1$. The matrix form transformation is:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \frac{1}{8} & 0 & -\frac{1}{2} & 0 \\ -\frac{1}{4} & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_5 \\ u_7 \end{bmatrix} \quad (3.18)$$

The modified master system is now formed through the congruent transformation (3.14). Note that the slave freedoms selected from each constraint must be distinct. For example the choice u_6 , u_4 and u_4 would be inadmissible as long the constraints are independent. This rule is easy to enforce when slave freedoms are chosen by hand, but can lead to implementation and numerical difficulties when it is programmed as an automated procedure, as further discussed later.

The 3 MFCs (3.16) with u_6 , u_4 and u_2 chosen as slaves and u_1 , u_2 and u_5 chosen as masters, may be presented in the partitioned matrix form:

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 4 & 0 \\ 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} u_3 \\ u_4 \\ u_6 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_5 \end{bmatrix} \quad (3.19)$$

This may be compactly written as:

$$\mathbf{A}_s \mathbf{u}_s + \mathbf{A}_m \mathbf{u}_m = \mathbf{0} \quad (3.20)$$

Solving for the slave freedoms gives

$$\mathbf{u}_s = -\mathbf{A}_s^{-1} \mathbf{A}_m \mathbf{u}_m \quad (3.21)$$

Expanding with zeros to fill out $\mathbf{u}$ and $\hat{\mathbf{u}}$ produces (3.18). Note that non-singularity of $\mathbf{A}_s$ is essential for this method to work.

3.2.3 Nonhomogeneous MFCs

Extension to nonhomogeneous constraints is immediate. In this case the transformation equation becomes nonhomogeneous. For example suppose that (3.11) has non-zero prescribed value:

$$u_2 - u_6 = 0.2 \quad (3.22)$$

Nonzero RHS values such as 0.2 in (3.22) may often be interpreted physically as "gaps" (thus the use of the symbol $\mathbf{g}$ in the matrix form). Chose u_6 again as slave: $u_6 = u_2 - 0.2$ and build the transformation:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -0.2 \\ 0 \end{bmatrix} \quad (3.23)$$

In the compact matrix form:

$$\mathbf{u} = \mathbf{T}\hat{\mathbf{u}} + \mathbf{g} \quad (3.24)$$

Here the constraint gap vector $\mathbf{g}$ is nonzero and $\mathbf{T}$ is the same as before. To get the modified system applying the shortcut rule, premultiply both sides of (3.24) by $\mathbf{T}^T \mathbf{K}$, replace $\mathbf{K}\mathbf{u}$ by $\mathbf{f}$ and pass the data to the RHS:

$$\hat{\mathbf{K}}\hat{\mathbf{u}} = \hat{\mathbf{f}}, \quad \text{in which} \quad \hat{\mathbf{K}} = \mathbf{T}^T \mathbf{K} \mathbf{T}, \quad \hat{\mathbf{f}} = \mathbf{T}^T (\mathbf{f} - \mathbf{K}\mathbf{g}). \quad (3.25)$$

Upon solving (3.25) for $\hat{\mathbf{u}}$, the complete displacement vector is recovered from (3.24). For the MFC (3.22) this technique gives the system:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} + K_{66} & K_{23} & 0 & K_{56} & K_{67} \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 \\ 0 & K_{56} & 0 & K_{45} & K_{55} & 0 \\ 0 & K_{67} & 0 & 0 & 0 & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 + f_6 - 0.2K_{66} \\ f_3 \\ f_4 \\ f_5 - 0.2K_{56} \\ f_7 - 0.2K_{67} \end{bmatrix} \quad (3.26)$$

3.2.4 The General Case

For implementation in general-purpose programs the **master-slave method** can be described as follows. The **DOFs** in $\mathbf{u}$ are classified into three types: **independent** or unconstrained, **masters** and **slaves**. The unconstrained DOFs are those that do not appear in any MFC. Label these sets as $\mathbf{u}_u$, $\mathbf{u}_m$ and $\mathbf{u}_s$, respectively, and partition the stiffness equations accordingly:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{um}^T & \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{us}^T & \mathbf{K}_{ms}^T & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix} \quad (3.27)$$

The MFCs may be written in matrix form as:

$$\mathbf{A}_m \mathbf{u}_m + \mathbf{A}_s \mathbf{u}_s = \mathbf{g}_A \quad (3.28)$$

where $\mathbf{A}_s$ is assumed square and nonsingular. If so we can solve for the slave freedoms:

$$\mathbf{u}_s = \mathbf{A}_s^{-1} \mathbf{A}_m \mathbf{u}_m + \mathbf{A}_s^{-1} \mathbf{g}_A \triangleq \mathbf{T} \mathbf{u}_m + \mathbf{g} \quad (3.29)$$

where $\triangleq$ means *equal by definition*. Inserting into the partitioned stiffness equation (3.27) and symmetrizing yields:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u - \mathbf{K}_{us}\mathbf{g} \\ \mathbf{f}_m - \mathbf{K}_{ms}\mathbf{g} \end{bmatrix} \quad (3.30)$$

where $\hat{\mathbf{K}}_{um} = \mathbf{K}_{um} + \mathbf{K}_{us}\mathbf{T}$
 and $\hat{\mathbf{K}}_{mm} = \mathbf{K}_{mm} + \mathbf{T}^T\mathbf{K}_{ms}^T + \mathbf{K}_{ms}\mathbf{T} + \mathbf{T}^T\mathbf{K}_{ss}\mathbf{T}$.

It is seen that the misleading simplicity of the handworked example is gone.

3.2.5 Retaining the Original Freedoms

A potential disadvantage of the master-slave method in computer work is that it requires a rearrangement of the original stiffness equations because $\hat{\mathbf{u}}$ is a subset of $\mathbf{u}$. The disadvantage can be annoying when sparse matrix storage schemes are used for the stiffness matrix, and becomes intolerable if secondary storage is used for that purpose.

With a bit of trickery it is possible to maintain the original freedom ordering. Let us display it for the example problem under (3.11). Instead of (3.12), use the *square* transformation:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ \bar{u}_6 \\ u_7 \end{bmatrix} \quad (3.31)$$

in which $\bar{u}_6$ is a *placeholder* for the slave freedom u_6 . The modified equations are:

$$\begin{bmatrix}
K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\
K_{12} & K_{22} + K_{66} & K_{23} & 0 & 0 & K_{56} & 0 & K_{67} \\
0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\
0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\
0 & K_{56} & 0 & K_{45} & K_{55} & 0 & 0 & 0 \\
0 & K_{67} & 0 & 0 & 0 & 0 & 0 & K_{77}
\end{bmatrix}
\begin{bmatrix}
u_1 \\
u_2 \\
u_3 \\
u_4 \\
u_5 \\
\bar{u}_6 \\
u_7
\end{bmatrix}
=
\begin{bmatrix}
f_1 \\
f_2 + f_6 \\
f_3 \\
f_4 \\
f_5 \\
0 \\
f_7
\end{bmatrix}
\tag{3.32}$$

which are submitted to the equation solver. If the solver is not trained to skip zero rows and columns, a one should be placed in the diagonal entry for the $\bar{u}_6$ equation. The solver will return $\bar{u}_6 = 0$, and this placeholder value is replaced by u_2 .

3.2.6 Model Reduction by Kinematic Constraints

The congruent transformation equations (3.14) and (3.25) have additional applications beyond the master-slave method. An important one is *model reduction by kinematic constraints*. Through this procedure the number of **DOFs** of a static or dynamic FEM model is reduced by a significant number, typically to 1% – 10% of the original number. This is done by taking a lot of slaves and a few masters. Only the masters are left after the transformation. The reduced model is commonly used in subsequent calculations as a component of a larger system, particularly during design or in parameter identification.

Example:

Consider the bar assembly of Figure 3.3. Assume that the only masters are the end motions u_1 and u_7 , as illustrated in Figure 3.3, and interpolate all freedoms linearly:

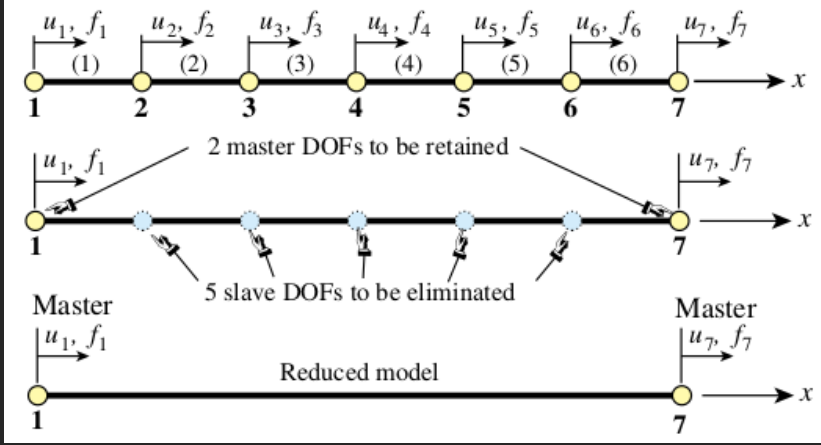


Figure 3.3: Model reduction of the example structure to the end freedoms

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 5 & 1 \\ 4 & 3 \\ 3 & 3 \\ 3 & 3 \\ 6 & 6 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_7 \end{bmatrix}, \quad \text{or} \quad \mathbf{u} = \mathbf{T}\hat{\mathbf{u}} \quad (3.33)$$

The reduced-order-model (**ROM**) equations are:

$$\hat{\mathbf{k}}\hat{\mathbf{u}} = \mathbf{T}^T \mathbf{K} \mathbf{T} \hat{\mathbf{u}} = \mathbf{T}^T \mathbf{f} = \hat{\mathbf{f}} \quad (3.34)$$

or in detail

$$\begin{bmatrix} \hat{K}_{11} & \hat{K}_{17} \\ \hat{K}_{17}^T & \hat{K}_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_7 \end{bmatrix} = \begin{bmatrix} \hat{f}_1 \\ \hat{f}_7 \end{bmatrix} \quad (3.35)$$

in which:

$$\begin{aligned}
\hat{K}_{11} &= \frac{1}{36} (36K_{11} + 60K_{12} + 25K_{22} + 40K_{23} + 16K_{33} + 24K_{34} \\
&\quad + 9K_{44} + 12K_{45} + 4K_{55} + 4K_{56} + K_{66}) \\
\hat{K}_{17} &= \frac{1}{36} (6K_{12} + 5K_{22} + 14K_{23} + 8K_{33} + 18K_{34} + 9K_{44} \\
&\quad + 18K_{45} + 8K_{55} + 14K_{56} + 5K_{66} + 6K_{67}) \\
\hat{K}_{77} &= \frac{1}{36} (K_{22} + 4K_{23} + 4K_{33} + 12K_{34} + 9K_{44} + 24K_{45} \\
&\quad + 16K_{55} + 40K_{56} + 25K_{66} + 60K_{67} + 36K_{77}) \\
\hat{f}_1 &= \frac{1}{6} (6f_1 + 5f_2 + 4f_3 + 3f_4 + 2f_5 + f_6) \\
\hat{f}_7 &= \frac{1}{6} (f_2 + 2f_3 + 3f_4 + 4f_5 + 5f_6 + 6f_7)
\end{aligned} \tag{3.36}$$

This reduces the order of the FEM model from 7 to 2. The **key feature** is that the masters are picked **a priori**, as the freedoms to be retained in the model for further use.

Model reduction can be also done by the **static condensation** (Guyan) method. As its name indicates, condensation is restricted to static analysis. On the other hand, for such problems it is exact whereas model reduction by kinematic constraints generally introduces approximations.

3.2.7 Assesment of the Master-Slave Method

The **Master-Slave Method** enjoys the advantage of being **exact** (except for inevitable solution errors from finite number precision) and of reducing the nuber of unknowns. The concept is also easy to explain and learn. The main implementation drawback is the complexity of the general case. The complexity is due to three factors:

1. The equations may have to be rearranged because of the disappearance

of the slave freedoms. This drawback can be alleviated, however, by the **placeholder** trick.

2. An auxiliary linear system has to be assembled and solved to produce the transformation matrix $\mathbf{T}$ and vector $\mathbf{g}$.
3. The transformation process may generate many additional matrix terms. If a sparse matrix storage scheme is used for $\mathbf{K}$, the logic for allocating memory and storing these entries can be difficult and expensive.

The level of complexity depends on the generality allowed as well as on programming decisions. If $\mathbf{K}$ is stored as full matrix and slave freedom coupling in the MFCs is disallowed the logic is simple (this is the case in model reduction, since each slave freedom appears in one and only one MFC). On the other hand, if arbitrary couplings are permitted and $\mathbf{K}$ is placed in secondary (disk) storage according to some sparse scheme, the complexity can become overwhelming.

Another, more subtle, drawback of this method is that it requires decisions as to which DOFs are to be treated as slaves. This can lead to implementation and numerical stability problems. Although for disjointed constraints the process can be programmed in reliable form, in more general cases of coupled constraint equations it can lead to incorrect decisions. For example, suppose that in the example problem you have the following two MFCs:

$$\begin{aligned}\frac{1}{6}u_2 + \frac{1}{2}u_4 &= u_6 \\ u_3 + 6u_6 &= u_7\end{aligned}\tag{3.37}$$

For numerical stability reasons it is usually better to pick as slaves the freedoms with largest coefficients. If this is done, the program would select u_6 as slave freedoms from both constraints. This leads to a contradiction because having two constraints we must eliminate two slave DOFs, not just one. The resulting modified system would in fact be inconsistent. Although this defect can be easily fixed by the program logic in this case, one can imagine the complexity burden if faced with hundreds of thousands of MFCs.

Serious numerical problems can arise if the MFCs are not independent. For example:

$$\begin{aligned}
\frac{1}{6}u_2 &= u_6 \\
\frac{1}{3}u_3 + 6u_6 &= u_7 \\
u_2 + u_3 - u_7 &= 0
\end{aligned} \tag{3.38}$$

The last constraint is an exact linear combination of the first two. If the program blindly choses u_2 , u_3 and u_7 as slaves, the modified system is incorrect because we eliminate three equations when in fact there are only two independent constraints. Exact linear dependence, as before, can be recognised by a rank analysis of the $\mathbf{A}_s$ matrix defined. In the floating-point arithmetic, however, such detection may fail because that kind of computation is inexact by nature (The safest technique to identify dependencies is to do a singular value decomposition (SVD) of $\mathbf{A}_s$. This can be, however, prohibitively expensive if one is dealing with hundreds of thousands of constraints.).

The complexity of slave selection is in fact equivalent to that of automatically selecting kinematic redundancies in the Force Method of structural analysis. **It has led implementators of programs that use this method to require masters and slaves to be prescribed in the input data, thus transferring the burden to users.**

The method is not generally extensible to nonlinear constraints without case by case programming.

In conclusion, the master-slave method is useful when a few simple linear constraints are imposed by hand. As a general purpose technique for FEM it suffers from complexity and lack of robustness. It is worth learning, however, because of the great importance of congruent transformations in *model reduction* for static and dynamic problems.

Bibliography

Felippa - IFEM (Chapter 8)

Zienkiewicz and Taylor - The Finite Element Method *4th ed.* 1988

3.3 The Penalty Method

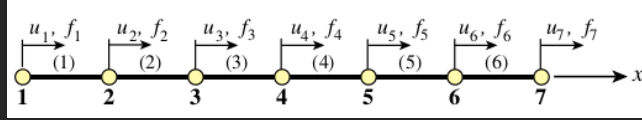


Figure 3.4: 1D problem discretized with six bar finite elements.

The penalty method will be first presented using a physical interpretation, leaving the mathematical formulation to a subsequent section. Consider again the bar example structure. To impose $u_2 = u_6$ imagine that nodes 2 and 6 are connected with a "fat" bar of axial stiffness w , labeled with element number 7, as shown in Figure 3.5. This bar is called **penalty element** and w is its **penalty weight**.

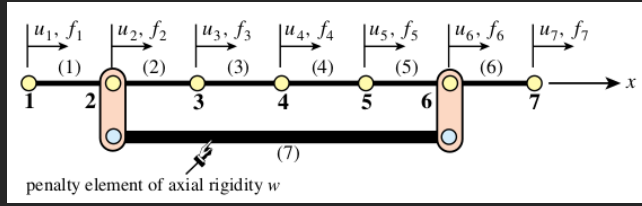


Figure 3.5: Adjunction of a fictitious penalty bar of axial stiffness w , identified as element 7, to enforce $u_2 = u_6$.

Such an element, albeit fictitious, can be treated exactly like another bar element insofar as continuing the assembly of the master stiffness equations. The penalty element stiffness equations $\mathbf{K}^{(7)}\mathbf{u}^{(7)} = \mathbf{f}^{(7)}$, are:

$$w \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} u_2 \\ u_6 \end{bmatrix} = \begin{bmatrix} f_2^{(7)} \\ f_6^{(7)} \end{bmatrix} \quad (3.39)$$

Because there is one freedom per node, the two local element freedoms map into global freedoms 2 and 6, respectively. Using the assembly rules we obtain the following modified master stiffness equations: $\hat{\mathbf{K}}\hat{\mathbf{u}} = \hat{\mathbf{f}}$, which shown in detail are:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} + w & K_{23} & 0 & 0 & -w & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 \\ 0 & -w & 0 & 0 & K_{56} & K_{66} + w & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.40)$$

This system can now be submitted to the equation solver. Note that $\hat{\mathbf{u}} \equiv \mathbf{u}$ and only $\mathbf{K}$ has changed.

3.3.1 Choosing the Penalty Weight

What happens when (3.40) is solved numerically? If a *finite* weight w is chosen the constraint $u_2 = u_6$ is approximately satisfied in the sense that one gets $u_2 - u_6 = e_g$, where $e_g \neq 0$. The "gap error" e_g is called the *constraint violation*. The magnitude $|e_g|$ of this violation depends on the weight: the larger w , the smaller the violation. More precisely, it can be shown that $|e_g|$ becomes proportional to $\frac{1}{w}$ as w gets to be sufficiently large. For example, raising w from, say, 10^6 to 10^7 can be expected to cut the constraint violation roughly by 10 if the physical stiffnesses are small compared to w .

Therefore it seems as if the proper strategy should be: try to make w as large as possible while respecting the computer overflow limits. However, this is misleading. As the penalty weight w tends to ∞ the modified stiffness matrix becomes more and more *ill-conditioned with respect to inversion*.

To make this point clear, suppose for definiteness that rigidities $E^e A^e / L^e$ of the actual bars $e = 1, \dots, 6$ are unity, that $w \gg 1$, and that the computer solving the stiffness equations has a floating-point precision of 16 decimal places. Numerical analysts characterize such precision by saying that $\epsilon_f = \mathcal{O}(10^{-16})$, where $|\epsilon_f|$ is the smallest power of 10 that perceptibly adds to 1 in floating-point arithmetic (this is usually done not for decimal numbers but base-2 binary numbers). The modified stiffness matrix of (3.40) becomes:

$$\hat{\mathbf{K}} = \begin{bmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2+w & -1 & 0 & 0 & -w & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & -w & 0 & 0 & -1 & 2+w & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 \end{bmatrix} \quad (3.41)$$

Clearly as $w \rightarrow \infty$ rows 2 and 6, as well as columns 2 and 6, tend to become linearly dependent, in fact the negative of each other. But **linear dependency means singularity!** Thus $\hat{\mathbf{K}}$ approaches singularity as $w \rightarrow \infty$. In fact, if w exceeds $1/\epsilon_f = 10^{16}$ the computer will not be able to distinguish $\hat{\mathbf{K}}$ from an exactly singular matrix. If $w < 10^{16}$, the effect will be seen in increasing solution errors affecting the computed displacements $\hat{\mathbf{u}}$ returned by the equation solver. These errors, however, tend to be more random in nature than the constraint violation error.

3.3.2 The Square Root Rule

Obviously we have two effects at odds with each other. Making w larger reduces the constraint violation error but increases the solution error. The best w is that which makes both errors roughly equal in absolute value. This tradeoff value is difficult to find aside of systematically running numerical experiments. In practice *square root rule* is often followed.

This rule can be stated as follows:

Suppose that the largest stiffness coefficient, before adding penalty elements, is of the order of 10^k and that the working machine precision is p digits (such order-of-magnitude estimates can be easily found by scanning just the diagonal of $\mathbf{K}$). Then choose penalty weights to be of order $10^{k+p/2}$ with the proviso that such a choice would not cause arithmetic overflow (if overflow occurs, the master stiffness matrix should be scaled throughout or a better choice of physical units made).

For the above example in which $k \approx 0$ and $p \approx 16$, the optimal w given by this rule would be $w \approx 10^8$. This w would yield a constraint violation and a solution

error of order 10^{-8} . Note that there is no simple way to do better than this accuracy aside from using **extended floating-point precision**. This is not easy to do when using standard low-level programming languages.

The name "square root@" arises because the recommended w is in fact $10^k \sqrt{10^p}$. It is seen that picking the weight by this rule requires knowledge of both stiffness magnitudes and floating-point hardware properties of the computer used, as well as the precision selected by the program.

3.3.3 Penalty Elements for General MFCs

For the constraint $u_2 = u_6$ the physical interpretation of the penalty element is clear. Points 2 and 6 must move in lockstep along x , which can be approximately enforced by the heavy bar device. But how about $3u_3 + u_5 - 4u_6 = 1$ or just $u_2 = -u_6$?

The treatment of more general constraints is linked to the theory of *Courant penalty functions* from the *variational calculus*. Consider the homogenous constraint:

$$4u_3 + u_5 - 4u_6 = 0 \quad (3.42)$$

Rewrite this equation in matrix form:

$$\begin{bmatrix} 3 & 1 & -4 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = 0 \quad (3.43)$$

and premultiply both sides by the transpose of the coefficient matrix:

$$\begin{bmatrix} 3 \\ 1 \\ -4 \end{bmatrix} \begin{bmatrix} 3 & 1 & -4 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} 9 & 3 & -12 \\ 3 & 1 & -4 \\ -12 & -4 & 16 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = \bar{\mathbf{K}}^e \mathbf{u} = 0 \quad (3.44)$$

Here $\bar{\mathbf{K}}^e$ is the *unscaled* stiffness matrix of the penalty element. This is now multiplied by the penalty weight w and assembled into the master stiffness

matrix following the usual rules. For the example problem, augmentint the master stiffness matrix with the w -scaled penalty element yields:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} + 9w & K_{34} & 3w & -12w & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 3w & K_{45} & K_{55} + w & K_{56} - 4w & 0 \\ 0 & 0 & -12w & 0 & K_{56} - 4w & K_{66} + 16w & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.45)$$

If the constraint is nonhomogeneous the force vector is also modified. To illustrate this effect, consider the MFC: $3u_3 + u_5 - 4u_6 = 1$. Rewrite this in matrix form as:

$$\begin{bmatrix} 3 & 1 & -4 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = 1 \quad (3.46)$$

Premultiply both sides by the transpose of the coefficient matrix:

$$\begin{bmatrix} 9 & 3 & -12 \\ 3 & 1 & -4 \\ -12 & -4 & 16 \end{bmatrix} \begin{bmatrix} u_3 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \\ -4 \end{bmatrix} \quad (3.47)$$

Scaling by w and assembling yields:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} + 9w & K_{34} & 3w & -12w & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 3w & K_{45} & K_{55} + w & K_{56} - 4w & 0 \\ 0 & 0 & -12w & 0 & K_{56} - 4w & K_{66} + 16w & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 + 3w \\ f_4 \\ f_5 + w \\ f_6 - 4w \\ f_7 \end{bmatrix} \quad (3.48)$$

3.3.4 The Theory Behind the Penalty Method Coefficient Matrix

The rule of *Courant penalty functions* comes from the following mathematical theory. Suppose we have a set of m linear MFCs. Using the matrix notation introduced, these will be stated as:

$$\mathbf{a}_p \mathbf{u} = b_p, \quad p = 1, \dots, m \quad (3.49)$$

where $\mathbf{u}$ contains all DOFs and each $\mathbf{a}_p$ is a row vector with the same length as $\mathbf{u}$. To incorporate the MFCs into the FEM model one selects a weight $w_p > 0$ for each constraint and constructs the so-called **Courant quadratic penalty function** or "penalty energy":

$$P = \sum_{p=1}^m P_p, \text{ with } P_p = \mathbf{u}^T \left(\frac{1}{2} \mathbf{a}_p^T \mathbf{a}_p \mathbf{u} - w_p \mathbf{a}_p^T b_p \right) = \frac{1}{2} \mathbf{u}^T \mathbf{K}^{(p)} \mathbf{u} - \mathbf{u}^T \mathbf{f}^{(p)} \quad (3.50)$$

where we have called $\mathbf{K}^{(p)} = w_p \mathbf{a}_p^T \mathbf{a}_p$ and $\mathbf{f}^{(p)} = w_p \mathbf{a}_p^T b_p$. P is added to the potential energy function $\Pi = \frac{1}{2} \mathbf{u}^T \mathbf{K} \mathbf{u} - \mathbf{u}^T \mathbf{f}$ to form the *augmented potential energy* $\Pi_a = \Pi + P$. Minimization of Π_a with respect to $\mathbf{u}$ yields:

$$\left(\mathbf{K} \mathbf{u} + \sum_{p=1}^m \mathbf{K}^{(p)} \right) \mathbf{u} = \mathbf{f} + \sum_{p=1}^m \mathbf{f}^{(p)} \quad (3.51)$$

Each term of the sum on p , which derives from term P_p may be viewed as contributed by a penalty element with globalised stiffness matrix $\mathbf{K}^{(p)} = w_p \mathbf{a}_p^T \mathbf{a}_p$ and globalised added force term $\mathbf{f}^{(p)} = w_p \mathbf{a}_p^T b_p$.

To use a even more compact form we may write the set of MFCs as $\mathbf{A} \mathbf{u} = \mathbf{b}$. Then the penalty augmented system can be written compactly as:

$$(\mathbf{K} + \mathbf{A}^T \mathbf{W} \mathbf{A}) \mathbf{u} = \mathbf{f} + \mathbf{W} \mathbf{A}^T \mathbf{b} \quad (3.52)$$

where $\mathbf{W}$ is a diagonal matrix of penalty weights. This compact form, however, conceals the configuration of the penalty elements.

3.3.5 Assessment of the Penalty Method

The main advantage of the penalty function method is its straightforward computer implementation. Looking at the modified systems it is obvious that master equations need not be rearranged. That is, $\mathbf{u}$ and $\hat{\mathbf{u}}$ are the same. Constraints may be programmed as "penalty elements", and stiffness and force contributions for these elements merged through the standard assembler. In fact using this method there is no need to distinguish between unconstrained and constrained equations! Once all elements - regular and penalty - are assembled, the system can be passed to the equation solver. **Single Freedom Constraints**, are usually processed separately for efficiency.

An important advantage with respect to the master-slave (elimination) method is its lack of sensitivity with respect to whether constraints are linearly dependent. To give a simplistic example, suppose that the constraint $u_2 = u_6$ appears twice. Then two penalty elements connecting 2 and 6 will be inserted, doubling the intended weight but not otherwise causing undue harm.

An advantage with respect to the Lagrange multiplier method is that positive definiteness is not lost. Such loss can affect the performance of certain numerical processes (for example solving the master stiffness equations by Cholesky factorisation or conjugate-gradients). Finally, it is worth noting that the penalty method is easily extensible to nonlinear constraints.

The main disadvantage, however, is a serious one: the choice of weight values that balance solution accuracy with the violation of constraint conditions. For simple cases the square root rule previously described often works, although its effective use calls for knowledge of the magnitude of stiffness coefficients. Such knowledge may be difficult to extract from general purpose "black box" program. For difficult cases selection of appropriate weights may require extensive numerical experimentation, wasting the user time with numerical games that have no bearing on the actual objective, which is getting a solution.

The deterioration of the condition number for the penalty-augmented stiffness matrix can have serious side effects in some solution procedures such as eigenvalue extraction or iterative solver.

Finally, even if optimal weights are selected, the combined solution error cannot be lowered beyond a threshold value.

From this assessment it is evident that penalty augmentation, although superior to the master-slave method from the standpoint of generality and ease of implementation, is no panacea.

3.4 Lagrange Multiplier Adjuction

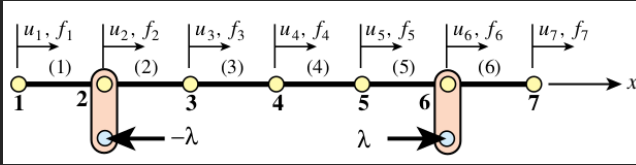


Figure 3.6: Physical interpretation of Lagrange multiplier adjuction to enforce the MFC $u_2 = u_6$

3.4.1 Physical Interpretation

As in the case of the penalty function method, the method Lagrange multiplier can be given a rigorous justification within the framework of variational calculus. But in the same spirit it will be introduced for the example structure from a physical standpoint that is particularly illuminating.

Consider again the constraint $u_2 = u_6$. Borrowing some ideas from the penalty method, imagine that nodes 2 and 6 are connected now by a *rigid* link rather than a flexible one. Thus the constraint is imposed exactly. But of course the penalty method with an infinite weight would "blow up".

We may remove the link if it is replaced by an appropriate reaction force pair $(-\lambda, +\lambda)$, as illustrated in Figure 3.6. These are called the *constraint forces*. Incorporating these forces into the original stiffness equations we get:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & 0 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 - \lambda \\ f_3 \\ f_4 \\ f_5 \\ f_6 + \lambda \\ f_7 \end{bmatrix} \quad (3.53)$$

This λ is called **Lagrange multiplier**. Because λ is an unknown, let us transfer it to the **LHS** by **appending** it to the vector of unknowns:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 1 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & -1 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ \lambda \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \end{bmatrix} \quad (3.54)$$

But now we have 7 equations for 8 unknowns. To render the system determinate, the constraint condition $u_2 - u_6 = 0$ is appended as eight equation:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 1 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & -1 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 \\ 0 & 1 & 0 & 0 & 0 - 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ \lambda \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ 0 \end{bmatrix} \quad (3.55)$$

This is called the **multiplier-augmented** system. Its coefficient matrix, which is symmetric, is called the **bordered stiffness matrix**. The process by which λ is appended to the vector of original unknowns is called **adjunction**. Solving this system provides the desired solution for the DOFs while also characterising the constraint forces through λ .

3.4.2 Lagrange Multipliers fro General MFCs

The general procedure will be stated first as a recipe. Suppose that we want to solve the example structure subjected to three MFCs:

$$u_2 - u_6 = 0, \quad 5u_2 - 8u_7 = 3, \quad 3u_3 + u_5 - 4u_6 = 1 \quad (3.56)$$

Adjoin these MFCs as the eighth, ninth, and tenth equations:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 & -8 \\ 0 & 0 & 3 & 0 & 1 & -4 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ 0 \\ 3 \\ 1 \end{bmatrix} \quad (3.57)$$

Three Lagrange multipliers: λ_1, λ_2 and λ_3 are required to care of three MFCs. Adjoin those unknowns to the nodal displacement vector. Symmetrize the coefficient matrix by appending 3 columns that are the transpose of the 3 last rows and filling the bottom right-hand corner with a 3×3 zero matrix:

$$\begin{bmatrix} K_{11} & K_{12} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ K_{12} & K_{22} & K_{23} & 0 & 0 & 0 & 0 & 1 & 5 & 0 \\ 0 & K_{23} & K_{33} & K_{34} & 0 & 0 & 0 & 0 & 0 & 3 \\ 0 & 0 & K_{34} & K_{44} & K_{45} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & K_{45} & K_{55} & K_{56} & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & K_{56} & K_{66} & K_{67} & -1 & 0 & -4 \\ 0 & 0 & 0 & 0 & 0 & K_{67} & K_{77} & 0 & -8 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 & 0 & 0 & -8 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 1 & -4 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \\ \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ 0 \\ 3 \\ 1 \end{bmatrix} \quad (3.58)$$

3.4.3 The Theory Behind

The recipe illustrated above comes from a well known technique of variational calculus. Using the matrix notation introduced, compactly denote the set of m MFCs by $\mathbf{A}\mathbf{u} = \mathbf{b}$, where $\mathbf{A}$ is $m \times n$. The potential energy of the unconstrained finite element model is $\Pi = \frac{1}{2}\mathbf{u}^T \mathbf{K}\mathbf{u} - \mathbf{u}^T \mathbf{f}$. To impose the constraint, adjoin m Lagrange multipliers collected in vector λ and form the Lagrangian

$$\mathcal{L}(\mathbf{u}, \lambda) = \Pi + \lambda(\mathbf{A}\mathbf{u} - \mathbf{b}) = \frac{1}{2}\mathbf{u}^T \mathbf{K}\mathbf{u} - \mathbf{u}^T \mathbf{f} + \lambda^T (\mathbf{A}\mathbf{u} - \mathbf{b}) \quad (3.59)$$

Extremizing $\mathcal{L}$ with respect to $\mathbf{u}$ and λ yields the multiplier-augmented form

$$\begin{bmatrix} \mathbf{K} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{b} \end{bmatrix} \quad (3.60)$$

The master stiffness matrix $\mathbf{K}$ is said to be **bordered** with $\mathbf{A}$ and $\mathbf{A}^T$. Solving this system provides $\mathbf{u}$ and λ . The latter can be interpreted as forces of constraint in the following sense: a removed constraint can be replaced by a system of forces characterised by λ multiplied by the constraint coefficients. More precisely, the constraint forces are $-\mathbf{A}^T \lambda$.

3.4.4 Assessment of the Lagrange Multiplier Method

In contrast to the penalty method, the method of Lagrange multipliers has the advantage of being exact (aside from computational errors due to finite precision arithmetic). It provides directly the constraint forces, which are of interest in many applications. It does not require guesses as regards to weights. As the penalty method, it can be extended without difficulty to nonlinear constraints.

It is not free of disadvantages. It introduces additional unknowns, requiring expansion of the original stiffness method, and more complicated storage allocation procedures. It renders the augmented stiffness matrix indefinite, an effect that may cause grief with some linear equation solving methods that rely on positive definiteness. Finally, as the master-slave method, it is sensitive to

the degree of linear independence of the constraints: if the constraint $u_2 = u_6$ is specified twice, the bordered stiffness is obviously singular.

On the whole this method appears to be the most elegant one for a general-purpose finite element program that is supposed to work as a "black box" by minimizing guesses and choices from its users. Its implementation, however, is not simple. Special care must be exercised to detect singularities due to constraint dependency and to account for the effect of loss of positive definiteness of the bordered stiffness on equation solver.

3.4.5 The Augmented Lagrangian Method

The general matrix of the penalty function and Lagrangian multiplier methods are given by expressions (3.51) and (3.60), respectively. A useful connection between these methods can be established as follows.

Because the lower diagonal block of the bordered stiffness matrix in (3.60) is null, it is not possible to directly eliminate λ . To make this possible, replace this block by $\epsilon \mathbf{S}^{-1}$, where $\mathbf{S}$ is a constraint-scaling diagonal matrix of appropriate order and ϵ is a small number. The reciprocal of ϵ is a large number called $w = 1/\epsilon$. To maintain exactness of the second equation, $\epsilon \mathbf{S}^{-1} \lambda$ is added to the RHS:

$$\begin{bmatrix} \mathbf{K} & \mathbf{A}^T \\ \mathbf{A} & \epsilon \mathbf{S}^{-1} \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \epsilon \mathbf{S}^{-1} \lambda^P \end{bmatrix} \quad (3.61)$$

Here superscript P (for "predicted" value) is attached to the λ on the RHS as a "tracer". We can now formally solve for λ and subsequently for $\mathbf{u}$. The results may be presented as:

$$\begin{aligned} (\mathbf{K} + w \mathbf{A}^T \mathbf{S} \mathbf{A}) \mathbf{u} &= \mathbf{f} + w \mathbf{A}^T \mathbf{S} \mathbf{b} - \mathbf{A}^T \lambda^P \\ \lambda &= \lambda^P + w \mathbf{S} (\mathbf{b} - \mathbf{A} \mathbf{u}) \end{aligned} \quad (3.62)$$

Setting $\lambda^P = \mathbf{0}$ in the first matrix equation yields:

$$(\mathbf{K} + w \mathbf{A}^T \mathbf{S} \mathbf{A}) \mathbf{u} = \mathbf{f} + w \mathbf{A}^T \mathbf{S} \mathbf{b} \quad (3.63)$$

on taking $\mathbf{W} = w\mathbf{S}$, the general matrix equation (3.51) of the penalty method is recovered.

This relation suggests the construction of *iterative procedures* in which one tries to *improve the accuracy of the penalty function while w is kept constant*. This strategy circumvents the aforementioned ill-conditioning problems when the weight w is gradually increased. One such method is easily constructed by inspecting (3.62). Using superscript k as an iteration index and keeping w fixed, solve equations (3.62) in tandem as follows:

$$\begin{aligned} (\mathbf{K} + \mathbf{A}^T \mathbf{W} \mathbf{A}) \mathbf{u}^k &= \mathbf{f} + \mathbf{A}^T \mathbf{W} \mathbf{b} - \mathbf{A}^T \boldsymbol{\lambda}^k \\ \boldsymbol{\lambda}^{k+1} &= \boldsymbol{\lambda}^k + \mathbf{W} (\mathbf{b} - \mathbf{A} \mathbf{u}^k) \end{aligned} \tag{3.64}$$

for $k = 0, 1, \dots$, beginning with $\boldsymbol{\lambda}^0 = 0$. Then $\mathbf{u}^0$ is the penalty solution. If the process converges one recovers the exact Lagrangian solution without having to solve the Lagrangian system (3.61) directly.

The family of iterative procedures that may be precipitated from (3.62) collectively pertains to the class of *augmented Lagrangian methods*.

3.5 Summary

The treatment of linear MFCs in finite element systems can be carried out by several methods. Three of these: **master-slave elimination**, **penalty augmentation** and **Lagrange multiplier** adjunction have been discussed.

It is emphasised that no method is uniformly satisfactory in terms of generality, robustness, numerical behavior and simplicity of implementation.

For a general purpose program that tries to attain "black box" behavior (that is minimal decisions on the part of users) the Lagrange multipliers has the edge. This edge is unfortunately blunted by a fairly complex computer implementation and by the loss of positive definiteness in the bordered stiffness matrix.

Chapter 4

Contact Formulations

Contact describes the interaction of two bodies whose surfaces may touch, transmit forces and subsequently separate or slide. At first sight this may look like another application of the **multifreedom constraints** treated in the previous chapter. There is, however, one fundamental difference.

An ordinary MFC is an *equality constraint*. Once imposed, it remains active throughout the analysis. Contact in the normal direction is generally a *unilateral constraint*: the surfaces must not penetrate each other, but they are allowed to separate. Consequently, the constraint is active only while the surfaces are closed.

The simplest scalar distinction is

$$\underbrace{a(\mathbf{u}) = 0}_{\text{equality constraint}} \quad \text{versus} \quad \underbrace{g_n(\mathbf{u}) \geq 0}_{\text{unilateral contact constraint}}. \quad (4.1)$$

Here g_n is the *normal gap*. In this chapter the sign convention is:

$$g_n > 0 \quad \text{open gap}, \quad g_n = 0 \quad \text{closed contact}, \quad g_n < 0 \quad \text{penetration.} \quad (4.2)$$

A contact definition is therefore not merely a special stiffness placed between two nodes. A complete definition has to specify at least four things:

1. how the geometrical gap and relative sliding are measured,
2. which normal and tangential motions are admissible,
3. how the corresponding constraints are imposed numerically,
4. how points on one discretized surface interact with the other surface.

Terms such as *frictionless*, *rough*, *frictional*, *no separation* and *bonded* primarily describe the admissible relative motion and the tractions that may be transmitted. They should not be confused with *penalty*, *Lagrange multiplier* and *augmented Lagrangian*, which describe numerical enforcement methods. Nor should they be confused with *node-to-surface* or *surface-to-surface*, which describe contact discretizations.

4.1 A One-Dimensional Contact Example

The basic behaviour of normal contact can be understood without surface geometry. Consider a node attached to a spring of stiffness k . A positive external force f moves the node to the right. A rigid wall is located at $u = g_0$, as sketched conceptually by

$$\text{spring } k \quad \longrightarrow \quad u \quad | \quad \text{rigid wall at } g_0. \quad (4.3)$$

The distance from the node to the wall is

$$g_n = g_0 - u. \quad (4.4)$$

Thus $u < g_0$ means that the node and wall are separated, $u = g_0$ means that they touch, and $u > g_0$ would represent an inadmissible penetration.

If the wall is absent, equilibrium gives

$$ku = f, \quad (4.5)$$

and therefore $u = f/k$. Two cases are possible.

1. If $f/k < g_0$, the unconstrained displacement does not reach the wall. The contact remains open and the wall exerts no force.
2. If $f/k \geq g_0$, the unconstrained displacement would cross the wall. Contact must become active, the displacement is limited to $u = g_0$, and the wall supplies a reaction force.

Let the compressive contact reaction be denoted by $p_n \geq 0$. It acts in the direction opposite to increasing u . Equilibrium of the node is

$$ku + p_n = f. \quad (4.6)$$

The exact solution is therefore

$$\begin{aligned} u &= \frac{f}{k}, \quad p_n = 0, & \text{if } f < kg_0, \\ u &= g_0, \quad p_n = f - kg_0, & \text{if } f \geq kg_0. \end{aligned} \quad (4.7)$$

This small example already contains the central difficulty of contact: before solving the equilibrium equations, it is generally not known whether the contact reaction is zero or whether the gap is zero. The set of active contact constraints is part of the solution.

4.2 The Unilateral Normal Constraint

For a frictionless normal contact pair the admissible states are described by three conditions:

$$g_n \geq 0, \quad p_n \geq 0, \quad p_n g_n = 0. \quad (4.8)$$

These are the *Karush–Kuhn–Tucker* conditions for unilateral contact. Their meaning is more important than their name.

1. $g_n \geq 0$ states that penetration is not admissible.
2. $p_n \geq 0$ states that ordinary contact can push but cannot pull. A tensile normal traction would require an adhesive or bonded interface.
3. $p_n g_n = 0$ is the *complementarity condition*. Either the gap is open and the pressure is zero, or the pressure is positive and the gap is closed. They cannot both be positive.

The two admissible states may be written explicitly as

$$\begin{aligned} \text{open: } & g_n > 0, \quad p_n = 0, \\ \text{closed: } & g_n = 0, \quad p_n \geq 0. \end{aligned} \quad (4.9)$$

At the instant of first touch both $g_n = 0$ and $p_n = 0$ are possible. This transition point separates the open and closed branches.

Equation (4.8) also explains why replacing contact by a permanent equality $g_n = 0$ is generally incorrect. Such an equality would prevent separation and could transmit tension. It would describe a *no-separation* or tied normal interface, not ordinary contact.

4.2.1 Contact Status as an Active Set

Suppose that a finite element model contains many possible contact locations. At a given nonlinear iteration the solver classifies each location as active or inactive:

$$\begin{aligned}\mathcal{A} &= \{i \mid \text{contact constraint } i \text{ is active}\}, \\ \mathcal{I} &= \{i \mid \text{contact constraint } i \text{ is inactive}\}.\end{aligned}\tag{4.10}$$

For an inactive constraint, $p_{n,i} = 0$. For an active constraint, the solver attempts to enforce $g_{n,i} = 0$, either exactly or approximately. After the displacement update, the status is checked again. A contact may close, open, stick or slip, so the active set changes during the nonlinear solution.

This status update is one reason why a structurally linear model may still require a nonlinear contact analysis. Even if the material law and the small-displacement stiffness matrix are linear, the switch between $p_n = 0$ and $g_n = 0$ is not.

4.3 Contact Kinematics

The one-dimensional gap (4.4) is replaced in a continuum model by the distance between two candidate surfaces. It is common to label one side *slave* and the other side *master*. These labels describe the search and projection procedure; they do not imply that one physical body is more important than the other.

Let a point on the slave surface have reference position $\mathbf{X}_s$ and current position

$$\mathbf{x}_s = \mathbf{X}_s + \mathbf{u}_s.\tag{4.11}$$

Let the master surface be parametrized by surface coordinates $\boldsymbol{\xi}$:

$$\mathbf{x}_m(\boldsymbol{\xi}) = \mathbf{X}_m(\boldsymbol{\xi}) + \mathbf{u}_m(\boldsymbol{\xi}).\tag{4.12}$$

A corresponding master point $\mathbf{x}_m(\bar{\boldsymbol{\xi}})$ is found, most often by a closest-point projection. If $\mathbf{n}_m$ is the outward unit normal of the master surface at that point, the normal gap is

$$g_n = \left(\mathbf{x}_s - \mathbf{x}_m(\bar{\boldsymbol{\xi}}) \right)^T \mathbf{n}_m. \quad (4.13)$$

The orientation of $\mathbf{n}_m$ must be chosen consistently with the adopted sign convention. Reversing the normal reverses the sign of g_n and of the normal traction definition.

4.3.1 Closest-Point Projection

For a smooth master surface, the closest point satisfies orthogonality of the slave-to-master vector to both surface tangent directions:

$$\left(\mathbf{x}_s - \mathbf{x}_m(\bar{\boldsymbol{\xi}}) \right)^T \frac{\partial \mathbf{x}_m}{\partial \xi_\alpha} = 0, \quad \alpha = 1, 2. \quad (4.14)$$

Equation (4.14) is itself nonlinear when the surface is curved or undergoing finite motion. In practice the projection has to determine both the contacted element or segment and the local coordinates inside that element.

If the projection falls outside the candidate face, the solver may test an adjacent face, edge or vertex. Consequently, contact search is a geometrical problem in addition to the mechanical equilibrium problem.

4.3.2 Normal and Tangential Decomposition

The normal projector and the tangential projector are

$$\mathbf{P}_n = \mathbf{n}\mathbf{n}^T, \quad \mathbf{P}_t = \mathbf{I} - \mathbf{n}\mathbf{n}^T. \quad (4.15)$$

Any relative displacement increment $\Delta \mathbf{u}_{rel}$ may be decomposed as

$$\Delta \mathbf{u}_{rel} = \underbrace{\mathbf{P}_n \Delta \mathbf{u}_{rel}}_{\text{normal part}} + \underbrace{\mathbf{P}_t \Delta \mathbf{u}_{rel}}_{\text{tangential part}}. \quad (4.16)$$

The incremental tangential slip is commonly written

$$\Delta \mathbf{g}_t = \mathbf{P}_t (\Delta \mathbf{u}_s - \Delta \mathbf{u}_m). \quad (4.17)$$

For finite sliding, the tangential basis rotates and the projected master point moves across element boundaries. Therefore $\mathbf{g}_t$ is normally updated incrementally rather than treated as a single difference between initial and current Cartesian coordinates.

4.3.3 Small and Finite Sliding

A *small-sliding* formulation assumes that the contacted point remains near its initial projection. The contact normal and the interpolation of the opposite surface can then be updated only mildly or even kept fixed within an increment. This is efficient when relative tangential motion is small compared with the local element dimensions.

A *finite-sliding* formulation allows the slave point to travel over many master faces. The closest point, surface normal, tangent basis and contact interpolation must be updated as the bodies move. This is more general but also more expensive and more sensitive to surface discretization.

Small versus finite sliding is a kinematic choice. It is independent of whether the interface is frictionless or frictional and independent of whether a penalty or multiplier method is used.

4.4 Normal Constraint Enforcement

The conditions (4.8) state the exact mechanical problem, but they do not by themselves prescribe how the algebraic equations should be formed. The same three families introduced for MFCs appear again:

1. penalty enforcement,
2. Lagrange multiplier enforcement,
3. augmented Lagrangian enforcement.

The important difference from ordinary MFCs is that these methods are applied only to the currently active contact constraints.

4.4.1 The Penalty Method

Define the positive-part or Macaulay bracket by

$$\langle x \rangle = \max(x, 0). \quad (4.18)$$

With the present gap convention, a simple normal penalty law is

$$p_n = \epsilon_n \langle -g_n \rangle, \quad (4.19)$$

where $\epsilon_n > 0$ is the normal penalty stiffness. If the gap is open, $g_n > 0$, then $p_n = 0$. If penetration occurs, $g_n < 0$, then the contact pressure grows in proportion to the penetration $-g_n$.

The corresponding penalty potential per contact point is

$$\Pi_c = \frac{1}{2} \epsilon_n \langle -g_n \rangle^2. \quad (4.20)$$

The contact force follows from variation of this potential with respect to the nodal displacements. If $\mathbf{B}_n = \partial g_n / \partial \mathbf{u}$ denotes the gap-displacement matrix, the contact contribution to the residual has the form

$$\mathbf{r}_c = -\mathbf{B}_n^T p_n, \quad (4.21)$$

up to the sign convention used for the global residual.

Penalty Solution of the One-Dimensional Example

For the node-wall example, penetration is $u - g_0$. Once contact is active, choose

$$p_n = \epsilon_n(u - g_0). \quad (4.22)$$

Substitution into equilibrium (4.6) gives

$$ku + \epsilon_n(u - g_0) = f, \quad (4.23)$$

and therefore

$$u = \frac{f + \epsilon_n g_0}{k + \epsilon_n}. \quad (4.24)$$

The penetration error is

$$u - g_0 = \frac{f - kg_0}{k + \epsilon_n}. \quad (4.25)$$

As $\epsilon_n \rightarrow \infty$, the penetration tends to zero and $u \rightarrow g_0$. For any finite ϵ_n , however, some penetration is required to generate the contact reaction. This is not a geometrical prediction of the physical model. It is the constraint violation introduced by penalty enforcement.

The tradeoff is the same as for penalty MFCs. A small ϵ_n gives a soft interface and excessive penetration. A very large ϵ_n improves constraint accuracy but increases the condition number of the tangent stiffness matrix and can impair Newton convergence.

4.4.2 Choosing the Normal Penalty Stiffness

There is no universal penalty stiffness that is suitable for every contact pair. The value has to be large relative to the normal compliance of the adjacent finite elements, but not so large that the contact terms dominate the structural stiffness by many unnecessary orders of magnitude.

A useful dimensional estimate is

$$\epsilon_n \sim \alpha \frac{E_{eff} A_c}{h_c}, \quad (4.26)$$

for a discrete contact point carrying representative area A_c , or

$$\bar{\epsilon}_n \sim \alpha \frac{E_{eff}}{h_c} \quad (4.27)$$

for a traction-based surface formulation. Here E_{eff} is an effective normal modulus of the contacting materials, h_c is a representative element length in the contact-normal direction, and α is a dimensionless scaling factor.

These expressions are scaling arguments, not exact laws. Element type, integration scheme, shell thickness, material incompressibility and the chosen surface discretization all affect the useful range. The computed penetration and non-linear convergence should therefore be checked rather than relying on a large default number alone.

4.4.3 The Lagrange Multiplier Method

A Lagrange multiplier can enforce the active normal constraint exactly. For a known active set, collect the active gaps in vector $\mathbf{g}_n$ and linearize them as

$$\Delta \mathbf{g}_n = \mathbf{A}_n \Delta \mathbf{u}, \quad (4.28)$$

where $\mathbf{A}_n$ is the active normal constraint matrix. The Newton correction has the bordered form

$$\begin{bmatrix} \mathbf{K}_T & \mathbf{A}_n^T \\ \mathbf{A}_n & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \lambda_n \end{bmatrix} = - \begin{bmatrix} \mathbf{r} \\ \mathbf{g}_n \end{bmatrix}. \quad (4.29)$$

The multiplier λ_n represents the contact pressure or an equivalent nodal contact force, depending on the discretization. The gap of an active constraint can be driven to zero without selecting a penalty stiffness.

This exactness has a price. Additional unknowns are introduced, the coefficient matrix becomes indefinite, and the active set must still be determined. A multiplier that becomes tensile is inadmissible for ordinary contact; the corresponding constraint must be released.

For the one-dimensional example, active contact gives the exact equations

$$\begin{bmatrix} k & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} u \\ p_n \end{bmatrix} = \begin{bmatrix} f \\ g_0 \end{bmatrix}. \quad (4.30)$$

The solution is $u = g_0$ and $p_n = f - kg_0$. If this computed p_n is negative, the assumed active status was wrong and the contact must be opened.

4.4.4 The Augmented Lagrangian Method

The augmented Lagrangian method combines a moderate penalty stiffness with an iteratively corrected pressure estimate. With iteration index j , one common normal update is

$$p_n^{j+1} = \langle p_n^j - \epsilon_n g_n^{j+1} \rangle. \quad (4.31)$$

If penetration remains, $g_n < 0$, the pressure estimate increases. If the gap opens, the positive-part projection prevents a tensile contact pressure and ultimately returns the pressure to zero.

The displacement problem inside an augmentation step resembles a penalty problem, but the multiplier estimate carries information from one augmentation to the next. Hence the gap can be reduced without increasing ϵ_n to an excessively large value.

The augmented Lagrangian method is therefore often a useful compromise:

1. it is more accurate than a penalty method using the same stiffness,
2. it avoids the full multiplier system in some implementations,
3. it requires additional status and augmentation iterations.

The exact implementation varies. Some formulations keep explicit multiplier unknowns, while others update pressure-like history variables outside the main Newton solve. The mechanical meaning remains the same: a penalty regularization is combined with iterative multiplier correction.

4.4.5 Assessment of Normal Enforcement Methods

The three methods should be compared by their numerical consequences rather than by the apparent elegance of their equations.

1. *Penalty enforcement* retains the original displacement unknowns and is relatively easy to assemble. The contact constraint is only approximate and its accuracy depends on the penalty stiffness.
2. *Lagrange multiplier enforcement* can satisfy the active gap exactly and gives the contact reactions directly. It adds unknowns, produces an indefinite system and requires careful treatment of dependent constraints.
3. *Augmented Lagrangian enforcement* reduces the dependence on a very large penalty stiffness, but adds an outer correction process and still requires contact-status decisions.

No method removes the underlying nonsmoothness caused by opening and closing. A sophisticated enforcement method cannot compensate for an incorrect contact normal, poor surface discretization or a failed search projection.

4.5 Tangential Contact and Friction

Normal contact decides whether the surfaces may approach or separate. Tangential contact decides whether they may slide and which shear traction is transmitted while they are closed.

Let $\mathbf{t}_t$ denote the tangential traction vector and p_n the compressive normal pressure. A friction law acts only while contact is closed. If the surfaces are open, both normal and tangential contact tractions vanish:

$$g_n > 0 \implies p_n = 0, \quad \mathbf{t}_t = \mathbf{0}. \quad (4.32)$$

4.5.1 Frictionless Contact

Frictionless contact imposes the unilateral normal condition but no tangential constraint:

$$g_n \geq 0, \quad p_n \geq 0, \quad p_n g_n = 0, \quad \mathbf{t}_t = \mathbf{0}. \quad (4.33)$$

The surfaces may separate and may slide freely. Compressive normal traction is transmitted, but no shear traction and no tensile normal traction are carried.

Frictionless does not mean that the bodies are disconnected. While closed, the normal relative motion is constrained and normal forces can be large. It only means that the tangential relative motion does not generate contact shear.

4.5.2 Coulomb Friction

The classical Coulomb law limits the tangential traction by the current normal pressure:

$$\phi = \|\mathbf{t}_t\| - \mu p_n \leq 0, \quad (4.34)$$

where $\mu \geq 0$ is the coefficient of friction. Two states are possible.

1. *Stick*: $\phi < 0$. The tangential traction lies inside the friction bound and the incremental tangential slip is constrained to zero, apart from any tangential penalty compliance.
2. *Slip*: $\phi = 0$. The traction magnitude reaches μp_n and its direction opposes the sliding direction.

For nonzero sliding velocity $\mathbf{v}_t$, the ideal slip law is

$$\mathbf{t}_t = -\mu p_n \frac{\mathbf{v}_t}{\|\mathbf{v}_t\|}. \quad (4.35)$$

The minus sign states that friction resists relative sliding. At zero velocity the direction is undefined, which is why the stick state has to be treated as an inequality rather than by direct substitution into (4.35).

4.5.3 A Tangential Trial-State Algorithm

The stick-slip update resembles an elastic predictor and plastic corrector. Let ϵ_t be a tangential penalty stiffness and let $\Delta \mathbf{g}_t$ be the tangential relative-displacement increment. A trial traction is

$$\mathbf{t}_t^{trial} = \mathbf{t}_t^n - \epsilon_t \Delta \mathbf{g}_t. \quad (4.36)$$

The trial friction function is

$$\phi^{trial} = \|\mathbf{t}_t^{trial}\| - \mu p_n. \quad (4.37)$$

If $\phi^{trial} \leq 0$, the contact sticks and

$$\mathbf{t}_t^{n+1} = \mathbf{t}_t^{trial}. \quad (4.38)$$

If $\phi^{trial} > 0$, the contact slips and the traction is returned to the friction limit:

$$\mathbf{t}_t^{n+1} = \mu p_n \frac{\mathbf{t}_t^{trial}}{\|\mathbf{t}_t^{trial}\|}. \quad (4.39)$$

The signs in (4.36) depend on the adopted traction and slip conventions. The important construction is invariant: first predict the traction assuming stick, then check the Coulomb bound, and if the bound is exceeded project the traction back to the admissible friction surface.

Because the limit μp_n changes with normal pressure, the normal and tangential problems are coupled. Opening the contact immediately removes the friction traction. A change in normal pressure changes the maximum transferable shear.

4.5.4 Regularized Friction

Ideal Coulomb friction is nonsmooth at the transition from stick to slip and at zero sliding velocity. Some formulations replace the discontinuous law by a smooth approximation, for example

$$\mathbf{t}_t = -\mu p_n \tanh\left(\frac{\|\mathbf{v}_t\|}{v_0}\right) \frac{\mathbf{v}_t}{\|\mathbf{v}_t\|}, \quad (4.40)$$

with a small regularization velocity v_0 . Such a law can improve nonlinear convergence, but it introduces artificial microslip and rate dependence. The regularization parameter is therefore part of the numerical model and should be reported when it materially affects the result.

4.6 Common Contact Definitions

The common names used in finite element programs can now be stated in terms of normal and tangential admissibility. The names are convenient, but their exact implementation may differ between programs. The solver-independent mechanical meaning is described below.

4.6.1 Frictionless

A frictionless interface has unilateral normal contact and zero tangential traction:

$$g_n \geq 0, \quad p_n \geq 0, \quad p_n g_n = 0, \quad \mathbf{t}_t = \mathbf{0}. \quad (4.41)$$

It can transmit compression. It cannot transmit tension or shear. It allows both separation and unrestricted sliding.

4.6.2 Frictional

A frictional interface has unilateral normal contact together with the Coulomb bound

$$\|\mathbf{t}_t\| \leq \mu p_n. \quad (4.42)$$

It can transmit compression and a pressure-dependent amount of shear. It allows separation. While closed, it may stick or slide depending on whether the required shear traction is below or at the friction limit.

The frictionless definition is recovered as the special case $\mu = 0$.

4.6.3 Rough

A rough interface prevents tangential sliding while the surfaces are in contact:

$$g_n = 0 \implies \Delta \mathbf{g}_t = \mathbf{0}. \quad (4.43)$$

It may be viewed as the limiting case of friction with an unbounded friction capacity, although implementations often impose the tangential constraint directly rather than using a very large value of μ .

Rough contact may transmit whatever tangential traction is required to prevent slip while the normal contact remains active. It still permits separation and normally cannot transmit tensile normal traction. This distinguishes rough contact from bonded contact.

4.6.4 No Separation

A no-separation interface prevents opening in the normal direction once the interface is closed, but allows tangential sliding:

$$g_n = 0, \quad \mathbf{t}_t = \mathbf{0} \quad (4.44)$$

for the ideal frictionless no-separation case. Because the normal relation is bilateral, the interface can develop both compressive and tensile normal reactions. It is therefore not ordinary unilateral contact.

The phrase “once closed” is important. Some implementations tie surfaces that are initially within a specified tolerance; others first allow them to close and then prevent reopening. Initial status and program convention must therefore be checked.

No separation is useful when two surfaces may slide but are physically retained against opening, for example by an idealized enclosure. It is not appropriate when real lift-off is expected.

4.6.5 Bonded

An ideal bonded interface enforces displacement continuity in both normal and tangential directions:

$$\mathbf{u}_s - \mathbf{u}_m = \mathbf{0}. \quad (4.45)$$

Equivalently,

$$g_n = 0, \quad \mathbf{g}_t = \mathbf{0}. \quad (4.46)$$

A bonded interface can transmit compression, tension and shear. It allows neither separation nor sliding. Mechanically it is a tied interface or a set of MFCs, even if it is created through contact-surface machinery.

Bonded contact should not be interpreted as a universal model of adhesive failure. The ideal constraint has no strength limit and no fracture energy. Debonding requires an additional interface law, such as a cohesive-zone model, a damage law or an explicit release criterion.

4.6.6 Comparison of the Definitions

The principal differences are summarized below. “Yes” means that the idealized definition admits the motion or force without adding another constitutive law.

Definition	Separation	Sliding	Tensile normal force	Shear transfer
Frictionless	Yes	Yes	No	No
Frictional	Yes	Conditional	No	Limited by μp_n
Rough	Yes	No while closed	No	Unlimited while closed
No separation	No	Yes	Yes	No, if frictionless
Bonded	No	No	Yes	Yes

This table describes the ideal mechanics, not the numerical enforcement. For example, bonded contact may be implemented by direct elimination, MFCs, penalty terms, multipliers or mortar constraints. Two models with the same word “bonded” may therefore have different constraint accuracy and conditioning.

4.6.7 Cohesive and Adhesive Interfaces

A cohesive interface is different from an ideal bond because the transmitted traction is related to an interface separation. In a simple uncoupled form,

$$t_n = t_n(\delta_n), \quad \mathbf{t}_t = \mathbf{t}_t(\boldsymbol{\delta}_t), \quad (4.47)$$

where δ_n and $\boldsymbol{\delta}_t$ are normal and tangential separations. The law may initially be stiff, reach a strength limit and then soften as damage grows. The area under a traction-separation curve represents fracture energy.

Thus a cohesive interface is a constitutive model with finite strength and possible failure. Bonded contact is an ideal kinematic constraint with unlimited strength unless an additional failure rule is supplied.

4.7 Contact Discretizations

The continuous contact conditions apply over surfaces, but a finite element model contains nodes, edges, faces and interpolation functions. A contact discretization determines where the gap is evaluated and how the resulting tractions are distributed to the nodes.

4.7.1 Node-to-Node Contact

Node-to-node contact connects predefined pairs of nodes. For a pair with normal $\mathbf{n}$, the gap may be written

$$g_n = (\mathbf{x}_s - \mathbf{x}_m)^T \mathbf{n}. \quad (4.48)$$

The method is simple and inexpensive, but it requires compatible node locations and a known pairing. It is poorly suited to general sliding because a node that moves tangentially no longer remains opposite its original partner.

Node-to-node contact is best understood as a collection of discrete contact springs or constraints, not as an accurate representation of arbitrary smooth surface interaction.

4.7.2 Node-to-Surface Contact

In node-to-surface contact, each slave node is projected onto a master face. The gap is evaluated between the slave node and the interpolated master point. The contact force acting at the projection is distributed to the master nodes through the face shape functions.

This permits a slave node to move over the master mesh and does not require matching nodes. However, the result is biased: penetration is prevented at the slave nodes, but not necessarily at points between them. A coarse slave surface may therefore pass locally through the master surface between slave nodes.

As a general practical rule, the more finely discretized or more deformable surface is often selected as slave, while the smoother, coarser or stiffer surface is selected as master. This is a useful starting point, not a universal law. Curvature, shell normals, expected sliding direction and element order also matter.

4.7.3 Surface-to-Surface Contact

Surface-to-surface formulations evaluate contact over slave segments or integration points rather than only at slave nodes. The contact virtual work may be written schematically as

$$\delta W_c = \int_{\Gamma_c} (p_n \delta g_n + \mathbf{t}_t^T \delta \mathbf{g}_t) d\Gamma. \quad (4.49)$$

Numerical quadrature converts this surface integral into contact contributions at integration points. Surface-to-surface contact generally represents pressure more smoothly and reduces nodal bias, especially for higher-order elements and curved surfaces.

The term does not uniquely identify one algorithm. Different programs may use one-pass, two-pass, segment-to-segment or mortar-like formulations under the broad surface-to-surface name.

4.7.4 Mortar Contact

Mortar methods impose contact constraints in a weak, integral sense using interpolation spaces for the contact pressure or multipliers. Instead of matching the gap independently at isolated nodes, a weighted residual condition is used:

$$\int_{\Gamma_c} N_{\lambda_i} g_n \, d\Gamma = 0 \quad (4.50)$$

for the active contact region, where N_{λ_i} are multiplier or mortar shape functions.

Mortar methods are well suited to nonmatching meshes and can provide improved conservation and reduced master-slave bias. Their formulation, integration and active-set treatment are more involved than classical node-to-surface contact.

4.7.5 One-Pass and Two-Pass Contact

A one-pass method projects one selected side onto the other. It is inherently asymmetric because exchanging master and slave changes the discrete equations.

A two-pass or symmetric method performs the constraint construction in both directions. This may reduce bias, but simply duplicating constraints can also overconstrain the interface, especially for matching meshes or nearly incompressible response. A properly designed symmetric or mortar formulation is not equivalent to blindly applying two independent node-to-surface pairs.

The words *symmetric* and *asymmetric* should therefore be read as properties of the discrete contact algorithm, not of the physical friction law.

4.8 Linearization and Nonlinear Solution

The finite element equilibrium equations including contact may be written

$$\mathbf{r}(\mathbf{u}) = \mathbf{f}_{int}(\mathbf{u}) + \mathbf{f}_c(\mathbf{u}) - \mathbf{f}_{ext} = \mathbf{0}. \quad (4.51)$$

A Newton iteration solves

$$\mathbf{K}_T^i \Delta \mathbf{u}^i = -\mathbf{r}^i, \quad \mathbf{u}^{i+1} = \mathbf{u}^i + \Delta \mathbf{u}^i, \quad (4.52)$$

where

$$\mathbf{K}_T = \frac{\partial \mathbf{f}_{int}}{\partial \mathbf{u}} + \frac{\partial \mathbf{f}_c}{\partial \mathbf{u}}. \quad (4.53)$$

The contact tangent contains more than a normal spring stiffness. Depending on the formulation it includes derivatives of the gap, surface normal, projection coordinates, pressure law and friction law. In finite sliding, changes of the contact geometry produce additional geometric terms.

A typical contact iteration consists conceptually of:

1. predict or update the current geometry,
2. search for candidate surface pairs,
3. compute projections, gaps and relative tangential increments,
4. classify each contact point as open, closed-stick or closed-slip,
5. assemble contact residuals and tangent contributions,
6. solve the Newton correction,
7. repeat until equilibrium and contact conditions are satisfied.

The actual order may differ, and some solvers update status only at selected stages to improve robustness.

4.8.1 Contact Chattering

A contact point may alternate between open and closed states in successive iterations. Similarly, a friction point may alternate between stick and slip. This behaviour is called *chattering*. It can occur near the transition where both branches are nearly admissible.

Chattering may be reduced by smaller load increments, line search, status tolerances, augmented Lagrangian updates or mild regularization. Excessive stabilization, however, can alter the physical force-displacement response.

4.8.2 Initial Gap and Initial Penetration

Before the first equilibrium iteration, the discretized surfaces may have an initial gap or may overlap because of geometry tolerances, mesh approximation, shell offsets or intentional interference.

An initial penetration may be treated in several conceptually different ways:

1. preserve it and generate an initial contact force,
2. shift the contact reference so that the initial configuration is considered just touching,
3. move nodes or surfaces to remove the overlap,
4. ramp the interference over several increments.

These choices do not describe the same model. Removing an initial penetration changes the reference geometry, while preserving it may introduce prestress. The adopted treatment must therefore be understood rather than accepted as a purely numerical setting.

4.8.3 Search Distance and Candidate Detection

To avoid testing every surface face against every other face, contact algorithms use spatial search structures and a finite candidate distance. A contact pair outside the search region is ignored during that search pass.

A search distance that is too small can miss rapidly closing or initially separated surfaces. An excessively large distance increases search cost and can create unintended candidate pairs. Search tolerance affects detection; it does not by itself define the physical normal stiffness.

4.9 Numerical Behaviour and Assessment

Contact results should not be assessed only by whether the nonlinear solver reports convergence. A converged solution may still contain excessive penalty penetration, unintended contact pairs or mesh-dependent pressure peaks.

4.9.1 Quantities to Check

Useful checks include:

1. maximum and representative normal penetration,
2. normal contact pressure and integrated contact force,
3. open, closed, sticking and sliding status distributions,
4. frictional traction relative to the bound μp_n ,
5. equilibrium residual and reaction-force balance,
6. sensitivity to contact mesh refinement,
7. sensitivity to penalty stiffness and load-step size.

A penetration should be judged relative to a meaningful geometrical scale, such as local element thickness, surface curvature radius or allowed interface deflection. Reporting only its absolute value can be misleading when model units or component sizes differ.

4.9.2 Pressure Singularities

High contact pressure near a sharp edge, corner or the end of a contact region may be a genuine mathematical singularity. Mesh refinement can then increase the peak pressure rather than converge it to a finite value. Integrated force, contact width or pressure away from the singular point may be more meaningful quantities.

A smooth pressure contour is not proof of correctness. Nodal averaging and postprocessing can hide local oscillations or violations of the contact conditions.

4.9.3 Mesh Compatibility and Bias

Nonmatching contact meshes are permissible, but the relative element sizes and orders influence pressure transfer. A very coarse surface contacting a very fine surface may produce oscillatory tractions or artificial local compliance.

Changing master and slave sides is a useful diagnostic for a biased one-pass formulation. Large changes in global response indicate that the contact discretization, not only the physical model, controls the answer.

4.9.4 Contact in Dynamic Problems

In dynamics, closing contact can introduce an impact. A purely displacement-based penalty law stores elastic contact energy

$$\Pi_c = \frac{1}{2} \epsilon_n \langle -g_n \rangle^2 \quad (4.54)$$

and may generate high-frequency oscillations. Contact damping or an impact law may be added, but such additions dissipate energy and must be distinguished from material damping.

The stable time step of an explicit method can be strongly reduced by a large penalty stiffness because the contact stiffness increases the highest system frequency. Thus the penalty value affects not only penetration but also computational cost.

For implicit dynamics, abrupt status changes and frictional slip can make the Newton iterations difficult. Time-step reduction often helps because it limits the amount of contact-status change within one increment.

4.10 Relation to Multifreedom Constraints

The connection to Chapter 3 can now be stated precisely.

A bonded interface is essentially a set of displacement equality constraints:

$$\mathbf{A}\mathbf{u} = \mathbf{0}. \quad (4.55)$$

A no-separation normal interface is a bilateral equality constraint on the normal gap. A sticking tangential state is an equality constraint on incremental tangential slip.

Ordinary normal contact is different because it combines an inequality with a reaction-sign condition and complementarity:

$$g_n \geq 0, \quad p_n \geq 0, \quad p_n g_n = 0. \quad (4.56)$$

Friction adds a second inequality and a second status decision:

$$\|\mathbf{t}_t\| - \mu p_n \leq 0. \quad (4.57)$$

Therefore contact may be regarded as a changing collection of MFC-like constraints whose membership is unknown in advance. Penalty, multiplier and augmented Lagrangian methods reappear, but they are embedded in geometry search, active-set logic and constitutive stick-slip updates.

This viewpoint is useful because it separates three questions that are often mixed together:

1. *What relative motion is physically allowed?* This determines frictionless, frictional, rough, no-separation or bonded behaviour.

2. *How is the constraint represented on the mesh?* This determines node-to-node, node-to-surface, surface-to-surface or mortar contact.
3. *How is the discrete constraint imposed?* This determines penalty, Lagrange multiplier, augmented Lagrangian or direct elimination treatment.

A contact setting is properly understood only when all three questions can be answered.

4.11 Summary

Contact is a unilateral interaction. The normal gap may remain positive with zero pressure, or the gap may close and develop a compressive pressure. These alternatives are expressed by

$$g_n \geq 0, \quad p_n \geq 0, \quad p_n g_n = 0. \quad (4.58)$$

The contact type describes admissible relative motion and transmitted traction:

1. frictionless contact allows separation and sliding and transmits only compression,
2. frictional contact adds a pressure-dependent shear limit,
3. rough contact prevents sliding while closed but still allows separation,
4. no-separation contact prevents opening but allows tangential sliding,
5. bonded contact prevents both opening and sliding and can transmit tension, compression and shear.

Penalty, Lagrange multiplier and augmented Lagrangian methods describe how the normal and tangential constraints are enforced numerically. Node-to-node, node-to-surface, surface-to-surface and mortar methods describe how the continuous interface is discretized.

No single contact definition or algorithm is uniformly best. The appropriate choice depends on the expected motion, required force transfer, mesh relation, constraint accuracy and nonlinear robustness. Contact settings should therefore be selected from their mechanical meaning first and tuned numerically second.

Chapter 5

Rigid Body Elements

5.1 RBE2

The rigid freebody relations between **slave** and **master** node are:

$$\begin{aligned}\mathbf{T}_s &= \mathbf{T}_m + \mathbf{R}_s \times \bar{\mathbf{x}} \\ \mathbf{R}_s &= \mathbf{R}_m\end{aligned}\tag{5.1}$$

Where $\bar{\mathbf{x}}$ is a vector from **slave** to **master**:

$$\bar{\mathbf{x}} = \begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{z} \end{bmatrix} = \begin{bmatrix} x_s - x_m \\ y_s - y_m \\ z_s - z_m \end{bmatrix}\tag{5.2}$$

Then *vector multiplication* (cross product) is defined as:

$$\begin{aligned}\mathbf{R}_s \times \mathbf{x}_s &= \mathbf{R}_m \times \mathbf{x} = \\ &= \begin{bmatrix} \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \times \begin{bmatrix} x_s - x_m \\ y_s - y_m \\ z_s - z_m \end{bmatrix} = \begin{bmatrix} \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \times \begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{z} \end{bmatrix} = \\ &= \begin{bmatrix} \psi_m(z_s - z_m) - \theta_m(y_s - y_m) \\ \theta_m(x_s - x_m) - \varphi_m(z_s - z_m) \\ \varphi_m(y_s - y_m) - \psi_m(x_s - x_m) \end{bmatrix} = \begin{bmatrix} \psi_m\bar{z} - \theta_m\bar{y} \\ \theta_m\bar{x} - \varphi_m\bar{z} \\ \varphi_m\bar{y} - \psi_m\bar{x} \end{bmatrix} = \\ &= \begin{bmatrix} 0 & \bar{z} & -\bar{y} \\ -\bar{z} & 0 & \bar{x} \\ \bar{y} & -\bar{x} & 0 \end{bmatrix} \begin{bmatrix} \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix}\end{aligned}\tag{5.3}$$

The **slave** equations can be written as:

$$\begin{bmatrix} u_s \\ v_s \\ w_s \\ \varphi_s \\ \psi_s \\ \theta_s \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & z_s - z_m & -(y_s - y_m) \\ 0 & 1 & 0 & -(z_s - z_m) & 0 & x_s - x_m \\ 0 & 0 & 1 & y_s - y_m & -(x_s - x_m) & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \quad (5.4)$$

or:

$$\begin{bmatrix} u_s \\ v_s \\ w_s \\ \varphi_s \\ \psi_s \\ \theta_s \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & \bar{z} & -\bar{y} \\ 0 & 1 & 0 & -\bar{z} & 0 & \bar{x} \\ 0 & 0 & 1 & \bar{y} & -\bar{x} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \quad (5.5)$$

5.1.1 RBE2 Coefficients Example:

Master Node:

$$\mathbf{X}_m = [1 \quad 1 \quad 1]^T \quad (5.6)$$

Slave Nodes:

$$\begin{aligned} \mathbf{X}_{s,1} &= [0 \quad 0 \quad 0]^T \\ \mathbf{X}_{s,1} &= [2 \quad 0 \quad 0]^T \\ \mathbf{X}_{s,1} &= [2 \quad 2 \quad 0]^T \\ \mathbf{X}_{s,1} &= [0 \quad 2 \quad 0]^T \end{aligned} \quad (5.7)$$

Then Slave Node 1 equations are:

$$\mathbf{x}_1 = [x_{s,1} - x_m \quad y_{s,1} - y_m \quad z_{s,1} - z_m]^T = [-1 \quad -1 \quad -1]^T \quad (5.8)$$

Other Slave Nodes:

$$\begin{aligned} \mathbf{x}_2 &= [2 - 1 \quad 0 - 1 \quad 0 - 1]^T = [1 \quad -1 \quad -1]^T \\ \mathbf{x}_3 &= [2 - 1 \quad 2 - 1 \quad 0 - 1]^T = [1 \quad 1 \quad -1]^T \\ \mathbf{x}_4 &= [0 - 1 \quad 2 - 1 \quad 0 - 1]^T = [-1 \quad 1 \quad -1]^T \end{aligned} \quad (5.9)$$

Therefore, after filling in the coefficients:

$$\begin{aligned}
 \begin{bmatrix} u_{s,1} \\ v_{s,1} \\ w_{s,1} \\ \varphi_{s,1} \\ \psi_{s,1} \\ \theta_{s,1} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \\
 \begin{bmatrix} u_{s,2} \\ v_{s,2} \\ w_{s,2} \\ \varphi_{s,2} \\ \psi_{s,2} \\ \theta_{s,2} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \\
 \begin{bmatrix} u_{s,3} \\ v_{s,3} \\ w_{s,3} \\ \varphi_{s,3} \\ \psi_{s,3} \\ \theta_{s,3} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \\
 \begin{bmatrix} u_{s,4} \\ v_{s,4} \\ w_{s,4} \\ \varphi_{s,4} \\ \psi_{s,4} \\ \theta_{s,4} \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix}
 \end{aligned} \tag{5.10}$$

When all put together:

$$\begin{bmatrix} u_{s,1} \\ v_{s,1} \\ w_{s,1} \\ \varphi_{s,1} \\ \psi_{s,1} \\ \theta_{s,1} \\ u_{s,2} \\ v_{s,2} \\ w_{s,2} \\ \varphi_{s,2} \\ \psi_{s,2} \\ \theta_{s,2} \\ u_{s,3} \\ v_{s,3} \\ w_{s,3} \\ \varphi_{s,3} \\ \psi_{s,3} \\ \theta_{s,3} \\ u_{s,4} \\ v_{s,4} \\ w_{s,4} \\ \varphi_{s,4} \\ \psi_{s,4} \\ \theta_{s,4} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & -1 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & -1 & -1 \\ 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_m \\ v_m \\ w_m \\ \varphi_m \\ \psi_m \\ \theta_m \end{bmatrix} \quad (5.11)$$

It can be read as:

$$\begin{aligned} u_{s,1} &= u_m - \psi_m + \theta_m \\ v_{s,1} &= v_m \varphi_m - \theta_m \\ &\vdots \\ \theta_{s,4} &= \theta_m \end{aligned} \quad (5.12)$$

Or:

$$\mathbf{u}_s = \bar{\mathbf{T}} \mathbf{u}_m \quad (5.13)$$

5.1.2 Editing the Master Stiffness Matrix

This equation should be then expanded to all DOFs, where in $\mathbf{T}$ the slave DOFs columns are missing and the DOFs that do not take part in RBEs have 1 on the diagonal. The resulting matrix should have the same number of rows as the master stiffness matrix.

Then the master stiffness equation is modified:

$$\begin{aligned}\hat{\mathbf{K}} &= \mathbf{T}^T \mathbf{K} \mathbf{T} \\ \hat{\mathbf{f}} &= \mathbf{T}^T \mathbf{f} \\ \hat{\mathbf{K}} \hat{\mathbf{u}} &= \hat{\mathbf{f}}\end{aligned}\tag{5.14}$$

If we'd like to make the procedure more general, partition the master stiffness equations such that:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{um}^T & \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{us}^T & \mathbf{K}_{ms}^T & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix}\tag{5.15}$$

Based on (5.14) we could probably partition the matrices as:

$$\begin{array}{c|c} & \begin{array}{cc} u & m \end{array} \\ \begin{array}{c} u \\ m \\ s \end{array} & \overline{\begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix}} \end{array}\tag{5.16}$$

If we expand (5.13) and partition the matrices based on (5.16) into independent or **unconstrained**, **master** and **slave** nodes, then the equation:

$$\mathbf{u} = \mathbf{T} \hat{\mathbf{u}}\tag{5.17}$$

becomes:

$$\begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix}\tag{5.18}$$

Finally the equation (5.14) can be rewritten as:

$$\begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \bar{\mathbf{T}}^T \end{bmatrix} \begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{mu} & \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{su} & \mathbf{K}_{sm} & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \bar{\mathbf{T}}^T \end{bmatrix} \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix} \quad (5.19)$$

Expanding and multiplying:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} & \mathbf{K}_{us} \\ \mathbf{K}_{mu} + \bar{\mathbf{T}}^T \mathbf{K}_{su} & \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{sm} & \mathbf{K}_{ms} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m + \bar{\mathbf{T}}^T \mathbf{f}_s \end{bmatrix} \quad (5.20)$$

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ \mathbf{K}_{mu} + \bar{\mathbf{T}}^T \mathbf{K}_{su} & \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{sm} + \mathbf{K}_{ms} \bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m + \bar{\mathbf{T}}^T \mathbf{f}_s \end{bmatrix} \quad (5.21)$$

which is equal to:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ (\mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}})^T & \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{ms}^T + \mathbf{K}_{ms} \bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m + \bar{\mathbf{T}}^T \mathbf{f}_s \end{bmatrix} \quad (5.22)$$

considering that $\mathbf{f}_s = \mathbf{0}$ the equations reduce to:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \end{bmatrix} \quad (5.23)$$

where:

$$\begin{aligned} \hat{\mathbf{K}}_{um} &= \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ \hat{\mathbf{K}}_{mm} &= \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{ms}^T + \mathbf{K}_{ms} \bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \\ &= \mathbf{K}_{mm} + \mathbf{K}_{ms} \bar{\mathbf{T}} + (\mathbf{K}_{ms} \bar{\mathbf{T}})^T + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{aligned} \quad (5.24)$$

Then if **master-slave** relations are written as (*general case*):

$$\mathbf{u}_s = \bar{\mathbf{T}}\mathbf{u}_m + \mathbf{g} \quad (5.25)$$

Inserting into the partitioned master stiffness equations:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u - \mathbf{K}_{us}\mathbf{g} \\ \mathbf{f}_m - \mathbf{K}_{ms}\mathbf{g} \end{bmatrix} \quad (5.26)$$

where:

$$\begin{aligned} \hat{\mathbf{K}}_{um} &= \mathbf{K}_{um} + \mathbf{K}_{us}\bar{\mathbf{T}} \\ \hat{\mathbf{K}}_{mm} &= \mathbf{K}_{mm} + \bar{\mathbf{T}}^T \mathbf{K}_{ms}^T + \mathbf{K}_{ms}\bar{\mathbf{T}} + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{aligned} \quad (5.27)$$

MPC Application

Afterwards the **MPCs** are applied:

$$\mathbf{u}_s = \bar{\mathbf{T}}\mathbf{u}_m \quad (5.28)$$

This, when applied to the whole model:

$$\begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} \quad (5.29)$$

with **transformation matrix** $\mathbf{T}$ being:

$$\mathbf{T} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \bar{\mathbf{T}} \end{bmatrix} \quad (5.30)$$

Applying the transformation matrix to get **modified master equation system**:

$$\begin{aligned} \mathbf{T}^T \mathbf{K}_g \mathbf{T} \hat{\mathbf{u}}_g &= \mathbf{T}^T \mathbf{f}_g \\ \hat{\mathbf{K}}_g \hat{\mathbf{u}}_g &= \hat{\mathbf{f}}_g \end{aligned} \quad (5.31)$$

Where $\hat{\mathbf{u}}_g = [\mathbf{u}_{g,u} \quad \mathbf{u}_{g,m}]^T$.

From now on, the subscript $_g$ (*as global*) is implied.

Rotated Nodal Basis

When a node or an SPC is defined as a **rotated/skewed** or in a different type of Coordinate system (e.g. cylindrical) a transformation matrix needs to be applied to such nodes.

Cartesian Coordinate system definition by angle (2D):

$$\mathbf{u}_l = \mathbf{T} \mathbf{u}_g \quad (5.32)$$

SPC Application

Then to solve the **master stiffness equations** for prescribed **BCs** partitioned to **unconstrained** and **master** DOFs such as:

$$\begin{bmatrix} \mathbf{K}_{uu} & \hat{\mathbf{K}}_{um} \\ \hat{\mathbf{K}}_{um}^T & \hat{\mathbf{K}}_{mm} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_u \\ \mathbf{f}_m \end{bmatrix} \quad (5.33)$$

where:

$$\begin{aligned} \hat{\mathbf{K}}_{um} &= \mathbf{K}_{um} + \mathbf{K}_{us} \bar{\mathbf{T}} \\ \hat{\mathbf{K}}_{mm} &= \mathbf{K}_{mm} + \mathbf{K}_{ms} \bar{\mathbf{T}} + (\mathbf{K}_{ms} \bar{\mathbf{T}})^T + \bar{\mathbf{T}}^T \mathbf{K}_{ss} \bar{\mathbf{T}} \end{aligned} \quad (5.34)$$

Any of these **DOFs** can be constrained, so this partitioning scheme loses all purposes. Therefore the matrix is repartitioned again into **independent** and **constrained** partitions:

$$\begin{bmatrix} \mathbf{K}_{ii} & \mathbf{K}_{ic} \\ \mathbf{K}_{ic}^T & \mathbf{K}_{cc} \end{bmatrix} \begin{bmatrix} \mathbf{u}_i \\ \mathbf{u}_c \end{bmatrix} = \begin{bmatrix} \mathbf{f}_i \\ \mathbf{f}_c \end{bmatrix} \quad (5.35)$$

Solving Linear Equations

Where the **unknowns** are $\mathbf{u}_i$, $\mathbf{f}_c$:

$$\begin{bmatrix} \mathbf{K}_{ii} & \mathbf{K}_{ic} \\ \mathbf{K}_{ic}^T & \mathbf{K}_{cc} \end{bmatrix} \begin{bmatrix} \boxed{\mathbf{u}_i} \\ \boxed{\mathbf{u}_c} \end{bmatrix} = \begin{bmatrix} \boxed{\mathbf{f}_i} \\ \boxed{\mathbf{f}_c} \end{bmatrix} \quad (5.36)$$

Unpacking the equations:

$$\begin{aligned} \mathbf{K}_{ii} \boxed{\mathbf{u}_i} + \mathbf{K}_{ic} \mathbf{u}_c &= \mathbf{f}_i \\ \mathbf{K}_{ic}^T \boxed{\mathbf{u}_i} + \mathbf{K}_{cc} \mathbf{u}_c &= \boxed{\mathbf{f}_c} \end{aligned} \quad (5.37)$$

Then first solve for the **unknown displacements**:

$$\begin{aligned} \mathbf{K}_{ii} \boxed{\mathbf{u}_i} &= \mathbf{f}_i - \mathbf{K}_{ic} \mathbf{u}_c \\ \boxed{\mathbf{u}_i} &= \mathbf{K}_{ii}^{-1} (\mathbf{f}_i - \mathbf{K}_{ic} \mathbf{u}_c) \end{aligned} \quad (5.38)$$

Lastly solve for the **unknown reaction forces**:

$$\boxed{\mathbf{f}_c} = \mathbf{K}_{ii} \mathbf{u}_i + \mathbf{K}_{ic} \mathbf{u}_c \quad (5.39)$$

Recovering Full Displacement Vector

The full displacement vector is then recovered:

$$\begin{aligned} \mathbf{T} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix} \\ \hat{\mathbf{u}} &= \begin{bmatrix} \mathbf{u}_i \\ \mathbf{u}_c \end{bmatrix} = \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} \\ \mathbf{u}_g &= \mathbf{T} \hat{\mathbf{u}} \\ \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \overline{\mathbf{T}} \end{bmatrix} \begin{bmatrix} \mathbf{u}_u \\ \mathbf{u}_m \end{bmatrix} \end{aligned} \quad (5.40)$$

5.2 RBE3

The first step is to calculate the **weighted COG** of the element, where

$$\mathbf{x}_{COG} = \frac{\sum w_i \mathbf{x}_i}{\sum w_i} \quad (5.41)$$

where w_i is a weighting factor of DOF i and $\mathbf{x}_i$ is the position of the node related with DOF i

Once the **COG** is found, the loading is translocated from the reference point $\mathbf{x}_{app}$ where it is applied to the **COG** by performing a equilibrium equivalency.

$$\mathbf{F}_{COG} = \mathbf{F}_{app} \quad (5.42)$$

$$\mathbf{M}_{COG} = \mathbf{M}_{app} + \mathbf{r} \times \mathbf{F}_{app} \quad (5.43)$$

Then the $\mathbf{F}_{COG}$ is reacted (shared) between all master nodes relative to their weight.

$$\mathbf{F}_i = \mathbf{F}_{COG} \frac{w_i}{\sum w_i} \quad (5.44)$$

The moment $\mathbf{M}_{COG}$ is assumed to be reacted (shared) by the mater DOFs according to the relative weight of each one of them and the relative position with regards to the *COG*. The reaction force that compensates the moment is in the direction perpendicular to the vector joining the *COG* with the position of the node where the master DOFs are:

$$\mathbf{P}_i = -w_i \frac{\mathbf{M}_{COG} \times \mathbf{r}_i}{\sum w_i \cdot \mathbf{r}_i \cdot \mathbf{r}_i} \quad (5.45)$$

where $\mathbf{r}_i$ is the vector from the *COG* position to the i -th node $\mathbf{r}_i = \mathbf{x}_i - \mathbf{x}_{COG}$.

The total reaction force on the master DOFs is the sum $\mathbf{F}_i + \mathbf{P}_i$ for each master DOF.

This force is not dependent on the stiffness of the model. It is only dependent on the weighting factors and the relative position of the master nodes.

To get the **slave** node displacements:

1. **COG** position
2. $\mathbf{r}_i$ vectors
3. compute the weighted average of the master displacements
4. an unknown rotation is assumed and the displacements on the master nodes are written as if they were calculated with a rigid body movement with the weighted average displacements and the unknown rotations for each node i :

$$\mathbf{U}_s = \frac{\sum w_i \mathbf{U}_i}{\sum w_i} + \mathbf{R}_s \times \mathbf{r}_i \quad (5.46)$$

where $\mathbf{U}$ are translational displacements and $\mathbf{R}$ are rotational displacements.

Chapter 6

Governing Equations

6.1 Material Matrices

Problem	Displacement components	Strain vector ϵ^T	Stress vector τ^T
Bar	u	$[\epsilon_{xx}]$	$[\sigma_{xx}]$
Beam	w	$[\kappa_{xx}]$	$[m_{xx}]$
Plane stress	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}]$
Plane strain	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}]$
Axisymmetric	u, v	$[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}, \epsilon_{zz}]$	$[\sigma_{xx}, \sigma_{yy}, \tau_{xy}, \sigma_{zz}]$
3D	u, v, w	$[\epsilon_{xx}, \epsilon_{yy}, \epsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{zx}]$	$[\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \tau_{xy}, \tau_{yz}, \tau_{zx}]$
Plate bending	w	$[\kappa_{xx}, \kappa_{yy}, \kappa_{xy}]$	$[m_{xx}, m_{yy}, m_{xy}]$

Notation: $\epsilon_{xx} = \frac{\partial u}{\partial x}$, $\epsilon_{yy} = \frac{\partial v}{\partial y}$, $\gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}$, $\dots$, $\kappa_{xx} = \frac{\partial^2 w}{\partial x^2}$, $\kappa_{yy} = \frac{\partial^2 w}{\partial y^2}$, $\kappa_{xy} = \frac{\partial^2 w}{\partial x \partial y}$.

Table 6.1: Corresponding kinematic and static variables in various problems.

Problem	Material matrix C
Bar	E
Beam	EI
Plane stress	$\frac{E}{1-\nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$
Plane strain	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1 & \frac{\nu}{1-\nu} & 0 \\ \frac{\nu}{1-\nu} & 1 & 0 \\ 0 & 0 & \frac{1-2\nu}{2(1-\nu)} \end{bmatrix}$
Axisymmetric	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1 & \frac{\nu}{1-\nu} & 0 & \frac{\nu}{1-\nu} \\ \frac{\nu}{1-\nu} & 1 & 0 & \frac{\nu}{1-\nu} \\ 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 \\ \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 0 & 1 \end{bmatrix}$
3D	$\frac{E(1-\nu)}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1 & \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 0 & 0 & 0 \\ \frac{\nu}{1-\nu} & 1 & \frac{\nu}{1-\nu} & 0 & 0 & 0 \\ \frac{\nu}{1-\nu} & \frac{\nu}{1-\nu} & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2(1-\nu)} \end{bmatrix}$
Plate bending	$\frac{Eh^3}{12(1-\nu^2)} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$

Notation: E = Young's modulus, ν = Poisson's ratio, h = thickness of plate, I = moment of inertia

Table 6.2: Generalised stress-strain matrices for isotropic materials

Chapter 7

Dynamics

7.1 Function properties

Sine and Cosine functions

Let a function $f(t)$ be in the form:

$$f(t) = A_1 \cos \omega t + B_1 \sin \omega t \quad (7.1)$$

then the following relation applies:

$$\begin{aligned} f(t) &= A \cos(\omega t - \phi) \\ A &= \sqrt{A_1^2 + B_1^2} \\ \phi &= \tan^{-1} \left(\frac{B_1}{A_1} \right) \end{aligned} \quad (7.2)$$

also:

$$\begin{aligned} \cos(\omega t - \phi) &= \cos \omega t \cos \phi + \sin \omega t \sin \phi \\ \sin(\omega t - \phi) &= \sin \omega t \cos \phi - \cos \omega t \sin \phi \end{aligned} \quad (7.3)$$

7.2 SDOF system

7.2.1 Definition

Let:

$$m\ddot{u}(t) + c\dot{u}(t) + ku(t) = f(t) \quad (7.4)$$

or:

$$m\ddot{a}(t) + c\dot{v}(t) + ku(t) = f(t) \quad (7.5)$$

be the **SDOF equation of motion**, then the following applies:

$$\begin{aligned} \omega^2 &= \frac{k}{m} \\ \omega &= \sqrt{\frac{k}{m}} \\ f &= \frac{\omega}{2\pi} \\ T &= \frac{1}{f} \end{aligned} \quad (7.6)$$

where:

ω is radial eigenfrequency [rad/s],

f is eigenfrequency [Hz] and

T is period [s]

The simplest vibratory system can be described by a single mass connected to a spring (and possibly a dashpot). The mass is allowed to travel only along the spring elongation direction. Such systems are called *Single Degree-of-Freedom* (**SDOF**) systems and are shown in the figure.

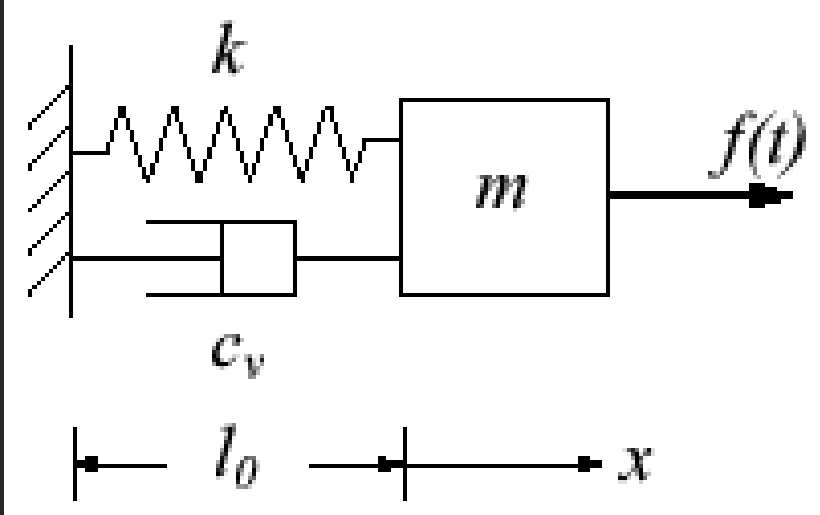


Figure 7.1: SDOF system with a linear spring and dashpot

7.2.2 Equation of Motion for SDOF Systems

SDOF vibration can be analyzed by **Newton's second law of motion**, $F = ma$. The analysis can be easily visualised with the aid of a **free body diagram**,

The resulting equation of motion is a **second order, non-homogeneous, ordinary differential equation**:

$$m\ddot{u} + c\dot{u} + ku = f(t) \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \quad (7.7)$$

with the initial conditions u_0 and $\dot{u}_0$.

7.2.3 Time Solution for Unforced Undamped SDOF Systems

<https://www.efunda.com/formulae/vibrations/sdof-free-undamped.cfm>

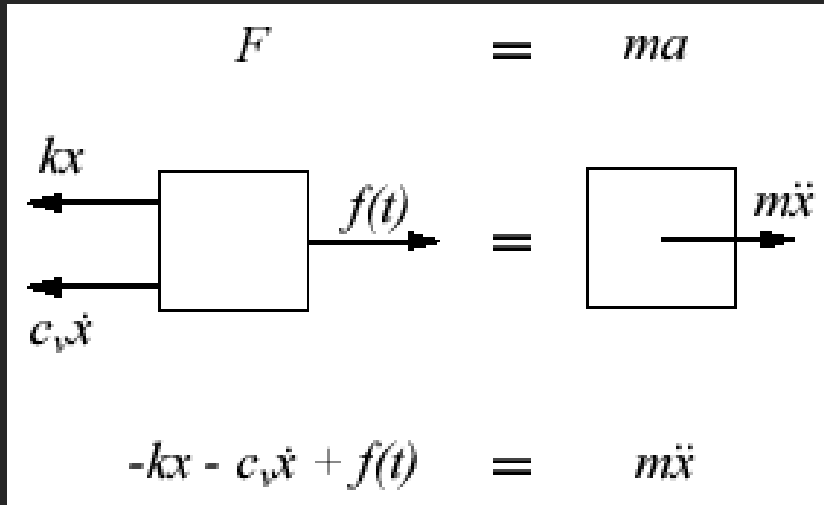


Figure 7.2: SDOF Free Body Diagram

The equation of motion derived in the definition can be simplified to:

$$m\ddot{u} + ku = 0 \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \quad (7.8)$$

This equation of motion is a **second order, homogeneous, ordinary differential equation** (ODE). If the mass and spring stiffness are constants, the ODE becomes a **linear, homogeneous ODE with constant coefficients** and can be solved by the Characteristic Equation method. The characteristic equation for this problem is:

$$ms^2 + k = 0 \quad (7.9)$$

which determines the 2 independent roots for the undamped vibration problem. The final solution (that contains the 2 independent roots from the characteristic equation and satisfies the initial conditions) is:

$$\begin{aligned}
u(t) &= c_1 e^{i\omega_n t} + c_2 e^{-i\omega_n t} \\
&= d_1 \cos \omega_n t + d_2 \sin \omega_n t \\
\implies u(t) &= u_0 \cos \omega_n t + \frac{\dot{u}_0}{\omega_n} \sin \omega_n t
\end{aligned} \tag{7.10}$$

The natural frequency ω_n is defined by:

$$\omega_n = \sqrt{\frac{k}{m}} \tag{7.11}$$

and depends only on the system mass and the spring stiffness (i.e. any damping will not change the natural frequency of a system).

Alternatively, the solution may be expressed by the equivalent form,

$$u(t) = A_0 \cos(\omega_n t - \phi_0) \tag{7.12}$$

where the amplitude A_0 and the initial phase ϕ_0 are given by:

$$\begin{aligned}
A_0 &= \sqrt{u_0^2 + \left(\frac{\dot{u}_0}{\omega_n}\right)^2} \\
\phi_0 &= \tan^{-1} \frac{\dot{u}_0}{u_0 \omega_n}
\end{aligned} \tag{7.13}$$

Note that an assumption of zero damping is typically not accurate. In reality, there almost always exists some resistance in vibratory systems. This resistance will damp the vibration and dissipate energy, the oscillatory motion caused by the initial disturbance will eventually be reduced to zero.

7.2.4 Time Solution for Unforced Damped SDOF Systems

<https://www.efunda.com/formulae/vibrations/sdof-free-damped.cfm>

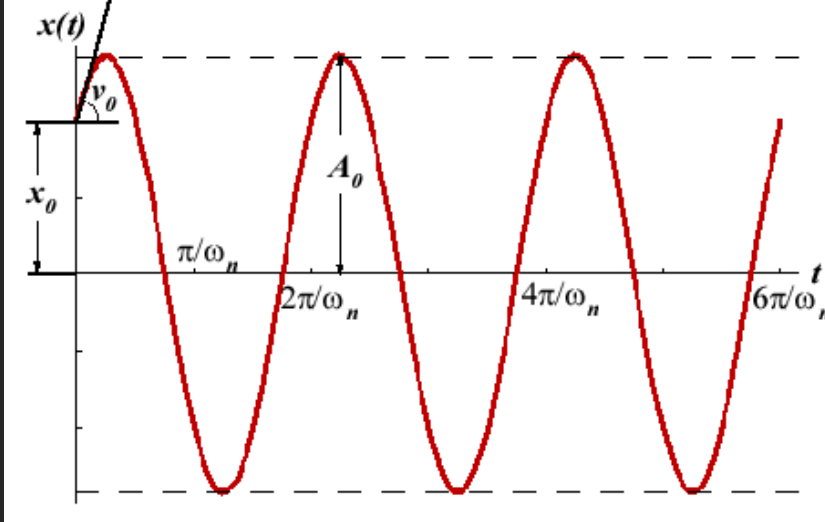


Figure 7.3: SDOF Undamped sample time behavior

Damping that produces a damping force proportional to the mass's velocity is commonly referred to as *viscous damping*, and is denoted graphically by a dashpot.

For an unforced damped **SDOF** system, the general equation of motion becomes:

$$m\ddot{u} + c\dot{u} + ku = 0 \quad \begin{cases} u(t=0) = u_0 \\ \dot{u}(t=0) = \dot{u}_0 \end{cases} \quad (7.14)$$

This equation of motion is a **second order, homogeneous, ODE**. If all parameters (mass, stiffness and viscous damping) are constants, the **ODE** becomes a **linear ODE with constant coefficients** and can be solved by the *Characteristic Equation method*. The characteristic equation for this problem is:

$$ms^2 + s_v s + k = 0 \quad (7.15)$$

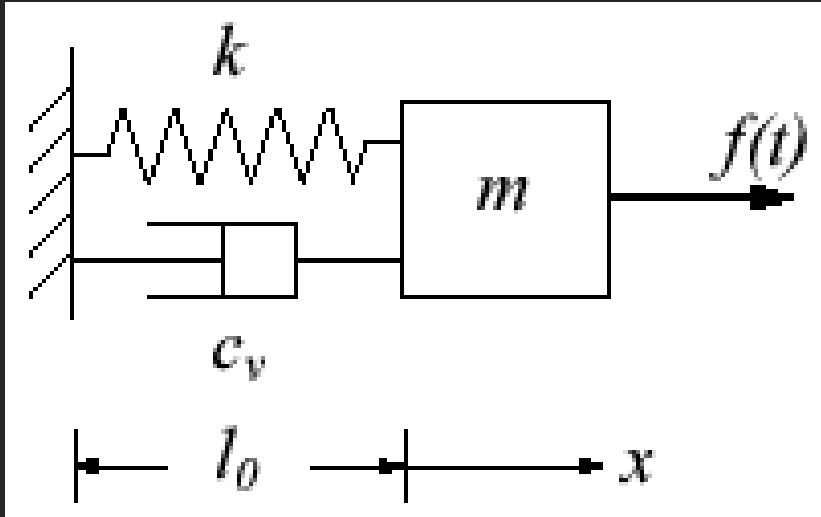


Figure 7.4: SDOF system with a linear spring and dashpot

which determines the 2 independent roots for the damped vibration problem. The roots to the characteristic equation fall into one of the following 3 cases:

- If $c_v^2 - 4mk < 0$, the system is termed **underdamped**. The roots of the characteristic equation are complex conjugates, corresponding to *oscillatory motion* with an *exponential decay* in amplitude.
- If $c_v^2 - 4mk = 0$, the system is termed **critically-damped**. The roots of the characteristic equation are repeated, corresponding to *simple decay motion* with at most *one overshoot* of the systems resting position.
- If $c_v^2 - 4mk > 0$, the system is termed **overdamped**. The roots of the characteristic equation are purely real and distinct, corresponding to *simple exponentially decaying motion*.

To simplify the solutions coming up, we define the critical damping c_c , the damping ratio ζ , and the damped vibration frequency ω_d as:

$$\begin{aligned}
c_c &= 2m\sqrt{\frac{k}{m}} = 2m\omega_n \\
\zeta &= \frac{c_v}{c_c} \\
\omega_d &= \sqrt{1 - \zeta^2}\omega_n
\end{aligned} \tag{7.16}$$

where the natural frequency of the system ω_n is given by:

$$\omega_n = \sqrt{\frac{k}{m}} \tag{7.17}$$

Note that ω_d will equal ω_n when the damping of the system is zero (i.e. undamped). The time solutions for the free **SDOF** system is presented below for each of the three case scenarios.

Time Solution of Unforced Underdamped SDOF Systems

When $c_v^2 - 4mk < 0$ (equivalent to $\zeta < 1$ or $c_v < c_c$), the characteristic equation has a pair of complex conjugate roots. The displacement solution for this kind of system is:

$$\begin{aligned}
u(t) &= c_1 e^{-\zeta + i\sqrt{1-\zeta^2}\omega_n t} + c_2 e^{-\zeta - i\sqrt{1-\zeta^2}\omega_n t} \\
&= e^{-\zeta\omega_n t} [d_1 \cos(\omega_d t) + d_2 \sin(\omega_d t)] \\
\Rightarrow u(t) &= \underbrace{e^{-\zeta\omega_n t}}_{\text{exponential decay}} \underbrace{\left[u_0 \cos(\omega_d t) + \frac{\dot{u}_0 + \zeta\omega_n u_0}{\omega_d} \sin(\omega_d t) \right]}_{\text{periodic motion}}
\end{aligned} \tag{7.18}$$

An alternate but equivalent solution is given by:

$$u(t) = A_0 \underbrace{e^{-\zeta\omega_n t}}_{\text{exponential decay}} \underbrace{\cos(\omega_d t - \phi_0)}_{\text{periodic motion}} \tag{7.19}$$

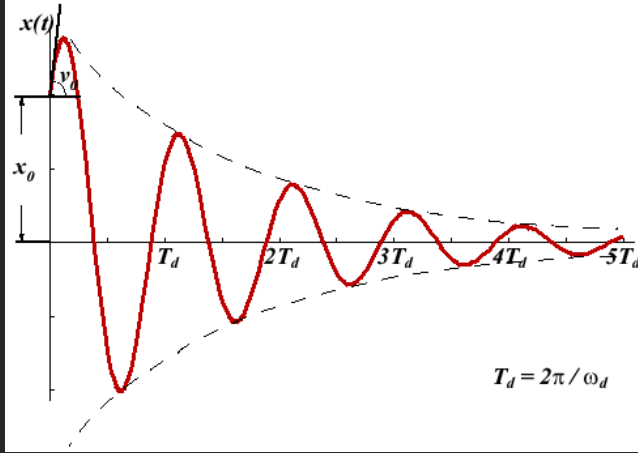


Figure 7.5: The displacement plot of an underdamped system

Note that the displacement amplitude decays exponentially (i.e. the natural logarithm of the amplitude ratio for any two displacements separated in time by a constant ratio is a constant):

$$\begin{aligned}
 \frac{A_k}{A_{k+1}} &= \frac{A_0 e^{-\zeta \omega_n (kT_d)} \cos(\phi_0)}{A_0 e^{-\zeta \omega_n [(k+1)T_d]} \cos(\phi_0)} \\
 &= \frac{e^{-\zeta \omega_n (kT_d)}}{e^{-\zeta \omega_n [(k+1)T_d]}} \\
 &= e^{\zeta \omega_n T_d} \\
 \Rightarrow \ln \left(\frac{A_k}{A_{k+1}} \right) &= \zeta \omega_n T_d = \zeta \omega_n \frac{2\pi}{\omega_d} = \frac{2\pi\zeta}{\sqrt{1-\zeta^2}}
 \end{aligned} \tag{7.20}$$

where $T_d = \frac{1}{f_d} = \frac{2\pi}{\omega_d}$ is the period of the damped vibration.

Time Solution of Unforced Critically-Damped Systems

When $c_v^2 - 4mk = 0$ (equivalent to $\zeta = 1$ or $c_v = c_c$), the characteristic equation has repeated real roots. The displacement solution for this kind of system is:

$$\begin{aligned}
 u(t) &= (c_1 + c_2 t) e^{-\omega_n t} \\
 \implies u(t) &= e^{-\omega_n t} [u_0 + (v_0 + \omega_n u_0) t]
 \end{aligned} \tag{7.21}$$

The critical damping factor c_c can be interpreted as the **minimum damping** that results in non-periodic motion (i.e. simple decay).

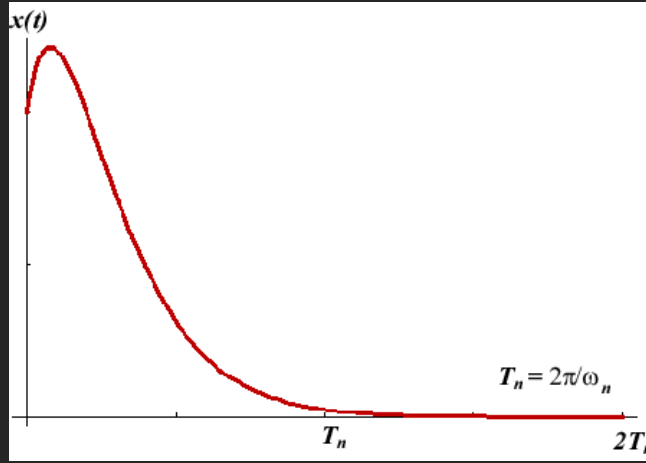


Figure 7.6: The displacement plot of a critically-damped system with positive initial displacement and velocity.

The displacement decays to a negligible level after one natural period T_n . Note that if the initial velocity v_0 is negative while the initial displacement u_0 is positive, there will exist one overshoot of the resting position in the displacement plot.

Time Solution of Unforced Overdamped SDOF Systems

When $c_v^2 - 4mk > 0$ (equivalent to $\zeta > 1$ or $c_v > c_c$), the characteristic equation has two distinct real roots. The displacement solution for this kind of system is:

$$\begin{aligned}
 u(t) &= c_1 e^{(-\zeta + \sqrt{\zeta^2 - 1})\omega_n t} + c_2 e^{(-\zeta - \sqrt{\zeta^2 - 1})\omega_n t} \\
 \Rightarrow u(t) &= \frac{u_0 \omega_n (\zeta + \sqrt{\zeta^2 - 1}) + v_0}{2\omega_n \sqrt{\zeta^2 - 1}} e^{(-\zeta + \sqrt{\zeta^2 - 1})\omega_n t} + \\
 &\quad + \frac{-u_0 \omega_n (\zeta - \sqrt{\zeta^2 - 1}) - v_0}{2\omega_n \sqrt{\zeta^2 - 1}} e^{(-\zeta - \sqrt{\zeta^2 - 1})\omega_n t}
 \end{aligned} \tag{7.22}$$

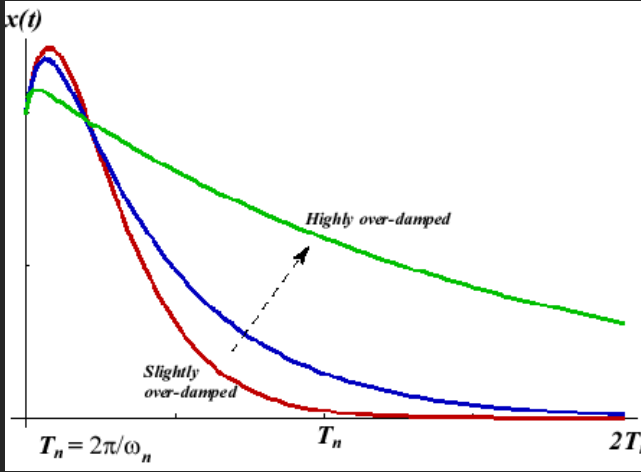


Figure 7.7: The displacement plot of an overdamped system response

The motion of an overdamped system is non-periodic, regardless of the initial conditions. The larger the damping, the longer time to decay from an initial disturbance.

If the system is heavily damped, $\zeta \gg 1$, the displacement solution takes the approximate form:

$$u(t) \approx u_0 + \frac{v_0}{2\zeta\omega_n} (1 - e^{-2\zeta\omega_n t}) \tag{7.23}$$

7.2.5 SDOF Systems under Harmonic Excitation

https://www.efunda.com/formulae/vibrations/sdof_harmo.cfm

When a **SDOF** System is forced by $f(t)$, the solution for the displacement $x(t)$ consists of two parts: the *complimentary solution*, and the *particular solution*. The complimentary solution for the problem is given by the **free vibration of an unforced SDOF System**. The **particular solution** depends on the nature of the forcing function.

When the forcing function is **harmonic** (i.e. it consists of at most a **sine** and **cosine** at the same **frequency**, a quantity that can be expressed by the complex exponential $e^{i\omega t}$), the method of **Undetermined Coefficients** can be used to find the particular solution. Non-harmonic forcing functions are handled by other techniques.

Consider the SDOF system forced by the harmonic function $f(t)$.

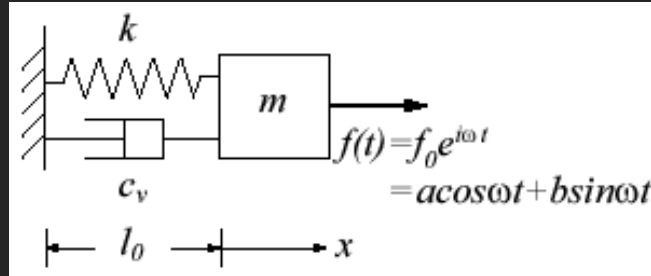


Figure 7.8: SDOF System forced by the harmonic function $f(t)$

The particular solution for this problem is found to be:

$$u_p(t) = \frac{f_0}{(k - m\omega^2) + ic\omega} e^{i\omega t} \quad (7.24)$$

The general solution is given by the sum of the complimentary and particular solutions multiplied by two weighting constants c_1 and c_2 ,

$$u(t) = c_1 u_c(t) + c_2 u_p(t) \quad (7.25)$$

The values of c_1 and c_2 are found by matching $u(t = 0)$ to the initial conditions.

Undamped SDOF Systems under Harmonic Excitation

For an undamped system ($c_v = 0$) the total displacement solution is:

$$\begin{aligned}
 u(t) &= d_1 \cos \omega_n t + d_2 \sin \omega_n t + \frac{f_0}{k - m\omega^2} e^{i\omega t} \\
 \implies u(t) &= \left(u_0 - \frac{f_0}{k - m\omega^2} \right) \cos \omega_n t \\
 &\quad + \left(\frac{v_0 - \frac{i\omega f_0}{k - m\omega^2}}{\omega_n} \right) \sin \omega_n t \\
 &\quad + \frac{f_0}{k - m\omega^2} e^{i\omega t}
 \end{aligned} \tag{7.26}$$

If the forcing frequency is close to the natural frequency, $\omega \approx \omega_n$, the system will exhibit **resonance** (very large displacements) due to near-zeros in the denominators of $u(t)$.

When the forcing frequency is equal to the natural frequency, we cannot use the $u(t)$ given above as it would give divide-by-zero. Instead we must use **L'Hôpital's Rule** to derive a solution free of zeros in the denominators.

$$\begin{aligned}
 u(t) &= u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t \\
 &\quad + \lim_{\omega \rightarrow \omega_n} \left\{ \frac{f_0}{k - m\omega^2} (e^{i\omega t} - \cos \omega_n t - i\omega \sin \omega_n t) \right\} \\
 &= u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t - \frac{f_0 \omega_n}{2k} (ite^{i\omega} nt - i \sin \omega_n t)
 \end{aligned} \tag{7.27}$$

To simplify $u(t)$, let's assume that the driving force consists only of the cosine function $f(t) = f_0 \cos \omega t$.

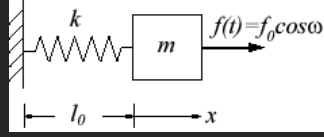


Figure 7.9: SDOF System forced by the harmonic function $f(t) = f_0 \cos \omega t$

The displacement solution reduces to:

$$\begin{aligned}
 u(t) &= u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t \\
 &+ \lim_{\omega \rightarrow \omega_n} \left\{ \frac{f_0}{k - m\omega^2} (\cos \omega t - \cos \omega_n t) \right\} \\
 &= \underbrace{u_0 \cos \omega_n t + \frac{v_0}{\omega_n} \sin \omega_n t}_{\text{free vibration (complementary)}} + \underbrace{\frac{f_0 \omega_n t}{2k} \sin \omega_n t}_{\substack{\text{amplitude} \\ \text{linearly} \\ \text{increased}}} \sin \omega_n t
 \end{aligned} \tag{7.28}$$

This solution contains one term multiplied by t . This term will cause the displacement amplitude to increase linearly with time as the forcing function pumps energy into the system.

The maximum displacement of an undamped system forced at its resonant frequency will increase unbounded according to the solution for $u(t)$ above. However, real systems will inject additional physics once displacements become large enough. These additional physics (nonlinear plastic deformation, heat transfer, buckling etc.) will serve to limit the maximum displacement exhibited by the system, and allow one to escape the "sudden death" impression that such systems will immediately fail.

Damped SDOF Systems under Harmonic Excitation

$$m\ddot{u} + c\dot{u} + ku = F_0 \cos(\omega t) \tag{7.29}$$

then **steady state** solution is in the form:

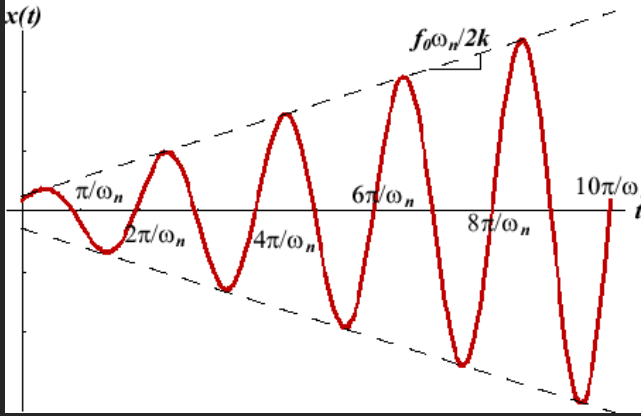


Figure 7.10: Resonance response of undamped SDOF with harmonic excitation

$$u_{s.s.} = u_0 \cos(\omega t - \phi) \quad (7.30)$$

If we plug the **steady state** solution to the **equation of motion**, we get:

$$\begin{aligned} u_o \left[(k - m\omega^2) \cos \omega t - \phi - c\omega \sin(\omega t - \phi) \right] &= F_0 \cos(\omega t) \quad | \times \frac{1}{k} \\ u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2} \right) \cos \omega t - \phi - 2\zeta \frac{\omega}{\omega_n} \sin(\omega t - \phi) \right] &= \frac{F_0}{k} \cos(\omega t) \end{aligned} \quad (7.31)$$

because:

$$\frac{k}{m} = \omega_n^2 \quad (7.32)$$

$$\zeta = \frac{c}{c_c} \quad (7.33)$$

$$c = 2m\sqrt{\frac{k}{m}} = 2m\omega_n \quad (7.34)$$

$$\frac{c}{k} = \frac{\zeta c_c}{k} = \frac{\zeta 2m\omega_n}{k} = \frac{2\zeta\omega_n^2}{\omega_n^2} = \frac{2\zeta}{\omega_n} \quad (7.35)$$

after utilising the sine and cosine relations:

$$\begin{aligned} \cos(\omega t - \phi) &= \cos \omega t \cos \phi + \sin \omega t \sin \phi \\ \sin(\omega t - \phi) &= \sin \omega t \cos \phi + \cos \omega t \sin \phi \end{aligned} \quad (7.36)$$

we get a set of two equatons:

$$\begin{aligned} u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2}\right) \cos \phi + 2\zeta \frac{\omega}{\omega_n} \sin \phi \right] \cos \omega t &= \frac{F_0}{k} \cos(\omega t) \\ u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2}\right) \sin \phi - 2\zeta \frac{\omega}{\omega_n} \cos \phi \right] \sin \omega t &= 0 \end{aligned} \quad (7.37)$$

and when solving just for amplitudes this reduces to a set of two algebraic equations which is then solved for u_0 and ϕ :

$$\begin{aligned} u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2}\right) \cos \phi + 2\zeta \frac{\omega}{\omega_n} \sin \phi \right] &= \frac{F_0}{k} \\ u_o \left[\left(1 - \frac{\omega^2}{\omega_n^2}\right) \sin \phi - 2\zeta \frac{\omega}{\omega_n} \cos \phi \right] &= 0 \end{aligned} \quad (7.38)$$

The solution (**transfer function**) is finally:

$$\begin{aligned} u_0 &= \frac{\frac{F_0}{k}}{\left[\left(1 - \frac{\omega^2}{\omega_n^2}\right)^2 + \left(2\zeta \frac{\omega}{\omega_n}\right)^2 \right]^{\frac{1}{2}}} \\ \phi &= \tan^{-1} \left(\frac{2\zeta \frac{\omega}{\omega_n}}{1 - \frac{\omega^2}{\omega_n^2}} \right) \end{aligned} \quad (7.39)$$

where:

u_0 is displacement amplitude

$\frac{F_0}{k}$ is displacement from static simulation.

7.3 Damping

7.3.1 Rayleigh Damping

<https://www.simscale.com/knowledge-base/rayleigh-damping-coefficients/>

Let **Rayleigh Damping** be defined as:

$$c = \alpha k + \beta m \quad (7.41)$$

where:

c is damping value [-],

α is stiffness dependent damping coefficient,

β is inertia (mass) dependent damping coefficient,

m is mass and

k is stiffness.

Thus, substituting this relation to the equation of motion:

$$\begin{aligned} m\ddot{u} + (\alpha k + \beta m) \dot{u} + ku &= f(t) & | \times \frac{1}{m} \\ \ddot{u} + \left(\alpha \frac{k}{m} + \beta \frac{m}{m} \right) \dot{u} + \frac{k}{m} u &= \frac{f(t)}{m} & | \frac{k}{m} = \omega_n^2 \\ \ddot{u} + (\alpha \omega_n^2 + \beta) \dot{u} + \omega_n^2 u &= \frac{f(t)}{m} \end{aligned} \quad (7.42)$$

Damping Ratio:

$$\begin{aligned} \zeta &= \frac{c}{2m\omega_n} \\ \zeta &= \frac{1}{2m\omega_n} (\alpha k + \beta m) \\ \zeta &= \frac{1}{2} \left(\alpha \omega_n + \frac{\beta}{\omega_n} \right) \end{aligned} \quad (7.43)$$

where:

ζ is damping ratio,

c is damping value [-],

m is mass and

ω_n is natural frequency [rad/s]

Substituting back:

$$\ddot{u} + 2\zeta\omega_n\dot{u} + \omega_n^2 ku = \frac{f(t)}{m} \quad (7.44)$$

When **Damping is proportional to inertia**:

In this case, the stiffness coefficient $\alpha = 0$ and thus:

$$\zeta = \frac{\beta}{2\omega_n} \quad (7.45)$$

For a given constant value of β , it is seen that the damping is inversely proportional to the natural frequency, as shown in the illustration:

Moreover, if one computes β from the damping ration ζ_1 at a given natural frequency ω_1 , all the natural frequencies below it will be amplified and the frequencies above it will be attenuated. The effect is more dramatic the farther the frequencies are from the reference value.

When **Damping is proportional to stiffness**:

In this case, the mass coefficient $\beta = 0$ and thus:

$$\zeta = \frac{1}{2}\alpha\omega_n \quad (7.46)$$

It is seen that, contrary to the first case, here the damping is directly proportional to the natural frequency:

If one computes α from the damping ratio ζ_1 at a natural frequency ω_1 , then the natural frequencies below will be attenuated and the frequencies above will be amplified.

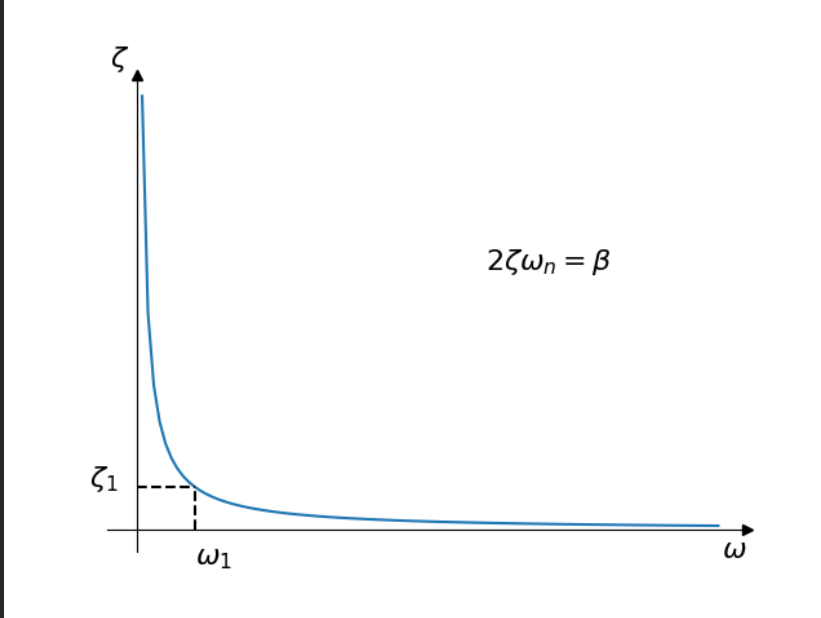


Figure 7.11: Schematic of damping proportional to inertia

General Case:

In the case of using the model with two parameters, the proportionality of damping against frequency is convex:

In this case one needs two damping ratios and two natural frequencies to create a pair of equations and solve for α and β . The model gives some flexibility on where to place the natural frequencies, but in general, frequencies too far away from the ones used in the computation will be amplified.

In the particular case of using equal damping ratios for the two frequencies, it is important to note that the damping ratio **will not be constant inside the range** defined by the sample points, but the inner frequencies will be attenuated. That is, the inner frequencies will have a lower damping ratio.

Computing the Rayleigh Damping Coefficients

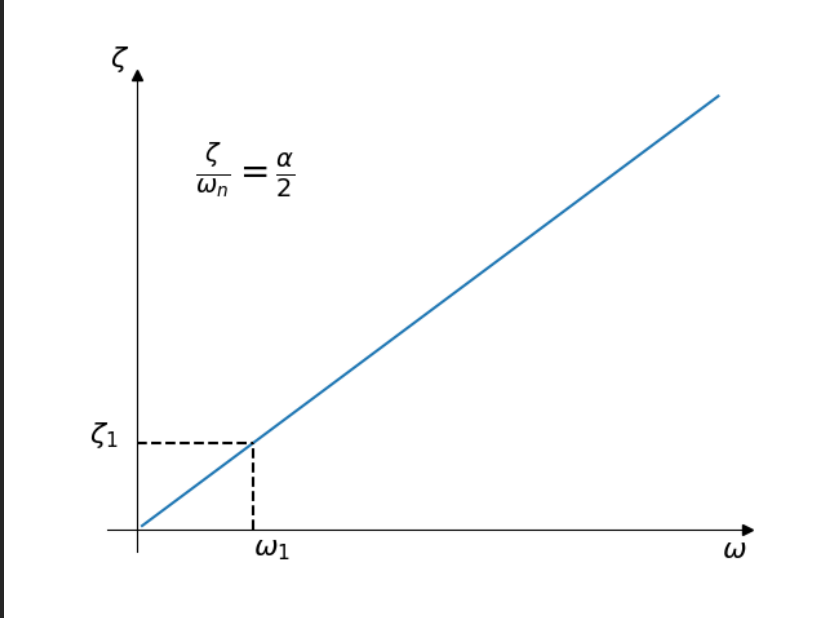


Figure 7.12: Schematic of damping proportional to stiffness

In the most common case, a transient response curve from the system is obtained and the damping ratio ζ_1 is determined for the lowest natural frequency ω_1 by measuring the (logarithmic) attenuation of successive peaks.

$$\begin{aligned}
 \zeta &= \frac{\delta}{\sqrt{\delta^2 + (2\pi)^2}} \\
 \delta &= \ln \frac{x_0}{x_1} \\
 f &= \frac{1}{T} = \frac{1}{t_1 - t_0} \\
 \omega &= 2\pi f
 \end{aligned} \tag{7.47}$$

It is then most common to assume the case of damping proportional to the stiffness, that is, $\beta = 0$, and the α stiffness coefficient is computed from:

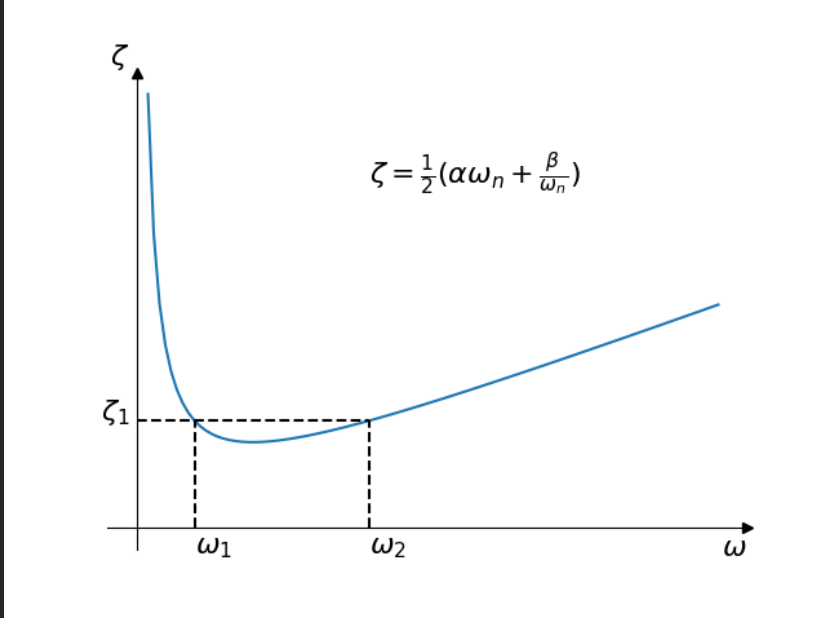


Figure 7.13: Schematic of full damping model

$$\alpha = \frac{2\zeta_1}{\omega_1} = \frac{\zeta + 1}{\pi f_1} \quad (7.48)$$

If the knowledge on the system indicates the case of damping decreasing with the frequency, then one can assume the case of damping proportional to the inertia, where $\alpha = 0$ and determine the mass coefficient β :

$$\beta = 2\zeta_1\omega_1 = 4\pi\zeta_1f_1 \quad (7.49)$$

If there is not such test data or knowledge of the system, or if one wishes to apply an approximate damping ration over a range of frequencies, then we can use the general case and build a system of two equations:

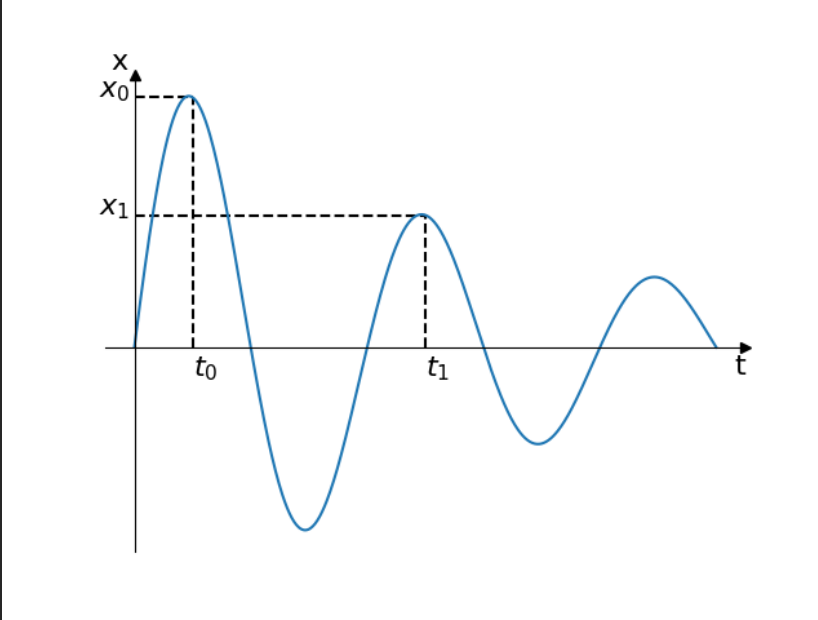


Figure 7.14: Determination of the damping ratio from the logarithmic decay

$$\begin{aligned}\zeta_1 &= \frac{1}{2} \left(\alpha \omega_1 + \frac{\beta}{\omega_1} \right) \\ \zeta_2 &= \frac{1}{2} \left(\alpha \omega_2 + \frac{\beta}{\omega_2} \right)\end{aligned}\tag{7.50}$$

Then solve for the unknown coefficients, keeping in mind the considerations given above for the general case and the influence of the model on natural frequencies inside and outside the range of interest. That is, perhaps one wants to achieve a mean damping ratio over the range, then compensate the attenuation by modifying the input damping ratios, or by performing some least-squares approximation from more than two frequency points.

7.4 Modal decomposition

7.4.1 Definition

Let:

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(t) \quad (7.51)$$

be the **equation of motion**, then:

$\mathbf{\Lambda}$ is a **spectral matrix**

$\mathbf{\Psi}$ is a **mode shape matrix**,

such that:

$$\mathbf{\Lambda} = \begin{bmatrix} \lambda_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_n^2 \end{bmatrix} \quad (7.52)$$

where λ_n^2 is the n-th eigenvalue of the problem $(\mathbf{K} - \lambda^2\mathbf{M})\mathbf{X} = 0$

and:

$$\mathbf{\Psi} = [\psi_1 \quad \dots \quad \psi_n] \quad (7.53)$$

where $\psi_n = \{\psi_{n,1}, \dots, \psi_{n,m}\}^T$ is the eigenvector of the n-th eigenvalue λ_n^2 .

The **displacement** $\mathbf{u}$ can be expressed as a linear combination of eigenvectors such that:

$$\begin{aligned} \mathbf{u}(t) &= \mathbf{\Psi}\mathbf{q}(t) \\ &= [\psi_1 \quad \dots \quad \psi_n] \mathbf{q}(t) \\ &= \begin{bmatrix} \psi_{1,1} & \dots & \psi_{n,1} \\ \vdots & \ddots & \vdots \\ \psi_{1,m} & \dots & \psi_{n,m} \end{bmatrix} \begin{bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{bmatrix} \end{aligned} \quad (7.54)$$

where $\mathbf{q}(t)$ is a vector of modal coefficients.

Then:

$$\begin{aligned}\dot{\mathbf{u}}(t) &= \Psi \dot{\mathbf{q}}(t) \\ \ddot{\mathbf{u}}(t) &= \Psi \ddot{\mathbf{q}}(t)\end{aligned}\tag{7.55}$$

First we substitute $\mathbf{u} = \Psi \mathbf{q}$ to the **equation of motion**:

$$\mathbf{M}\Psi\ddot{\mathbf{q}} + \mathbf{C}\Psi\dot{\mathbf{q}} + \mathbf{K}\Psi\mathbf{q} = \mathbf{F}(t)\tag{7.56}$$

By **premultiplying** with Ψ^T we get:

$$\Psi^T \mathbf{M} \Psi \ddot{\mathbf{q}} + \Psi^T \mathbf{C} \Psi \dot{\mathbf{q}} + \Psi^T \mathbf{K} \Psi \mathbf{q} = \Psi^T \mathbf{F}(t) = \mathbf{Q}(t)\tag{7.57}$$

where $\mathbf{Q}(t) = \Psi^T \mathbf{F}(t)$ is called a **modal load**.

7.4.2 Orthonormalised Modal Base

Then one can exploit the properties of **orthonormalised eigenvectors**, where:

$$\psi_r^T \psi_s = 0 \Leftrightarrow r \neq s\tag{7.58}$$

The **mass**, **damping** and **stiffness** matrices are therefore reduced to:

$$\begin{aligned}
\Psi^T \mathbf{M} \Psi &= \begin{bmatrix} m_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & m_n \end{bmatrix} \\
\Psi^T \mathbf{C} \Psi &= \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \\
\Psi^T \mathbf{K} \Psi &= \begin{bmatrix} k_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & k_n \end{bmatrix}
\end{aligned} \tag{7.59}$$

where:

m_i is the **modal mass** of the i-th shape

c_i is the **modal damping** of the i-th shape

k_i is the **modal stiffness** of the i-th shape

Note:

When are the eigenvectors **orthonormalised** to **mass**, the above equations reduce to:

$$\begin{aligned}
 \Psi^T \mathbf{M} \Psi &= \begin{bmatrix} 1.0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1.0 \end{bmatrix} = \mathbf{I} \\
 \Psi^T \mathbf{C} \Psi &= \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \\
 \Psi^T \mathbf{K} \Psi &= \begin{bmatrix} \lambda_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \lambda_n^2 \end{bmatrix} = \Lambda^2
 \end{aligned} \tag{7.60}$$

Mass normalisation:

$$\psi_{i, mass} = \frac{\psi_i}{\psi_i^T \mathbf{m}_i \psi_i} \tag{7.61}$$

where:

$\mathbf{m}_i$ is the i -th row of the **mass** matrix $\mathbf{M}$.

7.4.3 MDOF to SDOF

The MDOF **equation of motion** can be then rewritten as a set of **SDOF** equations:

$$\begin{aligned}
 \Psi^T \mathbf{M} \Psi \ddot{\mathbf{q}}(t) + \Psi^T \mathbf{C} \Psi \dot{\mathbf{q}}(t) + \Psi^T \mathbf{K} \Psi \mathbf{q}(t) &= \Psi^T \mathbf{F}(t) \\
 \mathbf{M}_\Psi \ddot{\mathbf{q}}(t) + \mathbf{C}_\Psi \dot{\mathbf{q}}(t) + \mathbf{K}_\Psi \mathbf{q}(t) &= \mathbf{Q}(t)
 \end{aligned} \tag{7.62}$$

Written explicitly:

$$\begin{aligned}
& \begin{bmatrix} m_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & m_n \end{bmatrix} \begin{bmatrix} \ddot{q}_1(t) \\ \vdots \\ \ddot{q}_n(t) \end{bmatrix} + \\
& \begin{bmatrix} c_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & c_n \end{bmatrix} \begin{bmatrix} \dot{q}_1(t) \\ \vdots \\ \dot{q}_n(t) \end{bmatrix} + \\
& \begin{bmatrix} k_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & k_n \end{bmatrix} \begin{bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{bmatrix} = \begin{bmatrix} Q_1(t) \\ \vdots \\ Q_n(t) \end{bmatrix}
\end{aligned} \tag{7.63}$$

It is important to note that all modal matrices (mass, damping and stiffness) are purely diagonal (with simple enough damping), so the **MDOF** problem reduces to a series of **SDOF** problems:

$$\begin{aligned}
m_1 \ddot{q}_1(t) + c_1 \dot{q}_1(t) + k_1 q_1(t) &= Q_1(t) \\
m_2 \ddot{q}_2(t) + c_2 \dot{q}_2(t) + k_2 q_2(t) &= Q_2(t) \\
&\vdots \\
m_n \ddot{q}_n(t) + c_n \dot{q}_n(t) + k_n q_n(t) &= Q_n(t)
\end{aligned} \tag{7.64}$$

which are solved separately and the results are **linearly** combined:

$$\begin{aligned}
\mathbf{u} &= \sum_{i=1}^n \psi_i q_i \\
\dot{\mathbf{u}} &= \sum_{i=1}^n \psi_i \dot{q}_i \\
\ddot{\mathbf{u}} &= \sum_{i=1}^n \psi_i \ddot{q}_i
\end{aligned} \tag{7.65}$$

Note: Above equation can be also written as:

$$\begin{aligned}\mathbf{u} &= \sum_{i=1}^n \psi_i q_{u,i} \\ \mathbf{v} &= \sum_{i=1}^n \psi_i q_{v,i} \\ \mathbf{a} &= \sum_{i=1}^n \psi_i q_{a,i}\end{aligned}\tag{7.66}$$

where:

$\mathbf{u}$ is displacement

$\mathbf{v}$ is velocity

$\mathbf{a}$ is acceleration

and

q_u is modal displacement

q_v is modal velocity

q_a is modal acceleration

7.4.4 Initial Conditions

To get the initial modal conditions, one simply:

$$\begin{aligned}\mathbf{q}_0 &= \Psi^{-1} \mathbf{u}_0 \\ \dot{\mathbf{q}}_0 &= \Psi^{-1} \dot{\mathbf{u}}_0 \\ \ddot{\mathbf{q}}_0 &= \Psi^{-1} \ddot{\mathbf{u}}_0\end{aligned}\tag{7.67}$$

where:

Ψ^{-1} is an inverse of eigenshape matrix $\Leftrightarrow$ all eigenvalues and eigenshapes are used.

Note: This is not usually so. Normally one would use **reduced modal base**, where only the first $\mathbf{n}$ eigenmodes are used. Then Ψ^{-1} is written as Ψ^+ and called the **pseudoinverse of mode shape matrix** defined (at least in **numpy** - `numpy.linalg.pinv()`) as:

The pseudo-inverse of a matrix $\mathbf{A}$, denoted $\mathbf{A}^+$, is defined as: “the matrix that ‘solves’ [the least-squares problem] $\mathbf{A}\mathbf{x} = \mathbf{b}$,” i.e., if $\bar{\mathbf{x}}$ is said solution, then $\mathbf{A}^+$ is that matrix such that $\bar{\mathbf{x}} = \mathbf{A}^+\mathbf{b}$.

It can be shown that if $\mathbf{Q}_1\mathbf{\Sigma}\mathbf{Q}_2^T = \mathbf{A}$ is the singular value decomposition of $\mathbf{A}$, then $\mathbf{A}^+ = \mathbf{Q}_2\mathbf{\Sigma}^+\mathbf{Q}_1^T$, where $\mathbf{Q}_{1,2}$ are orthogonal matrices, $\mathbf{\Sigma}$ is a diagonal matrix consisting of $\mathbf{A}$ ’s so-called singular values, (followed, typically, by zeros), and then $\mathbf{\Sigma}^+$ is simply the diagonal matrix consisting of the reciprocals of $\mathbf{A}$ ’s singular values (again, followed by zeros).

Chapter 8

Kinematics

8.1 Rigid Body Motion

A rigid freebody relation between **slave** (dependent) and **master** (independent) DOFs is calculated following the equation:

$$\begin{aligned}\mathbf{T}_s &= \mathbf{T}_m + \mathbf{R}_s \times \mathbf{x} \\ \mathbf{R}_s &= \mathbf{R}_m\end{aligned}\tag{8.1}$$

where $\mathbf{T}$ is a translations vector = $\{\mathbf{T}_1, \mathbf{T}_2, \mathbf{T}_3\}$, $\mathbf{R}$ is a rotations vector = $\{\mathbf{R}_1, \mathbf{R}_2, \mathbf{R}_3\}$, $\mathbf{x}$ is a vector from slave (base) to the master node (tip) = $\{\mathbf{x}_s - \mathbf{x}_m, \mathbf{y}_s - \mathbf{y}_m, \mathbf{z}_s - \mathbf{z}_m\}$ and $\times$ is a vector (cross) product.

Chapter 9

Iterative Methods

9.1 Root finding problems

9.1.1 Incremental Search Method

The approximate locations of the roots are best determined by plotting the function. Often a very rough plot, based on a few points, is sufficient to provide reasonable starting values.

The basic thought behind the incremental search method is simple: If $f(x_1)$ and $f(x_2)$ have opposite signs, then there is at least one root in the interval (x_1, x_2) . If the interval is small enough, it is likely to contain a single root. Thus the zeros of $f(x)$ can be detected by evaluating the function $f(x)$ at intervals Δx and looking for a change in sign.

There are several potential problems with the incremental search method:

- It is possible to miss two closely spaced roots if the search increment Δx is larger than the spacing of the roots
- A double root (two roots that coincide) will not be detected.

- Certain singularities (poles) of $f(x)$ can be mistaken for roots. For example, $f(x) = \tan(x)$ changes sign at $x \pm \frac{1}{2}n\pi$, $n = 1, 3, 5, \dots$. However, these locations are not true zeroes, because the function does not cross the x-axis.

Example of incremental function in python:

```

1 import math
2
3 def rootsearch(f: object, a: float, b: float, dx: float) -> float:
4     """function using incremental search on the interval <a, b>, in increments
5     of dx, to find the bounds <x1, x2> of the smallest root of f(x).
6
7     Args:
8         f (object): function to find the smallest root of
9         a (float): lower bound of the search region
10        b (float): upper bound of the search region
11        dx (float): search step
12
13    Returns:
14        x1 = x2 = None if no roots were detected.
15        x1 < x2 = the bounds specified by dx (x2 = x1 + dx)
16    """
17    x1, f1 = a, f(a)
18    x2, f2 = b, f(b)
19    while math.sign(f1) == math.sign(f2):
20        if x1 >= b:
21            return None, None
22        x1, f1 = x2, f2
23        x2, f2 = x2 + dx, f(x2 + dx)
24    else:
25        return x1, x3
26

```

9.1.2 Bisection Method

After a root $f(x) = 0$ has been bracketed in the interval $< x_1, x_2 >$, several methods can be used to close in on it. The method of **bisection** accomplishes it by *halving* the interval until the error is sufficiently small:

$$\begin{aligned}\varepsilon_{err} &= |x_2 - x_1| \\ \varepsilon_{err} &\leq \varepsilon_{tol}\end{aligned}\tag{9.1}$$

The method of **bisection** uses the same principle as the **incremental** search method. If there is a root $f(x) = 0$ in the interval $x \in < x_1, x_2 >$ then:

$$\text{sign}(f(x_1)) \neq \text{sign}(f(x_2))\tag{9.2}$$

To halve the interval, we compute $f(x_3)$, where $x_3 = \frac{1}{2}(x_1 + x_2)$ is the midpoint of the interval. If $\text{sign}(f(x_3)) \neq \text{sign}(f(x_2))$, then the root must be in the interval $< x_3, x_2 >$ and we replace the x_1 by x_3 . Otherwise the root must be in the interval $< x_1, x_3 >$ and we replace the x_2 by x_3 . In either case the interval is **halved**. The bisection is repeated until the interval has been reduced such that $\varepsilon_{err} \leq \varepsilon_{tol}$.

It is easy to compute the number of bisections needed to reach the prescribed ε_{tol} . The original interval Δx is reduced to $\Delta x/2$ after one bisection, $\Delta x/2^2$ after two bisections, and after n bisections it is $\Delta x/2^n$. Setting $\Delta x/2^n = \varepsilon$ and solving for n , we get:

$$n = \frac{\ln(\Delta x/\varepsilon)}{\ln 2}\tag{9.3}$$

Since n must be an integer, the ceiling of n is used (smallest integer greater n).

Python implementation:

```

1 import math
2
3 def bisection(f: callable, x1: float, x2: float,
4             deny_increase: bool = True, tol: float = 1.E-09) -> float:
5     """Root finding algorithm using a bisection method. The root must be
6     ↪ bracketed in
7     (x1, x2).
8
9     Args:
10         f (callable):          reference to the function to evaluate
11         x1 (float):            lower bounds of the root bracket
12         x2 (float):            upper bounds of the root bracket
13         deny_increase (bool): returns None if |f(x)| increases upon bisection
14                                (default = False)
15         tol (float):           allowed tolerance for root finding
16                                (default = 1.0E-09)
17
18     Returns:
19         (float): if root has been found
20         None:    otherwise
21
22     Raises:
23         ValueError: The root has not been bracketed.
24     """
25     f1 = f(x1)
26     if f1 == 0.: # jackpot -> no more searching
27         return x1
28
29     f2 = f(x2)
30     if f2 == 0.: # jackpot -> no more searching
31         return x2
32
33     if math.sign(f1) == math.sign(f2):
34         raise ValueError(f"The root has not been bracketed in <{x1:f}, {x2:f}>.")
35
36     n = int(math.ceil(math.log(abs(x2 - x1) / tol) / math.log(2)))

```



```

37     # the bisection itself
38     for i in range(n):
39         x3 = 0.5 * (x1 + x2)
40         f3 = f(x3)
41         # abs value of f(x) is larger than before
42         if deny_increase and ((abs(f3) > abs(f1)) or (abs(f3) > abs(f2))):
43             return None
44         if f3 == 0.: # jackpot -> no more searching
45             return x3
46         elif math.sign(f3) != math.sign(f2):
47             x1, f1 = x3, f3
48         else:
49             x2, f2 = x3, f3
50
51     # return the midpoint of the resulting span
52     return 0.5 * (x1 + x2)
53

```

9.1.3 Methods Based on Linear Interpolation

Secant and False Position Methods

The **secant** and **false position** methods are closely related. Both methods require two starting estimates of the root, say x_1 and x_2 . The function $f(x)$ is assumed to be approximately linear near the root, so that the improved value x_3 of the root can be estimated by linear interpolation between x_1 and x_2 .

This yields the relationship:

$$\frac{f_2}{x_3 - x_2} = \frac{f_1 - f_2}{x_2 - x_1} \quad (9.4)$$

where the notation $f_i = f(x_i)$ is used. Thus the improved estimate of the root is:

$$x_3 = x_2 - f_2 \frac{x_2 - x_1}{f_2 - f_1} \quad (9.5)$$

The difference between **false position** and **secant** method is in the manner the new estimate is dealt with.

- The **false position** method is similar to the **bisection** method where it is necessary that the root is **bracketed** by $x \in \langle x_1, x_2 \rangle$. When a new estimate x_3 is obtained, then if $\text{sign}(f_3) = \text{sign}(f_1)$, we let $x_1 \leftarrow x_3$, otherwise we let $x_2 \leftarrow x_3$. In this manner, the root is always bracketed in $\langle x_1, x_2 \rangle$. Then we run a new iteration until convergence criteria are met.
- The **secant** method is different in two ways. It does not require prior bracketing of the root, and it discards the oldest prior estimate of the root (i.e. after x_3 is computed, we let $x_1 \leftarrow x_2 \leftarrow x_3$).

The convergence of the **secant** method is **superlinear**, with the error behaving as $E_{k+1} = cE_k^{1.618}$, where the exponent 1.618... is called the "golden ratio".

Python implementation of the secant method:

```

1 import math
2
3 def secant(f: callable, x1: float, x2: float,
4           tol: float = 1.E-09, maxiter: int = 1000) -> float:
5     """Root finding algorithm using a secant method.
6
7     Args:
8         f (callable): reference to the function to evaluate
9         x1 (float):    lower bounds of the root bracket
10        x2 (float):    upper bounds of the root bracket
11        tol (float):   allowed tolerance for root finding
12                      (default = 1.0E-09)
13        maxiter (int): maximum number of iterations allowed (default = 1000)
14
15    Returns:
16        (float): if root has been found
17
18    Raises:
19        ValueError: No convergence is achievable as (f1 == f2, function is
20        ↪ constant)
21        ValueError: Maximum number of iterations reached.
22    """
23    f1 = f(x1)
24    if f1 == 0.: # jackpot -> no more searching
25        return x1
26
27    f2 = f(x2)
28    if f2 == 0.: # jackpot -> no more searching
29        return x2
30
31    # the secant method itself
32    for i in range(maxiter):
33        if f1 == f2:
34            raise ValueError(f"The function is constant in interval ({x1:f},
35            ↪ {x2:f}).")
36        x3 = x2 - f2 * (x2 - x1) / (f2 - f1)
37        f3 = f(x3)

```


Ridder's Method

Ridder's method is a clever modification of the **false position** method. Assuming the root is bracketed in (x_1, x_2) , we first compute $f_3 = f(x_3)$, where x_3 is the midpoint of the bracket.

It can be noted:

$$\begin{aligned} x_3 &= \frac{1}{2}(x_1 + x_2) \\ h &= \frac{1}{2}(x_2 - x_1) \end{aligned} \tag{9.6}$$

Next we introduce a new function $g(x)$ such, that:

$$g(x) = f(x)e^{(x-x_1)Q} \tag{9.7}$$

where the constant Q is determined by requiring the points (x_1, g_1) , (x_2, g_2) and (x_3, g_3) to lie on a straight line. As before, the notation is $g_i = g(x_i)$. The improved value x_4 is then obtained by linear interpolation of $g(x)$ rather than $f(x)$.

From 9.7 we obtain:

$$\begin{aligned} g_1 &= f_1 \\ g_2 &= f_2 e^{2hQ} \\ g_3 &= f_3 e^{hQ} \end{aligned} \tag{9.8}$$

where h is obtained from 9.6.

Using the requirement that the three points g_1 , g_2 and g_3 lie on a straight line or:

$$g_3 = \frac{g_1 + g_2}{2} \quad (9.9)$$

or:

$$f_3 e^{hQ} = \frac{1}{2} (f_1 + f_2 e^{2hQ}) \quad (9.10)$$

which is a quadratic equation in e^{hQ} . The solution is

$$e^{hQ} = \frac{f_3 \pm \sqrt{f_3^2 - f_1 f_2}}{f_2} \quad (9.11)$$

Linear interpolation based on points (x_1, g_1) and (x_3, g_3) now yields for the improved root:

$$\begin{aligned} x_4 &= x_3 - g_3 \frac{x_3 - x_1}{g_3 - g_1} \\ x_4 &= x_3 - f_3 e^{hQ} \frac{x_3 - x_1}{f_3 e^{hQ} - f_1} \end{aligned} \quad (9.12)$$

where in the last step the 9.8 is used. As the final step, a substitution for e^{hQ} is used from 9.11 and after some algebra the following solution is obtained:

$$x_4 = x_3 \pm (x_3 - x_1) \frac{f_3}{\sqrt{f_3^2 - f_1 f_2}} \quad (9.13)$$

it can be shown that the correct result is obtained by choosing the + sign if $f_1 - f_2 > 0$ and the minus sign if $f_1 - f_2 < 0$.

The 9.13 can be then adjusted as:

$$x_4 = x_3 + \text{sign}(f_1 - f_2) \times (x_3 - x_1) \frac{f_3}{\sqrt{f_3^2 - f_1 f_2}} \quad (9.14)$$

After the computation of x_4 , new brackets are determined for the root and 9.14 is applied again. The procedure is repeated until the difference between two successive values of x_4 becomes negligible.

The **Ridder's** iterative formula in 9.14 has a very useful property: If x_1 and x_2 bracket the root, then $x_4 \in \langle x_1, x_2 \rangle$ in all cases. In other words, once the root is bracketed, it stays bracketed.

Ridder's method can be shown to converge **quadratically**, making it faster than either the **secant** or the **false position** method.

It is the method to use if the derivative of $f(x)$ is impossible or difficult to compute.

Python implementation:

```

1 import math
2
3 def ridder(f: callable, x1: float, x2: float,
4           rtol: float = 1.E-09, maxiter: int = 1000) -> float:
5     """Root finding algorithm using a Ridder's algorithm. The root must be
6     bracketed in (x1, x2).
7
8     Args:
9         f (callable): reference to the function to evaluate
10        x1 (float):    lower bounds of the root bracket
11        x2 (float):    upper bounds of the root bracket
12        tol (float):   allowed tolerance for root finding
13                      absolute for found root, relative for
14                      change between root iterations
15                      (default = 1.0E-09)
16        maxiter (int): maximum number of iterations
17                      (default = 1000)
18
19    Returns:
20        (float): if root has been found
21        None:    if root is not bracketed
22
23    Raises:
24        ValueError: Maximum number of iterations reached.
25    """
26    f1 = f(x1)
27    if abs(f1) <= tol: # jackpot -> no more searching (absolute error)
28        return x1
29
30    f2 = f(x2)
31    if abs(f2) <= tol: # jackpot -> no more searching (absolute error)
32        return x2
33
34    if math.sign(f1) == math.sign(f2):
35        return None
36
37    x3 = 0.5 * (x1 + x2)

```



```

38
39     # the Ridder's method itself
40     xold = x1
41     for i in range(maxiter):
42         x3 = (x1 + x2) / 2
43         f3 = f(x3)
44         if abs(f3) <= tol: # jackpot -> no more searching (absolute error)
45             return x3
46         s = math.sqrt(f_3 ** 2 - f_1 * f_2)
47         if s == 0.:
48             return None
49         dx = (x3 - x1) * f3 / s
50         x4 = x3 + math.sign(f1 - f2) * dx
51         f4 = f(x4)
52         if abs(f4) <= tol: # jackpot -> no more searching (absolute error)
53             return x4
54
55         # test for convergence -> relative error
56         if abs(x4 - xold) <= tol * abs(x4):
57             return x4
58         xold = x4
59
60         # Re-bracket the root as tightly as possible
61         if math.sign(f3) == math.sign(f4):
62             if math.sign(f1) != math.sign(f4):
63                 # f1 0 f4 f3 f2
64                 x2, f2 = x4, f4
65             else:
66                 # f1 f3 f4 0 f2
67                 x1, f1 = x4, f4
68         else:
69             # f1 f3 0 f4 f2
70             x1, f1, x2, f2 = x3, f3, x4, f4
71
72     else:
73         raise ValueError(f"Maximum number of iterations ({maxiter:d}) reached!")
74

```

Newton-Raphson Method

The **Newton-Raphson** algorithm is the best known method of finding roots for a good reason: it is **simple** and **fast**. The only drawback of the method is that it uses the **derivative** $f'(x)$ of the function as well as the function $f(x)$ itself. Therefore, the **Newton-Raphson** method is usable only in problems where $f'(x)$ can be readily computed.

The **Newton-Raphson** formula can be derived from the **Taylor series** expansion of $f(x)$ about x :

$$f(x_{i+1}) = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \mathcal{O}(x_{i+1} - x_i)^2 \quad (9.15)$$

where $\mathcal{O}(z)$ is to be read as "of the order of z ". If x_{i+1} is a root of $f(x) = 0$. The formula 9.15 becomes:

$$0 = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \mathcal{O}(x_{i+1} - x_i)^2 \quad (9.16)$$

Assuming that $x_i \rightarrow x_{i+1}$, we can drop the last term in 9.16 and solve for x_{i+1} .

Then the **Newton-Raphson** formula is:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (9.17)$$

The formula 9.17 approximates $f(x)$ by the straight line that is tangent to the curve at x_i . This x_{i+1} is at the intersection of the x-axis and the tangent line.

The formula including $f'(x_i)$ can be also derived using the following discrete approach (using Δ as a finite neighborhood of x_i):

$$\begin{aligned} f'(x_1) &= \frac{f(x_i + \Delta/2) - f(x_i - \Delta/2)}{(x_i + \Delta/2) - (x_i - \Delta/2)} \\ f'(x_1) &= \frac{f(x_i + \Delta/2) - f(x_i - \Delta/2)}{\Delta} \end{aligned} \quad (9.18)$$

then setting $dx = \Delta$ and $dy = f(x_i + \Delta/2) - f(x_i - \Delta/2)$:

$$f'(x_1) = \frac{dy}{dx} \quad (9.19)$$

we can write:

$$x_{i+1} = x_i - dx \frac{f(x_i)}{dy} \quad (9.20)$$

The **algorithm** for the **Newton-Raphson** method is simple:

It repeatedly applies 9.17, starting with an initial value x_0 , until convergence criterion $|x_{i+1} - x_i| \leq \varepsilon_{tol}$ is reached, the ε_{tol} being the error tolerance. Only the **latest** value of x has to be stored.

The **algorithm** is here:

1. Let x be an estimate of the root of $f(x) = 0$.
2. Do until $|\Delta x| \leq \varepsilon_{tol}$:
 - (a) Compute $\Delta x = -f(x)/f'(x)$
 - (b) Let $x \leftarrow x + \Delta x$

The truncation error E in the **Newton-Raphson** formula can be shown to behave as

$$E_{i+1} = -\frac{f''(x)}{2f'(x)}E_i^2 \quad (9.21)$$

where x is the **root**. This indicates that the method converges *quadratically* (the error is the square of the error in the previous step). Consequently the number of significant figures is roughly doubled in every iteration.

Althou the **Newton-Raphson** method converges fast near the root, its global convergence characteristics are poor. The reason is that the tangent line is not always an acceptable approximation of the function. However, the method can be made nearly fail-safe by **combining** it with **bisection**.

The following *safe version* of the **Newton-Raphson** method assumes that the root to be computed is initially bracketed in $< x_1, x_2 >$. The midpoint of the bracket is used as the initial guess of the root. The brackets are updated after each iteration. If a **Newton-Raphson** iteration does not stay within the brackets, it is disregarded and replaced with bisection. Because the **Newton-Raphson** method uses the function $f(x)$ as well as its derivative, function routines for both (denoted as f and df) must be provided.

Python implementation:

```

1 import math
2
3 def newtonRaphson(f: callable, df: callable, x: float,
4                 tol: float = 1.E-09, maxiter: int = 1000) -> float:
5     for i in range(maxiter):
6         try:
7             dx = - f(x) / df(x)
8             except ZeroDivisionError as e:
9                 raise ValueError(f"Function derivation f'(x = {x:f}) = 0")
10        x += dx
11        if abs(dx) <= tol * abs(x):
12            return x
13
```

```

14     else:
15         raise ValueError(f"Maximum iteration number reached ({maxiter:d}).")
16
17 def newtonRaphsonImproved(f: callable, df: callable, x1: float, x2: float,
18                           tol: float = 1.E-09, maxiter: int = 1000) -> float:
19     f1 = f(x1)
20     if abs(f1) <= tol:
21         return x1
22
23     f2 = f(x2)
24     if abs(f2) <= tol:
25         return x2
26
27     if math.sign(f1) == math.sign(f2):
28         raise ValueError(f"Root is not bracketed by ({x1:f}, {x2:f}).")
29
30     x3 = (x1 + x2) / 2
31     for i in range(maxiter):
32         f3 = f(x3)
33         if abs(f3) <= tol:
34             return x3
35
36         # tighten the brackets on the root
37         if math.sign(f1) != math.sign(f3):
38             x2 = x3
39         else:
40             x1 = x3
41
42         # try a Newton-Raphson step
43         df3 = df(x3)
44
45         # if division by zero, push x3 out of bounds
46         try:
47             dx = -f3 / df3
48         except ZeroDivisionError as e:
49             dx = x2 - x1
50
51         x3 += dx
52
53         # if the result is out of brackets, use bisection
54         if (x2 - x3) * (x3 - x1) < 0.:

```


9.2 Minimizing function of Multiple variables

Downhill Simplex Method

Downhill Simplex method, also known as **Nelder-Mead** method is best used when optimizing (finding of **minimum**) a function of *multiple* variables, especially when derivatives (the **Jacobian**) are hard or impossible to obtain.

The method is **robust** but **slow**. Special consideration must be taken when preparing the function and when supplying constraints.

The **target function** to optimise usually takes the following form:

$$S(\mathbf{x}) = F(\mathbf{x}) + \sum_{i=1}^k \lambda_i (T_i - C_i(\mathbf{x}))^2 \quad (9.22)$$

where:

$S(\mathbf{x})$ is the optimised function

$\mathbf{x}$ is a vector of variables $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$

$F(\mathbf{x})$ is the function to optimise

T_i is the i-th constraint target value

$C_i(\mathbf{x})$ is the i-th constraint function, and

λ_i is the i-th constraint scaling weight factor, usually $\lambda \gg 1.0$.

The i-th constraint is $\lambda_i (T_i - C_i(\mathbf{x}))^2$ which, using the power of 2, is made to have higher weight the more the actual $C_i(\mathbf{x})$ value differs from the constraint target T_i to both sides of the spectra. The λ_i parameter is used to **scale** the not yet met convergence criterion even further. It is especially effective for values close to 0.

A **maximization** of a function $F(\mathbf{x})$ is actually equal to **minimization** of $-F(\mathbf{x})$.

The idea behind the **downhill simplex** method is to employ a moving **simplex** in the design space to surround the optimal point and then shrink the simplex

until its dimensions reach the specified error of tolerance. In n -dimensional space a **simplex** is a figure of $n + 1$ vertices connected by straight lines and bounded by polygonal faces. If $n = 1$, a **simplex** is a line. If $n = 2$, a **simplex** is a triangle, **tetrahedron** if $n = 3$.

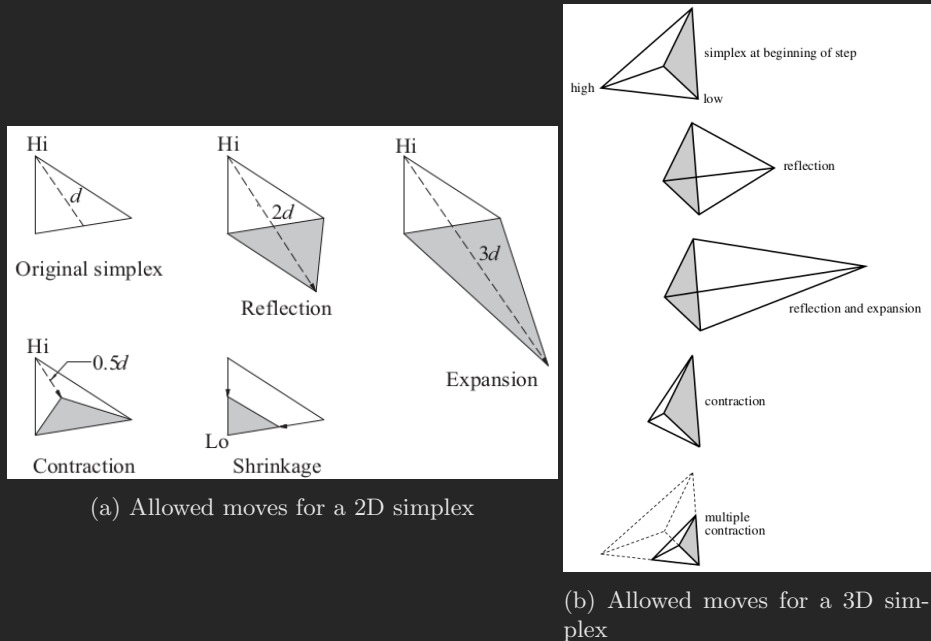


Figure 9.1: Allowed simplex moves for $n = 2$ and $n = 3$

The allowed moves of a **two-dimensional simplex** ($n = 2$) and **three-dimensional simplex** ($n = 3$) are illustrated on pictures 9.1.

By applying these moves in a suitable sequence, the **simplex** can **always** hunt down the minimum point, enclose it, and then shrink around it. The direction of a move is determined by the values of $S(\mathbf{x})$ (the function to be minimised, including constraints) at the **vertices**. The vertex with the **highest** value of S is labeled Hi , and Lo denotes the vertex with the **lowest** value.

The magnitude of a move is controlled by the distance d measured from the Hi vertex to the **centroid** of the opposing vertices.

The **centroid** is simply an arithmetic average of the other vertices.

$$\begin{aligned}
 x_{i_{Hi}} &= H_i \\
 x_c &= \sum_{i \neq i_{Hi}}^n x_i / (n - 1) \\
 d &= x_c - x_{i_{Hi}}
 \end{aligned} \tag{9.23}$$

The outline of the algorithm is as follows:

- Choose a starting simplex (using side parameter)
- Cycle until $d \leq \epsilon_{tol}$
 - find **vertices** v_{Hi} , v_{Lo} with the highest and lowest value of $S(\mathbf{x})$, respectively
 - compute $d = |v_{Hi} - \sum (v \neq v_{Hi}) / (n - 1)|$
 - Try **reflection** $v_n = v_{Hi} - 2d$
 - if $S(v_n) \leq S(v_{Lo})$: accept **reflection** $v_{Hi} = v_n$
 - Try **expansion**: $v_n = v_n - d$
 - if $S(v_n) \leq S(v_{Lo})$: accept **expansion** $v_{Hi} = v_n$
 - else:
 - if $S(v_n) \leq S(v_{Hi})$:
 - Try **contraction**: $v_n = v_{Hi} - d/2$
 - if $S(v_n) \leq S(v_{Hi})$: accept **contraction** $v_{Hi} = v_n$
 - else: use **shrinkage**: $v_i = (v_i + v_{Hi}) / 2 \quad \forall \quad v_i \neq v_{Hi}$

The **downhill simplex method** is much slower than other methods in most cases, but makes up for it in robustness. It often works for problems where other methods hang up.

The starting simplex for a three parameter optimization (with δ side parameter):

$$\mathbf{x}_0 = \{x_1, x_2, x_3\}$$

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 \\ x_1 + \delta & x_2 & x_3 \\ x_1 & x_2 + \delta & x_3 \\ x_1 & x_2 & x_3 + \delta \end{bmatrix} \quad (9.24)$$

Python implementation:

```

1 import numpy as np
2 import math
3
4 def downhill(F: callable, x0: np.ndarray,
5             side: float = 0.1, tol: float = 1.0e-6, maxiter: int = 500,
6             ro: float = 1.0, epsilon: float = 2.,
7             gamma: float = 0.5, sigma: float = 0.5) -> np.ndarray:
8     '''Downhill simplex method for minimization (Nelder-Mead method)
9
10     Args:
11         F (callable):    reference to scalar function that should be minimized
12         x0 (np.ndarray): starting vector x = initial estimate
13         side (float):    side length of the starting simplex (default = 0.1)
14         tol (float):     convergence toelarence (default = 1.0e-06)
15         maxiter (int):   maximum number of iterations (default = 500)
16         ro (float):      reflection coefficient      (default ro      = 1.0)
17         epsilon (float): expansion coefficient      (default epsilon = 2.0)
18         gamma (float):   contraction coefficient    (default gamma   = 0.5)
19         sigma (float):   shrink coefficient         (default sigma   = 0.5)
20
21     Returns:
22         (np.ndarray): the optimized values
23
24     '''
25     n = len(x0) # Number of variables
26

```

```

27     # Generate starting simplex
28     x = np.repeat(x0.reshape(1,-1), repeats=n+1, axis=0)
29     # move each parameter by 'side'
30     x[1:,:] += np.eye(n, n) * side
31
32     # Compute values of F at the vertices of the simplex
33     f = np.apply_along_axis(F, axis=1, arr=x)
34
35     # Main loop
36     for k in range(maxiter):
37         # Sort vertices from lowest to highest (indexes: lowest = 0, highest = n)
38         idx = np.argsort(f)
39         f, x = f[idx], x[idx]
40
41         # The centroid of all vertices except the highest  $x_{cog} = \sum x(i < n) / n$ 
42         # Compute the move vector d (from highest vertex to centroid of rest)
43         #  $d = x_{cog} - x_n$ 
44         d = np.sum(x[:-1], axis=0) / n - x[-1]
45
46         # Check for convergence
47         if math.sqrt(np.dot(d, d) / n) < tol:
48             print(f"[+] Convergence reached at iteration: {k:d}")
49             return x[0]
50
51         # Try reflection:  $x_r = x_{cog} + ro * (x_{cog} - x_n)$ 
52         xNew = x[-1] + (1.0 + ro) * d
53         fNew = F(xNew)
54         if fNew <= f[0]: # Accept reflection
55             x[-1], f[-1] = xNew, fNew
56
57         # Try expanding the reflection:  $x_e = x_{cog} + epsilon * (x_r - x_{cog})$ 
58         xNew = x[-1] + (epsilon - ro) * d
59         fNew = F(xNew)
60
61         if fNew <= f[0]: # Accept expansion
62             x[-1], f[-1] = xNew, fNew
63
64         # Try reflection again
65         elif fNew <= f[-1]: # Accept reflection
66             x[-1], f[-1] = xNew, fNew
67

```

```
68         else:
69             # Try contraction:  $w_c = w_{cog} + \gamma * (w_r - w_{cog})$ 
70             xNew = x[-1] + gamma * d
71             fNew = F(xNew)
72
73             if fNew <= f[-1]: # Accept contraction
74                 x[-1], f[-1] = xNew, fNew
75
76             else:
77                 # Use shrinkage:  $w_i = w_i + \sigma * (w_i - w_0)$ 
78                 x[1:] += sigma * (x[1:] - x[0])
79                 f[1:] = np.apply_along_axis(F, axis=1, arr=x[1:])
80
81     print(f"[-] Iteration limit exceeded: {maxiter:d}")
82     return x[0]
83
```

9.3 second order ODE's

Most of FEM problems are **second order Ordinary Differential Equations**, e.g:

$$\begin{aligned} m \frac{u''(t)}{d^2t} + c \frac{u'(t)}{dt} + ku(t) &= 0 \\ m\ddot{u} + c\dot{u} + ku &= 0 \\ ma + cv + ku &= 0 \end{aligned} \quad (9.25)$$

while the iterative methods are for **first order ODE's** only. A **substitution** technique is therefore used to transform the **second order ODE's** to a set of **first order ODE's** and these are then solved sequentially.

First we begin with a **second order ODE** in the form of:

$$a \frac{y''(x)}{d^2x} + b \frac{y'(x)}{dx} + cy = d, \quad \text{with } y(0) = y_0 \quad \text{and} \quad \frac{y'(0)}{dx} = y_1 \quad (9.26)$$

By stating:

$$\frac{dy}{dx} = z \quad (9.27)$$

and substituting back:

$$a \frac{dz}{dx} + bz + cy = d \quad (9.28)$$

now we can write the following set of equations:

$$\begin{aligned} z &= \frac{dy}{dx} \\ \frac{dz}{dx} &= \frac{d - cy - bz}{a} \end{aligned} \quad \begin{cases} y(0) = y_0 \\ z(0) = y_1 \end{cases} \quad (9.29)$$

9.4 Explicit

9.4.1 Euler's method

The **Euler's Method** is an iterative method to solve ODEs. This method's huge plus is it is fast. The minus is it gradually diverges from the ideal solution.

The **Euler's Method** uses the following formula:

$$y(t+h) = y(t) + hf(x, y) \quad (9.30)$$

to construct the tangent at point x and obtain the value of $y(x+h)$, whose slope is:

$$f(x, y) \quad \text{or} \quad \frac{dy}{dx} \quad (9.31)$$

In Euler's method, you can approximate the curve of the solution by the tangent in each interval (that is, by a sequence of short line segments), at steps of h .

In general, if you use small step size, the accuracy of the approximation increases.

General Formula

$$y_{i+1} = y_i + hf(x_i, y_i) \quad (9.32)$$

where:

y_{i+1} is the next estimated solution value,

y_i is the current value,

h is the interval between steps and

$f(x_i, y_i)$ is the value of the derivative at the current (x_i, y_i) point.

Pseudocode:

- define: $f(x, y)$

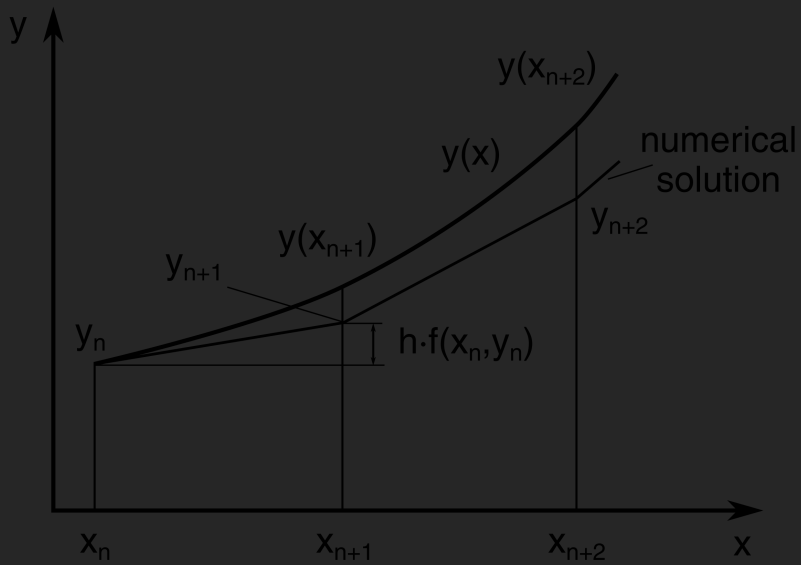


Figure 9.2: Euler's method schematic

- input: x_0, y_0
- input: h, n
- for j from 0 to $(n - 1)$ do
 - $y_{j+1} = y_j + hf(x_j, y_j)$
 - $x_{j+1} = x_j + h$
 - Print x_{j+1} and y_{j+1}
- End.

Note:

If thinking about a problem in time domain:

- define: $f(t, x)$
- input: t_0, x_0
- input: dt, n
- for j from 0 to $(n - 1)$ do
 - $x_{j+1} = x_j + dt f(t_j, x_j)$
 - $t_{j+1} = t_j + dt$
 - Print t_{j+1} and x_{j+1}
- end.

Python Code for vibration problem:

```

1  #!/usr/bin/python3
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5
6  def iterate(h, y0, func, rhs):
7      num_of_odes = y0.shape[0]
8      y1 = np.zeros(num_of_odes, dtype = float)
9      for i in range(num_of_odes-1):
10         y1[i] = y0[i] + y0[i+1] * h
11     y1[-1] = func(y1, rhs)
12     return y1
13
14
15 def euler(t, y, func, rhs):
16     N = t.shape[0]
17     for j in range(N-1):
18         dt = t[j+1] - t[j]
19         y[j+1,:] = iterate(dt, y[j,:], func, rhs[j])
20         # print(y[j+1,:])
21     return y
22
23
24 def euler_vibration():
25     m = 0.55 # tonnes
26     c = 3. # Ns/mm
27     k = 1000. # N/mm
28
29     freq = np.sqrt(k / m) / (2 * np.pi)
30
31     # equation to solve
32     # m * a + c * v + k * u = f
33     # a = (f - c * v - k * u) / m
34     acceleration = lambda u, f: (f - c * u[1] - k * u[0]) / m
35
36     T = 2.0 # seconds
37     dt = 0.0001 # timestep s

```

```

38     u_0 = 0.      # mm of initial displacement
39     v_0 = 0.      # mm/s of initial velocity
40
41     # times at which to solve
42     t = np.linspace(0, T, int(T/dt) + 1)
43
44     # force vector
45     F = 5000. # N of max impulse
46     t0 = 0.1 # impulse start time
47     t1 = 0.2 # impuls end time
48     f = np.zeros(t.shape[0], dtype=float)
49     # create a half sine impulse of force
50     for i in range(t.shape[0]):
51         if t[i] >= t0 and t[i] <= t1:
52             f[i] = F * np.sin((t[i] - t0) / (t1 - t0) * np.pi)
53         else:
54             f[i] = 0.
55
56     uva = np.zeros((t.shape[0],3), dtype=float)
57     uva[0,0] = u_0
58     uva[0,1] = v_0
59
60     uva = euler(t, uva, acceleration, f)
61
62     fig, axf = plt.subplots()
63     fig.subplots_adjust(right=0.60)
64
65     p1, = axf.plot(t, f, label='force', color='violet')
66     axu = axf.twinx()
67     p2, = axu.plot(t, uva[:,0], label='displacement', color='red')
68     axv = axf.twinx()
69     axv.spines.right.set_position(('axes', 1.20))
70     p3, = axv.plot(t, uva[:,1], label='velocity', color='blue')
71     axa = axf.twinx()
72     axa.spines.right.set_position(('axes', 1.40))
73     p4, = axa.plot(t, uva[:,2], label='acceleration', color='green')
74
75     axf.set_xlabel('Time [s]')
76     axf.set_ylabel('Force [N]')
77     axu.set_ylabel('Displacement [mm]')
78     axv.set_ylabel('Velocity [mm/s]')

```

```

79     axa.set_ylabel('Acceleration [mm/s2]')
80
81     axf.legend(handles=[p1, p2, p3, p4])
82
83     plt.show()
84
85 if __name__ == '__main__':
86     euler_vibration()
87

```

Which results to:

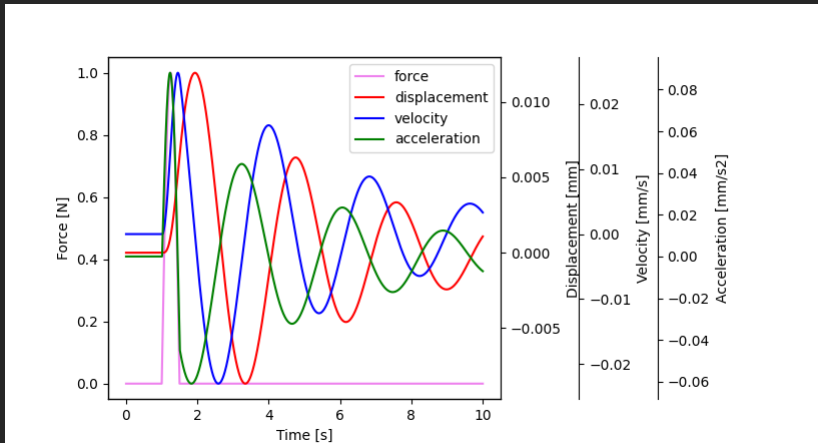


Figure 9.3: Euler's implementation for vibration problem example

The `iterate()` function:

```

1 def iterate(h, y0, func, rhs):
2     num_of_odes = y0.shape[0]
3     y1 = np.zeros(num_of_odes, dtype = float)
4     for i in range(num_of_odes-1):
5         y1[i] = y0[i] + y0[i+1] * h

```

```

6     y1[-1] = func(y1, rhs)
7     return y1
8

```

is written for general case of a set of ODEs. When considering the problem of vibration (set of 2 ODEs):

```

1 def iterate(h, u0, v0, a0, f, m, c, k):
2     u1 = u0 + h * v0
3     v1 = v0 + h * a0
4     # m * a + c * v + k * u = f
5     a1 = (f - c * v1 - k * u1) / m
6     return np.array([u1, v1, a1], dtype=float)

```

which is the iteration of a set of **first order ODEs**:

$$\begin{aligned}
 v &= \dot{u} \\
 \dot{v} &= \frac{f - c * v - k * u}{m}
 \end{aligned}
 \tag{9.33}$$

9.4.2 Runge-Kutta 4th order method

Runge-Kutta 4th order method is another explicit method for solving **first order ODEs**. The basic equation is:

$$\frac{dy}{dx} = f(x, y) \quad , y(0) = y_0 \quad (9.34)$$

The formula for the next value y_{i+1} after a step size equal to h is given by:

$$\begin{aligned} k_1 &= hf(x_i, y_i) \\ k_2 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right) \\ k_3 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right) \\ k_4 &= hf(x_i + h, y_i + k_3) \\ y_{i+1} &= y_i + \frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} + O(h^5) \end{aligned} \quad (9.35)$$

The formula basically computes next value y_{i+1} using current y_i plus **weighted average of four increments**:

- k_1 is the increment based on the slope at the beginning of the interval, using y
- k_2 is the increment based on the slope at the midpoint of the interval, using $y + hk_1/2$
- k_3 is the increment based on the slope at the midpoint, using $y + hk_2/2$
- k_4 is the increment based on the slope at the end of the interval, using $y + hk_3/2$

The method is a fourth order method, meaning that the local truncation error is on the order of $O(h^5)$, while the total accumulated error is of order $O(h^4)$.

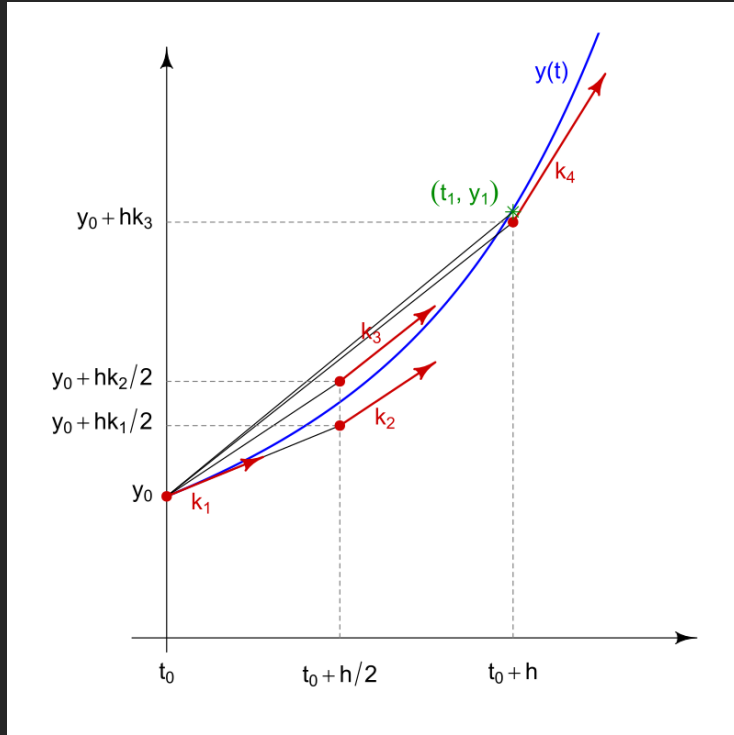


Figure 9.4: 4th Order Runge-Kutta's method schematic

Note:

The formula for the next value y_{i+1} can be also written as:

$$\begin{aligned}
 k_1 &= f(t_i, y_i) \\
 k_2 &= f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2} k_1\right) \\
 k_3 &= f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2} k_2\right) \\
 k_4 &= f(t_i + h, y_i + h k_3) \\
 y_{i+1} &= y_i + h \left(\frac{1}{6} k_1 + \frac{4}{6} k_2 + \frac{2}{6} k_3 + \frac{1}{6} k_4 \right) + O(h^5)
 \end{aligned} \tag{9.36}$$

This notation is more convenient to use when solving sets of ODEs for vibration problem, because it translates to:

$$\begin{aligned}
 u_1 &= u_0 \\
 v_1 &= v_0 \\
 a_1 &= (f - cv_0 - ku_0)/m \\
 u_2 &= u_0 + \frac{h}{2}v_1 \\
 v_2 &= v_0 + \frac{h}{2}a_1 \\
 a_2 &= (f - cv_1 - ku_1)/m \\
 u_3 &= u_0 + \frac{h}{2}v_2 \\
 v_3 &= v_0 + \frac{h}{2}a_2 \\
 a_3 &= (f - cv_2 - ku_2)/m \\
 u_4 &= u_0 + hv_3 \\
 v_4 &= v_0 + ha_3 \\
 a_4 &= (f - cv_3 - ku_3)/m \\
 u_{i+1} &= u_0 + \frac{h}{6}(v_1 + 2v_2 + 2v_3 + v_4) \\
 v_{i+1} &= v_0 + \frac{h}{6}(a_1 + 2a_2 + 2a_3 + a_4) \\
 a_{i+1} &= (f - cv_{i+1} - ku_{i+1})/m
 \end{aligned} \tag{9.37}$$

where each of the **ODEs** are solved sequentially from the values already known.

Python implementation for vibration problem:

```

1  #!/usr/bin/python3
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5
6  def iterate(h, y0, y, func, rhs):
7      num_of_odes = y0.shape[0]
8      y1 = np.zeros(num_of_odes, dtype = float)
9      for i in range(num_of_odes-1):
10         y1[i] = y0[i] + y[i+1] * h
11     y1[-1] = func(y1, rhs)
12     return y1
13
14
15 def runge_kutta_4(t, y, func, rhs):
16     N = t.shape[0]
17     for j in range(N-1):
18         dt = t[j+1] - t[j]
19         y1 = iterate(0., y[j,:], y[j,:], func, rhs[j])
20         y2 = iterate(dt/2, y[j,:], y1, func, 0.5 * (rhs[j+1] + rhs[j]))
21         y3 = iterate(dt/2, y[j,:], y2, func, 0.5 * (rhs[j+1] + rhs[j]))
22         y4 = iterate(dt, y[j,:], y3, func, rhs[j+1])
23
24         # next step solution
25         for i in range(y.shape[1]-1):
26             y[j+1,i] = y[j,i] + dt/6 * (y1[i+1] + 2 * y2[i+1] + 2 * y3[i+1] +
27                                     ↪ y4[i+1])
28         y[j+1,-1] = func(y[j+1,:), rhs[j+1])
29     return y
30
31 def rk4_vibration():
32     m = 10.    # tonnes
33     c = 5.     # Ns/mm
34     k = 50.    # N/mm
35
36     freq = np.sqrt(k / m) / (2 * np.pi)

```



```

37  print(f'f = {freq}')
38
39  # equation to solve
40  # m * a + c * v + k * u = f
41  # a = (f - c * v - k * u) / m
42  acceleration = lambda u, f: (f - c * u[1] - k * u[0]) / m
43
44  T = 10.      # seconds
45  dt = 0.01    # timestep s
46  u_0 = 0.     # mm of initial displacement
47  v_0 = 0.     # mm/s of initial velocity
48
49  # times at which to solve
50  t = np.linspace(0, T, int(T/dt) + 1)
51
52  # force vector
53  F = 1.       # N of max impulse
54  t0 = 1.      # impulse start time
55  t1 = 1.5     # impuls end time
56  f = np.zeros(t.shape[0], dtype=float)
57  # create a half sine impulse of force
58  for i in range(t.shape[0]):
59      if t[i] >= t0 and t[i] <= t1:
60          f[i] = F * np.sin((t[i] - t0) / (t1 - t0) * np.pi)
61      else:
62          f[i] = 0.
63
64  uva = np.zeros((t.shape[0],3), dtype=float)
65  uva[0,0] = u_0
66  uva[0,1] = v_0
67
68  uva = runge_kutta_4(t, uva, acceleration, f)
69
70  fig, axf = plt.subplots()
71  fig.subplots_adjust(right=0.60)
72
73  p1, = axf.plot(t, f, label='force', color='violet')
74  axu = axf.twinx()
75  p2, = axu.plot(t, uva[:,0], label='displacement', color='red')
76  axv = axf.twinx()
77  axv.spines.right.set_position(('axes', 1.20))

```

```

78     p3, = axv.plot(t, uva[:,1], label='velocity', color='blue')
79     axa = axf.twinx()
80     axa.spines.right.set_position(('axes', 1.40))
81     p4, = axa.plot(t, uva[:,2], label='acceleration', color='green')
82
83     axf.set_xlabel('Time [s]')
84     axf.set_ylabel('Force [N]')
85     axu.set_ylabel('Displacement [mm]')
86     axv.set_ylabel('Velocity [mm/s]')
87     axa.set_ylabel('Acceleration [mm/s2]')
88
89     axf.legend(handles=[p1, p2, p3, p4])
90
91     plt.show()
92
93 if __name__ == '__main__':
94     rk4_vibration()
95

```

Which results to:

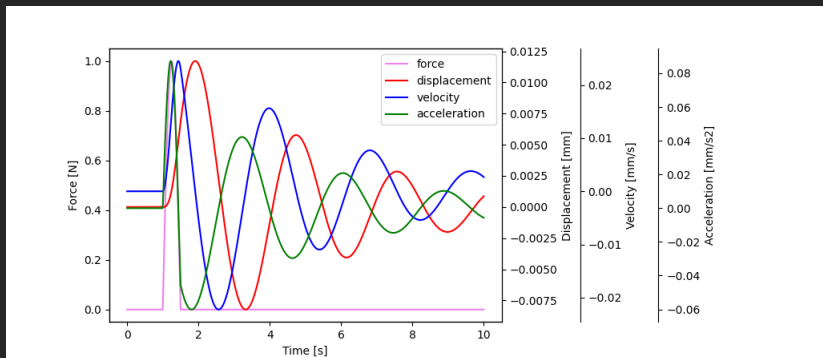


Figure 9.5: 4th order Runge-Kutta's implementation for vibration problem example

9.5 Implicit

The basic approach is an incremental step-by-step solution with an assumption that the solution for a discrete time t is known and that the solution for the discrete time $t + \Delta t$ is required, where Δt is a suitably chosen time increment. Hence, considering time $t + \Delta t$:

$$\mathbf{F}^{t+\Delta t} - \mathbf{N}^{t+\Delta t} = \mathbf{0} \quad (9.38)$$

Where $\mathbf{F}$ is the vector of **external forces** (loads) and $\mathbf{N}$ is the vector of **inner forces** (stresses).

Assume that $\mathbf{F}^{t+\Delta t}$ is independent of the deformations. Since the solution is known at time t , we can write:

$$\mathbf{N}^{t+\Delta t} = \mathbf{N}^t + \Delta \mathbf{N} \quad (9.39)$$

where $\Delta \mathbf{N}$ is the increment in nodal point forces corresponding to the increment in element displacements and stresses from time t to time $t + \Delta t$. This vector can be approximated using a **tangent stiffness matrix** $\mathbf{K}^t$ which corresponds to the geometric and material conditions at time t ,

$$\Delta \mathbf{N} \doteq \mathbf{K}^t \Delta \mathbf{u} \quad (9.40)$$

where $\Delta \mathbf{u}$ is a vector of incremental nodal point displacements and

$$\mathbf{K}^t = \frac{\partial \mathbf{N}^t}{\partial \mathbf{u}^t} \quad (9.41)$$

Hence, the **tangent stiffness matrix** corresponds to the derivative of the internal element nodal point forces $\mathbf{N}^t$ with respect to the nodal point displacements $\mathbf{u}^t$.

Substituting (9.40) and (9.41) into (9.38), we obtain

$$\mathbf{K}^t \Delta \mathbf{u} = \mathbf{F}^{t+\Delta t} - \mathbf{N}^t \quad (9.42)$$

and solving for $\mathbf{u}^{\Delta t}$, we can calculate an approximation to the displacements at time $t + \Delta t$,

$$\mathbf{u}^{t+\Delta t} \doteq \mathbf{u}^t + \Delta \mathbf{u} \quad (9.43)$$

The exact displacements at time $t + \Delta t$ are those that correspond to the applied loads $\mathbf{F}^{t+\Delta t}$. We calculate in (9.43) only an approximation to these displacements because (9.40) was used.

Having evaluated an approximation to the displacements corresponding to time $t + \Delta t$, we could now solve for an approximation to the stresses and corresponding nodal point forces at time $t + \Delta t$, and then proceed to the next time increment calculations. However, because of the assumption (9.40), such a solution may be subject to very significant errors and, depending on the time or load step sizes used, may indeed be unstable. In practice, it is therefore necessary to iterate until the solution of (9.38) is obtained to sufficient accuracy.

9.5.1 Newton-Raphson

Having calculated an *increment* in nodal point displacements, which defines a *new total* displacement vector, we can repeat the incremental solution using the currently known total displacements at time t .

The equations used in **Newton-Raphson** iteration are, *for* $i = 1, 2, 3, \dots$,

$$\begin{aligned} \mathbf{K}_{i-1}^{t+\Delta t} \Delta \mathbf{u}_i &= \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t} \\ \mathbf{u}_i^{t+\Delta t} &= \mathbf{u}_i^{t+\Delta t} + \mathbf{u}_{i-1}^{t+\Delta t} + \Delta \mathbf{u}_i \end{aligned} \quad (9.44)$$

with the initial conditions:

$$\begin{aligned}\mathbf{u}_0^{t+\Delta t} &= \mathbf{u}^t \\ \mathbf{K}_0^{t+\Delta t} &= \mathbf{K}^t \\ \mathbf{N}_0^{t+\Delta t} &= \mathbf{N}^t\end{aligned}\tag{9.45}$$

Note that in the first iteration, the relations in (9.44) reduce to the equations (9.42) and (9.43). Then, in subsequent iterations, the latest estimates for the nodal point displacements are used to evaluate the corresponding element stresses and nodal point forces $\mathbf{N}_{i-1}^{t+\Delta t}$ and **tangent stiffness matrix** $\mathbf{K}_{i-1}^{t+\Delta t}$.

The **tangent stiffness matrix** is achieved in the same way as the **global stiffness matrix**, but of the **deformed** geometry, incorporating the model nonlinearities (if applicable), such as:

- **geometric** non-linearities as large rotations, function-dependent springs, etc.
- **configuration** non-linearities as tension only or compression only elements, contact definitions (usually contacts should be realised on the **RHS** of the $\mathbf{Ku} = \mathbf{f}$ equation)
- **material** non-linearities as plasticity etc.

The out-of-balance load vector $\mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t}$ corresponds to a load vector that is not yet balanced by element stress, and hence an increment in the nodal point displacement is required. This updating of the nodal point displacement in the iteration is continued until out-of-balance loads and incremental displacements are small enough.

An important point is that the calculation of $\mathbf{N}_{i-1}^{t+\Delta t}$ from $\mathbf{u}_{i-1}^{t+\Delta t}$ is **crucial**. Any errors in this calculation will, in general, result in an incorrect response prediction.

The correct evaluation of the **tangent stiffness matrix** $\mathbf{K}_{i-1}^{t+\Delta t}$ is also important. The use of proper **tangent stiffness matrix** may be necessary for

convergence and, in general, will result in fewer iterations until convergence is reached.

However, because the expense involved in evaluating and factoring a new **tangent stiffness matrix**, in practice, it can be more efficient, depending on the nonlinearities present in the analysis, to evaluate a new **tangent stiffness matrix** only at certain times. Specifically, in the **modified Newton-Raphson method** a new **tangent stiffness matrix** is established only at the beginning of each load step, and in **quasi-Newton** methods **secant stiffness matrices** are used instead of the tangent stiffness matrix.

The use of the iterative solution requires appropriate convergence criteria. If inappropriate criteria are used, the iteration may be terminated before the necessary solution accuracy is reached or be continued after the required accuracy has been reached.

Full Newton-Raphson Procedure derivation

The finite element equilibrium requirements amount to finding the solution of the equations:

$$f(\mathbf{u}^*) = 0 \quad (9.46)$$

Where:

$$f(\mathbf{u}^*) = \mathbf{F}^{t+\Delta t}(\mathbf{u}^*) - \mathbf{N}^{t+\Delta t}(\mathbf{u}^*) \quad (9.47)$$

We denote here and in the following the complete array of the solution as $\mathbf{u}^*$ but realize that this vector may also contain variables other than displacements, for example, pressure variables and rotations.

Assume that in the iterative solution we have evaluated $\mathbf{u}_{i-1}^{t+\Delta t}$; then a Taylor series expansion gives:

$$\begin{aligned}
f(\mathbf{u}^*) &= f(\mathbf{u}_{i-1}^{t+\Delta t}) + \left[\frac{\partial N}{\partial \mathbf{u}} \right] \bigg|_{\mathbf{u}_{i-1}^{t+\Delta t}} (\mathbf{u}^* - \mathbf{u}_{i-1}^{t+\Delta t}) \\
&\quad + \text{higher order terms}
\end{aligned} \tag{9.48}$$

Substituting from (9.47) into (9.48) and using (9.46), we obtain:

$$\begin{aligned}
&\left[\frac{\partial \mathbf{N}}{\partial \mathbf{u}} \right] \bigg|_{\mathbf{u}_{i-1}^{t+\Delta t}} (\mathbf{u}^* - \mathbf{u}_{i-1}^{t+\Delta t}) + \text{higher order terms} \\
&= \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t}
\end{aligned} \tag{9.49}$$

where we assumed that the externally applied loads are deformation independent.

Neglecting the higher-order terms in (9.49), we can calculate an increment in the displacements,

$$\mathbf{K}_{i-1}^{t+\Delta t} \Delta \mathbf{u}_i = \mathbf{F}^{t+\Delta t} - \mathbf{N}_{i-1}^{t+\Delta t} \tag{9.50}$$

where $\mathbf{K}_{i-1}^{t+\Delta t}$ is the current **tangent stiffness matrix**

$$\mathbf{K}_{i-1}^{t+\Delta t} = \left[\frac{\partial \mathbf{N}}{\partial \mathbf{u}} \right] \bigg|_{\mathbf{u}_{i-1}^{t+\Delta t}} \tag{9.51}$$

and the improved displacement solution is:

$$\mathbf{u}_i^{t+\Delta t} = \mathbf{u}_{i-1}^{t+\Delta t} + \Delta \mathbf{u}_i \tag{9.52}$$

The relations in (9.50) and (9.52) constitute the Newton-Raphson solution of (9.38). Since an incremental analysis is performed with time (or load) steps of size Δt , the initial conditions in this iteration are $\mathbf{K}_0^{t+\Delta t} = \mathbf{K}^t$, $\mathbf{N}_0^{t+\Delta t} = \mathbf{N}^t$,

and $\mathbf{u}_0^{t+\Delta t} = \mathbf{u}^t$. The iteration is continued **until appropriate convergence criteria are satisfied**.

A characteristic of this iteration is that a new tangent stiffness matrix is calculated in **each** iteration, which is why this method is also referred to as the **full Newton-Raphson method**.

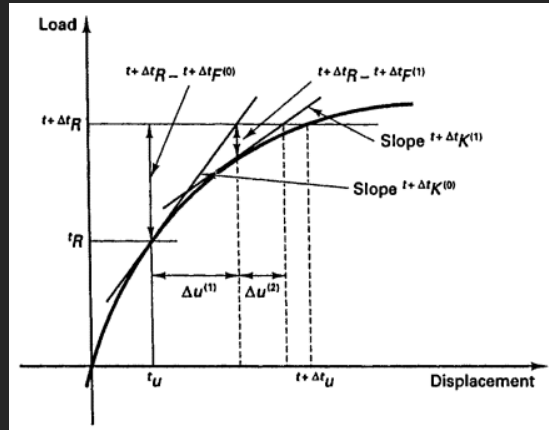


Figure 9.6: Newton-Raphson iteration in solution of SDOF system. $\mathbf{R}$ is load increment, $\mathbf{F}$ are internal forces, $\mathbf{u}$ are displacements.

Chapter 10

Model reduction

10.1 Guyan reduction

Guyan reduction, also known as **static condensation**, is a *dimensionality reduction* method which reduces the number of **DOFs** by ignoring the internal terms of the equilibrium equations and expressing the unloaded **DOFs** in terms of the loaded **DOFs**.

The static equilibrium equation:

$$\begin{bmatrix} \mathbf{K}_{mm} & \mathbf{K}_{ms} \\ \mathbf{K}_{sm} & \mathbf{K}_{ss} \end{bmatrix} \begin{bmatrix} \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{f}_m \\ \mathbf{f}_s \end{bmatrix} \quad (10.1)$$
$$\begin{aligned} \mathbf{K}_{mm} \mathbf{u}_m + \mathbf{K}_{ms} \mathbf{u}_s &= \mathbf{f}_m \\ \mathbf{K}_{sm} \mathbf{u}_m + \mathbf{K}_{ss} \mathbf{u}_s &= \mathbf{f}_s \end{aligned}$$

Where m denotes the **master** DOFs and s the **slave** DOFs. Stating that s section of the problem DOFs is unloaded, we can write that $\mathbf{f}_s = \mathbf{0}$. Then the slave DOFs are expressed by the following equation:

$$\mathbf{K}_{sm} \mathbf{u}_m + \mathbf{K}_{ss} \mathbf{u}_s = \mathbf{0} \quad (10.2)$$

Solving the above equation in terms of the independent (master) DOFs leads to the following dependency relations:

$$\mathbf{u}_s = -\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm}\mathbf{u}_m \quad (10.3)$$

Substituting the dependency relations to the upper (master) partition of the static equilibrium problem condenses away the slave DOFs, leading to the following reduced system of linear equations:

$$(\mathbf{K}_{mm} - \mathbf{K}_{ms}\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm})\mathbf{u}_m = \mathbf{f}_m \quad (10.4)$$

The above system of linear equations is equivalent to the original problem, but expressed in terms of **master** DOFs alone. Thus, the **Guyan** reduction method results in a reduced system by condensing away the **slave** DOFs.

The **Guyan reduction** can be also expressed as a *change of basis* which produces a low-dimensional representation of the original space, represented by the **master** DOFs. The linear transformation that maps the reduced space onto the full space is expressed as:

$$\begin{bmatrix} \mathbf{u}_m \\ \mathbf{u}_s \end{bmatrix} = \begin{bmatrix} \mathbf{I} \\ -\mathbf{K}_{ss}^{-1}\mathbf{K}_{sm} \end{bmatrix} \mathbf{u}_m = \mathbf{T}_G \mathbf{u}_m \quad (10.5)$$

where $\mathbf{T}_G$ represents the **Guyan** reduction *transformation matrix*. Thus, the reduced problem is represented as:

$$\mathbf{K}_G \mathbf{u}_m = \mathbf{f}_m \quad (10.6)$$

In the above equation, $\mathbf{K}_G$ represents the reduced system of linear equations that's obtained by applying the **Guyan reduction** transformation on the full system, which is expressed as:

$$\mathbf{K}_G = \mathbf{T}_G^T \mathbf{K} \mathbf{T}_G \quad (10.7)$$

where $\mathbf{T}_G$ has the dimension of $n_{DOF} \times n_m$.

10.2 Craig-Bampton reduction

The **Craig-Bampton reduction**, also known as the **fixed-interface component mode synthesis**, extends the **Guyan reduction** by supplementing the static deformation shapes with a truncated set of dynamic deformation shapes. It therefore retains the exact static relation between the interface and internal DOFs, while approximating the internal dynamic deformation by a modal base.

The dynamic equilibrium equation is partitioned into the **boundary** DOFs b and the **internal** DOFs i :

$$\begin{bmatrix} \mathbf{M}_{bb} & \mathbf{M}_{bi} \\ \mathbf{M}_{ib} & \mathbf{M}_{ii} \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{u}}_b \\ \ddot{\mathbf{u}}_i \end{bmatrix} + \begin{bmatrix} \mathbf{K}_{bb} & \mathbf{K}_{bi} \\ \mathbf{K}_{ib} & \mathbf{K}_{ii} \end{bmatrix} \begin{bmatrix} \mathbf{u}_b \\ \mathbf{u}_i \end{bmatrix} = \begin{bmatrix} \mathbf{f}_b \\ \mathbf{f}_i \end{bmatrix} \quad (10.8)$$

where the boundary DOFs are retained in physical coordinates and the internal DOFs are reduced. In the terminology used for the **Guyan reduction**, the boundary DOFs correspond to the master DOFs and the internal DOFs to the slave DOFs.

10.2.1 Constraint modes

The **constraint modes**, also called the **static constraint modes**, describe the static deformation of the internal DOFs caused by a unit displacement of each individual boundary DOF. The inertia terms and the externally applied forces on the internal DOFs are set to zero:

$$\mathbf{K}_{ib}\mathbf{u}_b + \mathbf{K}_{ii}\mathbf{u}_i = \mathbf{0} \quad (10.9)$$

The internal displacement is then expressed as:

$$\mathbf{u}_i = -\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib}\mathbf{u}_b = \Psi_c\mathbf{u}_b \quad (10.10)$$

where:

$$\Psi_c = -\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib} \quad (10.11)$$

is the **constraint mode matrix**. Each column of Ψ_c is the internal static displacement field corresponding to a unit displacement of one boundary DOF, while all remaining boundary DOFs are held fixed.

The inverse $\mathbf{K}_{ii}^{-1}$ is only a mathematical notation. Numerically, the constraint modes are obtained by solving the multiple right-hand-side problem:

$$\mathbf{K}_{ii}\Psi_c = -\mathbf{K}_{ib} \quad (10.12)$$

The matrix $\mathbf{K}_{ii}$ is factorised only once and the factorisation is reused for all boundary DOFs.

10.2.2 Fixed-interface normal modes

The dynamic deformation that cannot be represented by the constraint modes is approximated using the **fixed-interface normal modes**. These modes are calculated by fixing all boundary DOFs:

$$\mathbf{u}_b = \mathbf{0} \quad (10.13)$$

and solving the internal generalised eigenvalue problem:

$$(\mathbf{K}_{ii} - \omega_j^2 \mathbf{M}_{ii}) \boldsymbol{\varphi}_j = \mathbf{0} \quad (10.14)$$

The retained fixed-interface modes are assembled column-wise into the matrix:

$$\Phi_i = [\boldsymbol{\varphi}_1 \quad \boldsymbol{\varphi}_2 \quad \dots \quad \boldsymbol{\varphi}_{n_\varphi}] \quad (10.15)$$

where n_φ is the number of retained fixed-interface modes. When the modes are mass-normalised:

$$\begin{aligned}\Phi_i^T \mathbf{M}_{ii} \Phi_i &= \mathbf{I} \\ \Phi_i^T \mathbf{K}_{ii} \Phi_i &= \Omega_i^2\end{aligned}\tag{10.16}$$

where:

$$\Omega_i^2 = \begin{bmatrix} \omega_1^2 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \omega_{n_\varphi}^2 \end{bmatrix}\tag{10.17}$$

is the diagonal matrix of the retained fixed-interface eigenvalues.

10.2.3 Craig-Bampton transformation

The internal displacement is represented as a superposition of the static constraint modes and the fixed-interface normal modes:

$$\mathbf{u}_i = \Psi_c \mathbf{u}_b + \Phi_i \boldsymbol{\eta}\tag{10.18}$$

where $\boldsymbol{\eta}$ is the vector of the fixed-interface modal coordinates. The full physical displacement vector is therefore expressed as:

$$\begin{aligned}\begin{bmatrix} \mathbf{u}_b \\ \mathbf{u}_i \end{bmatrix} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \Psi_c & \Phi_i \end{bmatrix} \begin{bmatrix} \mathbf{u}_b \\ \boldsymbol{\eta} \end{bmatrix} \\ \mathbf{u} &= \mathbf{T}_{CB} \mathbf{q}_{CB}\end{aligned}\tag{10.19}$$

where:

$$\begin{aligned}\mathbf{T}_{CB} &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \Psi_c & \Phi_i \end{bmatrix} \\ \mathbf{q}_{CB} &= \begin{bmatrix} \mathbf{u}_b \\ \boldsymbol{\eta} \end{bmatrix}\end{aligned}\tag{10.20}$$

The Craig-Bampton coordinates therefore consist of:

- the physical displacements of all retained boundary DOFs,
- the modal amplitudes of the retained fixed-interface normal modes.

Substituting $\mathbf{u} = \mathbf{T}_{CB}\mathbf{q}_{CB}$ to the full equation of motion and premultiplying it with $\mathbf{T}_{CB}^T$ gives the reduced equation:

$$\mathbf{M}_{CB}\ddot{\mathbf{q}}_{CB} + \mathbf{C}_{CB}\dot{\mathbf{q}}_{CB} + \mathbf{K}_{CB}\mathbf{q}_{CB} = \mathbf{f}_{CB} \quad (10.21)$$

where:

$$\begin{aligned} \mathbf{M}_{CB} &= \mathbf{T}_{CB}^T \mathbf{M} \mathbf{T}_{CB} \\ \mathbf{C}_{CB} &= \mathbf{T}_{CB}^T \mathbf{C} \mathbf{T}_{CB} \\ \mathbf{K}_{CB} &= \mathbf{T}_{CB}^T \mathbf{K} \mathbf{T}_{CB} \\ \mathbf{f}_{CB} &= \mathbf{T}_{CB}^T \mathbf{f} \end{aligned} \quad (10.22)$$

For exact constraint modes and a symmetric stiffness matrix, the reduced stiffness matrix has the following block-diagonal form:

$$\mathbf{K}_{CB} = \begin{bmatrix} \mathbf{K}_G & \mathbf{0} \\ \mathbf{0} & \mathbf{\Omega}_i^2 \end{bmatrix} \quad (10.23)$$

where:

$$\mathbf{K}_G = \mathbf{K}_{bb} - \mathbf{K}_{bi}\mathbf{K}_{ii}^{-1}\mathbf{K}_{ib} \quad (10.24)$$

is the **Guyan-condensed stiffness matrix**. The corresponding mass matrix is generally not block-diagonal:

$$\mathbf{M}_{CB} = \begin{bmatrix} \mathbf{M}_{bb}^{CB} & \mathbf{M}_{b\varphi}^{CB} \\ \mathbf{M}_{\varphi b}^{CB} & \mathbf{I} \end{bmatrix} \quad (10.25)$$

with:

$$\begin{aligned} \mathbf{M}_{bb}^{CB} &= \mathbf{M}_{bb} + \mathbf{M}_{bi}\Psi_c + \Psi_c^T \mathbf{M}_{ib} + \Psi_c^T \mathbf{M}_{ii} \Psi_c \\ \mathbf{M}_{b\varphi}^{CB} &= (\mathbf{M}_{bi} + \Psi_c^T \mathbf{M}_{ii}) \Phi_i \\ \mathbf{M}_{\varphi b}^{CB} &= (\mathbf{M}_{b\varphi}^{CB})^T \end{aligned} \quad (10.26)$$

The coupling in the reduced mass matrix is the reason why the Craig-Bampton coordinates are not normal modes of the unconstrained component. The boundary coordinates remain physical and are dynamically coupled with the fixed-interface modal coordinates.

Note:

The Craig-Bampton transformation does not reduce the boundary DOFs. If a component has n_b boundary DOFs and n_φ retained fixed-interface modes, then the reduced model has:

$$n_{CB} = n_b + n_\varphi \quad (10.27)$$

generalised coordinates. A large number of interface nodes can therefore still result in a relatively large Craig-Bampton model.

10.3 Flexible body modal reduction

A **flexible body** used in a multibody or reduced-order simulation is often represented by a small set of free-interface normal modes instead of keeping all Craig-Bampton boundary coordinates as independent generalised coordinates. This can be obtained by performing a second modal reduction in the Craig-Bampton space.

The Craig-Bampton equation without damping and external forces is:

$$\mathbf{M}_{CB}\ddot{\mathbf{q}}_{CB} + \mathbf{K}_{CB}\mathbf{q}_{CB} = \mathbf{0} \quad (10.28)$$

No boundary coordinates are constrained in the second reduction pass. The free-interface generalised eigenvalue problem is therefore:

$$(\mathbf{K}_{CB} - \omega_j^2 \mathbf{M}_{CB}) \boldsymbol{\xi}_j = \mathbf{0} \quad (10.29)$$

The eigenvectors are assembled into the reduced free-interface mode matrix:

$$\boldsymbol{\Xi} = [\boldsymbol{\xi}_1 \quad \boldsymbol{\xi}_2 \quad \dots \quad \boldsymbol{\xi}_{n_f}] \quad (10.30)$$

where n_f is the number of retained free-interface modes. The Craig-Bampton coordinates are then represented by:

$$\mathbf{q}_{CB} = \boldsymbol{\Xi} \mathbf{q}_f \quad (10.31)$$

and the full physical displacement is recovered as:

$$\begin{aligned} \mathbf{u} &= \mathbf{T}_{CB} \mathbf{q}_{CB} \\ &= \mathbf{T}_{CB} \boldsymbol{\Xi} \mathbf{q}_f \\ &= \mathbf{T}_f \mathbf{q}_f \end{aligned} \quad (10.32)$$

where:

$$\mathbf{T}_f = \mathbf{T}_{CB} \boldsymbol{\Xi} \quad (10.33)$$

is the final **flexible body transformation matrix**. The columns of $\mathbf{T}_f$ are free-interface mode shapes expressed at the original FE DOFs. The interface

displacement is no longer an independent coordinate. It is approximated by the retained flexible body modes in the same way as the internal displacement.

The final reduced matrices are calculated as:

$$\begin{aligned}\mathbf{M}_f &= \mathbf{\Xi}^T \mathbf{M}_{CB} \mathbf{\Xi} = \mathbf{T}_f^T \mathbf{M} \mathbf{T}_f \\ \mathbf{K}_f &= \mathbf{\Xi}^T \mathbf{K}_{CB} \mathbf{\Xi} = \mathbf{T}_f^T \mathbf{K} \mathbf{T}_f \\ \mathbf{C}_f &= \mathbf{\Xi}^T \mathbf{C}_{CB} \mathbf{\Xi} = \mathbf{T}_f^T \mathbf{C} \mathbf{T}_f \\ \mathbf{f}_f &= \mathbf{\Xi}^T \mathbf{f}_{CB} = \mathbf{T}_f^T \mathbf{f}\end{aligned}\tag{10.34}$$

When the free-interface eigenvectors are mass-normalised:

$$\begin{aligned}\mathbf{\Xi}^T \mathbf{M}_{CB} \mathbf{\Xi} &= \mathbf{I} \\ \mathbf{\Xi}^T \mathbf{K}_{CB} \mathbf{\Xi} &= \mathbf{\Omega}_f^2\end{aligned}\tag{10.35}$$

The flexible body equation of motion then becomes:

$$\ddot{\mathbf{q}}_f + \mathbf{C}_f \dot{\mathbf{q}}_f + \mathbf{\Omega}_f^2 \mathbf{q}_f = \mathbf{f}_f\tag{10.36}$$

For classical modal damping, the damping matrix can be represented by a diagonal matrix:

$$\mathbf{C}_f = \begin{bmatrix} 2\zeta_1\omega_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 2\zeta_{n_f}\omega_{n_f} \end{bmatrix}\tag{10.37}$$

10.3.1 Rigid body modes

A free three-dimensional component has six **rigid body modes**: three translations and three rotations. Their eigenvalues are theoretically equal to zero because a rigid body displacement produces no elastic strain energy:

$$\mathbf{K}_{CB}\boldsymbol{\xi}_r = \mathbf{0} \quad (10.38)$$

In finite-precision arithmetic, the rigid body eigenvalues are usually small but not exactly zero. They must therefore be identified using a tolerance relative to the characteristic stiffness or to the first elastic eigenvalue, rather than by checking whether the calculated frequency is exactly zero.

The exact treatment of the rigid body modes depends on the formulation of the surrounding simulation:

- when the flexible component is described only by modal coordinates, the rigid body modes must be retained to allow free translation and rotation,
- when the multibody formulation describes the rigid body position and orientation separately, the six rigid body modes are not used as elastic coordinates; the retained elastic modes must nevertheless remain mass-orthogonal to the rigid body motion.

The second case is commonly used by floating-frame flexible body formulations. The large rigid motion is described by the reference frame of the body and the modal coordinates describe only the small elastic deformation relative to this frame.

10.3.2 Mode truncation

If all n_{CB} eigenvectors of the reduced free-interface eigenproblem are kept, the transformation Ξ represents only another basis of the same Craig-Bampton space. The second reduction becomes a dimensionality reduction only when the mode matrix is truncated:

$$n_f < n_{CB} \quad (10.39)$$

This provides the distinction between the two reduced representations:

- the Craig-Bampton model retains all boundary DOFs and therefore preserves an independent physical coordinate at every interface DOF,
- the flexible body modal model retains only selected free-interface modes and therefore approximates both the boundary and internal deformation by a smaller number of modal coordinates.

The accuracy of the final flexible body base is limited by both truncation stages. The fixed-interface base must first be rich enough to represent the required internal dynamics. The free-interface base is then selected according to the frequency range and deformation patterns required by the final simulation. Increasing the number of free-interface modes cannot recover deformation that was already excluded from the Craig-Bampton base.

10.3.3 Numerical procedure

A numerically efficient implementation of the complete reduction can be summarised as follows:

1. Reorder the FE matrices into boundary and internal DOFs.
2. Factorise $\mathbf{K}_{ii}$ and solve $\mathbf{K}_{ii}\boldsymbol{\Psi}_c = -\mathbf{K}_{ib}$ for all constraint modes.
3. Solve the lowest required eigenpairs of $\mathbf{K}_{ii}\boldsymbol{\varphi} = \omega^2\mathbf{M}_{ii}\boldsymbol{\varphi}$ with the boundary fixed.
4. Mass-normalise the fixed-interface modes and assemble $\mathbf{T}_{CB}$.
5. Calculate $\mathbf{M}_{CB}$, $\mathbf{K}_{CB}$ and, when required, $\mathbf{C}_{CB}$ by congruent transformation.
6. Solve the free-interface eigenproblem $\mathbf{K}_{CB}\boldsymbol{\xi} = \omega^2\mathbf{M}_{CB}\boldsymbol{\xi}$.
7. Identify the rigid body modes and select the required elastic modes.
8. Mass-normalise the retained eigenvectors and assemble $\mathbf{T}_f = \mathbf{T}_{CB}\boldsymbol{\Xi}$.
9. Calculate the final modal matrices and the load and output projection operators.

The following numerical considerations are important:

- The matrix inverse must not be formed explicitly. A sparse factorisation and multiple right-hand-side solve should be used for the constraint modes.
- The fixed-interface stiffness matrix $\mathbf{K}_{ii}$ must be nonsingular. A singular matrix indicates an internal rigid body motion or a mechanism that is not restrained by the selected boundary DOFs.
- Symmetric generalised eigensolvers preserve the mass orthogonality of distinct modes. An ordinary Euclidean Gram-Schmidt process is therefore not normally applied to the eigenvectors.
- Repeated or tightly clustered eigenvalues can require re-orthogonalisation of the corresponding mode block. This must use the mass inner product $\boldsymbol{\xi}_r^T \mathbf{M}_{CB} \boldsymbol{\xi}_s$.
- The Craig-Bampton matrices should be symmetrised only to remove small round-off asymmetry, not to hide an incorrectly assembled transformation.
- Mode selection should be based on the required frequency range and on the deformation and loading patterns, rather than on the mode count alone.

For a retained mode $\boldsymbol{\xi}_j$, the mass normalisation is performed as:

$$\boldsymbol{\xi}_{j, mass} = \frac{\boldsymbol{\xi}_j}{\sqrt{\boldsymbol{\xi}_j^T \mathbf{M}_{CB} \boldsymbol{\xi}_j}} \quad (10.40)$$

The orthogonality and the eigenpair residual can be checked using:

$$\begin{aligned} \mathbf{E}_M &= \boldsymbol{\Xi}^T \mathbf{M}_{CB} \boldsymbol{\Xi} - \mathbf{I} \\ \mathbf{E}_K &= \boldsymbol{\Xi}^T \mathbf{K}_{CB} \boldsymbol{\Xi} - \boldsymbol{\Omega}_f^2 \\ \mathbf{r}_j &= \mathbf{K}_{CB} \boldsymbol{\xi}_j - \omega_j^2 \mathbf{M}_{CB} \boldsymbol{\xi}_j \end{aligned} \quad (10.41)$$

A valid reduced base should have small off-diagonal entries in $\mathbf{E}_M$ and $\mathbf{E}_K$, and a small residual norm $\|\mathbf{r}_j\|$ relative to the norms of the stiffness and inertia terms.

Note:

The free-interface modes calculated in the Craig-Bampton space are not the original full FE free-free modes. They are approximations restricted to the subspace spanned by $\mathbf{T}_{CB}$. After mapping with $\mathbf{T}_f = \mathbf{T}_{CB}\mathbf{\Xi}$, they have the same physical meaning and converge towards the full FE free-free modes as the Craig-Bampton base is enriched.

Scope of the classical formulation:

The above derivation assumes a linear time-invariant system with symmetric mass and stiffness matrices. For a component linearised about a prestressed operating point, the tangent stiffness matrix can be used in the same reduction procedure.

When a velocity-dependent gyroscopic matrix $\mathbf{G}$ is retained, it must be transformed together with the other system matrices:

$$\mathbf{G}_{CB} = \mathbf{T}_{CB}^T \mathbf{G} \mathbf{T}_{CB} \quad (10.42)$$

The free-interface problem then contains the term $i\omega\mathbf{G}_{CB}$ and is no longer the symmetric real-valued eigenproblem described above. Its modes can be complex and the simple mass-orthogonality relations do not generally apply.

Chapter 11

Element Atlas

11.1 Matrices

The inversion formula for 3x3 matrix:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$
$$\mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} \quad (11.1)$$

where:

$$\begin{aligned}
A_{11} &= a_{22}a_{33} - a_{23}a_{32} \\
A_{22} &= a_{33}a_{11} - a_{31}a_{13} \\
A_{33} &= a_{11}a_{22} - a_{12}a_{21} \\
A_{12} &= a_{23}a_{31} - a_{21}a_{33} \\
A_{23} &= a_{31}a_{12} - a_{32}a_{11} \\
A_{31} &= a_{12}a_{23} - a_{13}a_{22} \\
A_{21} &= a_{32}a_{13} - a_{12}a_{33} \\
A_{32} &= a_{13}a_{21} - a_{23}a_{11} \\
A_{13} &= a_{21}a_{22} - a_{31}a_{22} \\
|A_{13}| &= a_{11}A_{11} + a_{12}A_{21} + a_{13}A_{31}
\end{aligned} \tag{11.2}$$

11.2 Numerical Integration

Note:

Quadrature is another term for **Numerical Integration**.

11.2.1 Newton-Cotes quadrature

In the most obvious procedure, points at which the function is to be formed are determined *a priori* - usually at equal intervals - and a polynomial passed through the values of the function at these points and exactly integrated.

As n values define a polynomial of degree $n - 1$, the errors will be of the order $O(h^n)$ where h is the element size. This leads to the well-known **Newton-Cotes** 'quadrature formulae'. The integrals can be written as:

$$I = \int_{-1}^1 f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \tag{11.3}$$

for the range of integration between -1 and $+1$. For example, if $n = 2$, we have the well-known trapezoidal rule:

$$I = f(-1) + f(1) \quad (11.4)$$

for $n = 3$, the well-known **Simpson** one-third rule:

$$I = \frac{1}{3} [f(-1) + 4f(0) + f(1)] \quad (11.5)$$

and for $n = 4$:

$$I = \frac{1}{4} \left[f(-1) + 3f\left(-\frac{1}{3}\right) + 3f\left(\frac{1}{3}\right) + f(1) \right] \quad (11.6)$$

11.2.2 Gauss quadrature

If in place of specifying the position of sampling points *a priori* we allow these to be located at points to be determined so as to aim for best accuracy, then for a given number of sampling points increased accuracy can be obtained. Indeed, if we again consider:

$$I = \int_{-1}^1 f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \quad (11.7)$$

and again assume a polynomial expression, it is easy to see that for n sampling points we have $2n$ unknowns (H_i and ξ_i) and hence a polynomial of degree $2n-1$ could be constructed and exactly integrated. The error is thus of order $O(h^{2n})$.

The simultaneous equations involved are difficult to solve, but some mathematical manipulation will show that the solution can be obtained explicitly in terms of **Legendre polynomials**. Thus this particular process is frequently known as **Gauss-Legendre** quadrature.

For purposes of finite elements analysis complex calculations are involved in determining the values of f , the function to be integrated. Thus the Gauss-type processes, requiring the least number of such evaluations, are ideally suited and are mostly used exclusively.

Other expressions for integration of functions of the type

$$I = \int_{-1}^1 w(\xi) f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \quad (11.8)$$

can be derived for prescribed forms of $w(\xi)$, again integrating up to a certain order of accuracy a polynomial expansion of $f(\xi)$.

11.2.3 Linear and Quadrilateral Elements

For 1D:

$$I = \int_{-1}^1 f(\xi) d\xi = \sum_1^n H_i f(\xi_i) \quad (11.9)$$

For 2D:

The most obvious way of obtaining the integral:

$$I = \int_{-1}^1 \int_{-1}^1 f(\xi, \eta) d\xi d\eta \quad (11.10)$$

Is to first evaluate the inner integral keeping η constant, i.e.:

$$\int_{-1}^1 f(\xi, \eta) d\xi = \sum_{j=1}^n H_j f(\xi_j, \eta) = \psi(\eta) \quad (11.11)$$

Evaluating the outer integral in a similar manner, we have:

$$\begin{aligned}
 I &= \int_{-1}^1 \psi(\eta) d\eta = \sum_{i=1}^n H_i \psi(\eta_i) \\
 &= \sum_{i=1}^n H_i \sum_{j=1}^n H_j f(\xi_j, \eta_i) \\
 &= \sum_{i=1}^n \sum_{j=1}^n H_i H_j f(\xi_j, \eta_i)
 \end{aligned} \tag{11.12}$$

For 3D:

For a 3D element we have similarly:

$$\begin{aligned}
 I &= \int_{-1}^1 \int_{-1}^1 \int_{-1}^1 f(\xi, \eta, \mu) d\xi d\eta d\mu \\
 &= \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n H_i H_j H_k f(\xi_i, \eta_j, \mu_k)
 \end{aligned} \tag{11.13}$$

In the above, the number of integrating points in each direction was assumed to be the same. Clearly this is not necessary and on occasion it may be an advantage to use different numbers in each direction of integration.

To get the **Gauss-Legendre** points and weights in python use:

```

1 import numpy as np
2
3 # number of integration points
4 N = 3
5
6 points, weights = np.polynomial.legendre.leggauss(N)

```

integration point	polynomial order	weight
0	n = 1	2.0
$-1/\sqrt{3}$	n = 2	1.0
$1/\sqrt{3}$		1.0
$-\sqrt{0.6}$	n = 3	5/9
0.0		8/9
$\sqrt{0.6}$		5/9
-0.861 136 311 594 953	n = 4	0.347 854 845 137 454
-0.339 981 043 584 856		0.652 145 154 862 546
0.339 981 043 584 856		0.652 145 154 862 546
0.861 136 311 594 953		0.347 854 845 137 454
-0.906 179 845 938 664	n = 5	0.236 926 885 056 189
-0.538 469 310 105 683		0.478 628 670 499 366
0.000 000 000 000 000		0.568 888 888 888 889
0.538 469 310 105 683		0.478 628 670 499 366
0.906 179 845 938 664		0.236 926 885 056 189
-0.932 469 514 203 152	n = 6	0.171 324 492 379 170
-0.661 209 386 466 265		0.360 761 573 048 139
-0.238 619 186 083 197		0.467 913 934 572 691
0.238 619 186 083 197		0.467 913 934 572 691
0.661 209 386 466 265		0.360 761 573 048 139
0.932 469 514 203 152		0.171 324 492 379 170
-0.949 107 912 342 759	n = 7	0.129 484 966 168 870
-0.741 531 185 599 394		0.279 705 391 489 277
-0.405 845 151 377 397		0.381 830 050 505 119
0.000 000 000 000 000		0.471 959 183 673 469
0.405 845 151 377 397		0.381 830 050 505 119
0.741 531 185 599 394		0.279 705 391 489 277
0.949 107 912 342 759		0.129 484 966 168 870

Table 11.1: Gauss-Legendre Integration Points and Weights

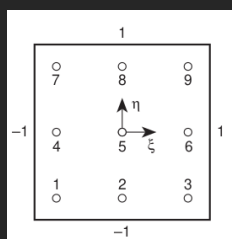


Figure 11.1: Integration points for $n = 3$ in a square region. Exact for a polynomial of fifth order in each direction

11.2.4 Triangular and Tetrahedral elements:

2D

For a triangle, in terms of the **area coordinates** the integrals are of the form:

$$I = \int_0^1 \int_0^{1-\xi} f(\xi, \eta, \mu) d\eta d\xi, \quad \mu = 1 - \xi - \eta \quad (11.14)$$

Once again we could use n Gauss points and arrive at a summation expression of the type used in the previous section. However, the limits of integration now involve the variable itself and it is convenient to use alternative sampling points for the second integration by use of a special Gauss expression for integrals of the type given by equation (11.8) in which w is a linear function. These have been devised by *Radau* and used successfully in finite element context. It is, however, much more desirable (and aesthetically pleasing) to use special formulae in which no bias is given to any of the natural coordinates (ξ, η, μ) . Such formulae were first derived by *Hammer et al.* and *Felippa* and a series of necessary sampling points and weights.

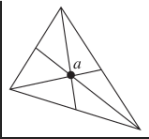
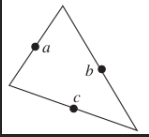
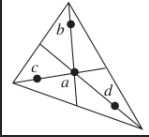
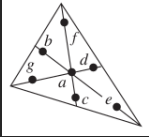
Order	Figure	Error	Points	Triangular Coordinates	Weights
Linear		$R = O(h^2)$	a	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$	1
Quadratic		$R = O(h^3)$	a b c	$\frac{1}{2}, \frac{1}{2}, 0$ $0, \frac{1}{2}, \frac{1}{2}$ $\frac{1}{2}, 0, \frac{1}{2}$	$\frac{1}{3}$ $\frac{1}{3}$ $\frac{1}{3}$
Cubic		$R = O(h^4)$	a b c d	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$ 0.6, 0.2, 0.2 0.2, 0.6, 0.2 0.2, 0.2, 0.6	$-\frac{27}{48}$ $\frac{25}{48}$ $\frac{25}{48}$ $\frac{25}{48}$
Quintic		$R = O(h^6)$	a b c d e f g with: $\alpha_1 = 0.059\ 715\ 871\ 7$ $\beta_1 = 0.470\ 142\ 064\ 1$ $\alpha_2 = 0.797\ 426\ 985\ 3$ $\beta_2 = 0.101\ 286\ 507\ 3$	$\frac{1}{3}, \frac{1}{3}, \frac{1}{3}$ $\alpha_1, \beta_1, \beta_1$ $\beta_1, \alpha_1, \beta_1$ $\beta_1, \beta_1, \alpha_1$ $\alpha_2, \beta_2, \beta_2$ $\beta_2, \alpha_2, \beta_2$ $\beta_2, \beta_2, \alpha_2$	0.2250000000 0.1323941527 0.1323941527 0.1323941527 0.1259391805 0.1259391805 0.1259391805

Table 11.2: Triangular Element integration points and weights

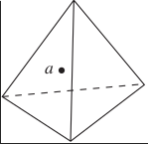
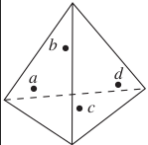
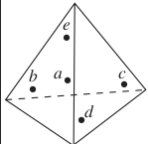
Order	Figure	Error	Points	Tetrahedral Coordinates	Weights
Linear		$R = O(h^2)$	a	$\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}$	1
Quadratic		$R = O(h^6)$	a b c d	$\alpha, \beta, \beta, \beta$ $\beta, \alpha, \beta, \beta$ $\beta, \beta, \alpha, \beta$ $\beta, \beta, \beta, \beta$	$\frac{1}{4}$ $\frac{1}{4}$ $\frac{1}{4}$ $\frac{1}{4}$
			with: $\alpha = 0.585 \ 410 \ 20$ $\beta = 0.138 \ 196 \ 60$		
Cubic		$R = O(h^4)$	a b c d e	$\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}$ $\frac{1}{2}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}$ $\frac{1}{6}, \frac{1}{2}, \frac{1}{6}, \frac{1}{6}$ $\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6}$ $\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{2}$	$-\frac{4}{5}$ $\frac{9}{20}$ $\frac{9}{20}$ $\frac{9}{20}$ $\frac{9}{20}$

Table 11.3: Tetrahedral Element integration points and weights

11.3 General Element Matrices Procedure

The procedure to generate any element stiffness, mass and load matrices can be generalised to the following steps:

1. Create Element domain in natural coordinates:
 - -1 to 1 for quadrilateral elements
 - 0 to 1 for triangular and linear elements
2. select the number of Gauss points based on the order of the element
 - 1 or 2 for linear elements
 - 3 for quadratic elements

Example:

```
1 import numpy as np
2 gp, gw = np.polynomial.legendre.leggauss(3)
```

3. Generate the shape functions $\mathbf{N}_n^e$ in natural coordinates based on the number of Gauss points, dimension of the element and its domain.

The $\mathbf{N}_n^e$ matrix has the dimensions of:

- number of integration points = number of rows
- number of shape functions = number of columns

Therefore:

$$\mathbf{N}_n^e = \begin{bmatrix} N_{1,1}^e & \cdots & N_{n,1}^e \\ \vdots & \vdots & \vdots \\ N_{1,m}^e & \cdots & N_{n,m}^e \end{bmatrix} \quad (11.15)$$

where $1 \dots m$ are the integration points and $1 \dots n$ are the shape functions.

4. Transform the shape functions from natural coordinates to global coordinates

$$\mathbf{N}_g^e = \mathbf{N}_n^e \mathbf{x}^e$$

$$\mathbf{N}_g^e = \begin{bmatrix} N_{1,1}^e(\xi) & \dots & N_{n,1}^e(\xi) \\ \vdots & \vdots & \vdots \\ N_{1,m}^e(\xi) & \dots & N_{n,m}^e(\xi) \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix} \quad (11.16)$$

The $\mathbf{x}^e$ matrix is a matrix of global coordinates of the element nodes.

The resulting matrix has the dimension of:

- number of integration points = number of rows
- number of coordinates = number of columns (3D = 3, 2D = 2)

5. Generate the shape functions derivatives $\mathbf{B}_n^e$ in natural coordinates

The shape functions derivatives matrix has 3 dimensions being *number of integration points* $\times$ *number of natural coordinates* $\times$ *number of shape functions*

6. Create the Matrix of Jacobians from the shape functions derivatives:

$$\mathbf{J} = \mathbf{B}_n^e \mathbf{x}^e \quad (11.17)$$

The Matrix of Jacobians has the dimension of *number of integration points* $\times$ *number of global coordinates* $\times$ *number of global coordinates*, where number of global coordinates means 2 for 2D and 3 for 3D problem.

7. Get the Matrix of Jacobian determinants and an inverse Jacobian:

$$\mathbf{J}^{-1} \mathbf{J}_d = \det \mathbf{J} \quad (11.18)$$

Example:

```
1 d_jacobi = np.linalg.det(jacobi)
2 i_jacobi = np.linalg.inv(jacobi)
```

The Jacobian determinant is computed for each integration point and has a dimension of *number of integration points* $\times 1$

The inverse Jacobian has the same dimension as the Jacobian (**must have**)

8. Transform the shape function derviations from natural coordinates to global coordinates

$$\mathbf{B}_g^e = \mathbf{J}^{-1} \mathbf{B}_n^e \quad (11.19)$$

9. Create the Material Stiffness matrix $\mathbf{C}$

10. Finally for each integration point:

3D:

$$\begin{aligned} \mathbf{B}_{i,x}^e &= \mathbf{B}_g(i, 0) \\ \mathbf{B}_{i,y}^e &= \mathbf{B}_g(i, 1) \\ \mathbf{B}_{i,z}^e &= \mathbf{B}_g(i, 2) \end{aligned} \quad (11.20)$$

where $\mathbf{B}_{i,x}^e$, $\mathbf{B}_{i,y}^e$ and $\mathbf{B}_{i,z}^e$ are vectors of legth = *number of shape functions*

Then matrix $\mathbf{B}$ is:

$$\mathbf{B}_i = \begin{bmatrix} \mathbf{B}_{i,x}^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{i,y}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{B}_{i,z}^e \\ \mathbf{B}_{i,y}^e & \mathbf{B}_{i,x}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{i,z}^e & \mathbf{B}_{i,y}^e \\ \mathbf{B}_{i,z}^e & \mathbf{0} & \mathbf{B}_{i,x}^e \end{bmatrix} \quad (11.21)$$

and matrix $\mathbf{N}$ is:

$$\mathbf{N}_i = \begin{bmatrix} \mathbf{N}_{n,i}^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{N}_{n,i}^e & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{N}_{n,i}^e \end{bmatrix} \quad (11.22)$$

where $\mathbf{0}$ is a zero vector of length = *number of shape functions* and $\mathbf{N}_{n,i}^e$ is a vector of shape functions in natural coordinates pertaining to the i -th integration point.

Then:

$$(a) \quad \mathbf{K}_i^e = \mathbf{B}_i^T \mathbf{C} \mathbf{B}_i \det \mathbf{J}_i w_i \quad (11.23)$$

$$(b) \quad \mathbf{M}_i^e = \rho \mathbf{N}_i^T \mathbf{N}_i \det \mathbf{J}_i w_i \quad (11.24)$$

$$(c) \quad \mathbf{F}_i^e = \mathbf{N}_i^T \mathbf{F} \det \mathbf{J}_i w_i \quad (11.25)$$

$$\text{where } \mathbf{F} = \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix}$$

where w_i is the multiple of gaussian weights for the respective integration point (for 3D case brick where the natural coordinates are ξ, η, μ the $w_i = w_{\xi_i} w_{\eta_i} w_{\mu_i}$

11. Finally:

$$(a) \quad \mathbf{K}^e = \sum_{i=1}^n \mathbf{K}_i^e \quad (11.26)$$

$$(b) \quad \mathbf{M}^e = \sum_{i=1}^n \mathbf{M}_i^e \quad (11.27)$$

$$(c) \quad \mathbf{F}^e = \sum_{i=1}^n \mathbf{F}_i^e \quad (11.28)$$

11.4 ROD

11.4.1 Element Formulation

The **rod** element is a 1-D linear element. It has 2 nodes, and 3 dofs for each node u_x, u_y, u_z (or u, v, w). Rotation **DOFs** are not constrained.

Natural coordinates

The natural coordinate ξ of the **ROD** element goes from 0 to 1. The 2-noded rod element is defined by:

$$\begin{bmatrix} 1 \\ x \\ y \\ z \\ u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ x_1 & x_2 \\ y_1 & y_2 \\ z_1 & z_2 \\ u_{x1} & u_{x2} \\ u_{y1} & u_{y2} \\ u_{z1} & u_{z2} \end{bmatrix} \begin{bmatrix} N_1^e \\ N_2^e \end{bmatrix} \quad (11.29)$$

Shape Functions

The **ROD** natural coordinates are:

node	ξ
1	0
2	1

Table 11.4: ROD corners natural coordinates

The shape functions are:

$$\begin{aligned} N_1^e &= 1 - \xi \\ N_2^e &= \xi \end{aligned} \quad (11.30)$$

Partial Derivatives

The shape function derivatives with respect to the natural coordinate:

$$\begin{aligned}\frac{\partial N_1^e}{\partial \xi} &= -1 \\ \frac{\partial N_2^e}{\partial \xi} &= 1\end{aligned}\tag{11.31}$$

The Jacobian

The derivatives of the shape functions are given by the usual chain rule formula:

$$\begin{aligned}\frac{\partial N_i^e}{\partial x} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial x} \\ \frac{\partial N_i^e}{\partial y} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial y} \\ \frac{\partial N_i^e}{\partial z} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial z}\end{aligned}\tag{11.32}$$

In matrix form:

$$\begin{bmatrix} \frac{\partial N_i^e}{\partial x} \\ \frac{\partial N_i^e}{\partial y} \\ \frac{\partial N_i^e}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{\partial \xi}{\partial x} \\ \frac{\partial \xi}{\partial y} \\ \frac{\partial \xi}{\partial z} \end{bmatrix} \left[\frac{\partial N_i^e}{\partial \xi} \right]\tag{11.33}$$

The 3×1 matrix above is $\mathbf{J}^{-1}$, the inverse of:

$$\mathbf{J} = \frac{\partial(x, y, z)}{\partial(\xi)} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \end{bmatrix}\tag{11.34}$$

Given the element coordinates the Jacobian can be computed:

$$\mathbf{J}_i = \begin{bmatrix} \frac{\partial N_i^e}{\partial \xi} & \frac{\partial N_i^e}{\partial \xi} & \frac{\partial N_i^e}{\partial \xi} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}\tag{11.35}$$

end then numerically inverted for $\mathbf{J}^{-1}$.

If the $\mathbf{J}$ is not a square matrix, then **Moore-Penrose pseudoinverse** applies:

$$\mathbf{J}^{-1} = \mathbf{J}^+ = (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \quad (11.36)$$

in python:

```

1      import numpy as np
2
3      J_i = np.linalg.pinv(J)
```

Assembling the **Jacobian** together gives essentially that for a 1D case

$$\mathbf{J} = l \quad (11.37)$$

and therefore:

$$\mathbf{J}^{-1} = \frac{1}{l} \quad (11.38)$$

$$\det J = l \quad (11.39)$$

afterwards for each integration point:

$$\mathbf{N}_g^i = \mathbf{N}^i \mathbf{u} \quad (11.40)$$

$$\mathbf{B}_g^i = \mathbf{J}^{i-1} \frac{\partial N^i}{\partial \xi} \quad (11.41)$$

then:

$$\begin{aligned}
\mathbf{K}^e &\stackrel{+}{=} \mathbf{B}_g^{iT} \mathbf{D} \mathbf{B}_g^i \det \mathbf{J}^i w^i \\
\mathbf{M}^e &\stackrel{+}{=} \rho \mathbf{N}^{iT} \mathbf{N}^i \det \mathbf{J}^i w^i \\
\mathbf{F}^e &\stackrel{+}{=} \mathbf{N}^{iT} \mathbf{F} \det \mathbf{J}^i w^i
\end{aligned} \quad (11.42)$$

11.4.2 Example 1D

If assembling the element in local CSYS, then $u_y = 0$ and $u_z = 0$. Simplifying the equations gives that:

$$\mathbf{N} = [N_1^e \quad N_2^e] = [1 - \xi \quad \xi] \quad (11.43)$$

and

$$u^e(\xi) = \mathbf{N} \begin{bmatrix} u_1^e \\ u_2^e \end{bmatrix} \quad (11.44)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = [-1 \quad 1] \quad (11.45)$$

the strain-displacement relation is given by:

$$\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u} \quad (11.46)$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight is $w = 2.0$)

$$\mathbf{N} = [0.5 \quad 0.5] \quad (11.47)$$

Shape Functions in global coordinates:

$$\mathbf{N} = 0.5u_1 + 0.5u_2 = 0.5(u_1 + u_2) \quad (11.48)$$

Shape Function Derivatives in Natural Coordinates:

$$\mathbf{B} = [-1 \quad 1] \quad (11.49)$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = u_2 - u_1 = l \quad (11.50)$$

Jacobian Determinant:

$$\det \mathbf{J} = l \quad (11.51)$$

Jacobian Inverse:

$$\mathbf{J}^{-1} = \frac{1}{l} \quad (11.52)$$

Shape function derivatives in global coordinates:

$$\mathbf{B}_g = \mathbf{J}^{-1}\mathbf{B} = \frac{1}{l} \begin{bmatrix} -1 & 1 \end{bmatrix} \quad (11.53)$$

Material Stiffness Matrix

$$\mathbf{D} = EA \quad (11.54)$$

Then for each integration point (here 1):

$$\begin{aligned} \mathbf{K}^e &\pm \mathbf{B}_g^T \mathbf{D} \mathbf{B}_g \det \mathbf{J} w \\ \mathbf{K}^e &\pm \frac{1}{l} \begin{bmatrix} -1 \\ 1 \end{bmatrix} EA \frac{1}{l} \begin{bmatrix} -1 & 1 \end{bmatrix} l 1 \\ \mathbf{K}^e &\pm \frac{EA}{l} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} \\ \mathbf{K}^e &\pm \frac{EA}{l} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \end{aligned} \quad (11.55)$$

11.4.3 Example 2D

If assembling the element in 2D, then $u_z = 0$. Simplifying the equations gives that:

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & N_2^e & 0 \\ 0 & N_1^e & 0 & N_2^e \end{bmatrix} = \begin{bmatrix} 1 - \xi & 0 & \xi & 0 \\ 0 & 1 - \xi & 0 & \xi \end{bmatrix} \quad (11.56)$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \end{bmatrix} \quad (11.57)$$

or if:

$$\mathbf{N} = [N_1^e \quad N_2^e] = [1 - \xi \quad \xi] \quad (11.58)$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N} \begin{bmatrix} u_{x1}^e & u_{y1}^e \\ u_{x2}^e & u_{y2}^e \end{bmatrix} \quad (11.59)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \quad (11.60)$$

the strain-displacement relation is given by:

$$\boldsymbol{\epsilon} = \mathbf{B} \mathbf{u} \quad (11.61)$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight $w = 2.0$) then numerically:

$$\mathbf{N} = [0.5 \quad 0.5] \quad (11.62)$$

Shape Functions in global coordinates:

$$\mathbf{N} = \begin{bmatrix} 1 - \xi & 0 & \xi & 0 \\ 0 & 1 - \xi & 0 & \xi \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \end{bmatrix} = \begin{bmatrix} 0.5u_{x1} + 0.5u_{x2} \\ 0.5u_{y1} + 0.5u_{y2} \end{bmatrix} \tag{11.63}$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate **B** matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \tag{11.64}$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \end{bmatrix} = \begin{bmatrix} u_{x2}^e - u_{x1}^e \\ u_{y2}^e - u_{y1}^e \end{bmatrix} = \begin{bmatrix} l_x \\ l_y \end{bmatrix} \tag{11.65}$$

??

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

ll

$$\det \mathbf{J} = ||\mathbf{J}|| = \sqrt{J_1^2 + J_2^2} = \sqrt{l_x^2 + l_y^2} = l \tag{11.66}$$

??

$$\mathbf{K}^e \stackrel{+}{=} \mathbf{B}_g^T \mathbf{D} \mathbf{B}_g \det \mathbf{J} w \quad (11.71)$$

$$\begin{aligned} \mathbf{K}^e &\stackrel{+}{=} \frac{1}{l} \begin{bmatrix} -c \\ -s \\ c \\ s \end{bmatrix} EA \frac{1}{l} \begin{bmatrix} -c & -s & c & s \end{bmatrix} lw \\ &\stackrel{+}{=} \frac{EA}{l} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix} w \end{aligned} \quad (11.72)$$

which is the same result as when the element stiffness matrix is derived in local coordinate system for a situation where ξ axis coincides with the element x axis and the u_y component is 0. Afterwards, the element **stiffness** and **mass** matrices as well as **forces** vector are transformed to the global CSYS using a transformation matrix $\mathbf{T}$:

$$\mathbf{K}_l^e = \frac{EA}{l} \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (11.73)$$

$$\mathbf{T} = \begin{bmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & c & s \\ 0 & 0 & -s & c \end{bmatrix} \quad (11.74)$$

then the global element stiffness matrix is defined:

$$\begin{aligned}
\mathbf{K}_g^e &= \mathbf{T}^T \mathbf{K}_l^e \mathbf{T} \\
&= \frac{EA}{l} \begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} c & s & 0 & 0 \\ -s & c & 0 & 0 \\ 0 & 0 & c & s \\ 0 & 0 & -s & c \end{bmatrix} \\
&= \frac{EA}{l} \begin{bmatrix} c & -s & 0 & 0 \\ s & c & 0 & 0 \\ 0 & 0 & c & -s \\ 0 & 0 & s & c \end{bmatrix} \begin{bmatrix} c & s & -c & -s \\ 0 & 0 & 0 & 0 \\ -c & -s & c & s \\ 0 & 0 & 0 & 0 \end{bmatrix} \\
&= \frac{EA}{l} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix}
\end{aligned} \tag{11.75}$$

11.4.4 Example 3D

If assembling the element in 3D, then $u_x \neq 0$, $u_y \neq 0$ and $u_z \neq 0$. The full equations give that:

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & 0 & N_2^e & 0 & 0 \\ 0 & N_1^e & 0 & 0 & N_2^e & 0 \\ 0 & 0 & N_1^e & 0 & 0 & N_2^e \end{bmatrix} = \begin{bmatrix} 1-\xi & 0 & 0 & \xi & 0 & 0 \\ 0 & 1-\xi & 0 & 0 & \xi & 0 \\ 0 & 0 & 1-\xi & 0 & 0 & \xi \end{bmatrix} \tag{11.76}$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \\ u_z^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{z1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{z2}^e \end{bmatrix} \tag{11.77}$$

or if:

$$\mathbf{N} = [N_1^e \quad N_2^e] = [1-\xi \quad \xi] \tag{11.78}$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \\ u_z^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e & u_{y1}^e & u_{z1}^e \\ u_{x2}^e & u_{y2}^e & u_{z2}^e \end{bmatrix} \quad (11.79)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix} \quad (11.80)$$

the strain-displacement relation is given by:

$$\boldsymbol{\epsilon} = \mathbf{B}\mathbf{u} \quad (11.81)$$

Integrating the element at midpoint $\xi = 0.5$ with a weight of $w = 1.0$ (for interval $\langle 0, 1 \rangle$, for an interval $\langle -1, 1 \rangle$ the weight $w = 2.0$) then numerically:

$$\mathbf{N} = [0.5 \quad 0.5] \quad (11.82)$$

Shape Functions in global coordinates:

$$\mathbf{N} = \begin{bmatrix} 1 - \xi & 0 & 0 & \xi & 0 & 0 \\ 0 & 1 - \xi & 0 & 0 & \xi & 0 \\ 0 & 0 & 1 - \xi & 0 & 0 & \xi \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{z1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{z2}^e \end{bmatrix} = \begin{bmatrix} 0.5u_{x1} + 0.5u_{x2} \\ 0.5u_{y1} + 0.5u_{y2} \\ 0.5u_{z1} + 0.5u_{z2} \end{bmatrix} \quad (11.83)$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate $\mathbf{B}$ matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix} \quad (11.84)$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}_{\xi=0.5} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{z1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{z2}^e \end{bmatrix} = \begin{bmatrix} u_{x2}^e - u_{x1}^e \\ u_{y2}^e - u_{y1}^e \\ u_{z2}^e - u_{z1}^e \end{bmatrix} = \begin{bmatrix} l_x \\ l_y \\ l_z \end{bmatrix} \quad (11.85)$$

??

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

ll

$$\det \mathbf{J} = \|\mathbf{J}\| = \sqrt{J_1^2 + J_2^2 + J_3^2} = \sqrt{l_x^2 + l_y^2 + l_z^2} = l \quad (11.86)$$

??

Jacobian Inverse (when the **Jacobian matrix** is a row vector, its inverse is just its **transpose** divided by its determinant squared):

Moore-Penrose pseudoinverse:

$$\begin{aligned}
 \boxed{\mathbf{J}^{-1}} &= \mathbf{J}^+ = (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \\
 &= \left(\begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \begin{bmatrix} l_x \\ l_y \\ l_z \end{bmatrix} \right)^{-1} \begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
 &= (l_x^2 + l_y^2 + l_z^2)^{-1} \begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
 &= \frac{1}{l_x^2 + l_y^2 + l_z^2} \begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
 &= \frac{1}{\sqrt{l_x^2 + l_y^2 + l_z^2}^2} \begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \\
 &= \boxed{\frac{1}{l^2} \mathbf{J}^T}
 \end{aligned} \tag{11.87}$$

ℓ ℓ

$$\mathbf{J}^{-1} = \frac{1}{l^2} \begin{bmatrix} l_x & l_y & l_z \end{bmatrix} \tag{11.88}$$

Shape function derivatives in global coordinates:

$$\begin{aligned}
 \mathbf{B}_g &= \mathbf{J}^{-1} \mathbf{B} = \frac{1}{l^2} \begin{bmatrix} l_x & l_y & l_z \end{bmatrix}_{\xi=0.5} \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 1 \end{bmatrix}_{\xi=0.5} \\
 &= \frac{1}{l^2} \begin{bmatrix} -l_x & -l_y & -l_z & l_x & l_y & l_z \end{bmatrix}
 \end{aligned} \tag{11.89}$$

Material Stiffness Matrix

$$\mathbf{D} = EA \tag{11.90}$$

Then for each integration point (here 1):

$$\mathbf{K}^e \stackrel{+}{=} \mathbf{B}_g^T \mathbf{D} \mathbf{B}_g \det \mathbf{J} w \quad (11.91)$$

$$\begin{aligned} \mathbf{K}^e \stackrel{+}{=} \frac{1}{l^2} \begin{bmatrix} -l_x \\ -l_y \\ -l_z \\ l_x \\ l_y \\ l_z \end{bmatrix} EA \frac{1}{l^2} \begin{bmatrix} -l_x & -l_y & -l_z & l_x & l_y & l_z \end{bmatrix} l w \\ \stackrel{+}{=} \frac{EA}{l^3} \begin{bmatrix} l_x^2 & l_x l_y & l_x l_z & -l_x^2 & -l_x l_y & -l_x l_z \\ l_x l_y & l_y^2 & l_y l_z & -l_x l_y & -l_y^2 & -l_y l_z \\ l_x l_z & l_y l_z & l_z^2 & -l_x l_z & -l_y l_z & -l_z^2 \\ -l_x^2 & -l_x l_y & -l_x l_z & l_x^2 & l_x l_y & l_x l_z \\ -l_x l_y & -l_y^2 & -l_y l_z & l_x l_y & l_y^2 & l_y l_z \\ -l_x l_z & -l_y l_z & -l_z^2 & l_x l_z & l_y l_z & l_z^2 \end{bmatrix} w \end{aligned} \quad (11.92)$$

11.4.5 Example 2D quadratic

If assembling the element in 2D, then and $u_z = 0$.

The node numbering scheme is 1 – 3 – 2 and the natural coordinate is defined $\xi \in \langle -1, 1 \rangle$.

The **quadratic** shape functions are:

$$\begin{aligned} N_1^e &= -\frac{1}{2}\xi(1-\xi) \\ N_2^e &= \frac{1}{2}\xi(1+\xi) \\ N_3^e &= (1+\xi)(1-\xi) \end{aligned} \quad (11.93)$$

$$\mathbf{N} = \begin{bmatrix} N_1^e & 0 & N_2^e & 0 & N_3^e & 0 \\ 0 & N_1^e & 0 & N_2^e & 0 & N_3^e \end{bmatrix} \quad (11.94)$$

$$\mathbf{N} = \begin{bmatrix} -\frac{1}{2}\xi(1-\xi) & 0 & \frac{1}{2}\xi(1+\xi) & 0 & (1+\xi)(1-\xi) & 0 \\ -\frac{1}{2}\xi(1-\xi) & -\frac{1}{2}\xi(1-\xi) & 0 & \frac{1}{2}\xi(1+\xi) & 0 & (1+\xi)(1-\xi) \end{bmatrix} \quad (11.95)$$

$$\mathbf{N} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 & 0 \\ 0 & \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 \end{bmatrix} \quad (11.96)$$

and

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{x3}^e \\ u_{y3}^e \end{bmatrix} \quad (11.97)$$

or if:

$$\mathbf{N} = \begin{bmatrix} N_1^e & N_2^e & N_3^e \end{bmatrix} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & \frac{\xi^2}{2} + \frac{\xi}{2} & 1 - \xi^2 \end{bmatrix} \quad (11.98)$$

then:

$$\begin{bmatrix} u_x^e(\xi) \\ u_y^e(\xi) \end{bmatrix} = \mathbf{N}(\xi) \begin{bmatrix} u_{x1}^e & u_{y1}^e \\ u_{x2}^e & u_{y2}^e \\ u_{x3}^e & u_{y3}^e \end{bmatrix} \quad (11.99)$$

shape function derivatives:

$$\mathbf{B} = \frac{\partial \mathbf{N}}{\partial \xi} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \quad (11.100)$$

the strain-displacement relation is given by:

$$\boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u} \quad (11.101)$$

Integrating the element at two **Gauss-Legendre** points of the natural coordinate $\xi = [-0.5773502692, 0.5773502692]$ with weights of $w = 1.0$ then numerically:

To get the correct interpolation points and weights, python can be used:

```

1      import numpy as np
2
3      xi, w = np.polynomial.legendre.leggauss(2)
```

for $\xi^1 = -0.5773502692$

$$\mathbf{N}^1 = \begin{bmatrix} 0.45534 & -0.12201 & 0.66667 \end{bmatrix} \quad (11.102)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{N}^2 = \begin{bmatrix} -0.12201 & 0.45534 & 0.66667 \end{bmatrix} \quad (11.103)$$

Shape Functions in global coordinates:

$$\begin{bmatrix} u_{x1}^i \\ u_{y1}^i \end{bmatrix} = \begin{bmatrix} \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 & 0 \\ 0 & \frac{\xi^2}{2} - \frac{\xi}{2} & 0 & \frac{\xi^2}{2} + \frac{\xi}{2} & 0 & 1 - \xi^2 \end{bmatrix}_{\xi^i} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{x3}^e \\ u_{y3}^e \end{bmatrix} \quad (11.104)$$

$$\begin{bmatrix} u_x^1 \\ u_y^1 \end{bmatrix} = \begin{bmatrix} 0.45534u_{x1} - 0.12201u_{x2} + 0.66667u_{x3} \\ -0.12201u_{y1} + 0.45534u_{y2} + 0.66667u_{y3} \end{bmatrix} \quad (11.105)$$

$$\begin{bmatrix} u_x^2 \\ u_y^2 \end{bmatrix} = \begin{bmatrix} -0.12201u_{x1} + 0.45534u_{x2} + 0.66667u_{x3} \\ 0.45534u_{y1} - 0.12201u_{y2} + 0.66667u_{y3} \end{bmatrix} \quad (11.106)$$

Shape Function Derivatives in Natural Coordinates (there is a row for each integration point if the coordinates are specified as matrix, not a column vector. otherwise for each integration point there is a separate $\mathbf{B}$ matrix):

$$\frac{\partial \mathbf{N}}{\partial \xi} = \mathbf{B} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \quad (11.107)$$

Jacobian:

$$\mathbf{J} = \mathbf{B}\mathbf{u} = \begin{bmatrix} \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi & 0 \\ 0 & \xi - \frac{1}{2} & 0 & \xi + \frac{1}{2} & 0 & -2\xi \end{bmatrix} \begin{bmatrix} u_{x1}^e \\ u_{y1}^e \\ u_{x2}^e \\ u_{y2}^e \\ u_{x3}^e \\ u_{y3}^e \end{bmatrix} \quad (11.108)$$

for $\xi^1 = -0.5773502692$

$$\mathbf{J}^1 = \begin{bmatrix} -1.07735u_{x1} - 0.07735u_{x2} + 1.15470u_3 \\ -1.07735u_{y1} - 0.07735u_{y2} + 1.15470u_{y3} \end{bmatrix} \quad (11.109)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{J}^2 = \begin{bmatrix} 0.07735u_{x1} + 1.07735u_{x2} - 1.15470u_3 \\ 0.07735u_{y1} + 1.07735u_{y2} - 1.15470u_{y3} \end{bmatrix} \quad (11.110)$$

Jacobian Determinant (when the **Jacobian matrix** is a vector, its determinant is simply its **length**):

$$\det \mathbf{J} = \|\mathbf{J}\| \quad (11.111)$$

for $\xi^1 = -0.5773502692$

$$\det \mathbf{J}^1 = \sqrt{(-1.07735u_{x1} - 0.07735u_{x2} + 1.15470u_3)^2 + (-1.07735u_{y1} - 0.07735u_{y2} + 1.15470u_{y3})^2} \quad (11.112)$$

for $\xi^2 = 0.5773502692$

$$\det \mathbf{J}^2 = \sqrt{(0.07735u_{x1} + 1.07735u_{x2} - 1.15470u_3)^2 + (0.07735u_{y1} + 1.07735u_{y2} - 1.15470u_{y3})^2} \quad (11.113)$$

Jacobian Inverse (when the **Jacobian matrix** is a row vector, its inverse is just its **transpose** divided by its determinant squared):

Moore-Penrose pseudoinverse:

$$\mathbf{J}^{-1} = \mathbf{J}^+ = (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \quad (11.114)$$

for $\xi^1 = -0.5773502692$

$$\mathbf{J}^{1-1} = \frac{1}{\det \mathbf{J}^{12}} \mathbf{J}^{1T} \quad (11.115)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{J}^{2-1} = \frac{1}{\det \mathbf{J}^{22}} \mathbf{J}^{2T} \quad (11.116)$$

Shape function derivatives in global coordinates:

$$\mathbf{B}_g = \mathbf{J}^{-1} \mathbf{B} \quad (11.117)$$

for $\xi^1 = -0.5773502692$

$$\mathbf{B}_g^1 = \mathbf{J}^{1-1} \mathbf{B}^1 \quad (11.118)$$

for $\xi^2 = 0.5773502692$

$$\mathbf{B}_g^2 = \mathbf{J}^{2-1} \mathbf{B}^2 \quad (11.119)$$

Material Stiffness Matrix

$$\mathbf{D} = EA \quad (11.120)$$

Then the stiffness matrix is composed as:

$$\mathbf{K}^e = \sum_{i=1}^2 \mathbf{B}_g^{iT} \mathbf{D} \mathbf{B}_g^i \det \mathbf{J}^i w^i \quad (11.121)$$

11.4.6 Python Implementation

```

1  #!/usr/bin/python3
2
3  import numpy as np
4  np.set_printoptions(precision=6, suppress=True)
5
6  def rod2(xyz, E, A, rho, fx, fy, fz, gauss_points=1):
7      print('\nStarting ROD element generation:\n')
8      coors = xyz.reshape(-1, 1)
9      print('----- Input')
10     print('coors =\n{0}'.format(coors))
11     print(f'{A = } mm2\n{rho = } t/mm2\n{fx = } N/mm\n{fy = } N/mm' +
12         f'\n{fz = } N/mm\n{gauss_points = }\n')
13

```

```

14     domain = np.array([[ -1],
15                        [ 1]], dtype=float)
16     print('Domain: {0}\n{1}\n'.format(domain.shape, domain))
17
18     print('----- Gauss-Legendre')
19     # gauss-legendre integration
20     ip, iw = np.polynomial.legendre.leggauss(gauss_points)
21     print('Gauss-Legendre Integration\nip = {0}\n{1}\n'.format(ip, iw))
22
23     full_domain = np.vstack((domain, ip.reshape(-1, 1)))
24
25     print('----- Shape Functions')
26     # shape functions in natural coordinates
27     xi = ip.T
28     print('xi = {0}\n'.format(xi))
29     psi = []
30     for i in range(xi.shape[0]):
31         psi.append(np.array([[1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2, 0, 0],
32                             [0, 1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2, 0],
33                             [0, 0, 1/2 - xi[i]/2, 0, 0, 1/2 + xi[i]/2]],
34                             dtype=float))
35     psi = np.array(psi)
36     # psi = psi.transpose(0, 2, 1)
37     print('Shape Functions: {0}\n{1}\n'.format(psi.shape, psi))
38
39     # integration points in global coordinates
40     psi_g = psi @ coors
41     print('Integration Points in '
42           'Global Coordinates: {0}\n{1}\n'.format(psi_g.shape, psi_g))
43
44     # shape function derivatives in natural coordinates
45     dpsi = []
46     for i in range(xi.shape[0]):
47         dpsi.append(np.array([[ -1/2, 0, 0, 1/2, 0, 0],
48                               [0, -1/2, 0, 0, 1/2, 0],
49                               [0, 0, -1/2, 0, 0, 1/2]], dtype=float))
50     dpsi = np.array(dpsi)
51     print('Shape Functions Derivatives: {0}\n{1}\n'.format(dpsi.shape, dpsi))
52
53     print('----- Jacobians')
54     # Jacobian Matrix

```

```

55     jacobi = dps_i @ coors
56     print('Jacobian Matrix: {0}\n{1}\n'.format(jacobi.shape, jacobi))
57
58     # Jacobian Determinants
59     d_jacobi = []
60     for i in range(xi.shape[0]):
61         d_jacobi.append(np.linalg.norm(jacobi[i]))
62     d_jacobi = np.array(d_jacobi)
63     print('Determinant of Jacobian: {0}\n{1}\n'.format(d_jacobi.shape, d_jacobi))
64
65     # Inverse Jacobian
66     i_jacobi = np.linalg.pinv(jacobi)
67     print('Inverse Jacobian Matrix: {0}\n{1}\n'.format(i_jacobi.shape, i_jacobi))
68
69     # Shape Function Derivatives in Global Coordinates
70     dps_i_g = i_jacobi @ dps_i
71     print('Shape Function Derivatives in Global Coordinates:
72     ↪ {0}\n{1}\n'.format(dps_i_g.shape, dps_i_g))
73
74     # material stiffness
75     D = np.array([[E * A]], dtype=float)
76     print('Material Stiffness Matrix: {0}\n{1}\n'.format(D.shape, D))
77
78     # create element stiffness and mass matrix
79     Ke = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
80     Me = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
81     Fe = np.zeros((domain.shape[0] * 3, 1), dtype=float)
82
83     print('----- System Matrices')
84     # iterate over integration points
85     F = np.array([[fx], [fy], [fz]], dtype=float)
86     for i in range(xi.shape[0]):
87         B = dps_i_g[i]
88         N = psi[i]
89
90         Ke += (B.T @ D @ B) * d_jacobi[i] * iw[i]
91         Me += rho * (N.T @ N) * d_jacobi[i] * iw[i]
92         Fe += (N.T @ F) * d_jacobi[i] * iw[i]
93
94     print('Element Stiffness Matrix: {0}\n{1}\n'.format(Ke.shape, Ke))
95     print('Element Mass Matrix: {0}\n{1}\n'.format(Me.shape, Me))

```

```

95     print('Element Volume Force Vector: {0}\n{1}\n'.format(Fe.shape, Fe))
96
97     print('----- Displacements')
98     # xyz displacements at end nodes
99     u = np.array([0.,
100                  0.,
101                  0.,
102                  .001,
103                  .001,
104                  .001], dtype=float).reshape(-1, 1)
105     print('Displacements {0}\n{1}\n'.format(u.shape, u))
106
107     print('----- Strains')
108     eps = []
109     for i in range(xi.shape[0]):
110         eps.append(dpsi_g[i] @ u)
111     eps = np.array(eps)
112     print('Strains {0}\n{1}\n'.format(eps.shape, eps))
113
114     print('----- Stresses')
115     sig = D @ eps
116     print('Stresses {0}\n{1}\n'.format(sig.shape, sig))
117
118     return Ke, Me, Fe
119
120
121
122 if __name__ == '__main__':
123     coor = np.array([[ 0, 0., 0.],
124                     [1000., 0., 0.]], dtype=float)
125
126     E = 210000.0 # MPa steel
127     A = 200.     # steel
128     rho = 9.81E-9 # steel t/mm3
129
130     fx = 1. # volumetric continuous load
131     fy = 0. # volumetric continuous load
132     fz = 1. # volumetric continuous load
133
134     rod2(coor, E, A, rho, fx, fy, fz, 1)

```


11.5 HEX8

11.5.1 Element Formulation

The **Hexahedral** element is a 3-D quadrilateral **trilinear** element, otherwise known as a **brick**. Topologically is hexahedron equivalent to a cube. It has eight corners, twelve edges and six faces. Especially the **HEX8** element has 8 interpolation points.

Natural coordinates

The **natural coordinate system** of a hexahedral element are called **isoparametric hexahedral coordinates**, denoted ξ , η and μ . The coordinates go from -1 to 1 and span from each face to the opposite one. Each coordinate is 0 on the plane at midpoint of opposing faces called the **median** face. This choice of limits is to facilitate the use of standard *Gauss integration formulas*.

Corner Numbering Rules

The eight corners of a hexahedron are locally numbered $1, 2, \dots, 8$. The corner numbering rule is:

- select one face and numbers the corners of this face $1 \dots 4$ in a **counter-clockwise** direction.
- number the corners directly opposite to corners $1, 2, 3, 4$ as $5, 6, 7, 8$ respectively.

The purpose of this numbering manner is so that a positive volume (or more precisely, a positive **Jacobian determinant** at every point).

The definition of ξ , η and μ can be now made more precise:

- ξ goes from -1 from (center of) face 1485 to $+1$ on face 2376

- η goes from -1 from (center of) face 1265 to $+1$ on face 3487
- μ goes from -1 from (center of) face 1234 to $+1$ on face 5678

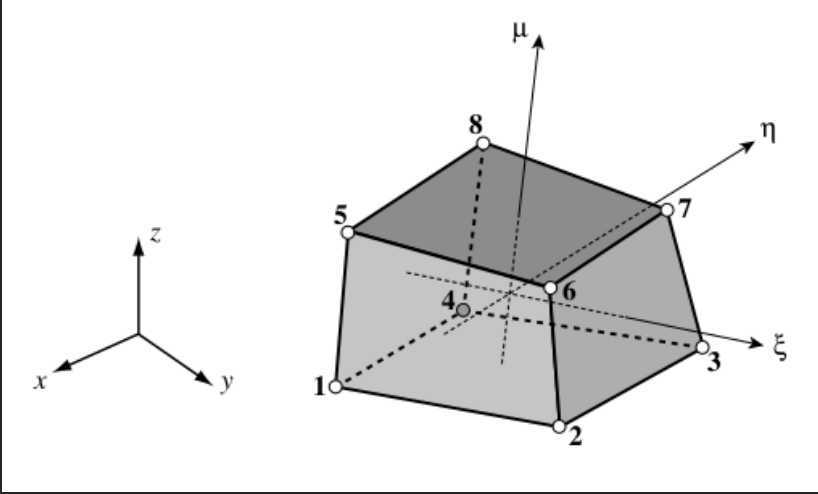


Figure 11.2: The 8-node hexahedron and the natural coordinates η , ξ and μ .

Element Definition

The 8-noded hexahedral element is defined by:

$$\begin{bmatrix} 1 \\ x \\ y \\ z \\ u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 \\ y_1 & y_2 & y_3 & y_4 & y_5 & y_6 & y_7 & y_8 \\ y_1 & y_2 & y_3 & y_4 & y_5 & y_6 & y_7 & y_8 \\ u_{x1} & u_{x2} & u_{x3} & u_{x4} & u_{x5} & u_{x6} & u_{x7} & u_{x8} \\ u_{y1} & u_{y2} & u_{y3} & u_{y4} & u_{y5} & u_{y6} & u_{y7} & u_{y8} \\ u_{z1} & u_{z2} & u_{z3} & u_{z4} & u_{z5} & u_{z6} & u_{z7} & u_{z8} \end{bmatrix} \begin{bmatrix} N_1^e \\ N_2^e \\ N_3^e \\ N_4^e \\ N_5^e \\ N_6^e \\ N_7^e \\ N_8^e \end{bmatrix} \quad (11.122)$$

The hexahedron corners natural coordinates are:

Partial Derivatives

The calculation of the shape functions derivatives with respect to the natural coordinates:

$$\begin{aligned}
 \frac{\partial N_1^e}{\partial \xi} &= -\frac{1}{8} (1 - \eta) (1 - \mu) \\
 \frac{\partial N_2^e}{\partial \xi} &= \frac{1}{8} (1 - \eta) (1 - \mu) \\
 \frac{\partial N_3^e}{\partial \xi} &= \frac{1}{8} (1 + \eta) (1 - \mu) \\
 \frac{\partial N_4^e}{\partial \xi} &= -\frac{1}{8} (1 + \eta) (1 - \mu) \\
 \frac{\partial N_5^e}{\partial \xi} &= -\frac{1}{8} (1 - \eta) (1 + \mu) \\
 \frac{\partial N_6^e}{\partial \xi} &= \frac{1}{8} (1 - \eta) (1 + \mu) \\
 \frac{\partial N_7^e}{\partial \xi} &= \frac{1}{8} (1 + \eta) (1 + \mu) \\
 \frac{\partial N_8^e}{\partial \xi} &= -\frac{1}{8} (1 + \eta) (1 + \mu)
 \end{aligned} \tag{11.125}$$

Note:

The partial derivatives can be also written as:

$$\begin{aligned}
 \frac{\partial N_i^e}{\partial \xi} &= \frac{1}{8} \frac{\xi_i}{|\xi_i|} (1 + \eta \eta_i) (1 + \mu \mu_i) \\
 \frac{\partial N_i^e}{\partial \eta} &= \frac{1}{8} \frac{\eta_i}{|\eta_i|} (1 + \xi \xi_i) (1 + \mu \mu_i) \\
 \frac{\partial N_i^e}{\partial \mu} &= \frac{1}{8} \frac{\mu_i}{|\mu_i|} (1 + \xi \xi_i) (1 + \eta \eta_i)
 \end{aligned} \tag{11.128}$$

The Jacobian:

The derivatives of the shape functions are given by the usual chain rule formulas:

$$\begin{aligned}
 \frac{\partial N_i^e}{\partial x} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial x} + \frac{\partial N_i^e}{\partial \eta} \frac{\partial \eta}{\partial x} + \frac{\partial N_i^e}{\partial \mu} \frac{\partial \mu}{\partial x} \\
 \frac{\partial N_i^e}{\partial y} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial y} + \frac{\partial N_i^e}{\partial \eta} \frac{\partial \eta}{\partial y} + \frac{\partial N_i^e}{\partial \mu} \frac{\partial \mu}{\partial y} \\
 \frac{\partial N_i^e}{\partial z} &= \frac{\partial N_i^e}{\partial \xi} \frac{\partial \xi}{\partial z} + \frac{\partial N_i^e}{\partial \eta} \frac{\partial \eta}{\partial z} + \frac{\partial N_i^e}{\partial \mu} \frac{\partial \mu}{\partial z}
 \end{aligned} \tag{11.129}$$

In matrix form:

$$\begin{bmatrix} \frac{\partial N_i^e}{\partial x} \\ \frac{\partial N_i^e}{\partial y} \\ \frac{\partial N_i^e}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{\partial \xi}{\partial x} & \frac{\partial \eta}{\partial x} & \frac{\partial \mu}{\partial x} \\ \frac{\partial \xi}{\partial y} & \frac{\partial \eta}{\partial y} & \frac{\partial \mu}{\partial y} \\ \frac{\partial \xi}{\partial z} & \frac{\partial \eta}{\partial z} & \frac{\partial \mu}{\partial z} \end{bmatrix} \begin{bmatrix} \frac{\partial N_i^e}{\partial \xi} \\ \frac{\partial N_i^e}{\partial \eta} \\ \frac{\partial N_i^e}{\partial \mu} \end{bmatrix} \tag{11.130}$$

The 3×3 matrix above is $\mathbf{J}^{-1}$, the inverse of:

$$\mathbf{J} = \frac{\partial (x, y, z)}{\partial (\xi, \eta, \mu)} = \begin{bmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial y}{\partial \xi} & \frac{\partial z}{\partial \xi} \\ \frac{\partial x}{\partial \eta} & \frac{\partial y}{\partial \eta} & \frac{\partial z}{\partial \eta} \\ \frac{\partial x}{\partial \mu} & \frac{\partial y}{\partial \mu} & \frac{\partial z}{\partial \mu} \end{bmatrix} \tag{11.131}$$

Matrix $\mathbf{J}$ is called the *Jacobian matrix* of (x, y, z) with respect to (ξ, η, μ) . In the finite element literature, matrices $\mathbf{J}$ and $\mathbf{J}^{-1}$ are called simply the *Jacobian* and *inverse Jacobian*, respectively, although such a short name is sometimes ambiguous. The notation

$$\begin{aligned}\mathbf{J} &= \frac{\partial (x, y, z)}{\partial (\xi, \eta, \mu)} \\ \mathbf{J}^{-1} &= \frac{\partial (\xi, \eta, \mu)}{\partial (x, y, z)}\end{aligned}\tag{11.132}$$

is standard in multivariable calculus and suggests that the Jacobian may be viewed as a generalisation of the ordinary derivative to which it reduces for a scalar function $\mathbf{x} = x(\xi)$.

Computing the Jacobian Matrix

The isoparametric definition of hexahedron element geometry is:

$$\begin{aligned}x &= x_i N_i^e \\ y &= y_i N_i^e \\ z &= z_i N_i^e\end{aligned}\tag{11.133}$$

where the summation convention is understood to apply over $i = 1, 2, \dots, n$, in which n denotes the number of element nodes.

Note: This for a given hexahedral element gives an $n \times 3$ matrix.

Differentiating these relations with respect to the hexahedron coordinates we construct the matrix $\mathbf{J}$ as follows:

$$\mathbf{J} = \begin{bmatrix} x_i \frac{\partial N_i^e}{\partial \xi} & y_i \frac{\partial N_i^e}{\partial \xi} & z_i \frac{\partial N_i^e}{\partial \xi} \\ x_i \frac{\partial N_i^e}{\partial \eta} & y_i \frac{\partial N_i^e}{\partial \eta} & z_i \frac{\partial N_i^e}{\partial \eta} \\ x_i \frac{\partial N_i^e}{\partial \mu} & y_i \frac{\partial N_i^e}{\partial \mu} & z_i \frac{\partial N_i^e}{\partial \mu} \end{bmatrix} \quad (11.134)$$

or:

$$\mathbf{J}_i = \begin{bmatrix} \frac{\partial N_1^e}{\partial \xi} & \frac{\partial N_2^e}{\partial \xi} & \cdots & \frac{\partial N_n^e}{\partial \xi} \\ \frac{\partial N_1^e}{\partial \eta} & \frac{\partial N_2^e}{\partial \eta} & \cdots & \frac{\partial N_n^e}{\partial \eta} \\ \frac{\partial N_1^e}{\partial \mu} & \frac{\partial N_2^e}{\partial \mu} & \cdots & \frac{\partial N_n^e}{\partial \mu} \end{bmatrix} \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_n & y_n & z_n \end{bmatrix} \quad (11.135)$$

Given a point of hexahedron coordinates (ξ, η, μ) the Jacobian $\mathbf{J}$ can be easily formed using the above formula, and numerically inverted to form $\mathbf{J}^{-1}$.

The Strain-Displacement Matrix

Having obtained the shape function derivatives, the matrix $\mathbf{B}$ for hexahedron element displays the usual structure for 3D elements:

$$\mathbf{B} = \mathbf{D}\Phi = \begin{bmatrix} \frac{\partial}{\partial x} & 0 & 0 \\ 0 & \frac{\partial}{\partial y} & 0 \\ 0 & 0 & \frac{\partial}{\partial z} \\ \frac{\partial}{\partial y} & \frac{\partial}{\partial x} & 0 \\ 0 & \frac{\partial}{\partial z} & \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} & 0 & \frac{\partial}{\partial x} \end{bmatrix} \begin{bmatrix} \mathbf{q} & 0 & 0 \\ 0 & \mathbf{q} & 0 \\ 0 & 0 & \mathbf{q} \end{bmatrix} = \begin{bmatrix} \mathbf{q}_x & 0 & 0 \\ 0 & \mathbf{q}_y & 0 \\ 0 & 0 & \mathbf{q}_z \\ \mathbf{q}_y & \mathbf{q}_x & 0 \\ 0 & \mathbf{q}_z & \mathbf{q}_y \\ \mathbf{q}_z & 0 & \mathbf{q}_x \end{bmatrix} \quad (11.136)$$

where:

$$\begin{aligned} \mathbf{q} &= [N_1^e \quad \cdots \quad N_n^e] \\ \mathbf{q}_x &= \left[\frac{\partial N_1^e}{\partial x} \quad \cdots \quad \frac{\partial N_n^e}{\partial x} \right] \\ \mathbf{q}_y &= \left[\frac{\partial N_1^e}{\partial y} \quad \cdots \quad \frac{\partial N_n^e}{\partial y} \right] \\ \mathbf{q}_z &= \left[\frac{\partial N_1^e}{\partial z} \quad \cdots \quad \frac{\partial N_n^e}{\partial z} \right] \end{aligned} \quad (11.137)$$

are row vectors of length n , n being the number of nodes in the element.

Stiffness Matrix Evaluation

The element stiffness Matrix is given by:

$$\mathbf{K}^e = \int_{V^e} \mathbf{B}^T \mathbf{E} \mathbf{B} dV^e \quad (11.138)$$

As in two-dimensional case, this is replaced by a numerical integration formula which now involves a triple loop over conventional Gauss quadrature rules. Assuming that the stress-strain matrix $\mathbf{E}$ is constant over the element,

$$\mathbf{K}^e = \sum_{i=1}^{p_1} \sum_{j=1}^{p_2} \sum_{k=1}^{p_3} w_i w_j w_k \mathbf{B}_{ijk}^T \mathbf{E} \mathbf{B}_{ijk} \mathbf{J}_{ijk} \quad (11.139)$$

Here p_1 , p_2 and p_3 are the number of Gauss points in the ξ , η and μ direction, respectively, while $\mathbf{B}_{ijk}$ and $\mathbf{J}_{ijk}$ are abbreviations for:

$$\begin{aligned} \mathbf{B}_{ijk} &= \mathbf{B}(\xi_i, \eta_i, \mu_i) \\ \mathbf{J}_{ijk} &= \det \mathbf{J}(\xi_i, \eta_i, \mu_i) \end{aligned} \quad (11.140)$$

Usually the number of integration points is taken the same in all directions: $p = p_1 = p_2 = p_3$. The total number of Gauss points is thus p^3 . Each point adds at most 6 to the stiffness matrix rank. The minimum rank-sufficient rules for the 8-node and 20-node hexahedra are $p = 2$ and $p = 3$, respectively.

11.5.2 Python implementation

```

1 #!/usr/bin/python3
2
3 import numpy as np
4 np.set_printoptions(precision=2) # , suppress=True)
5
```

```

6 def hex8_stiffness_mass_load(coors: np.ndarray,
7                               E: float,
8                               nu: float,
9                               rho: float,
10                              fx: float = 0.,
11                              fy: float = 0.,
12                              fz: float = 0.):
13     """
14     In:
15         coors - 8 x 3 coordinate matrix
16                [[x1,y1,z1], ... , [xn,yn,zn]]
17         E     - Youngs's Modulus
18         nu    - Poisson's Constant
19         rho   - Density
20         fx    - Volumetric Load in x direction
21         fy    - Volumetric Load in y direction
22         fz    - Volumetric Load in z direction
23     """
24     domain = np.array([[-1., -1., -1.],
25                       [1., -1., -1.],
26                       [1., 1., -1.],
27                       [-1., 1., -1.],
28                       [-1., -1., 1.],
29                       [1., -1., 1.],
30                       [1., 1., 1.],
31                       [-1., 1., 1.]], dtype=float)
32     print('Domain: {0}\n{1}'.format(domain.shape, domain))
33
34     # gaussian interpolation
35     gauss_points = 3
36     g_points, g_weights = np.polynomial.legendre.leggauss(gauss_points)
37     integration_points = []
38     interpolation_weights = []
39     for i, xi in enumerate(g_points):
40         for j, eta in enumerate(g_points):
41             for k, mu in enumerate(g_points):
42                 integration_points.append([xi, eta, mu])
43                 interpolation_weights.append(g_weights[i] *
44                                             g_weights[j] *
45                                             g_weights[k])
46     integration_points = np.array(integration_points, dtype=float)

```

```

47 interpolation_weights = np.array(interpolation_weights, dtype=float)
48 print('Gauss Integration '
49       '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
50                                           integration_points,
51                                           interpolation_weights))
52
53 full_domain = np.vstack((domain, integration_points))
54
55 # Shape Functions in Natural Coordinates
56 xi = integration_points.T[0]
57 eta = integration_points.T[1]
58 mu = integration_points.T[2]
59 psi = np.zeros((8, integration_points.shape[0]), dtype=float)
60 for i in range(8):
61     psi[i] = 1/8 * (1 + xi * domain[i, 0]) *
62                  (1 + eta * domain[i, 1]) *
63                  (1 + mu * domain[i, 2])
64
65 psi = psi.T
66
67 print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
68
69 # Shape Functions in Global Coordinates
70 psi_g = psi @ coors
71
72 print('Shape Functions in '
73       'Global Coordinates: {0}\n{1}'.format(psi_g.shape, psi_g))
74
75 # Shape Functions Derivatives in Natural Coordinates
76 dps_i = 1 / 8 * np.array([[(eta - 1.0) * (1.0 - mu), # 1
77                             (xi - 1) * (1 - mu),
78                             -(1 - xi) * (1 - eta)],
79                             [(1 - eta) * (1 - mu), # 2
80                             (-1 - xi) * (1 - mu),
81                             -(1 + xi) * (1 - eta)],
82                             [(1 + eta) * (1 - mu), # 3
83                             (1 + xi) * (1 - mu),
84                             -(1 + xi) * (1 + eta)],
85                             [(-1.0 - eta) * (1 - mu), # 4
86                             (1 - xi) * (1 - mu),
87                             -(1 - xi) * (1 + eta)],
88                             [(1 - eta) * (-1 - mu), # 5
89                             -(1 - xi) * (1 + mu),
90                             (1 - xi) * (1 - eta)],
91                             [(-1.0 - eta) * (-1 - mu), # 6
92                             (-1 - xi) * (1 + mu),
93                             (1 + xi) * (1 - eta)],
94                             [(1 + eta) * (-1 - mu), # 7
95                             (1 + xi) * (1 + mu),
96                             (1 + xi) * (1 + eta)],
97                             [(1 - eta) * (-1 - mu), # 8
98                             (-1 - xi) * (1 - mu),
99                             (1 - xi) * (1 - eta)]]

```

```

88             [(1 - eta) * (1 + mu),      # 6
89             -(1 + xi) * (1 + mu),
90             (1 + xi) * (1 - eta)],
91             [(1 + eta) * (1 + mu),      # 7
92             (1 + xi) * (1 + mu),
93             (1 + xi) * (1 + eta)],
94             [-(1 + eta) * (1 + mu),     # 8
95             (1 - xi) * (1 + mu),
96             (1 - xi) * (1 + eta)]]
97     dps_i = dps_i.T
98     print('Shape Functions Derivatives: '
99           '{0}\n{1}'.format(dps_i.shape, dps_i))
100
101     # Jacobian Matrix
102     jacob_i = dps_i @ coors
103     print('Jacobian Matrix: {0}\n{1}'.format(jacob_i.shape, jacob_i))
104
105     # Jacobian Determinants
106     d_jacob_i = np.linalg.det(jacob_i)
107     print('Determinant of Jacobian: '
108           '{0}\n{1}'.format(d_jacob_i.shape, d_jacob_i))
109
110     # Inverse Jacobian
111     i_jacob_i = np.linalg.inv(jacob_i)
112     print('Inverse Jacobian Matrix: '
113           '{0}\n{1}'.format(i_jacob_i.shape, i_jacob_i))
114
115     # Shape Function Derivatives in Global Coordinates
116     dps_g = i_jacob_i @ dps_i
117     print('Shape Function Derivatives in Global Coordinates: '
118           '{0}\n{1}'.format(dps_g.shape, dps_g))
119
120     # Material Stiffness Matrix
121     C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
122         np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
123                 [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
124                 [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
125                 [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
126                 [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
127                 [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
128                 dtype=float)

```

```

129     print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
130
131     # Create Element Stiffness and Mass Matrix
132     Ke = np.zeros((domain.size, domain.size), dtype=float)
133     Me = np.zeros((domain.size, domain.size), dtype=float)
134     Fe = np.zeros((domain.size, 1), dtype=float)
135     # o = np.zeros(domain.shape[0], dtype=float)
136
137     # iterate over Gauss points of domain, * means unpack values
138     for i in range(integration_points.shape[0]):
139         # this is smart but arranges dofs by component, not by node
140         # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
141         # B = np.array([
142         #     [*dpsi_g[i, 0, :], *o, *o],
143         #     [*o, *dpsi_g[i, 1, :], *o],
144         #     [*o, *o, *dpsi_g[i, 2, :]],
145         #     [*dpsi_g[i, 2, :], *o, *dpsi_g[i, 0, :]],
146         #     [*o, *dpsi_g[i, 2, :], *dpsi_g[i, 1, :]],
147         #     [*dpsi_g[i, 1, :], *dpsi_g[i, 0, :], *o]], dtype=float)
148
149         # dofs ordered by node
150         # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
151         dpx = dpsi_g[i,0,:]
152         dpy = dpsi_g[i,1,:]
153         dpz = dpsi_g[i,2,:]
154         # B = np.array(
155         #
156         ↪ [[dpx[0],0,0,dpx[1],0,0,dpx[2],0,0,dpx[3],0,0,dpx[4],0,0,dpx[5],0,0,dpx[6],0,0,dpx[7]
157         #
158         ↪ [0,dpy[0],0,0,dpy[1],0,0,dpy[2],0,0,dpy[3],0,0,dpy[4],0,0,dpy[5],0,0,dpy[6],0,0,dpy[7]
159         #
160         ↪ [0,0,dpz[0],0,0,dpz[1],0,0,dpz[2],0,0,dpz[3],0,0,dpz[4],0,0,dpz[5],0,0,dpz[6],0,0,dpz[7]
161         #
162         ↪ [dpy[0],dpx[0],0,dpy[1],dpx[1],0,dpy[2],dpx[2],0,dpy[3],dpx[3],0,dpy[4],dpx[4],0,dpy[5],dpx[5],0,
163         ↪ [0,dpz[0],dpy[0],0,dpz[1],dpy[1],0,dpz[2],dpy[2],0,dpz[3],dpy[3],0,dpz[4],dpy[4],0,dpz[5],dpy[5],0,
164         #
165         ↪ [dpz[0],0,dpx[0],dpz[1],0,dpx[1],dpz[2],0,dpx[2],dpz[3],0,dpx[3],dpz[4],0,dpx[4],dpz[5],0,dpx[5],dpz[6],0,dpx[6],dpz[7],0,dpx[7]]
166         #
167         dtype=_FLOAT)
168     B = np.array([np.column_stack((dpx, o, o)).flatten(),
169                  np.column_stack((o, dpy, o)).flatten(),

```

```

164         np.column_stack(( o,   o, dpz)).flatten(),
165         np.column_stack((dpy, dpx,  o)).flatten(),
166         np.column_stack(( o, dpz, dpy)).flatten(),
167         np.column_stack((dpz,   o, dpx)).flatten()], dtype=float)
168
169     # this is smart but arranges dofs by component, not by node
170     # component-wise dof ordering (x1, .. , y1, .. y4, z1, .. , z4)
171     # N = np.array([
172     #     [*psi[i], *o, *o],
173     #     [*o, *psi[i], *o],
174     #     [*o, *o, *psi[i]]],
175     #     dtype=float)
176
177     # node wise dof ordering (x1, y1, z1, .. , x4, y4, z4)
178     # N =
179     ↪ np.array([[p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7],0,0],
180     #
181     ↪ [0,p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7],0],
182     #
183     ↪ [0,0,p[0],0,0,p[1],0,0,p[2],0,0,p[3],0,0,p[4],0,0,p[5],0,0,p[6],0,0,p[7]]],
184     #
185     dtype=_FLOAT)
186     N = np.array([np.column_stack((p, o, o)).flatten(),
187         np.column_stack((o, p, o)).flatten(),
188         np.column_stack((p, o, o)).flatten()], dtype=float)
189
190     F = np.array([[fx], [fy], [fz]], dtype=float)
191
192     Ke += (B.T @ C @ B) * d_jacobi[i] * interpolation_weights[i]
193     Me += rho * (N.T @ N) * d_jacobi[i] * interpolation_weights[i]
194     Fe += (N.T @ F) * d_jacobi[i] * interpolation_weights[i]
195
196     print('Element Stiffness Matrix: {0}\n{1}'.format(Ke.shape, Ke))
197     print('Element Mass Matrix: {0}\n{1}'.format(Me.shape, Me))
198     print('Element Volume Force Vector: {0}\n{1}'.format(Fe.shape, Fe))

```

And to get stresses:

```

1 #!/usr/bin/python3

```

```

2
3 import numpy as np
4 np.set_printoptions(precision=2) # , suppress=True)
5
6 def hex8_stresses(coors: np.ndarray,
7                  disp: np.ndarray,
8                  E: float,
9                  nu: float):
10     """
11     In:
12         coors - 8 x 3 coordinate matrix
13              [[x1,y1,z1], ... , [xn,yn,zn]]
14         disp - 8 x 3 coordinate matrix
15              [[u1,v1,w1], ... , [un,vn,wn]]
16         E     - Youngs's Modulus
17         nu    - Poisson's Constant
18     """
19     domain = np.array([[ -1., -1., -1.],
20                       [ 1., -1., -1.],
21                       [ 1., 1., -1.],
22                       [-1., 1., -1.],
23                       [-1., -1., 1.],
24                       [ 1., -1., 1.],
25                       [ 1., 1., 1.],
26                       [-1., 1., 1.]], dtype=float)
27     print('Domain: {0}\n{1}'.format(domain.shape, domain))
28
29     # gaussian interpolation
30     gauss_points = 3
31     g_points, g_weights = np.polynomial.legendre.leggauss(gauss_points)
32     integration_points = []
33     interpolation_weights = []
34     for i, xi in enumerate(g_points):
35         for j, eta in enumerate(g_points):
36             for k, mu in enumerate(g_points):
37                 integration_points.append([xi, eta, mu])
38                 interpolation_weights.append(g_weights[i] *
39                                           g_weights[j] *
40                                           g_weights[k])
41     integration_points = np.array(integration_points, dtype=float)
42     interpolation_weights = np.array(interpolation_weights, dtype=float)

```



```

43 print('Gauss Integration '
44       '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
45                                           integration_points,
46                                           interpolation_weights))
47
48 full_domain = np.vstack((domain, integration_points))
49
50 # Shape Functions in Natural Coordinates
51 xi = integration_points.T[0]
52 eta = integration_points.T[1]
53 mu = integration_points.T[2]
54 psi = np.zeros((8, integration_points.shape[0]), dtype=float)
55 for i in range(8):
56     psi[i] = 1/8 * (1 + xi * domain[i, 0]) *
57                   (1 + eta * domain[i, 1]) *
58                   (1 + mu * domain[i, 2])
59
60 psi = psi.T
61 print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
62
63 # Shape Functions in Global Coordinates
64 psi_g = psi @ coors
65 print('Shape Functions in '
66       'Global Coordinates: {0}\n{1}'.format(psi_g.shape, psi_g))
67
68 # Shape Functions Derivatives in Natural Coordinates
69 dps1 = 1 / 8 * np.array([[ (eta - 1.0) * (1.0 - mu), # 1
70                           (xi - 1) * (1 - mu),
71                           -(1 - xi) * (1 - eta)],
72                          [(1 - eta) * (1 - mu), # 2
73                           (-1 - xi) * (1 - mu),
74                           -(1 + xi) * (1 - eta)],
75                          [(1 + eta) * (1 - mu), # 3
76                           (1 + xi) * (1 - mu),
77                           -(1 + xi) * (1 + eta)],
78                          [(-1.0 - eta) * (1 - mu), # 4
79                           (1 - xi) * (1 - mu),
80                           -(1 - xi) * (1 + eta)],
81                          [(1 - eta) * (-1 - mu), # 5
82                           -(1 - xi) * (1 + mu),
83                           (1 - xi) * (1 - eta)],
84                          [(1 - eta) * (1 + mu), # 6

```

```

84         -(1 + xi) * (1 + mu),
85         (1 + xi) * (1 - eta)],
86     [(1 + eta) * (1 + mu),      # 7
87      (1 + xi) * (1 + mu),
88      (1 + xi) * (1 + eta)],
89     [-(1 + eta) * (1 + mu),    # 8
90      (1 - xi) * (1 + mu),
91      (1 - xi) * (1 + eta)]]
92
93 dpsi = dpsi.T
94 print('Shape Functions Derivatives: '
95       '{0}\n{1}'.format(dpsi.shape, dpsi))
96
97 # Jacobian Matrix
98 jacobi = dpsi @ coors
99 print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))
100
101 # Jacobian Determinants
102 d_jacobi = np.linalg.det(jacobi)
103 print('Determinant of Jacobian: '
104       '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))
105
106 # Inverse Jacobian
107 i_jacobi = np.linalg.inv(jacobi)
108 print('Inverse Jacobian Matrix: '
109       '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))
110
111 # Shape Function Derivatives in Global Coordinates
112 dpsi_g = i_jacobi @ dpsi
113 print('Shape Function Derivatives in Global Coordinates: '
114       '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))
115
116 # Material Stiffness Matrix
117 C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
118     np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
119              [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
120              [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
121              [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
122              [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
123              [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
124             dtype=float)
125 print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))

```

```
125
126     du = disp.T @ np.transpose(dpsi_g, axes=[0, 2, 1])
127
128     exx = du[:, 0, 0]
129     eyy = du[:, 1, 1]
130     ezz = du[:, 2, 2]
131     exy = du[:, 0, 1] + du[:, 1, 0]
132     eyz = du[:, 1, 2] + du[:, 2, 1]
133     exz = du[:, 0, 2] + du[:, 2, 0]
134     epsilons = np.array([exx, eyy, ezz, exy, eyz, exz])
135
136     sigmas = (C @ epsilons).T
137     epsilons = epsilons.T
138     print('Element Strains: {0}\n{1}'.format(epsilons.shape, epsilons))
139     print('Element Stresses: {0}\n{1}'.format(stresses.shape, stresses))
```

11.6 TET4 Element

11.6.1 Element Formulation

For a linear tetrahedron no numerical integration is needed (there can be only one integration point, shape functions derivatives are equal to 1).

The linear tetrahedron is not often used for stress analysis because of its poor performance. Its main value in structural and solid mechanics is educational: it serves as a vehicle to introduce the basic steps of formulation of 3D solid elements, particularly with regards to the use of natural coordinate systems and node numbering conventions. It should be noted that 3D visualisation is notoriously more difficult than 2D, so it is needed to procede more slowly here.

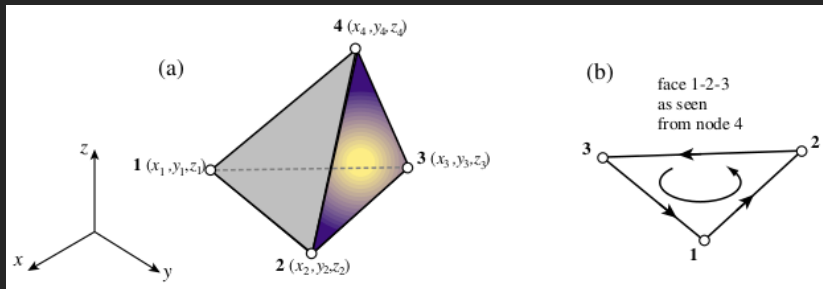


Figure 11.3: (a) The linear tetrahedron element TET4, (b) Node numbering convention.

Tetrahedron Geometry

The tetrahedron geometry is fully defined by giving the location of the four corner nodes with respect to Right-Hand Coordinate Sysetm notation (x, y, z) :


```

38
39     elif gauss_points == 5:
40         integration_points = np.array([[1/4, 1/4, 1/4, 1/4],
41                                       [1/2, 1/6, 1/6, 1/6],
42                                       [1/6, 1/2, 1/6, 1/6],
43                                       [1/6, 1/6, 1/2, 1/6],
44                                       [1/6, 1/6, 1/6, 1/2]], dtype=float)
45         integration_weights = np.array([-4/5, 9/20, 9/20, 9/20, 9/20],
46                                       dtype=float) * 1/6
47
48     print('Gauss Integration '
49           '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
50                                             integration_points,
51                                             integration_weights))
52
53     full_domain = np.vstack((domain, integration_points))
54
55     # Shape Functions in Natural Coordinates
56     xi = integration_points.T[0]
57     eta = integration_points.T[1]
58     mu = integration_points.T[2]
59     # zeta is not used to enforce
60     # (1 - xi - eta - mu - zeta) = 0
61     # zeta = integration_points.T[3]
62     psi = np.zeros((4, integration_points.shape[0]), dtype=float)
63     # another way of writing that psi = [xi, eta, mu, zeta]
64     # for i in range(4):
65     #     psi[i] = (1 + xi * domain[i, 0]) * (1 + eta * domain[i, 1]) * (1 + mu * domain[i, 2]) * (1 + zeta * domain[i, 3])
66     psi[0] = xi
67     psi[1] = eta
68     psi[2] = mu
69     psi[3] = 1 - xi - eta - mu # = zeta
70     psi = psi.T
71     print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
72
73     # Shape Functions in Global Coordinates
74     psi_g = psi @ coors
75     print('Shape Functions in Global Coordinates: '
76           '{0}\n{1}'.format(psi_g.shape, psi_g))
77

```

```

78  # Shape Functions Derivatives in Natural Coordinates
79  dpsi = np.zeros((integration_points.shape[0], 4, 3), dtype=float)
80  dpsi[:,0,0] = 1
81  dpsi[:,1,1] = 1
82  dpsi[:,2,2] = 1
83  dpsi[:,3,:] = -1
84  dpsi = dpsi.transpose((0, 2, 1))
85  print('Shape Functions Derivatives: '
86        '{0}\n{1}'.format(dpsi.shape, dpsi))
87
88  # Jacobian Matrix
89  jacobi = dpsi @ coors
90  print('Jacobian Matrix: {0}\n{1}'.format(jacobi.shape, jacobi))
91
92  # Jacobian Determinants
93  d_jacobi = np.linalg.det(jacobi)
94  print('Determinant of Jacobian: '
95        '{0}\n{1}'.format(d_jacobi.shape, d_jacobi))
96
97  # Inverse Jacobian
98  i_jacobi = np.linalg.inv(jacobi)
99  print('Inverse Jacobian Matrix: '
100        '{0}\n{1}'.format(i_jacobi.shape, i_jacobi))
101
102  # Shape Function Derivatives in Global Coordinates
103  dpsi_g = i_jacobi @ dpsi
104  print('Shape Function Derivatives in Global Coordinates: '
105        '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))
106
107  # Material Stiffness Matrix
108  C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
109      np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
110               [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
111               [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
112               [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
113               [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
114               [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
115               dtype=float)
116  print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
117
118  # Create Element Stiffness and Mass Matrix

```

```

119 Ke = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
120 Me = np.zeros((domain.shape[0] * 3, domain.shape[0] * 3), dtype=float)
121 Fe = np.zeros((domain.shape[0] * 3, 1), dtype=float)
122 o = np.zeros(domain.shape[0], dtype=float)
123
124 # iterate over Gauss points of domain, * means unpack values
125 for i in range(integration_points.shape[0]):
126     # component-wise dof ordering (x1, .., y1, .. y4, z1, .., z4)
127     # B = np.array([[*dpsi_g[i, 0, :], *o, *o],
128     #               [*o, *dpsi_g[i, 1, :], *o],
129     #               [*o, *o, *dpsi_g[i, 2, :]],
130     #               [*dpsi_g[i, 2, :], *o, *dpsi_g[i, 0, :]],
131     #               [*o, *dpsi_g[i, 2, :], *dpsi_g[i, 1, :]],
132     #               [*dpsi_g[i, 1, :], *dpsi_g[i, 0, :], *o]],
133     #               dtype=float)
134
135     # node wise dof ordering (x1, y1, z1, .., x4, y4, z4)
136     dpx = dpsi_g[i,0,:]
137     dpy = dpsi_g[i,1,:]
138     dpz = dpsi_g[i,2,:]
139     B = np.array(
140         [[dpx[0],0,0,dpx[1],0,0,dpx[2],0,0,dpx[3],0,0],
141          [0,dpy[0],0,0,dpy[1],0,0,dpy[2],0,0,dpy[3],0],
142          [0,0,dpz[0],0,0,dpz[1],0,0,dpz[2],0,0,dpz[3]],
143          [dpy[0],dpx[0],0,dpy[1],dpx[1],0,dpy[2],dpx[2],0,dpy[3],dpx[3],0],
144          [0,dpz[0],dpy[0],0,dpz[1],dpy[1],0,dpz[2],dpy[2],0,dpz[3],dpy[3]],
145          [dpz[0],0,dpx[0],dpz[1],0,dpx[1],dpz[2],0,dpx[2],dpz[3],0,dpx[3]],
146          dtype=float)
147
148     # component-wise dof ordering (x1, .., y1, .. y4, z1, .., z4)
149     # N = np.array([[*psi[i], *o, *o],
150     #               [*o, *psi[i], *o],
151     #               [*o, *o, *psi[i]]], dtype=float)
152
153     # node wise dof ordering (x1, y1, z1, .., x4, y4, z4)
154     N = np.array([[psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3],0,0],
155                  [0,psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3],0],
156                  [0,0,psi[i,0],0,0,psi[i,1],0,0,psi[i,2],0,0,psi[i,3]],
157                  dtype=float)
158
159     F = np.array([[fx], [fy], [fz]], dtype=float)

```



```

160
161     print((B.T @ C @ B).shape)
162     print(Ke.shape)
163     Ke += (B.T @ C @ B) * d_jacobi[i] * integration_weights[i]
164     Me += rho * (N.T @ N) * d_jacobi[i] * integration_weights[i]
165     Fe += (N.T @ F) * d_jacobi[i] * integration_weights[i]
166
167     print('Element Stiffness Matrix: {0}\n{1}'.format(Ke.shape, Ke))
168     print('Element Mass Matrix: {0}\n{1}'.format(Me.shape, Me))
169     print('Element Volume Force Vector: {0}\n{1}'.format(Fe.shape, Fe))

```

And to get stresses:

```

1  #!/usr/bin/python3
2
3  import numpy as np
4  np.set_printoptions(precision=2) # , suppress=True)
5
6  def tet4_stresses(coors: np.ndarray,
7                   disp: np.ndarray,
8                   E: float,
9                   nu: float):
10
11     """
12     In:
13         coors - 4 x 3 coordinate matrix
14                [[x1,y1,z1], ... , [xn,yn,zn]]
15         disp - 4 x 3 coordinate matrix
16                [[u1,v1,w1], ... , [un,vn,wn]]
17         E     - Youngs's Modulus
18         nu    - Poisson's Constant
19     """
20     domain = np.array([[1., 0., 0., 0.],
21                       [0., 1., 0., 0.],
22                       [0., 0., 1., 0.],
23                       [0., 0., 0., 1.]], dtype=float)
24     print('Domain: {0}\n{1}'.format(domain.shape, domain))
25
26     # gaussian interpolation

```

```

26     integration_points = None
27     integration_weights = None
28     if gauss_points == 1:
29         integration_points = np.array([1/4, 1/4, 1/4, 1/4],
30                                       dtype=float).reshape(1, 4)
31         integration_weights = np.array([1 * 1 * 1 * 1],
32                                       dtype=float) * 1/6
33
34     elif gauss_points == 4:
35         a = 0.58541020
36         b = 0.13819660
37         integration_points = np.array([[a, b, b, b],
38                                       [b, a, b, b],
39                                       [b, b, a, b],
40                                       [b, b, b, a]], dtype=float)
41         integration_weights = np.array([1/4, 1/4, 1/4, 1/4],
42                                       dtype=float) * 1/6
43
44     elif gauss_points == 5:
45         integration_points = np.array([[1/4, 1/4, 1/4, 1/4],
46                                       [1/2, 1/6, 1/6, 1/6],
47                                       [1/6, 1/2, 1/6, 1/6],
48                                       [1/6, 1/6, 1/2, 1/6],
49                                       [1/6, 1/6, 1/6, 1/2]], dtype=float)
50         integration_weights = np.array([-4/5, 9/20, 9/20, 9/20, 9/20],
51                                       dtype=float) * 1/6
52
53     print('Gauss Integration '
54           '{0}:\n{1}\nWeights:\n{2}'.format(gauss_points,
55                                               integration_points,
56                                               integration_weights))
57
58     full_domain = np.vstack((domain, integration_points))
59
60     # Shape Functions in Natural Coordinates
61     xi = integration_points.T[0]
62     eta = integration_points.T[1]
63     mu = integration_points.T[2]
64     # zeta is not used to enforce
65     # (1 - xi - eta - mu - zeta) = 0
66     # zeta = integration_points.T[3]

```

```

67  psi = np.zeros((4, integration_points.shape[0]), dtype=float)
68  # another way of writing that psi = [xi, eta, mu, zeta]
69  # for i in range(4):
70  #     psi[i] = (1 + xi * domain[i, 0]) * (1 + eta * domain[i, 1]) * (1 + mu *
71  #         ↪ domain[i, 2]) * (1 + zeta * domain[i, 3])
72  psi[0] = xi
73  psi[1] = eta
74  psi[2] = mu
75  psi[3] = 1 - xi - eta - mu # = zeta
76  psi = psi.T
77  print('Shape Functions: {0}\n{1}'.format(psi.shape, psi))
78
79  # Shape Functions in Global Coordinates
80  psi_g = psi @ coors
81  print('Shape Functions in Global Coordinates: '
82  #     '{0}\n{1}'.format(psi_g.shape, psi_g))
83
84  # Shape Functions Derivatives in Natural Coordinates
85  dps = np.zeros((integration_points.shape[0], 4, 3), dtype=float)
86  dps[:,0,0] = 1
87  dps[:,1,1] = 1
88  dps[:,2,2] = 1
89  dps[:,3,:] = -1
90  dps = dps.transpose((0, 2, 1))
91  print('Shape Functions Derivatives: '
92  #     '{0}\n{1}'.format(dps.shape, dps))
93
94  # Jacobian Matrix
95  jacob = dps @ coors
96  print('Jacobian Matrix: {0}\n{1}'.format(jacob.shape, jacob))
97
98  # Jacobian Determinants
99  d_jacob = np.linalg.det(jacob)
100 print('Determinant of Jacobian: '
101 #     '{0}\n{1}'.format(d_jacob.shape, d_jacob))
102
103 # Inverse Jacobian
104 i_jacob = np.linalg.inv(jacob)
105 print('Inverse Jacobian Matrix: '
106 #     '{0}\n{1}'.format(i_jacob.shape, i_jacob))

```

```

107     # Shape Function Derivatives in Global Coordinates
108     dpsi_g = i_jacobi @ dpsi
109     print('Shape Function Derivatives in Global Coordinates: '
110           '{0}\n{1}'.format(dpsi_g.shape, dpsi_g))
111
112     # Material Stiffness Matrix
113     C = E / ((1.0 + nu) * (1.0 - 2.0 * nu)) *
114         np.array([[1.0 - nu, nu, nu, 0.0, 0.0, 0.0],
115                 [nu, 1.0 - nu, nu, 0.0, 0.0, 0.0],
116                 [nu, nu, 1.0 - nu, 0.0, 0.0, 0.0],
117                 [0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0, 0.0],
118                 [0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0, 0.0],
119                 [0.0, 0.0, 0.0, 0.0, 0.0, (1.0 - 2.0 * nu) / 2.0]],
120                 dtype=float)
121     print('Material Stiffness Matrix: {0}\n{1}'.format(C.shape, C))
122     du = disp.T @ np.transpose(dpsi_g, axes=[0, 2, 1])
123
124     exx = du[:, 0, 0]
125     eyy = du[:, 1, 1]
126     ezz = du[:, 2, 2]
127     exy = du[:, 0, 1] + du[:, 1, 0]
128     eyz = du[:, 1, 2] + du[:, 2, 1]
129     exz = du[:, 0, 2] + du[:, 2, 0]
130     epsilons = np.array([exx, eyy, ezz, exy, eyz, exz])
131
132     sigmas = (C @ epsilons).T
133     epsilons = epsilons.T
134     print('Element Strains: {0}\n{1}'.format(epsilons.shape, epsilons))
135     print('Element Stresses: {0}\n{1}'.format(stresses.shape, stresses))

```

Chapter 12

Numerics

12.1 Numerical Spaces

The vectors used in numerical methods do not always consist of a finite list of numbers. In the finite element method, for example, the unknown displacement is first considered as a function over the body and only later approximated by a finite set of nodal values. A *space* specifies which objects are admitted as vectors, which operations can be performed on them and how their size or mutual distance is measured.

The spaces introduced below form a useful hierarchy. A **Banach space** is complete with respect to a norm. A **Hilbert space** is a Banach space whose norm is generated by an inner product. A **Sobolev space** is a function space in which the function and a prescribed number of its weak derivatives are integrable. For the exponent $p = 2$, a Sobolev space is also a Hilbert space. A **Krylov space**, on the other hand, is a finite-dimensional subspace generated from a matrix and a starting vector during an iterative calculation.

12.1.1 Hilbert Space

A **Hilbert space** is an inner product space that is complete with respect to the norm induced by its inner product. The definition contains two distinct ideas:

1. The *inner product* $\langle u, v \rangle$ measures both the length of vectors and the angle between them.
2. *Completeness* means that a converging sequence of vectors does not leave the space. More precisely, every Cauchy sequence in the space has a limit that also belongs to the space.

The inner product induces the norm:

$$\|u\| = \sqrt{\langle u, u \rangle}. \quad (12.1)$$

The familiar Euclidean space $\mathbb{R}^n$ with

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v} \quad (12.2)$$

is a finite-dimensional Hilbert space. In numerical mechanics the more important examples are often spaces of functions. The space $L^2(\Omega)$ consists of functions whose square is integrable over the domain Ω . Its inner product is:

$$\langle u, v \rangle_{L^2(\Omega)} = \int_{\Omega} u(\mathbf{x})v(\mathbf{x}) \, d\Omega, \quad (12.3)$$

and the corresponding norm is:

$$\|u\|_{L^2(\Omega)} = \sqrt{\int_{\Omega} |u(\mathbf{x})|^2 \, d\Omega}. \quad (12.4)$$

Completeness is important because numerical methods construct sequences of approximations. If these approximations converge in the chosen norm, completeness guarantees that their limit is still an admissible member of the space.

Without this property a method could converge towards an object that is not contained in the space in which the problem was formulated.

The inner product also makes the geometric concepts used in finite-dimensional linear algebra available for functions. Two functions are orthogonal if their inner product is zero, projection means finding the nearest member of a subspace, and an orthonormal basis can be constructed in the same sense as for vectors in $\mathbb{R}^n$. Modal expansion, Fourier series and the Galerkin method all use this Hilbert-space geometry.

12.1.2 Sobolev Space

Classical derivatives require a function to be smooth at every point. Finite element functions are usually only piecewise smooth: their value may be continuous between elements while their derivative changes abruptly at an element boundary. **Sobolev spaces** provide a setting in which such functions can still be differentiated in a weaker, integral sense.

The Sobolev space $W^{k,p}(\Omega)$ contains functions for which the function itself and all weak derivatives up to order k belong to $L^p(\Omega)$:

$$W^{k,p}(\Omega) = \{u \in L^p(\Omega) ; D^\alpha u \in L^p(\Omega) \text{ for all } |\alpha| \leq k\}. \quad (12.5)$$

Here α is a multi-index and $D^\alpha u$ denotes the corresponding weak derivative. A weak derivative is defined through integration by parts. A function v is the weak derivative of u in the x_i direction if:

$$\int_{\Omega} u \frac{\partial \varphi}{\partial x_i} d\Omega = - \int_{\Omega} v \varphi d\Omega \quad (12.6)$$

for every sufficiently smooth test function φ that vanishes at the boundary of its support. The derivative is therefore identified by how the function acts inside an integral, not by requiring a pointwise derivative to exist everywhere.

For $1 \leq p < \infty$, a commonly used norm on $W^{k,p}(\Omega)$ is:

$$\|u\|_{W^{k,p}(\Omega)} = \left(\sum_{|\alpha| \leq k} \|D^\alpha u\|_{L^p(\Omega)}^p \right)^{1/p}. \quad (12.7)$$

For $p = 2$ the notation

$$H^k(\Omega) = W^{k,2}(\Omega) \quad (12.8)$$

is commonly used. The space $H^k(\Omega)$ is a Hilbert space with the inner product:

$$\langle u, v \rangle_{H^k(\Omega)} = \sum_{|\alpha| \leq k} \int_{\Omega} D^\alpha u D^\alpha v \, d\Omega. \quad (12.9)$$

For the standard displacement formulation of linear elasticity the strain contains first derivatives of displacement. Finite strain energy therefore requires the displacement field to have square-integrable first weak derivatives. The natural solution space is consequently based on $H^1(\Omega)$. This does not mean that the displacement derivative must be continuous between elements. It means that the displacement and its first weak derivatives must be square-integrable. Conventional displacement finite elements satisfy this requirement by maintaining displacement continuity while permitting jumps in strain and stress across element boundaries.

Boundary conditions further restrict the space. If a displacement is prescribed on the part Γ_u of the boundary, the admissible trial functions belong to an affine subset of $H^1(\Omega)$, whereas the corresponding test functions vanish on Γ_u . This distinction between trial and test spaces is the functional-space counterpart of inserting prescribed nodal displacements into a finite-dimensional system of equations.

12.1.3 Banach Space

A **Banach space** is a normed vector space that is complete with respect to its norm. A norm assigns a non-negative size $\|u\|$ to every vector and satisfies:

$$\begin{aligned}
\|u\| &\geq 0, & \|u\| = 0 &\Leftrightarrow u = 0, \\
\|\alpha u\| &= & |\alpha| \|u\|, \\
\|u + v\| &\leq & \|u\| + \|v\|.
\end{aligned} \tag{12.10}$$

Completeness has the same meaning as for Hilbert spaces: every Cauchy sequence has a limit in the space. The difference is that a Banach space needs only a norm; it does not need an inner product. Consequently, notions depending on angles and orthogonality are not automatically available.

Every Hilbert space is a Banach space because its inner product induces a norm through Equation (12.1). The converse is not generally true. For example, $L^p(\Omega)$ is a Banach space for $1 \leq p \leq \infty$, but it is a Hilbert space only for $p = 2$. The L^1 and L^∞ norms measure the size of a function but cannot be generated by an inner product.

This distinction matters in numerical methods. Problems posed in Hilbert spaces can use orthogonal projection, energy minimisation and conjugacy directly. Problems posed in general Banach spaces may still be well defined and complete, but their analysis and numerical algorithms cannot rely on Euclidean geometry without introducing additional structure.

12.1.4 Krylov Space

A **Krylov space**, more commonly called a **Krylov subspace**, is a finite-dimensional space generated by repeatedly applying a matrix $\mathbf{A}$ to a starting vector $\mathbf{b}$. The Krylov subspace of order m is:

$$\mathcal{K}_m(\mathbf{A}, \mathbf{b}) = \text{span} \{ \mathbf{b}, \mathbf{A}\mathbf{b}, \mathbf{A}^2\mathbf{b}, \dots, \mathbf{A}^{m-1}\mathbf{b} \}. \tag{12.11}$$

The vectors in Equation (12.11) are not usually formed by explicitly computing powers of $\mathbf{A}$. Instead, one matrix-vector product is performed at each iteration:

$$\mathbf{v}_{j+1} = \mathbf{A}\mathbf{v}_j. \quad (12.12)$$

The newly generated vector is then normally orthogonalised against the existing basis. The Arnoldi algorithm performs this operation for a general matrix, while the Lanczos algorithm exploits symmetry to obtain a shorter recurrence.

For the linear system

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \quad (12.13)$$

an iterative Krylov method searches for an approximation of the form:

$$\mathbf{x}_m = \mathbf{x}_0 + \mathbf{z}_m, \quad \mathbf{z}_m \in \mathcal{K}_m(\mathbf{A}, \mathbf{r}_0), \quad (12.14)$$

where $\mathbf{r}_0 = \mathbf{b} - \mathbf{A}\mathbf{x}_0$ is the initial residual. Different methods choose $\mathbf{z}_m$ by imposing different conditions on the new residual. The conjugate-gradient method, GMRES and MINRES therefore use related approximation spaces but different projection conditions.

Krylov spaces are also used for eigenvalue calculations. The large matrix is projected onto the much smaller subspace, and eigenvalues of the projected matrix approximate selected eigenvalues of the original problem. This is particularly useful in finite element analysis because a sparse matrix-vector product can be computed without forming dense matrix powers or a complete factorisation.

The dimension of $\mathcal{K}_m$ can be smaller than m if a newly generated vector is linearly dependent on the previous vectors. In exact arithmetic this means that the subspace has become invariant under $\mathbf{A}$. In finite precision, loss of orthogonality and nearly dependent vectors must be handled carefully, which connects Krylov methods directly to the orthogonalisation algorithms in the following section.

12.2 Orthogonalisation

12.2.1 Gram-Schmidt process

In linear algebra the **Gram-Schmidt algorithm** is a way of finding two or more vectors perpendicular to each other. It is a method of constructing an orthonormal basis from a set of vectors in an inner product space, most commonly the Euclidean space $\mathbb{R}^n$ equipped with a standard inner product.

The Gram-Schmidt process takes a finite, *linearly independent* set of vectors $\mathbf{S} = \{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ for $k \leq n$ and generates an **orthogonal** set $\mathbf{S}' = \{\mathbf{u}_1, \dots, \mathbf{u}_n\}$ that spans the same k -dimensional subspace of $\mathbb{R}^n$ as $\mathbf{S}$.

The application of the Gram-Schmidt process to the column vectors of a full column rank matrix yields the **QR decomposition**.

The vector projection of a vector $\mathbf{v}$ onto $\mathbf{u}$ is defined as:

$$proj_{\mathbf{u}}(\mathbf{v}) = \frac{\langle \mathbf{v}, \mathbf{u} \rangle}{\langle \mathbf{u}, \mathbf{u} \rangle} \mathbf{u} \quad (12.15)$$

where $\langle \mathbf{v}, \mathbf{u} \rangle$ denotes the **dot product** of the vectors $\mathbf{u}$ and $\mathbf{v}$. This means that the $proj_{\mathbf{u}}(\mathbf{v})$ is the orthogonal projection of $\mathbf{v}$ onto the line spanned by $\mathbf{u}$. It can be also thought of as the part of the vector $\mathbf{v}$ that is common with vector $\mathbf{u}$. If the two vectors are perpendicular, then the dot product is equal 0.

By subtracting the $proj_{\mathbf{u}}(\mathbf{v})$ from vector $\mathbf{v}$ we obtain a new vector that is perpendicular to the vector $\mathbf{u}$.

Given k nonzero linearly independent vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ the **Gram-Schmidt** process defines the vectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ as follows:

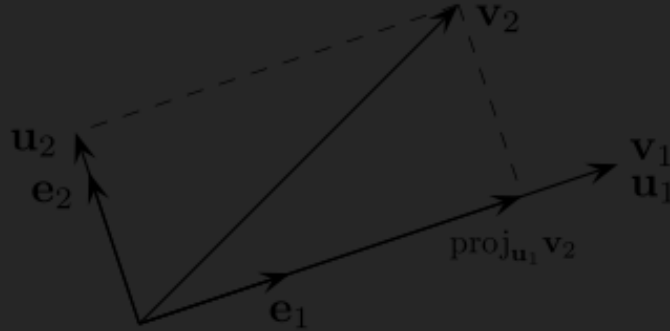


Figure 12.1: first two steps of the Gram-Schmidt process

$$\begin{aligned}
 \mathbf{u}_1 &= \mathbf{v}_1 & \mathbf{e}_1 &= \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} \\
 \mathbf{u}_2 &= \mathbf{v}_2 - \text{proj}_{\mathbf{u}_1}(\mathbf{v}_2) & \mathbf{e}_2 &= \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} \\
 \mathbf{u}_3 &= \mathbf{v}_3 - \text{proj}_{\mathbf{u}_1}(\mathbf{v}_3) - \text{proj}_{\mathbf{u}_2}(\mathbf{v}_3) & \mathbf{e}_3 &= \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} \\
 \vdots & & \vdots & \\
 \mathbf{u}_i &= \mathbf{v}_i - \sum_{j=1}^{i-1} \text{proj}_{\mathbf{u}_j}(\mathbf{v}_i) & \mathbf{e}_i &= \frac{\mathbf{u}_i}{\|\mathbf{u}_i\|}
 \end{aligned} \tag{12.16}$$

The sequence $\mathbf{u}_1, \dots, \mathbf{u}_k$ is the required system of orthogonal vectors, and the normalised vectors $\mathbf{e}_1, \dots, \mathbf{e}_k$ form an **orthonormal set**. The calculation of $\mathbf{u}_1, \dots, \mathbf{u}_k$ is known as **Gram-Schmidt orthogonalisation** and the calculation of sequence $\mathbf{e}_1, \dots, \mathbf{e}_k$ is called **Gram-Schmidt orthonormalisation**.

Python implementation:

```

1 import numpy as np
2

```

```

3 def GramSchmidtRows(A: np.ndarray, normalise: bool = False):
4     """Perform the Gram-Schmidt algorithm on the rows of matrix A so that all
5     rows are orthogonalised to the preceding rows"""
6
7     # copy the matrix
8     G = A.copy()
9
10    # row to orthogonalise
11    for j in range(1, G.shape[0]):
12        # the predecessors
13        for i in range(j):
14            # unit vector representation of preceding row
15            unit_row = G[i,:] / np.linalg.norm(G[i,:])
16            # projection of current row onto the preceding rows
17            G[j,:] -= unit_row * np.dot(G[j,:], unit_row)
18        if normalise:
19            G[j,:] /= np.linalg.norm(G[j,:])
20
21    return G
22
23 def GramSchmidtCols(A: np.ndarray, normalise : bool = False):
24     """Perform the Gram-Schmidt algorithm on the columns of matrix A so that all
25     columns are orthogonalised to the preceding rows"""
26
27     # copy the matrix
28     G = A.copy()
29
30    # column to orthogonalise
31    for j in range(1, G.shape[1]):
32        # the predecessors
33        for i in range(j):
34            # unit vector representation of preceding column
35            unit_col = G[:,i] / np.linalg.norm(G[:,i])
36            # projection of current column onto the preceding columns
37            G[:,j] -= unit_col * np.dot(G[:,j], unit_col)
38        if normalise:
39            G[:,j] /= np.linalg.norm(G[:,j])
40
41    return G

```

12.2.2 Modified Gram-Schmidt process

The **Gram-Schmidt orthogonalisation** is prone to **rounding errors** and is therefore considered **numerically unstable**. For the process as described above it is considered particularly bad.

The **Gram-Schmidt process** can be stabilised by a small modification, sometimes referred to as **modified Gram-Schmidt process** or MGS. This approach gives the same result as the original formula in exact arithmetic and introduces smaller errors in finite-precision arithmetic.

Instead of computing the $\mathbf{u}_i$ as:

$$\mathbf{u}_i = \mathbf{v}_i - \sum_{j=1}^{i-1} \text{proj}_{\mathbf{u}_j}(\mathbf{v}_i) \quad (12.17)$$

it is computed as:

$$\begin{aligned} \mathbf{u}_i^{(1)} &= \mathbf{v}_i - \text{proj}_{\mathbf{u}_1}(\mathbf{v}_i) \\ \mathbf{u}_i^{(2)} &= \mathbf{u}_i^{(1)} - \text{proj}_{\mathbf{u}_2}(\mathbf{u}_i^{(1)}) \\ &\vdots \\ \mathbf{u}_i^{(i-1)} &= \mathbf{u}_i^{(i-2)} - \text{proj}_{\mathbf{u}_{i-1}}(\mathbf{u}_i^{(i-2)}) \\ \mathbf{e}_i &= \frac{\mathbf{u}_i^{(i-1)}}{\|\mathbf{u}_i^{(i-1)}\|} \end{aligned} \quad (12.18)$$

Another description would be that the orthogonalisation is performed as a multistep process where in first step all vectors but first are made orthogonal to the first vectors, in the second step all vectors but the first two are made orthogonal to the second vector (already orthogonal to 1st one), and so on until all are processed.

Python implementation:

```

1 import numpy as np
2
3 def ModifiedGramSchmidtRows(A: np.ndarray, normalise: bool = False):
4     """Perform the Modified Gram-Schmidt algorithm on the rows of matrix A
5     so that all rows are orthogonalised to the preceding rows"""
6
7     # copy the matrix
8     G = A.copy()
9
10    # vector to orthogonalise against
11    for i in range(G.shape[0]-1):
12        # simplify dot product
13        unit_row = G[i,:] / np.linalg.norm(G[i,:])
14        if normalise:
15            G[i,:] = unit_row
16        # all later vectors
17        for j in range(i+1, G.shape[0]):
18            # subtract the projection
19            G[j,:] -= unit_row * np.dot(G[j,:], unit_row)
20
21    return G
22
23 def ModifiedGramSchmidtCols(A: np.ndarray, normalise: bool = False):
24     """Perform the Modified Gram-Schmidt algorithm on the columns of matrix A
25     so that all columns are orthogonalised to the preceding rows"""
26
27     # copy the matrix
28     G = A.copy()
29
30    # vector to orthogonalise against
31    for i in range(G.shape[1]-1):
32        # simplify dot product
33        unit_col = G[:,i] / np.linalg.norm(G[:,i])
34        if normalise:
35            G[:,i] = unit_col
36        # all later vectors
37        for j in range(i+1, G.shape[1]):
38            # subtract the projection
39            G[:,j] -= unit_col * np.dot(G[:,j], unit_col)
40

```

```
41     return G
```

12.3 Factorisation

12.3.1 Cholesky Factorisation

Cholesky Factorisation is a matrix operation such that a matrix $\mathbf{A}$ is decomposed to a multiplication of a lower triangular matrix $\mathbf{L}$ and its conjugate transpose $\mathbf{L}^T$.

The **Cholesky Factorisation** is applicable to a **Hermitian, positive definite** matrices.

$$\mathbf{A} = \mathbf{L}\mathbf{L}^* \quad (12.19)$$

Every Hermitian positive definite matrix (and thus every real-valued symmetric positive definite matrix) has a unique Cholesky factorisation.

If $\mathbf{A}$ can be written as $\mathbf{L}\mathbf{L}^*$ for some invertible $\mathbf{L}$, lower triangular or otherwise, then $\mathbf{A}$ is Hermitian and positive definite.

When $\mathbf{A}$ is a real matrix (hence symmetric and positive definite), the factorisation may be written:

$$\mathbf{A} = \mathbf{L}\mathbf{L}^T \quad (12.20)$$

A Python example of *Cholesky-Banachiewicz* factorisation algorithm:

```
1 import numpy as np
2
3 def Cholesky_Banachiewicz(A: np.ndarray) -> np.ndarray:
4     """Perform the Choleski factorisation of matrix A such that:
```



```

5
6     A = L @ L.T
7
8     where:
9         L is lower triangular matrix
10
11     Args:
12         A (np.ndarray): a real, symmetric, positive-definite matrix
13
14     Returns:
15         (np.ndarray): the result of the decomposition stored as a lower
16             ⇐ triangular
17             matrix.
18     """
19     L = np.zeros(A.shape, dtype=A.dtype)
20     for i in range(A.shape[0]):
21         for j in range(i+1):
22             sum = 0
23             for k in range(j):
24                 sum += L[i,k] * L[j,k]
25
26             if i == j:
27                 L[i,j] = np.sqrt(A[i,i] - sum)
28             else:
29                 L[i,j] = 1. / L[j,j] * (A[i,j] - sum)
30
31     return L

```

And the solution of matrix equation:

$$\begin{aligned}
 \mathbf{Ax} &= \mathbf{b} \\
 \mathbf{A} &= \mathbf{LL}^T \\
 \mathbf{LL}^T \mathbf{x} &= \mathbf{b} \quad , \text{substitute } \mathbf{y} = \mathbf{L}^T \mathbf{x} \\
 \mathbf{Ly} &= \mathbf{b} \quad , \text{solve using forward substitution} \\
 \mathbf{L}^T \mathbf{x} &= \mathbf{y} \quad , \text{solve using backward substitution}
 \end{aligned}
 \tag{12.21}$$

the $\mathbf{b}$ can be either a vector or a matrix.

Python example:

```

1 import numpy as np
2
3 def Solve_Triangular(A: np.ndarray, b: np.ndarray, kind="lower") -> np.ndarray:
4     """Solve  $Ax = b$  using either forward or backward substitution. For lower
5     triangular matrix the forward substitution is used, for upper triangular
6     matrix the backward substitution is used.
7     """
8     b = b.copy()
9     flatten = False
10    if len(b.shape) == 1:
11        b = b.reshape(-1,1)
12        flatten = True
13
14    m = A.shape[0]
15    n = b.shape[1]
16    x = np.zeros(b.shape, dtype=b.dtype)
17
18    if kind == "lower":
19        for i in range(m):           # rows of A
20            x[i,:] = b[i,:]          # perform operations on whole row of b
21            for j in range(i):       # columns of A from start to diagonal
22                x[i,:] -= A[i,j] * x[j,:] # perform operations on whole row of b
23            x[i,:] /= A[i,i]         # divide by the element on the diagonal
24
25    elif kind == "upper":
26        for i in reversed(range(m)): # rows of A from end
27            x[i,:] = b[i,:]          # perform operations on whole row of b
28            for j in range(i+1, m):  # columns of A from diagonal up to end
29                x[i,:] -= A[i,j] * x[j,:] # perform operations on whole row of b
30            x[i,:] /= A[i,i]         # divide by the element on the diagonal
31
32    else:
33        raise NotImplementedError(f"Substitution for other than 'lower' or
34        ↪ 'upper' triangular not implemented: '{kind:s}'")
35
36    if flatten:
37        return x.flatten()

```

```

37     else:
38         return x
39
40
41 def Cholesky_Solve(L: np.ndarray, b: np.ndarray) -> np.ndarray:
42     """Solve the matrix equation using results of Cholesky decomposition
43
44         Ax = b
45         A = L @ L.T
46         L @ L.T @ x = b
47         L @ z = b
48         L.T @ x = z
49
50     Args:
51         L (np.ndarray): a lower triangular matrix from Cholesky decomposition
52         b (np.ndarray): right hand side of the equation
53
54     Returns:
55         (np.ndarray): the solution on the left hand side
56     """
57     z = Solve_Triangular(L, b, "lower")
58     x = Solve_Triangular(L.T, z, "upper")
59     return x
60

```

12.3.2 LU Factorisation

The **LU factorisation** writes a square matrix as a product of a lower triangular matrix and an upper triangular matrix:

$$\mathbf{A} = \mathbf{L}\mathbf{U} \quad (12.22)$$

The factorisation is obtained by applying Gaussian elimination to $\mathbf{A}$. The multipliers used to remove the entries below a pivot are stored in $\mathbf{L}$, while the resulting upper triangular matrix is $\mathbf{U}$. Using a unit diagonal in $\mathbf{L}$ removes the otherwise arbitrary scaling between the two factors and gives the **Doolittle form**:

$$L_{ii} = 1 \quad (12.23)$$

Once the factors are available, the equation $\mathbf{Ax} = \mathbf{b}$ is solved in two triangular steps:

$$\begin{aligned} \mathbf{LUx} &= \mathbf{b} \\ \mathbf{Ly} &= \mathbf{b}, \quad \text{solve by forward substitution} \\ \mathbf{Ux} &= \mathbf{y}, \quad \text{solve by backward substitution} \end{aligned} \quad (12.24)$$

The factorisation is more expensive than one triangular solution, but it may be reused for many right hand sides. This is important in finite element analysis, where the same tangent matrix may be used for several load vectors or for several operations within one iteration.

LU Factorisation without pivoting

For the Doolittle form, comparison of the entries in $\mathbf{A} = \mathbf{LU}$ gives:

$$\begin{aligned} U_{kj} &= A_{kj} - \sum_{s=1}^{k-1} L_{ks} U_{sj}, \quad j = k, \dots, n \\ L_{ik} &= \frac{1}{U_{kk}} \left(A_{ik} - \sum_{s=1}^{k-1} L_{is} U_{sk} \right), \quad i = k+1, \dots, n \end{aligned} \quad (12.25)$$

The first equation forms row k of $\mathbf{U}$. The second equation then forms column k of $\mathbf{L}$. The calculation requires every pivot U_{kk} to be nonzero. A nonsingular matrix can nevertheless produce a zero pivot. For example:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad (12.26)$$

is nonsingular, but elimination cannot start without exchanging rows. Even when a pivot is merely very small, division by it can amplify rounding errors. Factorisation without pivoting is therefore used only when the matrix structure ensures safe pivots, or when this has been verified separately.

Python implementation:

```

1 import numpy as np
2
3
4 def LU_No_Pivoting(A: np.ndarray,
5                   tolerance: float = 1.0e-14):
6     """Factorise a square matrix A = L @ U without pivoting.
7
8     L has a unit diagonal. No NumPy linear algebra factorisation or
9     solution routine is used.
10    """
11    A = np.asarray(A)
12    n = A.shape[0]
13
14    if A.shape != (n, n):
15        raise ValueError("A must be square")
16
17    L = np.eye(n, dtype=A.dtype)
18    U = np.zeros_like(A)
19
20    for k in range(n):
21        # Form row k of U.
22        for j in range(k, n):
23            total = 0.0
24            for s in range(k):
25                total += L[k, s] * U[s, j]
26            U[k, j] = A[k, j] - total
27
28            if abs(U[k, k]) <= tolerance:
29                raise ZeroDivisionError(
30                    "zero or numerically small pivot; pivoting is required"
31                )
32

```

```

33     # Form column k of L.
34     for i in range(k + 1, n):
35         total = 0.0
36         for s in range(k):
37             total += L[i, s] * U[s, k]
38         L[i, k] = (A[i, k] - total) / U[k, k]
39
40     return L, U
41
42
43 def LU_Solve(L: np.ndarray,
44             U: np.ndarray,
45             b: np.ndarray) -> np.ndarray:
46     """Solve L @ U @ x = b using the triangular solver defined above."""
47     y = Solve_Triangular(L, b, kind="lower")
48     x = Solve_Triangular(U, y, kind="upper")
49     return x

```

LU Factorisation with row pivoting

Partial pivoting exchanges rows before each elimination step. In column k , the row containing the largest available absolute entry is selected:

$$p = \operatorname{argmax}_{i=k,\dots,n} |U_{ik}| \quad (12.27)$$

The corresponding rows are exchanged and the factorisation becomes:

$$\mathbf{PA} = \mathbf{LU} \quad (12.28)$$

where $\mathbf{P}$ is a permutation matrix. A permutation matrix only changes the order of equations. It is orthogonal, so that:

$$\mathbf{P}^{-1} = \mathbf{P}^T \quad (12.29)$$

The row exchange avoids division by zero whenever a usable pivot exists in the current column. Selecting the largest candidate also limits the elimination multipliers:

$$|L_{ik}| = \left| \frac{U_{ik}}{U_{kk}} \right| \leq 1 \quad (12.30)$$

Partial pivoting is the usual general-purpose form of LU factorisation. To solve $\mathbf{Ax} = \mathbf{b}$, the right hand side must be permuted in the same way:

$$\begin{aligned} \mathbf{PAx} &= \mathbf{Pb} \\ \mathbf{LUx} &= \mathbf{Pb} \end{aligned} \quad (12.31)$$

Python implementation:

```

1 import numpy as np
2
3
4 def LU_Partial_Pivoting(A: np.ndarray,
5                       tolerance: float = 1.0e-14):
6     """Factorise P @ A = L @ U using partial row pivoting."""
7     A = np.array(A, copy=True)
8     n = A.shape[0]
9
10    if A.shape != (n, n):
11        raise ValueError("A must be square")
12
13    U = A.copy()
14    L = np.eye(n, dtype=A.dtype)
15    P = np.eye(n, dtype=A.dtype)
16
17    for k in range(n - 1):
18        # Find the largest available entry in column k.
19        pivot = k
20        for i in range(k + 1, n):

```

```

21         if abs(U[i, k]) > abs(U[pivot, k]):
22             pivot = i
23
24     if abs(U[pivot, k]) <= tolerance:
25         raise ZeroDivisionError("matrix is singular to working precision")
26
27     if pivot != k:
28         U[[k, pivot], :] = U[[pivot, k], :]
29         P[[k, pivot], :] = P[[pivot, k], :]
30
31         # The already computed part of L belongs to the exchanged rows.
32         if k > 0:
33             L[[k, pivot], :k] = L[[pivot, k], :k]
34
35         # Gaussian elimination below the pivot.
36         for i in range(k + 1, n):
37             L[i, k] = U[i, k] / U[k, k]
38             U[i, k:] -= L[i, k] * U[k, k:]
39             U[i, k] = 0.0
40
41     if abs(U[-1, -1]) <= tolerance:
42         raise ZeroDivisionError("matrix is singular to working precision")
43
44     return P, L, U
45
46
47 def LU_Partial_Solve(P: np.ndarray,
48                     L: np.ndarray,
49                     U: np.ndarray,
50                     b: np.ndarray) -> np.ndarray:
51     """Solve  $A @ x = b$  from  $P @ A = L @ U$ ."""
52     y = Solve_Triangular(L, P @ b, kind="lower")
53     x = Solve_Triangular(U, y, kind="upper")
54     return x

```

LU Factorisation with complete pivoting

Complete pivoting searches the whole remaining submatrix rather than only the current column:

$$(p, q) = \underset{i,j=k,\dots,n}{\operatorname{argmax}} |U_{ij}| \quad (12.32)$$

Both rows and columns are exchanged. The resulting factorisation is:

$$\mathbf{PAQ} = \mathbf{LU} \quad (12.33)$$

The row permutation $\mathbf{P}$ reorders equations, while the column permutation $\mathbf{Q}$ reorders unknowns. Complete pivoting usually produces smaller element growth than partial pivoting, but searching the complete trailing submatrix is more expensive and column permutations complicate bookkeeping. It is therefore used when additional robustness is more important than the extra work.

For a linear system:

$$\begin{aligned} \mathbf{PAQ} &= \mathbf{LU} \\ \mathbf{LUQ}^T \mathbf{x} &= \mathbf{Pb} \end{aligned} \quad (12.34)$$

After the triangular solves, the original ordering is restored by $\mathbf{x} = \mathbf{Qy}$.

Python implementation:

```

1 import numpy as np
2
3
4 def LU_Complete_Pivoting(A: np.ndarray,
5                           tolerance: float = 1.0e-14):
6     """Factorise P @ A @ Q = L @ U using complete pivoting."""
```

```

7     A = np.array(A, copy=True)
8     n = A.shape[0]
9
10    if A.shape != (n, n):
11        raise ValueError("A must be square")
12
13    U = A.copy()
14    L = np.eye(n, dtype=A.dtype)
15    P = np.eye(n, dtype=A.dtype)
16    Q = np.eye(n, dtype=A.dtype)
17
18    for k in range(n - 1):
19        pivot_row = k
20        pivot_col = k
21        pivot_value = abs(U[k, k])
22
23        for i in range(k, n):
24            for j in range(k, n):
25                if abs(U[i, j]) > pivot_value:
26                    pivot_row = i
27                    pivot_col = j
28                    pivot_value = abs(U[i, j])
29
30        if pivot_value <= tolerance:
31            raise ZeroDivisionError("matrix is singular to working precision")
32
33        if pivot_row != k:
34            U[[k, pivot_row], :] = U[[pivot_row, k], :]
35            P[[k, pivot_row], :] = P[[pivot_row, k], :]
36            if k > 0:
37                L[[k, pivot_row], :k] = L[[pivot_row, k], :k]
38
39        if pivot_col != k:
40            U[:, [k, pivot_col]] = U[:, [pivot_col, k]]
41            Q[:, [k, pivot_col]] = Q[:, [pivot_col, k]]
42
43        for i in range(k + 1, n):
44            L[i, k] = U[i, k] / U[k, k]
45            U[i, k:] -= L[i, k] * U[k, k:]
46            U[i, k] = 0.0
47

```

```

48     if abs(U[-1, -1]) <= tolerance:
49         raise ZeroDivisionError("matrix is singular to working precision")
50
51     return P, L, U, Q
52
53
54 def LU_Complete_Solve(P: np.ndarray,
55                       L: np.ndarray,
56                       U: np.ndarray,
57                       Q: np.ndarray,
58                       b: np.ndarray) -> np.ndarray:
59     """Solve A @ x = b from P @ A @ Q = L @ U."""
60     z = Solve_Triangular(L, P @ b, kind="lower")
61     y = Solve_Triangular(U, z, kind="upper")
62     x = Q @ y
63     return x

```

12.3.3 QR Factorisation

The **QR factorisation** writes an $m \times n$ matrix with $m \geq n$ as:

$$\mathbf{A} = \mathbf{Q}\mathbf{R} \quad (12.35)$$

where the columns of $\mathbf{Q}$ are orthonormal and $\mathbf{R}$ is upper triangular:

$$\mathbf{Q}^*\mathbf{Q} = \mathbf{I} \quad (12.36)$$

In the **thin QR factorisation**, $\mathbf{Q}$ has size $m \times n$ and $\mathbf{R}$ has size $n \times n$. In the full factorisation, $\mathbf{Q}$ is $m \times m$ and $\mathbf{R}$ is $m \times n$.

Because multiplication by an orthogonal matrix does not change the Euclidean norm, QR factorisation transforms a least-squares problem without squaring its condition number:

$$\begin{aligned}
\underset{\mathbf{x}}{\text{minimise}} \|\mathbf{Ax} - \mathbf{b}\|_2 &= \underset{\mathbf{x}}{\text{minimise}} \|\mathbf{Q}^* \mathbf{Ax} - \mathbf{Q}^* \mathbf{b}\|_2 \\
&= \underset{\mathbf{x}}{\text{minimise}} \|\mathbf{Rx} - \mathbf{Q}^* \mathbf{b}\|_2
\end{aligned} \tag{12.37}$$

This is numerically preferable to forming the normal equations $\mathbf{A}^* \mathbf{Ax} = \mathbf{A}^* \mathbf{b}$.

QR Factorisation using Gram-Schmidt algorithm

Gram-Schmidt orthogonalisation of the columns of $\mathbf{A}$ gives $\mathbf{Q}$. Let $\mathbf{a}_j$ be column j of $\mathbf{A}$. Then:

$$\begin{aligned}
R_{ij} &= \mathbf{q}_i^* \mathbf{a}_j, \quad i < j \\
\mathbf{v}_j &= \mathbf{a}_j - \sum_{i=1}^{j-1} R_{ij} \mathbf{q}_i \\
R_{jj} &= \|\mathbf{v}_j\|_2 \\
\mathbf{q}_j &= \frac{\mathbf{v}_j}{R_{jj}}
\end{aligned} \tag{12.38}$$

The coefficients removed from column j form column j of $\mathbf{R}$. If $R_{jj} = 0$, the new column is linearly dependent on the previous columns and a full-rank thin QR factorisation does not exist.

The following code uses the modified ordering of the Gram-Schmidt operations, which is less sensitive to rounding errors than evaluating all projections from the original column at once.

```

1 import numpy as np
2
3
4 def Vector_Norm(x: np.ndarray) -> float:
5     """Euclidean norm without using numpy.linalg."""

```

```

6     value = np.dot(np.conjugate(x), x)
7     return float(np.sqrt(np.real(value)))
8
9
10 def QR_Gram_Schmidt(A: np.ndarray,
11                      tolerance: float = 1.0e-14):
12     """Compute the thin QR factorisation A = Q @ R."""
13     A = np.asarray(A)
14     m, n = A.shape
15
16     if m < n:
17         raise ValueError("the example assumes m >= n")
18
19     Q = np.zeros((m, n), dtype=A.dtype)
20     R = np.zeros((n, n), dtype=A.dtype)
21
22     for j in range(n):
23         v = A[:, j].copy()
24
25         for i in range(j):
26             R[i, j] = np.dot(np.conjugate(Q[:, i]), v)
27             v -= R[i, j] * Q[:, i]
28
29         R[j, j] = Vector_Norm(v)
30         if R[j, j] <= tolerance:
31             raise ValueError("columns of A are linearly dependent")
32
33         Q[:, j] = v / R[j, j]
34
35     return Q, R

```

QR Factorisation using Givens rotations

A **Givens rotation** acts only in a plane spanned by two coordinate axes. For real numbers a and b , choose:

$$c = \frac{a}{r}, \quad s = \frac{b}{r}, \quad r = \sqrt{a^2 + b^2} \quad (12.39)$$

and define:

$$\mathbf{G} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \quad (12.40)$$

Then:

$$\mathbf{G} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix} \quad (12.41)$$

Embedding this 2×2 block into an identity matrix allows one entry below the diagonal of $\mathbf{A}$ to be removed without changing the other rows. A sequence of rotations gives:

$$\mathbf{G}_p \cdots \mathbf{G}_2 \mathbf{G}_1 \mathbf{A} = \mathbf{R} \quad (12.42)$$

and therefore:

$$\mathbf{A} = \mathbf{Q}\mathbf{R}, \quad \mathbf{Q} = \mathbf{G}_1^T \mathbf{G}_2^T \cdots \mathbf{G}_p^T \quad (12.43)$$

Givens rotations are especially useful when the matrix is sparse or when only a small number of new entries must be eliminated, because each rotation modifies only two rows.

Python implementation:

```

1 import numpy as np
2
3
4 def Givens_Parameters(a: float, b: float):
5     """Return c and s so [[c,s],[-s,c]] @ [a,b] = [r,0]."""
6     r = float(np.hypot(abs(a), abs(b)))
7     if r == 0.0:
8         return 1.0, 0.0

```

```

9     return a / r, b / r
10
11
12 def QR_Givens(A: np.ndarray,
13               tolerance: float = 1.0e-14):
14     """Compute A = Q @ R using real Givens rotations."""
15     R = np.array(A, dtype=float, copy=True)
16     m, n = R.shape
17     Q = np.eye(m)
18
19     for j in range(min(m, n)):
20         # Work upwards so each rotation removes one entry in column j.
21         for i in reversed(range(j + 1, m)):
22             if abs(R[i, j]) <= tolerance:
23                 continue
24
25             c, s = Givens_Parameters(R[i - 1, j], R[i, j])
26
27             row_1 = R[i - 1, j:].copy()
28             row_2 = R[i, j:].copy()
29             R[i - 1, j:] = c * row_1 + s * row_2
30             R[i, j:] = -s * row_1 + c * row_2
31
32             # Q <- Q @ G.T, applied only to two columns.
33             col_1 = Q[:, i - 1].copy()
34             col_2 = Q[:, i].copy()
35             Q[:, i - 1] = c * col_1 + s * col_2
36             Q[:, i] = -s * col_1 + c * col_2
37
38     R[abs(R) < tolerance] = 0.0
39     return Q, R

```

QR Factorisation using Householder reflections

A **Householder reflection** maps a vector onto a coordinate axis in one operation. For a unit vector $\mathbf{v}$:

$$\mathbf{H} = \mathbf{I} - 2\mathbf{v}\mathbf{v}^* \quad (12.44)$$

The matrix is Hermitian and unitary:

$$\mathbf{H}^* = \mathbf{H}, \quad \mathbf{H}^* \mathbf{H} = \mathbf{I} \quad (12.45)$$

For a column segment $\mathbf{x}$, the vector $\mathbf{v}$ is selected so that:

$$\mathbf{H}\mathbf{x} = -\text{sign}(x_1)\|\mathbf{x}\|_2\mathbf{e}_1 \quad (12.46)$$

The sign is chosen to avoid subtracting nearly equal numbers while constructing $\mathbf{v}$. Each reflection removes all entries below one diagonal position. A matrix with n columns therefore needs at most n reflections. Compared with Gram-Schmidt, Householder QR usually gives better numerical orthogonality and is the standard dense-matrix method.

Python implementation:

```

1 import numpy as np
2
3
4 def Householder_Vector(x: np.ndarray,
5                       tolerance: float = 1.0e-14):
6     """Return a unit vector v defining H = I - 2 v v*."""
7     x = np.asarray(x)
8     norm_x = Vector_Norm(x)
9
10    if norm_x <= tolerance:
11        return np.zeros_like(x), False
12
13    phase = 1.0 if x[0] == 0.0 else x[0] / abs(x[0])
14    v = x.copy()
15    v[0] += phase * norm_x
16
17    norm_v = Vector_Norm(v)
18    if norm_v <= tolerance:
19        return np.zeros_like(x), False
20
```



```

21     return v / norm_v, True
22
23
24 def QR_Householder(A: np.ndarray,
25                    tolerance: float = 1.0e-14):
26     """Compute the full factorisation  $A = Q @ R$ ."""
27     R = np.array(A, copy=True)
28     m, n = R.shape
29     Q = np.eye(m, dtype=R.dtype)
30
31     for k in range(min(m, n)):
32         v, active = Householder_Vector(R[k:, k], tolerance)
33         if not active:
34             continue
35
36         #  $R \leftarrow H @ R$ , applied only to the active trailing block.
37         projection = np.dot(np.conjugate(v), R[k:, k:])
38         R[k:, k:] -= 2.0 * np.outer(v, projection)
39
40         #  $Q \leftarrow Q @ H$ .  $H$  is Hermitian, hence  $H^* = H$ .
41         projection = np.dot(Q[:, k:], v)
42         Q[:, k:] -= 2.0 * np.outer(projection, np.conjugate(v))
43
44     R[abs(R) < tolerance] = 0.0
45     return Q, R

```

QR Factorisation using Householder reflections with column pivoting

If columns of $\mathbf{A}$ are nearly linearly dependent, a small diagonal value in $\mathbf{R}$ may occur before the independent directions have been processed. **Column-pivoted QR** chooses at every step the remaining column with the largest residual norm. The factorisation is written:

$$\mathbf{AP} = \mathbf{QR} \tag{12.47}$$

where $\mathbf{P}$ is a column permutation matrix. The diagonal entries of $\mathbf{R}$ then tend to decrease in magnitude. This reveals the numerical rank:

$$|R_{11}| \geq |R_{22}| \geq \dots \quad (12.48)$$

The relation is not guaranteed to be exact for every matrix, but the diagonal usually separates well-resolved and nearly dependent directions. This makes pivoted QR useful for rank detection, least-squares problems and selecting independent interface or measurement coordinates.

Python implementation:

```

1 import numpy as np
2
3
4 def QR_Householder_Pivoting(A: np.ndarray,
5                             tolerance: float = 1.0e-14):
6     """Compute  $A @ P = Q @ R$  using Householder column pivoting."""
7     R = np.array(A, copy=True)
8     m, n = R.shape
9     Q = np.eye(m, dtype=R.dtype)
10    P = np.eye(n, dtype=R.dtype)
11
12    for k in range(min(m, n)):
13        # Recompute the residual column norms for clarity.
14        pivot = k
15        largest_norm_squared = -1.0
16        for j in range(k, n):
17            value = np.dot(np.conjugate(R[k:, j]), R[k:, j])
18            norm_squared = float(np.real(value))
19            if norm_squared > largest_norm_squared:
20                largest_norm_squared = norm_squared
21                pivot = j
22
23        if pivot != k:
24            R[:, [k, pivot]] = R[:, [pivot, k]]
25            P[:, [k, pivot]] = P[:, [pivot, k]]
26
27        v, active = Householder_Vector(R[k:, k], tolerance)
28        if not active:
29            continue

```

```

30
31     projection = np.dot(np.conjugate(v), R[k:, k:])
32     R[k:, k:] -= 2.0 * np.outer(v, projection)
33
34     projection = np.dot(Q[:, k:], v)
35     Q[:, k:] -= 2.0 * np.outer(projection, np.conjugate(v))
36
37     R[abs(R) < tolerance] = 0.0
38     return Q, R, P

```

12.3.4 Conversion to Hessenberg form using Givens rotations

The QR factorisation applies orthogonal transformations from the left and changes $\mathbf{A}$ into an upper triangular matrix. An eigenvalue problem requires a **similarity transformation**, because similarity preserves eigenvalues:

$$\mathbf{H} = \mathbf{Q}^* \mathbf{A} \mathbf{Q} \quad (12.49)$$

An **upper Hessenberg matrix** has zeros below its first subdiagonal:

$$H_{ij} = 0 \quad \text{for } i > j + 1 \quad (12.50)$$

A Givens rotation is first applied from the left to remove one entry below the first subdiagonal and then from the right with its transpose. The right multiplication is essential: without it the operation would not be a similarity transformation. Repeating this process gives a Hessenberg matrix with the same eigenvalues as $\mathbf{A}$. If $\mathbf{A}$ is symmetric, symmetry is preserved and the Hessenberg matrix is tridiagonal.

Python implementation:

```

1 import numpy as np
2

```

```

3
4 def Hessenberg_Givens(A: np.ndarray,
5                       tolerance: float = 1.0e-14):
6     """Compute  $H = Q^T @ A @ Q$  using real Givens rotations."""
7     H = np.array(A, dtype=float, copy=True)
8     n = H.shape[0]
9
10    if H.shape != (n, n):
11        raise ValueError("A must be square")
12
13    Q = np.eye(n)
14
15    for k in range(n - 2):
16        for i in reversed(range(k + 2, n)):
17            if abs(H[i, k]) <= tolerance:
18                continue
19
20            c, s = Givens_Parameters(H[i - 1, k], H[i, k])
21
22            #  $H \leftarrow G @ H$ : rotate rows  $i-1$  and  $i$ .
23            row_1 = H[i - 1, :].copy()
24            row_2 = H[i, :].copy()
25            H[i - 1, :] = c * row_1 + s * row_2
26            H[i, :] = -s * row_1 + c * row_2
27
28            #  $H \leftarrow H @ G.T$ : rotate the matching columns.
29            col_1 = H[:, i - 1].copy()
30            col_2 = H[:, i].copy()
31            H[:, i - 1] = c * col_1 + s * col_2
32            H[:, i] = -s * col_1 + c * col_2
33
34            #  $Q \leftarrow Q @ G.T$ .
35            col_1 = Q[:, i - 1].copy()
36            col_2 = Q[:, i].copy()
37            Q[:, i - 1] = c * col_1 + s * col_2
38            Q[:, i] = -s * col_1 + c * col_2
39
40    H[abs(H) < tolerance] = 0.0
41    return Q, H

```

12.3.5 Conversion to Hessenberg form using Householder reflections

A Householder reflection can remove a complete column segment in one operation. At step k , a reflector is constructed from:

$$\mathbf{x} = \mathbf{A}_{k+1:n,k} \quad (12.51)$$

and embedded into the identity matrix:

$$\hat{\mathbf{H}}_k = \begin{bmatrix} \mathbf{I}_k & \mathbf{0} \\ \mathbf{0} & \mathbf{H}_k \end{bmatrix} \quad (12.52)$$

The similarity update is:

$$\mathbf{A}_{k+1} = \hat{\mathbf{H}}_k^* \mathbf{A}_k \hat{\mathbf{H}}_k \quad (12.53)$$

Since a Householder reflector is Hermitian, its conjugate transpose is the reflector itself. For dense matrices this method requires fewer transformations than the Givens method and is normally preferred. Again, a symmetric matrix is reduced to symmetric tridiagonal form.

Python implementation:

```

1 import numpy as np
2
3
4 def Hessenberg_Householder(A: np.ndarray,
5                             tolerance: float = 1.0e-14):
6     """Compute H = Q* @ A @ Q by Householder similarity transforms."""
7     H = np.array(A, copy=True)
8     n = H.shape[0]
9
10    if H.shape != (n, n):
```

```

11         raise ValueError("A must be square")
12
13     Q = np.eye(n, dtype=H.dtype)
14
15     for k in range(n - 2):
16         v, active = Householder_Vector(H[k + 1:, k], tolerance)
17         if not active:
18             continue
19
20         # Left multiplication by the embedded reflector.
21         projection = np.dot(np.conjugate(v), H[k + 1:, k])
22         H[k + 1:, :] -= 2.0 * np.outer(v, projection)
23
24         # Right multiplication by the same reflector.
25         projection = np.dot(H[:, k + 1:], v)
26         H[:, k + 1:] -= 2.0 * np.outer(
27             projection, np.conjugate(v)
28         )
29
30         # Accumulate Q.
31         projection = np.dot(Q[:, k + 1:], v)
32         Q[:, k + 1:] -= 2.0 * np.outer(
33             projection, np.conjugate(v)
34         )
35
36     H[abs(H) < tolerance] = 0.0
37     return Q, H

```

12.3.6 Jacobi similarity rotations

A **Jacobi rotation** is a symmetric plane rotation used on both sides of a real symmetric matrix:

$$\mathbf{A}_{new} = \mathbf{J}^T \mathbf{A} \mathbf{J} \quad (12.54)$$

For selected indices p and q , the nontrivial part of $\mathbf{J}$ is:

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \quad (12.55)$$

The angle can be selected so that A_{pq} and A_{qp} become zero. Unlike Householder Hessenberg reduction, however, a later Jacobi rotation can recreate entries that an earlier rotation removed. Jacobi rotations are therefore not a finite general-purpose Hessenberg-reduction algorithm. Their natural use is to reduce a symmetric matrix directly towards diagonal form, as described in the next subsection.

A single Jacobi similarity step is useful for understanding the operation:

```

1 import numpy as np
2
3
4 def Jacobi_Rotation(A: np.ndarray, p: int, q: int):
5     """Apply one Jacobi similarity rotation to a real symmetric matrix."""
6     A = np.array(A, dtype=float, copy=True)
7     n = A.shape[0]
8
9     if A.shape != (n, n):
10         raise ValueError("A must be square")
11     if p == q:
12         return A, np.eye(n)
13
14     theta = 0.5 * np.arctan2(
15         2.0 * A[p, q], A[p, p] - A[q, q]
16     )
17     c = float(np.cos(theta))
18     s = float(np.sin(theta))
19
20     J = np.eye(n)
21     J[p, p] = c
22     J[q, q] = c
23     J[p, q] = -s
24     J[q, p] = s
25
26     return J.T @ A @ J, J

```

12.3.7 Jacobi rotations to diagonal matrix

For a real symmetric matrix, repeated Jacobi rotations reduce the off-diagonal entries while preserving symmetry and eigenvalues. When the process converges:

$$\mathbf{D} = \mathbf{V}^T \mathbf{A} \mathbf{V} \quad (12.56)$$

where $\mathbf{D}$ is diagonal and the columns of $\mathbf{V}$ are orthonormal eigenvectors. The diagonal entries of $\mathbf{D}$ are the eigenvalues.

A simple strategy selects the largest off-diagonal entry in every iteration:

$$(p, q) = \underset{p < q}{\operatorname{argmax}} |A_{pq}| \quad (12.57)$$

The rotation angle is obtained from:

$$\tan(2\theta) = \frac{2A_{pq}}{A_{pp} - A_{qq}} \quad (12.58)$$

Using `atan2` avoids an explicit division and preserves the correct quadrant. The method is easy to understand and produces accurate eigenvectors, but each rotation changes rows and columns of a dense matrix. It is therefore mainly useful for small or medium symmetric problems and as a reference implementation.

Python implementation:

```

1 import numpy as np
2
3
4 def Jacobi_Eigenvalue(A: np.ndarray,
5                       tolerance: float = 1.0e-12,
6                       max_iterations: int = 10000):
7     """Diagonalise a real symmetric matrix with Jacobi rotations."""
8     D = np.array(A, dtype=float, copy=True)
```



```

9     n = D.shape[0]
10
11     if D.shape != (n, n):
12         raise ValueError("A must be square")
13
14     V = np.eye(n)
15
16     for iteration in range(max_iterations):
17         # Locate the largest off-diagonal entry.
18         p = 0
19         q = 1
20         largest = 0.0
21         for i in range(n - 1):
22             for j in range(i + 1, n):
23                 if abs(D[i, j]) > largest:
24                     largest = abs(D[i, j])
25                     p = i
26                     q = j
27
28         if largest <= tolerance:
29             break
30
31         theta = 0.5 * np.arctan2(
32             2.0 * D[p, q], D[p, p] - D[q, q]
33         )
34         c = float(np.cos(theta))
35         s = float(np.sin(theta))
36
37         J = np.eye(n)
38         J[p, p] = c
39         J[q, q] = c
40         J[p, q] = -s
41         J[q, p] = s
42
43         D = J.T @ D @ J
44         V = V @ J
45
46         # Remove round-off remaining in the selected location.
47         D[p, q] = 0.0
48         D[q, p] = 0.0
49     else:

```

```

50         raise RuntimeError("Jacobi iteration did not converge")
51
52     eigenvalues = np.diag(D).copy()
53     return eigenvalues, V

```

12.3.8 Arnoldi algorithm

The **Arnoldi algorithm** constructs an orthonormal basis of a Krylov subspace for a general square matrix:

$$\mathcal{K}_m(\mathbf{A}, \mathbf{b}) = \text{span} \{ \mathbf{b}, \mathbf{A}\mathbf{b}, \dots, \mathbf{A}^{m-1}\mathbf{b} \} \quad (12.59)$$

Directly forming the powers of $\mathbf{A}$ would rapidly produce nearly dependent vectors. Arnoldi instead multiplies the newest basis vector by $\mathbf{A}$ and orthogonalises the result against all preceding vectors. This gives:

$$\mathbf{A}\mathbf{V}_m = \mathbf{V}_{m+1}\bar{\mathbf{H}}_m \quad (12.60)$$

where $\mathbf{V}_{m+1}$ has orthonormal columns and $\bar{\mathbf{H}}_m$ is upper Hessenberg. The projected matrix:

$$\mathbf{H}_m = \mathbf{V}_m^* \mathbf{A} \mathbf{V}_m \quad (12.61)$$

is small. Its eigenvalues are called **Ritz values** and approximate selected eigenvalues of $\mathbf{A}$. The same relation is the basis of GMRES for linear systems.

If the new vector has zero norm, the Krylov subspace has become invariant under $\mathbf{A}$. This is called a **happy breakdown**: no further basis vectors are required in exact arithmetic.

Python implementation:

```

1 import numpy as np
2

```

```

3
4 def Arnoldi(A: np.ndarray,
5             b: np.ndarray,
6             steps: int,
7             tolerance: float = 1.0e-14):
8     """Construct an Arnoldi basis and upper Hessenberg projection."""
9     A = np.asarray(A)
10    b = np.asarray(b)
11    n = A.shape[0]
12    m = min(steps, n)
13    dtype = np.result_type(A, b)
14
15    V = np.zeros((n, m + 1), dtype=dtype)
16    H = np.zeros((m + 1, m), dtype=dtype)
17
18    beta = Vector_Norm(b)
19    if beta <= tolerance:
20        raise ValueError("the starting vector must be nonzero")
21
22    V[:, 0] = b / beta
23    used_steps = m
24
25    for j in range(m):
26        w = A @ V[:, j]
27
28        # Modified Gram-Schmidt orthogonalisation.
29        for i in range(j + 1):
30            H[i, j] = np.dot(np.conjugate(V[:, i]), w)
31            w -= H[i, j] * V[:, i]
32
33        H[j + 1, j] = Vector_Norm(w)
34
35        if H[j + 1, j] <= tolerance:
36            used_steps = j + 1
37            break
38
39        V[:, j + 1] = w / H[j + 1, j]
40
41    return (
42        V[:, :used_steps + 1],
43        H[:used_steps + 1, :used_steps]

```

12.3.9 Lanczos algorithm

For a Hermitian matrix, the Arnoldi projected matrix is also Hermitian. A matrix that is both Hermitian and upper Hessenberg must be tridiagonal. This removes the need to orthogonalise against every preceding vector and gives the **Lanczos three-term recurrence**:

$$\begin{aligned}
 \mathbf{w}_j &= \mathbf{A}\mathbf{v}_j - \beta_{j-1}\mathbf{v}_{j-1} \\
 \alpha_j &= \mathbf{v}_j^* \mathbf{w}_j \\
 \mathbf{w}_j &\leftarrow \mathbf{w}_j - \alpha_j \mathbf{v}_j \\
 \beta_j &= \|\mathbf{w}_j\|_2 \\
 \mathbf{v}_{j+1} &= \frac{\mathbf{w}_j}{\beta_j}
 \end{aligned} \tag{12.62}$$

The resulting relation is:

$$\mathbf{A}\mathbf{V}_m = \mathbf{V}_m \mathbf{T}_m + \beta_m \mathbf{v}_{m+1} \mathbf{e}_m^T \tag{12.63}$$

where $\mathbf{T}_m$ is real symmetric tridiagonal for a real symmetric $\mathbf{A}$. Lanczos is well suited to large sparse structural eigenvalue problems because it requires only matrix-vector products and stores a limited number of vectors.

In exact arithmetic, the three-term recurrence preserves orthogonality. In finite precision, converged directions may reappear and produce duplicate Ritz values. The example therefore includes optional full reorthogonalisation. This increases the cost but makes the educational implementation reliable.

Python implementation:

```

1 import numpy as np
2

```

```

3
4 def Lanczos(A: np.ndarray,
5             b: np.ndarray,
6             steps: int,
7             tolerance: float = 1.0e-14,
8             reorthogonalise: bool = True):
9     """Construct a Lanczos basis and symmetric tridiagonal projection."""
10    A = np.asarray(A)
11    b = np.asarray(b)
12    n = A.shape[0]
13    m = min(steps, n)
14    dtype = np.result_type(A, b)
15
16    V = np.zeros((n, m), dtype=dtype)
17    alpha = np.zeros(m)
18    beta = np.zeros(max(m - 1, 0))
19
20    norm_b = Vector_Norm(b)
21    if norm_b <= tolerance:
22        raise ValueError("the starting vector must be nonzero")
23
24    v = b / norm_b
25    v_previous = np.zeros_like(v)
26    beta_previous = 0.0
27    used_steps = m
28
29    for j in range(m):
30        V[:, j] = v
31        w = A @ v - beta_previous * v_previous
32
33        alpha[j] = float(np.real(np.dot(np.conjugate(v), w)))
34        w -= alpha[j] * v
35
36        if reorthogonalise:
37            for i in range(j + 1):
38                coefficient = np.dot(np.conjugate(V[:, i]), w)
39                w -= coefficient * V[:, i]
40
41        if j == m - 1:
42            break
43

```

```

44     beta[j] = Vector_Norm(w)
45     if beta[j] <= tolerance:
46         used_steps = j + 1
47         break
48
49     v_previous = v
50     v = w / beta[j]
51     beta_previous = beta[j]
52
53     T = np.diag(alpha[:used_steps])
54     if used_steps > 1:
55         T += np.diag(beta[:used_steps - 1], 1)
56         T += np.diag(beta[:used_steps - 1], -1)
57
58     return V[:, :used_steps], T

```

12.3.10 Schur algorithm

Every square complex matrix has a **Schur decomposition**:

$$\mathbf{A} = \mathbf{Z}\mathbf{T}\mathbf{Z}^* \quad (12.64)$$

where $\mathbf{Z}$ is unitary and $\mathbf{T}$ is upper triangular. The eigenvalues of $\mathbf{A}$ appear on the diagonal of $\mathbf{T}$. For real arithmetic, a real matrix is generally reduced to **real Schur form**, which is upper quasi-triangular and may contain 2×2 diagonal blocks representing complex-conjugate eigenvalue pairs.

The QR algorithm first reduces $\mathbf{A}$ to Hessenberg form and then repeatedly performs shifted QR steps:

$$\begin{aligned} \mathbf{H}_k - \mu_k \mathbf{I} &= \mathbf{Q}_k \mathbf{R}_k \\ \mathbf{H}_{k+1} &= \mathbf{R}_k \mathbf{Q}_k + \mu_k \mathbf{I} \end{aligned} \quad (12.65)$$

Since:

$$\mathbf{H}_{k+1} = \mathbf{Q}_k^* \mathbf{H}_k \mathbf{Q}_k \quad (12.66)$$

all iterates are similar and have the same eigenvalues. The shift μ_k accelerates convergence. When a subdiagonal entry becomes small, the corresponding eigenvalue is **deflated** and the iteration continues on the remaining leading block.

The following educational implementation converts the matrix to complex arithmetic. It therefore converges to triangular complex Schur form even when the original real matrix has complex eigenvalues. Production eigensolvers use more sophisticated double shifts, aggressive deflation and carefully blocked operations.

```

1 import numpy as np
2
3
4 def Schur_QR(A: np.ndarray,
5             tolerance: float = 1.0e-10,
6             max_iterations: int = 5000):
7     """Compute a simple complex Schur form using shifted QR iteration.
8
9     This example reuses Hessenberg_Householder and QR_Householder.
10    """
11    A = np.array(A, dtype=complex, copy=True)
12    n = A.shape[0]
13
14    if A.shape != (n, n):
15        raise ValueError("A must be square")
16
17    Z, H = Hessenberg_Householder(A, tolerance * 0.01)
18    active = n
19    iterations = 0
20
21    while active > 1:
22        scale = (
23            abs(H[active - 2, active - 2])
24            + abs(H[active - 1, active - 1])

```

```

25         + 1.0
26     )
27
28     if abs(H[active - 1, active - 2]) <= tolerance * scale:
29         H[active - 1, active - 2] = 0.0
30         active -= 1
31         continue
32
33     if iterations >= max_iterations:
34         raise RuntimeError("shifted QR iteration did not converge")
35
36     # A simple Rayleigh shift from the last diagonal entry.
37     shift = H[active - 1, active - 1]
38     shifted = (
39         H[:active, :active]
40         - shift * np.eye(active, dtype=complex)
41     )
42     Q_step, R_step = QR_Householder(
43         shifted, tolerance * 0.01
44     )
45
46     # Apply the block similarity transformation to the whole matrix.
47     H[:active, :] = np.conjugate(Q_step).T @ H[:active, :]
48     H[:, :active] = H[:, :active] @ Q_step
49     Z[:, :active] = Z[:, :active] @ Q_step
50
51     iterations += 1
52
53     H[abs(H) < tolerance] = 0.0
54     return Z, H

```

12.3.11 Singular Value Decomposition (SVD)

For any real or complex $m \times n$ matrix, the singular value decomposition is:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^* \quad (12.67)$$

The columns of $\mathbf{U}$ and $\mathbf{V}$ are orthonormal. The diagonal entries of $\mathbf{\Sigma}$ are nonnegative singular values:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0 \quad (12.68)$$

The right singular vectors are eigenvectors of $\mathbf{A}^* \mathbf{A}$, while the left singular vectors are eigenvectors of $\mathbf{A} \mathbf{A}^*$:

$$\begin{aligned} \mathbf{A}^* \mathbf{A} \mathbf{v}_i &= \sigma_i^2 \mathbf{v}_i \\ \mathbf{A} \mathbf{A}^* \mathbf{u}_i &= \sigma_i^2 \mathbf{u}_i \\ \mathbf{A} \mathbf{v}_i &= \sigma_i \mathbf{u}_i \end{aligned} \quad (12.69)$$

The SVD exposes matrix rank, null spaces and conditioning. It also gives the best rank- r approximation in the Euclidean and Frobenius norms:

$$\mathbf{A}_r = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^* \quad (12.70)$$

The algorithms below form $\mathbf{A}^* \mathbf{A}$ implicitly through repeated matrix-vector or matrix-block products. This is useful for explaining the relation to eigenvalue iteration, but it squares the condition number and is not the preferred production SVD algorithm. Robust libraries use bidiagonal reduction followed by specialised iterations.

SVD using Power Iteration

Power iteration applied to $\mathbf{A}^* \mathbf{A}$ converges to the dominant right singular vector:

$$\mathbf{v}^{(k+1)} = \frac{\mathbf{A}^* \mathbf{A} \mathbf{v}^{(k)}}{\|\mathbf{A}^* \mathbf{A} \mathbf{v}^{(k)}\|_2} \quad (12.71)$$

After convergence:

$$\begin{aligned}\sigma &= \|\mathbf{A}\mathbf{v}\|_2 \\ \mathbf{u} &= \frac{\mathbf{A}\mathbf{v}}{\sigma}\end{aligned}\tag{12.72}$$

The rank-one part is then removed:

$$\mathbf{A} \leftarrow \mathbf{A} - \sigma \mathbf{u} \mathbf{v}^* \tag{12.73}$$

and the iteration is repeated for the next singular triplet. This **deflation** is simple but accumulated rounding errors can reduce orthogonality, especially for clustered or repeated singular values.

Python implementation:

```

1 import numpy as np
2
3
4 def SVD_Power(A: np.ndarray,
5               rank: int = None,
6               tolerance: float = 1.0e-10,
7               max_iterations: int = 10000):
8     """Approximate a thin SVD by power iteration and rank-one deflation."""
9     B = np.array(A, dtype=float, copy=True)
10    m, n = B.shape
11    maximum_rank = min(m, n)
12    r = maximum_rank if rank is None else min(rank, maximum_rank)
13
14    U = np.zeros((m, r))
15    singular_values = np.zeros(r)
16    V = np.zeros((n, r))
17    used_rank = r
18
19    for k in range(r):
20        # A deterministic nonzero initial vector.
21        v = np.arange(1, n + 1, dtype=float) + k

```

```

22     v /= Vector_Norm(v)
23
24     for iteration in range(max_iterations):
25         w = B.T @ (B @ v)
26         norm_w = Vector_Norm(w)
27
28         if norm_w <= tolerance:
29             used_rank = k
30             break
31
32     v_new = w / norm_w
33
34     # The sign of a singular vector is arbitrary.
35     same_sign = Vector_Norm(v_new - v)
36     opposite_sign = Vector_Norm(v_new + v)
37     v = v_new
38
39     if min(same_sign, opposite_sign) <= tolerance:
40         break
41     else:
42         raise RuntimeError("power iteration did not converge")
43
44     if used_rank == k:
45         break
46
47     Av = B @ v
48     sigma = Vector_Norm(Av)
49     if sigma <= tolerance:
50         used_rank = k
51         break
52
53     u = Av / sigma
54     U[:, k] = u
55     singular_values[k] = sigma
56     V[:, k] = v
57
58     # Remove the converged rank-one contribution.
59     B -= sigma * np.outer(u, v)
60
61     return (
62         U[:, :used_rank],

```



```

6             max_iterations: int = 10000):
7     """Approximate a thin SVD using block orthogonal iteration.
8
9     The example reuses QR_Householder and does not call numpy.linalg.
10    """
11    A = np.array(A, dtype=float, copy=True)
12    m, n = A.shape
13    r = min(m, n)
14
15    # Construct a deterministic full-rank starting block.
16    seed = np.empty((n, r))
17    for i in range(n):
18        for j in range(r):
19            seed[i, j] = 1.0 / (1.0 + i + j)
20            if i == j:
21                seed[i, j] += 1.0
22
23    Q, unused_R = QR_Householder(seed)
24    Q = Q[:, :r]
25
26    for iteration in range(max_iterations):
27        Y = A.T @ (A @ Q)
28        Q_new, unused_R = QR_Householder(Y)
29        Q_new = Q_new[:, :r]
30
31        # Compare columns while accounting for arbitrary signs.
32        error = 0.0
33        for j in range(r):
34            same_sign = Vector_Norm(Q_new[:, j] - Q[:, j])
35            opposite_sign = Vector_Norm(Q_new[:, j] + Q[:, j])
36            error = max(error, min(same_sign, opposite_sign))
37
38        Q = Q_new
39        if error <= tolerance:
40            break
41    else:
42        raise RuntimeError("orthogonal iteration did not converge")
43
44    U = np.zeros((m, r))
45    singular_values = np.zeros(r)
46

```

```

47     for j in range(r):
48         Aq = A @ Q[:, j]
49         singular_values[j] = Vector_Norm(Aq)
50         if singular_values[j] > tolerance:
51             U[:, j] = Aq / singular_values[j]
52
53     # Sort the singular triplets from largest to smallest.
54     order = np.argsort(-singular_values)
55     singular_values = singular_values[order]
56     U = U[:, order]
57     Q = Q[:, order]
58
59     return U, singular_values, Q.T

```

12.3.12 Positive semidefinite matrices

If a Hermitian matrix $\mathbf{A}$ is only positive semidefinite, instead of positive definite, then it still has a decomposition of the form $\mathbf{A} = \mathbf{L}\mathbf{L}^*$ where the diagonal entries of $\mathbf{L}$ are allowed to be zero. The decomposition need not be unique, for example:

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{L}\mathbf{L}^* \quad (12.76)$$

$$\mathbf{L} = \begin{bmatrix} 0 & 0 \\ \cos \theta & \sin \theta \end{bmatrix} \quad (12.77)$$

However, if the rank of $\mathbf{A}$ is r , then there is a unique lower triangular $\mathbf{L}$ with exactly r positive diagonal elements and $n - r$ columns containing all zeros.

Alternatively, the decomposition can be made unique when a pivoting choice is fixed. Formally, if $\mathbf{A}$ is an $n \times n$ positive semidefinite matrix of rank r , then there is at least one permutation matrix $\mathbf{P}$ such that $\mathbf{PAP}^T$ has a unique decomposition of the form

$$\mathbf{PAP}^T = \mathbf{L}\mathbf{L}^* \quad (12.78)$$

with

$$\mathbf{L} = \begin{bmatrix} \mathbf{L}_1 & \mathbf{0} \\ \mathbf{L}_2 & \mathbf{0} \end{bmatrix} \quad (12.79)$$

where $\mathbf{L}_1$ is a $r \times r$ lower triangular matrix with positive diagonal.

Example

$$\begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix} \begin{bmatrix} 2 & 6 & -8 \\ 0 & 1 & 5 \\ 0 & 0 & 3 \end{bmatrix} \quad (12.80)$$

12.3.13 Applications

Solution of a system of linear equations

The Cholesky decomposition is mainly used for the numerical solution of **linear equations** $\mathbf{Ax} = \mathbf{b}$. If $\mathbf{A}$ is symmetric and positive definite, then we can solve $\mathbf{Ax} = \mathbf{b}$ by first computing the Cholesky decomposition $\mathbf{A} = \mathbf{LL}^*$, then solving $\mathbf{Ly} = \mathbf{b}$ for $\mathbf{y}$ by forward substitution and finally solving $\mathbf{L}^*\mathbf{x} = \mathbf{y}$ for $\mathbf{x}$ by back substitution.

$$\begin{aligned} \mathbf{Ax} &= \mathbf{b} \\ \mathbf{LL}^*\mathbf{x} &= \mathbf{b}, \text{ substitute } \mathbf{L}^*\mathbf{x} = \mathbf{y} \\ \mathbf{Ly} &= \mathbf{b}, \text{ then solve} \\ \mathbf{L}^*\mathbf{x} &= \mathbf{y} \end{aligned} \quad (12.81)$$

For linear systems that can be put into symmetric form, the Cholesky decomposition is the method choice, for superior efficiency and numerical stability. Compared to the $\mathbf{LU}$ decomposition, it is roughly twice as efficient.

12.3.14 Matrix Inversion

A non-Hermitian matrix $\mathbf{B}$ can be inverted using the following identity, where $\mathbf{B}\mathbf{B}^*$ will always be Hermitian:

$$\mathbf{B}^{-1} = \mathbf{B}^*(\mathbf{B}\mathbf{B}^*)^{-1} \quad (12.82)$$

The above stated matrix relation is also called the **pseudoinverse** of a matrix. The following equality holds:

$$\mathbf{B}^*(\mathbf{B}\mathbf{B}^*)^{-1} = (\mathbf{B}^*\mathbf{B})^{-1}\mathbf{B}^* \quad (12.83)$$

The best way to compute the **pseudoinverse** is to use **singular value decomposition (SVD)**. With $\mathbf{A}$ being $\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^*$, where $\mathbf{U}$ and $\mathbf{V}$ (both $n \times n$) are orthogonal and $\mathbf{S}$ ($m \times n$) is **diagonal** with real, non-negative *singular values* $\sigma_{i,1} = 1, \dots, n$. We find that

$$\mathbf{A}^* = \mathbf{V}(\mathbf{S}^*\mathbf{S})^{-1}\mathbf{S}^*\mathbf{U}^* \quad (12.84)$$

If the rank r of $\mathbf{A}$ is smaller than n , the inverse of $\mathbf{S}^*\mathbf{S}$ does not exist, and one uses only the first r singular values. $\mathbf{S}$ then becomes an $r \times r$ matrix and $\mathbf{U}$, $\mathbf{V}$ shrink accordingly.

12.4 Linear Regression

12.4.1 Derivation

Linear Regression is an approach for predicting a response using a **single feature**. In linear regression we assume that two variables i. e. independent and dependent are linearly related.

Generally we define $\mathbf{x}$ as a **feature vector**:

$$\mathbf{x} = [x_1, x_2, \dots, x_n] \quad (12.85)$$

and $\mathbf{y}$ as **response vector**:

$$\mathbf{y} = [y_1, y_2, \dots, y_n] \quad (12.86)$$

then a predicted response:

$$h(x_i) = \beta_0 + \beta_1 \times x_i \quad (12.87)$$

where:

x_i represents the i -th observation

β_1 represents the slope, and

β_0 represents the intercept of the regression line.

To predict a new value y_i , we need to consider:

$$y_i = \beta_0 + \beta_1 \times x_i + \epsilon_i = h(x_i) + \epsilon_i \quad (12.88)$$

where:

ϵ_i is the **error** of the estimate.

The error function is therefore obtained:

$$\epsilon_i = y_i - h(x_i) \quad (12.89)$$

The β_0 and β_1 coefficients are obtained by minimizing the residual **error** of the estimate using e.g. the **least square method**:

$$\begin{aligned}
J(\beta_0, \beta_1) &= \frac{1}{2n} \sum_{i=1}^n \epsilon_i^2 \\
&= \frac{1}{2n} \sum_{i=1}^n (y_i - h(x_i))^2 \\
&= \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i)^2
\end{aligned} \tag{12.90}$$

To find the coefficients we can derivate the cost function, first with regards to β_0 :

$$\frac{\partial J}{\partial \beta_0} \left[\frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i)^2 \right] = 0 \tag{12.91}$$

where using the chain rule we can write:

$$\begin{aligned}
u &= y_i - \beta_0 - \beta_1 \times x_i \\
J &= \frac{1}{2n} \sum_{i=1}^n u^2 \\
\frac{\partial J}{\partial \beta_0} &= \frac{\partial J}{\partial u} \times \frac{\partial u}{\partial \beta_0} \\
\frac{\partial J}{\partial u} &= 2u \\
\frac{\partial u}{\partial \beta_0} &= -1 \\
\frac{\partial J}{\partial \beta_0} &= \frac{1}{2n} \times (-2) \times \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n y_i - \frac{1}{n} \sum_{i=1}^n \beta_0 - \frac{1}{n} \beta_1 \sum_{i=1}^n x_i \\
0 &= \bar{y} - \beta_0 - \beta_1 \bar{x} \\
\beta_0 &= \bar{y} - \beta_1 \bar{x}
\end{aligned} \tag{12.92}$$

To find β_1 :

$$\frac{\partial J}{\partial \beta_1} \left[\frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 \times x_i)^2 \right] = 0 \tag{12.93}$$

$$\begin{aligned}
u &= y_i - \beta_0 - \beta_1 x_i \\
J &= \frac{1}{2n} \sum_{i=1}^n u^2 \\
\frac{\partial J}{\partial \beta_1} &= \frac{\partial J}{\partial u} \times \frac{\partial u}{\partial \beta_1} \\
\frac{\partial J}{\partial u} &= 2u \\
\frac{\partial u}{\partial \beta_1} &= -x \\
\frac{\partial J}{\partial \beta_1} &= \frac{1}{2n} \times (-2) \sum_{i=1}^n x_i \times (y_i - \beta_0 - \beta_1 x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) \\
0 &= \frac{1}{n} \sum_{i=1}^n (x_i y_i - \beta_0 x_i - \beta_1 x_i^2)
\end{aligned} \tag{12.94}$$

Substituting the value of β_0 :

$$\begin{aligned}
\beta_0 &= \bar{y} - \beta_1 \bar{x} \\
0 &= \frac{1}{n} \sum_{i=1}^n (x_i y_i - (\bar{y} - \beta_1 \bar{x}) x_i - \beta_1 x_i^2) \\
0 &= \frac{1}{n} \sum_{i=1}^n (x_i y_i - \bar{y} x_i + \beta_1 \bar{x} x_i - \beta_1 x_i^2) \\
0 &= \frac{1}{n} \left(\sum_{i=1}^n (x_i y_i - \bar{y} x_i) - \beta_1 \sum_{i=1}^n (x_i^2 - \bar{x} x_i) \right) \\
\beta_1 &= \frac{\frac{1}{n} \sum_{i=1}^n (x_i y_i - \bar{y} x_i)}{\frac{1}{n} \sum_{i=1}^n (x_i^2 - \bar{x} x_i)}
\end{aligned} \tag{12.95}$$

Simplifying the formula using:

$$\begin{aligned}\bar{y} &= \frac{1}{n} \sum_{i=1}^n y_i \\ \bar{x} &= \frac{1}{n} \sum_{i=1}^n x_i\end{aligned}\tag{12.96}$$

we get:

$$\begin{aligned}\beta_1 &= \frac{\frac{1}{n} \sum_{i=1}^n x_i y_i - \bar{x} \bar{y}}{\frac{1}{n} \sum_{i=1}^n x_i^2 - \bar{x}^2} \\ \beta_1 &= \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2}\end{aligned}\tag{12.97}$$

12.4.2 Result

So, to recapitulate, to get a linear regression coefficients for a dataset:

$$\begin{aligned}\mathbf{x} &= [x_1, x_2, \dots, x_n] \\ \mathbf{y} &= [y_1, y_2, \dots, y_n]\end{aligned}\tag{12.98}$$

we may use equations:

$$\begin{aligned}\beta_1 &= \frac{\sum_{i=1}^n x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^n x_i^2 - n \bar{x}^2} \\ \beta_0 &= \bar{y} - \beta_1 \bar{x}\end{aligned}\tag{12.99}$$

with an error:

$$\epsilon_i = y_i - \beta_0 - \beta_1 x_i\tag{12.100}$$

12.5 Interpolation and Mapping of Values

Values often have to be transferred from one set of points or one mesh to another. Examples are the transfer of pressure from a fluid mesh to a structural mesh, the evaluation of measured data at finite element nodes, the mapping of temperatures between analyses and the comparison of results from two different discretisations.

Let the source data be known at positions $\mathbf{x}_i$:

$$u_i = u(\mathbf{x}_i), \quad i = 1, \dots, n_s \quad (12.101)$$

and let $\mathbf{x}$ be a target position. The mapping operation constructs an estimate:

$$\hat{u}(\mathbf{x}) \quad (12.102)$$

from the available source values. In many methods this estimate is a weighted sum:

$$\hat{u}(\mathbf{x}) = \sum_{i=1}^{n_s} w_i(\mathbf{x}) u_i \quad (12.103)$$

with weights that depend on the geometry and on the selected interpolation method.

A useful first requirement is the reproduction of a constant field. If all source values are equal to c , the mapped value should also be c . This is obtained when:

$$\sum_{i=1}^{n_s} w_i(\mathbf{x}) = 1 \quad (12.104)$$

A stronger requirement is the reproduction of a linear field. If:

$$u(\mathbf{x}) = a_0 + \mathbf{a}^T \mathbf{x} \quad (12.105)$$

then an exactly linearly consistent interpolation should satisfy:

$$\begin{aligned}\sum_i w_i(\mathbf{x}) &= 1 \\ \sum_i w_i(\mathbf{x}) \mathbf{x}_i &= \mathbf{x}\end{aligned}\tag{12.106}$$

The first equation reproduces the constant part and the second equation reproduces the linear part.

12.5.1 Interpolation, Approximation and Extrapolation

The words interpolation, approximation and extrapolation describe different situations:

- **Interpolation** evaluates a field between known values. An interpolating method normally reproduces the source values exactly.
- **Approximation** constructs a smooth or otherwise restricted field that need not pass through every source value. This is useful when the data contain noise.
- **Extrapolation** evaluates the field outside the region covered by the source data. The farther the target lies outside this region, the less information is available to determine the value.

For a point cloud, the natural interpolation region is usually its convex hull. For a finite element mesh, it is the union of the source elements. Inside this region, surrounding source values constrain the estimate from several directions. Outside it, all source points may lie on one side of the target and the result depends more strongly on assumptions made by the method.

A second distinction concerns what the mapping should preserve. Pointwise mapping attempts to reproduce the value at each target point. Conservative mapping instead preserves an integral quantity such as total force, heat, mass or flux. The two objectives are not generally equivalent.

12.5.2 Nearest-Neighbour Mapping

The simplest mapping assigns the value of the closest source point. Let:

$$j = \arg \min_i \|\mathbf{x} - \mathbf{x}_i\| \quad (12.107)$$

then:

$$\hat{u}(\mathbf{x}) = u_j \quad (12.108)$$

The method is piecewise constant. Space is divided into Voronoi cells and every target point in one cell receives the value of the source point belonging to that cell.

Nearest-neighbour mapping is suitable for discrete quantities such as material numbers, region identifiers and contact status. It does not create values that were absent from the source data and therefore cannot overshoot. For a smooth physical field, however, it produces jumps at the Voronoi boundaries and only first-order geometric accuracy.

Python implementation:

```

1 import numpy as np
2
3
4 def Nearest_Neighbour(source_points: np.ndarray,
5                       source_values: np.ndarray,
6                       target_points: np.ndarray) -> np.ndarray:
7     """Map values from source points to target points by nearest neighbour."""
8     source_points = np.asarray(source_points, dtype=float)
9     source_values = np.asarray(source_values)
10    target_points = np.asarray(target_points, dtype=float)
11
12    if source_points.ndim != 2 or target_points.ndim != 2:
13        raise ValueError("Point arrays must have shape (number, dimension).")
14    if source_points.shape[0] != source_values.shape[0]:

```



```

15         raise ValueError("Each source point must have one source value.")
16     if source_points.shape[1] != target_points.shape[1]:
17         raise ValueError("Source and target dimensions must agree.")
18
19     result_shape = (target_points.shape[0],) + source_values.shape[1:]
20     result = np.empty(result_shape, dtype=source_values.dtype)
21
22     for t, point in enumerate(target_points):
23         best_index = 0
24         best_distance_squared = float("inf")
25
26         for i, source_point in enumerate(source_points):
27             difference = point - source_point
28             distance_squared = np.dot(difference, difference)
29             if distance_squared < best_distance_squared:
30                 best_distance_squared = distance_squared
31                 best_index = i
32
33         result[t] = source_values[best_index]
34
35     return result
36
37
38 source_points = np.array([[0.0], [1.0], [2.0]])
39 source_values = np.array([0.0, 10.0, 20.0])
40 target_points = np.array([[0.2], [0.8], [1.6]])
41
42 mapped = Nearest_Neighbour(source_points, source_values, target_points)
43 print(mapped) # [ 0. 10. 20.]

```

A direct search compares every target point with every source point and costs $\mathcal{O}(n_s n_t)$. For large point clouds the same rule is normally accelerated by a spatial search structure such as a uniform grid, a k -d tree or a bounding volume hierarchy.

12.5.3 Inverse-Distance Weighting

Nearest-neighbour mapping uses only one source value. A smoother estimate is obtained when several nearby points contribute with weights that decrease with distance. For inverse-distance weighting:

$$\tilde{w}_i(\mathbf{x}) = \frac{1}{(\|\mathbf{x} - \mathbf{x}_i\| + \epsilon)^p} \quad (12.109)$$

and the normalized weights are:

$$w_i(\mathbf{x}) = \frac{\tilde{w}_i(\mathbf{x})}{\sum_j \tilde{w}_j(\mathbf{x})} \quad (12.110)$$

so that:

$$\hat{u}(\mathbf{x}) = \frac{\sum_i \tilde{w}_i(\mathbf{x}) u_i}{\sum_i \tilde{w}_i(\mathbf{x})} \quad (12.111)$$

The exponent p controls how local the interpolation is. A larger exponent gives more importance to the nearest points. The value $p = 2$ is common, but it is not universally optimal.

If the target coincides with a source point, its value should be returned exactly instead of relying on the regularisation ϵ . In practice the sum is also restricted to the k nearest points or to a radius of influence. This reduces cost and prevents distant data from affecting a local estimate.

Inverse-distance weighting reproduces constant fields because its normalized weights sum to one. It does not generally reproduce linear fields exactly. It is also sensitive to nonuniform point density: a dense cluster can receive a large total weight merely because it contains many points.

Python implementation using the k nearest source points:

```
1 import numpy as np
2
```

```

3
4 def Inverse_Distance_Weighting(source_points: np.ndarray,
5                                source_values: np.ndarray,
6                                target_points: np.ndarray,
7                                power: float = 2.0,
8                                neighbours: int | None = None,
9                                tolerance: float = 1.0e-12) -> np.ndarray:
10     """Map scalar or component-wise values by inverse-distance weighting."""
11     source_points = np.asarray(source_points, dtype=float)
12     source_values = np.asarray(source_values, dtype=float)
13     target_points = np.asarray(target_points, dtype=float)
14
15     if source_points.shape[0] != source_values.shape[0]:
16         raise ValueError("Each source point must have one source value.")
17     if power <= 0.0:
18         raise ValueError("The distance exponent must be positive.")
19
20     n_source = source_points.shape[0]
21     if neighbours is None:
22         neighbours = n_source
23     neighbours = max(1, min(int(neighbours), n_source))
24
25     result_shape = (target_points.shape[0],) + source_values.shape[1:]
26     result = np.empty(result_shape, dtype=float)
27
28     for t, point in enumerate(target_points):
29         difference = source_points - point
30         distance_squared = np.sum(difference * difference, axis=1)
31         closest = int(np.argmin(distance_squared))
32
33         if distance_squared[closest] <= tolerance * tolerance:
34             result[t] = source_values[closest]
35             continue
36
37         indices = np.argsort(distance_squared)[:neighbours]
38         distances = np.sqrt(distance_squared[indices])
39         weights = 1.0 / distances**power
40         weights /= np.sum(weights)
41
42         values = source_values[indices]
43         result[t] = np.tensordot(weights, values, axes=(0, 0))

```

```

44
45     return result
46
47
48 source_points = np.array([[0.0], [1.0], [2.0]])
49 source_values = np.array([0.0, 1.0, 4.0])
50 target_points = np.array([[0.5], [1.5]])
51
52 mapped = Inverse_Distance_Weighting(
53     source_points,
54     source_values,
55     target_points,
56     power=2.0,
57     neighbours=2,
58 )
59 print(mapped)

```

12.5.4 One-Dimensional Interpolation

When the source coordinates are ordered along one axis, the interval containing the target can be found directly. Let:

$$x_i \leq x \leq x_{i+1} \quad (12.112)$$

and define the local coordinate:

$$\xi = \frac{x - x_i}{x_{i+1} - x_i} \quad (12.113)$$

For a target inside the interval, $0 \leq \xi \leq 1$. Piecewise linear interpolation is:

$$\hat{u}(x) = (1 - \xi)u_i + \xi u_{i+1} \quad (12.114)$$

The interpolation is continuous, reproduces every source value and reproduces linear functions exactly. Its derivative is constant within each interval and may jump at a source point.

If the same equation is evaluated with $\xi < 0$ or $\xi > 1$, it becomes linear extrapolation. This preserves the slope of the first or last interval, but the magnitude can grow without bound as the target moves away from the known data.

Python implementation:

```

1 import numpy as np
2
3
4 def Piecewise_Linear_Interpolation(x_source: np.ndarray,
5                                   u_source: np.ndarray,
6                                   x_target: np.ndarray,
7                                   extrapolation: str = "linear") -> np.ndarray:
8     """Interpolate ordered one-dimensional data component by component."""
9     x_source = np.asarray(x_source, dtype=float)
10    u_source = np.asarray(u_source, dtype=float)
11    x_target = np.asarray(x_target, dtype=float)
12
13    if x_source.ndim != 1 or x_source.size < 2:
14        raise ValueError("At least two one-dimensional source coordinates are
15        ↪ required.")
16    if u_source.shape[0] != x_source.size:
17        raise ValueError("Each source coordinate must have one source value.")
18    if np.any(x_source[1:] <= x_source[:-1]):
19        raise ValueError("Source coordinates must be strictly increasing.")
20    if extrapolation not in ("linear", "constant", "error"):
21        raise ValueError("Unknown extrapolation rule.")
22
23    result_shape = x_target.shape + u_source.shape[1:]
24    result = np.empty(result_shape, dtype=float)
25
26    flat_target = x_target.reshape(-1)
27    flat_result = result.reshape((flat_target.size,) + u_source.shape[1:])
28
29    for j, x in enumerate(flat_target):
30        if x < x_source[0]:
31            if extrapolation == "error":
32                raise ValueError("Target lies below the source interval.")
33            if extrapolation == "constant":

```

```

33         flat_result[j] = u_source[0]
34         continue
35     i = 0
36     elif x > x_source[-1]:
37         if extrapolation == "error":
38             raise ValueError("Target lies above the source interval.")
39         if extrapolation == "constant":
40             flat_result[j] = u_source[-1]
41             continue
42         i = x_source.size - 2
43     else:
44         i = int(np.searchsorted(x_source, x, side="right") - 1)
45         i = min(max(i, 0), x_source.size - 2)
46
47         xi = (x - x_source[i]) / (x_source[i + 1] - x_source[i])
48         flat_result[j] = (1.0 - xi) * u_source[i] + xi * u_source[i + 1]
49
50     return result
51
52
53 x_source = np.array([0.0, 1.0, 3.0])
54 u_source = np.array([0.0, 2.0, 4.0])
55 x_target = np.array([-0.5, 0.5, 2.0, 4.0])
56
57 print(Piecewise_Linear_Interpolation(
58     x_source,
59     u_source,
60     x_target,
61     extrapolation="linear",
62 ))

```

Higher-order interpolation uses more than two neighbouring points. A global polynomial through n points may oscillate strongly, especially near the ends of a long interval. Piecewise polynomials avoid this problem.

A cubic spline is piecewise cubic and usually enforces continuity of the first and second derivatives. It is smooth, but it can overshoot between monotone source values. For constitutive curves and other data that must remain monotone, a monotone cubic Hermite method is often safer. Akima interpolation is another

local cubic construction that reduces oscillation near abrupt changes of slope.

12.5.5 Interpolation on Structured Grids

For a Cartesian grid, interpolation may be performed successively along each coordinate. In a rectangular cell define:

$$\begin{aligned}\xi &= \frac{x - x_0}{x_1 - x_0} \\ \eta &= \frac{y - y_0}{y_1 - y_0}\end{aligned}\tag{12.115}$$

The four bilinear shape functions are:

$$\begin{aligned}N_{00} &= (1 - \xi)(1 - \eta) \\ N_{10} &= \xi(1 - \eta) \\ N_{11} &= \xi\eta \\ N_{01} &= (1 - \xi)\eta\end{aligned}\tag{12.116}$$

and:

$$\hat{u}(\xi, \eta) = N_{00}u_{00} + N_{10}u_{10} + N_{11}u_{11} + N_{01}u_{01}\tag{12.117}$$

Bilinear interpolation reproduces functions of the form:

$$u(x, y) = a_0 + a_1x + a_2y + a_3xy\tag{12.118}$$

within an axis-aligned rectangular cell. It is linear along every line parallel to one coordinate axis, but it is not generally linear along an arbitrary diagonal.

For a three-dimensional rectangular cell, trilinear interpolation uses eight corner values:

$$\hat{u}(\xi, \eta, \zeta) = \sum_{i=0}^1 \sum_{j=0}^1 \sum_{k=0}^1 N_i(\xi) N_j(\eta) N_k(\zeta) u_{ijk} \quad (12.119)$$

where:

$$N_0(s) = 1 - s, \quad N_1(s) = s \quad (12.120)$$

The same tensor-product idea extends to bicubic and tricubic interpolation, where values and derivatives from neighbouring grid points are used to obtain a smoother field.

12.5.6 Interpolation on Finite Elements

A source finite element mesh already defines an interpolation field. The mapping problem therefore consists of two steps:

1. find the source element containing the target point,
2. evaluate the source element shape functions at that point.

The element search can dominate the total cost. Testing every target against every source element is rarely acceptable for large meshes. Bounding boxes, uniform spatial bins, octrees and bounding volume hierarchies are commonly used to create a short list of candidate elements.

Barycentric Coordinates in a Triangle

For a triangle with nodes $\mathbf{x}_1$, $\mathbf{x}_2$ and $\mathbf{x}_3$, the target point is written as:

$$\mathbf{x} = \lambda_1 \mathbf{x}_1 + \lambda_2 \mathbf{x}_2 + \lambda_3 \mathbf{x}_3 \quad (12.121)$$

with:

$$\lambda_1 + \lambda_2 + \lambda_3 = 1 \quad (12.122)$$

The barycentric coordinates are also the linear triangle shape functions. The mapped value is:

$$\hat{u}(\mathbf{x}) = \lambda_1 u_1 + \lambda_2 u_2 + \lambda_3 u_3 \quad (12.123)$$

The point lies inside or on the triangle when:

$$0 \leq \lambda_i \leq 1 \quad (12.124)$$

apart from a numerical tolerance. If one coordinate is negative, the point lies outside the edge opposite that node. Evaluating the same equation still gives an affine extrapolation.

For coordinates $\mathbf{x}_i = [x_i, y_i]^T$, solve:

$$\begin{bmatrix} x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (12.125)$$

For a triangle it is convenient to evaluate the closed-form determinant expressions directly.

Python implementation:

```

1 import numpy as np
2
3
4 def Triangle_Barycentric_Coordinates(vertices: np.ndarray,
5                                     point: np.ndarray,
6                                     tolerance: float = 1.0e-14) -> np.ndarray:
7     """Return the three barycentric coordinates of a two-dimensional point."""
8     vertices = np.asarray(vertices, dtype=float)
```

```

9     point = np.asarray(point, dtype=float)
10
11     if vertices.shape != (3, 2) or point.shape != (2,):
12         raise ValueError("Expected three 2D vertices and one 2D point.")
13
14     x1, y1 = vertices[0]
15     x2, y2 = vertices[1]
16     x3, y3 = vertices[2]
17     x, y = point
18
19     denominator = (y2 - y3) * (x1 - x3) + (x3 - x2) * (y1 - y3)
20     if abs(denominator) <= tolerance:
21         raise ValueError("The triangle is degenerate.")
22
23     lambda_1 = ((y2 - y3) * (x - x3)
24                + (x3 - x2) * (y - y3)) / denominator
25     lambda_2 = ((y3 - y1) * (x - x3)
26                + (x1 - x3) * (y - y3)) / denominator
27     lambda_3 = 1.0 - lambda_1 - lambda_2
28
29     return np.array([lambda_1, lambda_2, lambda_3])
30
31
32 def Triangle_Interpolation(vertices: np.ndarray,
33                            values: np.ndarray,
34                            point: np.ndarray,
35                            allow_extrapolation: bool = False,
36                            tolerance: float = 1.0e-12):
37     """Interpolate scalar or component-wise values in a linear triangle."""
38     values = np.asarray(values, dtype=float)
39     barycentric = Triangle_Barycentric_Coordinates(vertices, point)
40
41     if not allow_extrapolation:
42         if np.any(barycentric < -tolerance) or np.any(barycentric > 1.0 +
43             ↪ tolerance):
44             raise ValueError("The target point lies outside the triangle.")
45
46     return np.tensordot(barycentric, values, axes=(0, 0))
47
48 vertices = np.array([[0.0, 0.0],

```

```

49         [2.0, 0.0],
50         [0.0, 1.0]])
51 values = np.array([1.0, 5.0, 4.0])
52 point = np.array([0.5, 0.25])
53
54 weights = Triangle_Barycentric_Coordinates(vertices, point)
55 value = Triangle_Interpolation(vertices, values, point)
56 print(weights, value)

```

The same idea applies to a tetrahedron using four barycentric coordinates. A linear simplex is especially convenient because the natural coordinates are obtained from one linear system and the interpolation reproduces all affine fields exactly.

Isoparametric Shape Functions

For a general finite element, both geometry and field values are interpolated using shape functions in natural coordinates $\boldsymbol{\xi}$:

$$\begin{aligned}
 \mathbf{x}(\boldsymbol{\xi}) &= \sum_{a=1}^{n_e} N_a(\boldsymbol{\xi}) \mathbf{x}_a \\
 \hat{u}(\boldsymbol{\xi}) &= \sum_{a=1}^{n_e} N_a(\boldsymbol{\xi}) u_a
 \end{aligned} \tag{12.126}$$

The target position $\mathbf{x}$ is known, but its natural coordinate $\boldsymbol{\xi}$ is not. It is obtained from the inverse isoparametric mapping:

$$\mathbf{r}(\boldsymbol{\xi}) = \sum_a N_a(\boldsymbol{\xi}) \mathbf{x}_a - \mathbf{x} = \mathbf{0} \tag{12.127}$$

For nonlinear geometry interpolation this equation is solved iteratively. A Newton step is:

$$\boldsymbol{\xi}^{(k+1)} = \boldsymbol{\xi}^{(k)} - \mathbf{J}^{-1}(\boldsymbol{\xi}^{(k)}) \mathbf{r}(\boldsymbol{\xi}^{(k)}) \quad (12.128)$$

where:

$$\mathbf{J} = \frac{\partial \mathbf{x}}{\partial \boldsymbol{\xi}} \quad (12.129)$$

is the geometric Jacobian. A converged natural coordinate must still be checked against the element domain. Newton convergence alone does not prove that the target lies inside the element.

For a four-node quadrilateral with natural coordinates $-1 \leq \xi, \eta \leq 1$, the shape functions are:

$$\begin{aligned} N_1 &= \frac{1}{4}(1 - \xi)(1 - \eta) \\ N_2 &= \frac{1}{4}(1 + \xi)(1 - \eta) \\ N_3 &= \frac{1}{4}(1 + \xi)(1 + \eta) \\ N_4 &= \frac{1}{4}(1 - \xi)(1 + \eta) \end{aligned} \quad (12.130)$$

Python implementation including the inverse mapping:

```

1 import numpy as np
2
3
4 def Quad4_Shape_Functions(xi: float, eta: float):
5     N = 0.25 * np.array([
6         (1.0 - xi) * (1.0 - eta),
7         (1.0 + xi) * (1.0 - eta),
8         (1.0 + xi) * (1.0 + eta),
9         (1.0 - xi) * (1.0 + eta),

```

```

10     ])
11
12     dN_dxi = 0.25 * np.array([
13         -(1.0 - eta),
14         (1.0 - eta),
15         (1.0 + eta),
16         -(1.0 + eta),
17     ])
18
19     dN_deta = 0.25 * np.array([
20         -(1.0 - xi),
21         -(1.0 + xi),
22         (1.0 + xi),
23         (1.0 - xi),
24     ])
25     return N, dN_dxi, dN_deta
26
27
28 def Solve_2x2(A: np.ndarray, b: np.ndarray,
29              tolerance: float = 1.0e-14) -> np.ndarray:
30     determinant = A[0, 0] * A[1, 1] - A[0, 1] * A[1, 0]
31     if abs(determinant) <= tolerance:
32         raise ValueError("Singular two by two system.")
33
34     return np.array([
35         (b[0] * A[1, 1] - A[0, 1] * b[1]) / determinant,
36         (-b[0] * A[1, 0] + A[0, 0] * b[1]) / determinant,
37     ])
38
39
40 def Quad4_Natural_Coordinates(vertices: np.ndarray,
41                               point: np.ndarray,
42                               tolerance: float = 1.0e-12,
43                               maximum_iterations: int = 25) -> np.ndarray:
44     """Find natural coordinates of a point in a bilinear quadrilateral."""
45     vertices = np.asarray(vertices, dtype=float)
46     point = np.asarray(point, dtype=float)
47     natural = np.zeros(2, dtype=float)
48
49     for _ in range(maximum_iterations):
50         xi, eta = natural

```

```

51     N, dN_dxi, dN_deta = Quad4_Shape_Functions(xi, eta)
52
53     mapped_point = N @ vertices
54     residual = mapped_point - point
55     if np.sqrt(np.dot(residual, residual)) <= tolerance:
56         return natural
57
58     Jacobian = np.column_stack((dN_dxi @ vertices,
59                                dN_deta @ vertices))
60     increment = Solve_2x2(Jacobian, residual)
61     natural -= increment
62
63     if np.sqrt(np.dot(increment, increment)) <= tolerance:
64         return natural
65
66     raise RuntimeError("Inverse isoparametric mapping did not converge.")
67
68
69 def Quad4_Interpolation(vertices: np.ndarray,
70                          values: np.ndarray,
71                          point: np.ndarray,
72                          allow_extrapolation: bool = False,
73                          tolerance: float = 1.0e-10):
74     natural = Quad4_Natural_Coordinates(vertices, point)
75     xi, eta = natural
76
77     if not allow_extrapolation:
78         if abs(xi) > 1.0 + tolerance or abs(eta) > 1.0 + tolerance:
79             raise ValueError("The target point lies outside the quadrilateral.")
80
81     N, _, _ = Quad4_Shape_Functions(xi, eta)
82     return np.tensordot(N, np.asarray(values, dtype=float), axes=(0, 0))
83
84
85 vertices = np.array([[0.0, 0.0],
86                      [2.0, 0.2],
87                      [1.8, 1.2],
88                      [-0.1, 1.0]])
89 values = np.array([0.0, 2.0, 4.0, 1.0])
90 point = np.array([0.9, 0.55])
91

```

```

92 print(Quad4_Natural_Coordinates(vertices, point))
93 print(Quad4_Interpolation(vertices, values, point))

```

12.5.7 Extrapolation Outside the Source Domain

Extrapolation requires an assumption about the field where no source data exist. The selected rule should therefore be stated explicitly instead of being hidden inside the mapping implementation.

Nearest-Boundary Projection

A target point outside the source domain may be projected onto the closest source boundary point:

$$\mathbf{x}_p = \arg \min_{\mathbf{y} \in \Gamma_s} \|\mathbf{x} - \mathbf{y}\| \quad (12.131)$$

and the field is extended by:

$$\hat{u}(\mathbf{x}) = u(\mathbf{x}_p) \quad (12.132)$$

The value is constant in the outward normal direction. This rule is bounded and often suitable when a surface result is to be mapped to nearby offset points. It does not continue the normal gradient.

For a polygonal or finite element boundary, the closest point is found by projecting onto candidate edges or faces and clamping the local coordinates to their valid range.

Affine Extrapolation

The shape functions of a linear element can be evaluated outside the element. For a triangle, for example, one or more barycentric coordinates become negative, but:

$$\hat{u}(\mathbf{x}) = \sum_i \lambda_i(\mathbf{x}) u_i \quad (12.133)$$

still continues the same affine field. This is exact when the true field is linear, but it may produce large values far from the element. A distance limit or a transition to a bounded far-field value is therefore commonly applied.

Artificial Bounding Domain

Unrestricted extrapolation may be replaced by interpolation over an enlarged artificial domain. Construct a box Ω_b around the source data and add support points at its corners. Assign these artificial points a prescribed far-field value, for example the source mean:

$$\bar{u} = \frac{1}{n_s} \sum_{i=1}^{n_s} u_i \quad (12.134)$$

The augmented data set is:

$$\begin{aligned} \mathbf{X}_{aug} &= [\mathbf{X}_{source}; \mathbf{X}_{box}] \\ \mathbf{u}_{aug} &= [\mathbf{u}_{source}; \bar{u}, \dots, \bar{u}] \end{aligned} \quad (12.135)$$

A target outside the original source hull but inside the box is then inside the hull of the augmented points. A scattered-data interpolant, a triangulation, an RBF or a moving least-squares method can use these added points.

The box does not add measured information. It imposes a boundary assumption. Its size controls how quickly the estimate approaches the assigned far-field value. A close box creates a strong transition; a distant box allows the local source trend to continue farther. Assigning one global mean is bounded and neutral, but a physical far-field value, a local boundary average or zero may be more appropriate for a particular quantity.

Adding only corners is sufficient to enclose the convex hull, but it may not control the field well along a long box edge. Additional points can therefore be placed on the faces. The distribution of artificial points should not be so dense that it overwhelms the real data in methods whose weights depend on point density.

The following example augments a two-dimensional point cloud with mean-valued box corners and uses the inverse-distance weighting routine defined above for the final interpolation:

```

1 import numpy as np
2
3
4 def Add_Mean_Valued_Box(source_points: np.ndarray,
5                         source_values: np.ndarray,
6                         margin: float = 1.0):
7     """Add the corners of an expanded 2D box to a scattered data set."""
8     source_points = np.asarray(source_points, dtype=float)
9     source_values = np.asarray(source_values, dtype=float)
10
11     if source_points.ndim != 2 or source_points.shape[1] != 2:
12         raise ValueError("This example expects two-dimensional source points.")
13     if margin <= 0.0:
14         raise ValueError("The box margin must be positive.")
15
16     minimum = np.min(source_points, axis=0)
17     maximum = np.max(source_points, axis=0)
18     extent = maximum - minimum
19
20     # A zero extent can occur when all points share one coordinate.
21     scale = max(float(np.max(extent)), 1.0)
22     padding = margin * np.where(extent > 0.0, extent, scale)
23     lower = minimum - padding
24     upper = maximum + padding
25
26     corners = np.array([
27         [lower[0], lower[1]],
28         [upper[0], lower[1]],
29         [upper[0], upper[1]],

```

```

30         [lower[0], upper[1]],
31     ])
32
33     mean_value = np.mean(source_values, axis=0)
34     corner_values = np.empty((4,) + source_values.shape[1:], dtype=float)
35     corner_values[...] = mean_value
36
37     augmented_points = np.concatenate((source_points, corners), axis=0)
38     augmented_values = np.concatenate((source_values, corner_values), axis=0)
39     return augmented_points, augmented_values
40
41
42 source_points = np.array([[0.0, 0.0],
43                           [1.0, 0.0],
44                           [0.0, 1.0],
45                           [1.0, 1.0]])
46 source_values = np.array([0.0, 2.0, 4.0, 6.0])
47
48 target_points = np.array([[1.2, 0.5],
49                           [2.5, 0.5]])
50
51 points, values = Add_Mean_Valued_Box(
52     source_points,
53     source_values,
54     margin=2.0,
55 )
56
57 mapped = Inverse_Distance_Weighting(
58     points,
59     values,
60     target_points,
61     power=2.0,
62     neighbours=6,
63 )
64 print(mapped)

```

The result depends on both the box and the selected interpolant. The method should therefore be validated by varying the box size and checking whether the mapped values in the region of interest remain stable.

12.5.8 Interpolation of Scattered Data

When only scattered source points are available, no element connectivity exists a priori. The interpolation method must construct a neighbourhood or a global basis from the point positions.

Delaunay and Natural-Neighbour Interpolation

A Delaunay triangulation divides the convex hull of a point cloud into simplices. In two dimensions these are triangles; in three dimensions they are tetrahedra. The triangulation is chosen so that no source point lies inside the circumcircle or circumsphere of a simplex.

After the containing simplex has been found, ordinary piecewise linear interpolation uses its barycentric coordinates. The result is continuous and linearly consistent, but its gradient changes between neighbouring simplices.

Natural-neighbour interpolation is based on the dual Voronoi diagram. Insert the target point into the diagram. The area or volume taken from each neighbouring Voronoi cell defines its weight. The weights are local, positive inside the convex hull and sum to one. The method is smoother than piecewise linear Delaunay interpolation and has no arbitrary distance exponent.

Neither method defines a unique value outside the convex hull. Extrapolation must be handled separately, for example by boundary projection or by adding an artificial bounding domain.

Radial Basis Function Interpolation

A radial basis function depends only on the distance from a centre. A global RBF interpolant is:

$$\hat{u}(\mathbf{x}) = \sum_{i=1}^{n_s} \lambda_i \phi(\|\mathbf{x} - \mathbf{x}_i\|) + \mathbf{p}^T(\mathbf{x})\mathbf{c} \quad (12.136)$$

The polynomial term supplies the required polynomial consistency. For an affine term in d dimensions:

$$\mathbf{p}(\mathbf{x}) = [1, x_1, \dots, x_d]^T \quad (12.137)$$

The coefficients follow from:

$$\begin{bmatrix} \Phi & \mathbf{P} \\ \mathbf{P}^T & \mathbf{0} \end{bmatrix} \begin{bmatrix} \lambda \\ \mathbf{c} \end{bmatrix} = \begin{bmatrix} \mathbf{u} \\ \mathbf{0} \end{bmatrix} \quad (12.138)$$

where:

$$\begin{aligned} \Phi_{ij} &= \phi(\|\mathbf{x}_i - \mathbf{x}_j\|) \\ P_{ij} &= p_j(\mathbf{x}_i) \end{aligned} \quad (12.139)$$

Common radial functions include:

$$\begin{aligned} \phi(r) &= r^3 && \text{cubic polyharmonic spline} \\ \phi(r) &= r^2 \ln r && \text{thin-plate spline} \\ \phi(r) &= \exp[-(\epsilon r)^2] && \text{Gaussian} \\ \phi(r) &= \sqrt{1 + (\epsilon r)^2} && \text{multiquadric} \end{aligned} \quad (12.140)$$

Global RBF interpolation is smooth and works in any spatial dimension, but its matrix is dense. The factorisation cost grows approximately with n_s^3 and the storage with n_s^2 . Compactly supported radial functions produce sparse matrices and are therefore preferable for large point sets.

The following implementation uses the cubic kernel $\phi(r) = r^3$ and an affine polynomial term. The Gaussian elimination is included so the numerical steps remain visible:

```

1 import numpy as np
2
3
4 def Gaussian_Elimination(A: np.ndarray,
5                           b: np.ndarray,
6                           tolerance: float = 1.0e-14) -> np.ndarray:
7     """Solve a dense linear system by elimination with partial pivoting."""
8     A = np.asarray(A, dtype=float).copy()
9     b = np.asarray(b, dtype=float).copy()
10
11     flatten = b.ndim == 1
12     if flatten:
13         b = b.reshape(-1, 1)
14     if A.ndim != 2 or A.shape[0] != A.shape[1]:
15         raise ValueError("The coefficient matrix must be square.")
16     if b.shape[0] != A.shape[0]:
17         raise ValueError("The right hand side has an incompatible size.")
18
19     n = A.shape[0]
20     for k in range(n - 1):
21         pivot = k + int(np.argmax(np.abs(A[k:, k])))
22         if abs(A[pivot, k]) <= tolerance:
23             raise ValueError("The interpolation system is singular.")
24
25         if pivot != k:
26             A[[k, pivot], :] = A[[pivot, k], :]
27             b[[k, pivot], :] = b[[pivot, k], :]
28
29         for i in range(k + 1, n):
30             factor = A[i, k] / A[k, k]
31             A[i, k:] -= factor * A[k, k:]
32             b[i, :] -= factor * b[k, :]
33
34     if abs(A[-1, -1]) <= tolerance:
35         raise ValueError("The interpolation system is singular.")
36
37     x = np.zeros_like(b)
38     for i in range(n - 1, -1, -1):
39         residual = b[i, :].copy()
40         for j in range(i + 1, n):

```

```

41         residual -= A[i, j] * x[j, :]
42         x[i, :] = residual / A[i, i]
43
44     return x[:, 0] if flatten else x
45
46
47 def Cubic_RBF_Fit(source_points: np.ndarray,
48                   source_values: np.ndarray):
49     """Fit a cubic RBF with an affine polynomial term."""
50     points = np.asarray(source_points, dtype=float)
51     values = np.asarray(source_values, dtype=float)
52
53     if points.ndim != 2 or values.shape[0] != points.shape[0]:
54         raise ValueError("Source point and value counts must agree.")
55
56     n, dimension = points.shape
57     difference = points[:, None, :] - points[None, :, :]
58     radius = np.sqrt(np.sum(difference * difference, axis=2))
59     Phi = radius**3
60     P = np.column_stack((np.ones(n), points))
61
62     system = np.zeros((n + dimension + 1, n + dimension + 1))
63     system[:, :n] = Phi
64     system[:, n:] = P
65     system[n:, :n] = P.T
66
67     right_shape = (n + dimension + 1,) + values.shape[1:]
68     right = np.zeros(right_shape, dtype=float)
69     right[:, n] = values
70
71     coefficients = Gaussian_Elimination(system, right)
72     radial_coefficients = coefficients[:n]
73     polynomial_coefficients = coefficients[n:]
74     return radial_coefficients, polynomial_coefficients
75
76
77 def Cubic_RBF_Evaluate(source_points: np.ndarray,
78                        radial_coefficients: np.ndarray,
79                        polynomial_coefficients: np.ndarray,
80                        target_points: np.ndarray) -> np.ndarray:
81     points = np.asarray(source_points, dtype=float)

```

```

82     targets = np.asarray(target_points, dtype=float)
83
84     difference = targets[:, None, :] - points[None, :, :]
85     radius = np.sqrt(np.sum(difference * difference, axis=2))
86     Phi_target = radius**3
87     P_target = np.column_stack((np.ones(targets.shape[0]), targets))
88
89     return (np.tensordot(Phi_target, radial_coefficients, axes=(1, 0))
90           + np.tensordot(P_target, polynomial_coefficients, axes=(1, 0)))
91
92
93 source_points = np.array([[0.0, 0.0],
94                           [1.0, 0.0],
95                           [0.0, 1.0],
96                           [1.0, 1.0],
97                           [0.5, 0.4]])
98 source_values = np.array([0.0, 1.0, 1.0, 2.0, 0.7])
99 target_points = np.array([[0.25, 0.25], [0.75, 0.60]])
100
101 radial, polynomial = Cubic_RBF_Fit(source_points, source_values)
102 mapped = Cubic_RBF_Evaluate(
103     source_points,
104     radial,
105     polynomial,
106     target_points,
107 )
108 print(mapped)

```

The polynomial side conditions are not an optional decoration. They remove nonuniqueness for conditionally positive definite kernels and ensure that an affine source field is reproduced exactly. The source points must also support the selected polynomial basis; for an affine basis in two dimensions they must not all lie on one line.

Moving Least Squares

Moving least squares fits a local polynomial around every target point. Let:

$$u_h(\mathbf{y}; \mathbf{x}) = \mathbf{p}^T(\mathbf{y})\mathbf{a}(\mathbf{x}) \quad (12.141)$$

where $\mathbf{x}$ is the target and $\mathbf{y}$ is a source coordinate. The coefficient vector is chosen to minimize:

$$J(\mathbf{a}) = \sum_{i=1}^{n_s} w(\|\mathbf{x} - \mathbf{x}_i\|) [\mathbf{p}^T(\mathbf{x}_i)\mathbf{a} - u_i]^2 \quad (12.142)$$

Differentiation gives the local normal equations:

$$\mathbf{A}(\mathbf{x})\mathbf{a}(\mathbf{x}) = \mathbf{b}(\mathbf{x}) \quad (12.143)$$

with:

$$\begin{aligned} \mathbf{A}(\mathbf{x}) &= \sum_i w_i \mathbf{p}(\mathbf{x}_i) \mathbf{p}^T(\mathbf{x}_i) \\ \mathbf{b}(\mathbf{x}) &= \sum_i w_i \mathbf{p}(\mathbf{x}_i) u_i \end{aligned} \quad (12.144)$$

and the mapped value is:

$$\hat{u}(\mathbf{x}) = \mathbf{p}^T(\mathbf{x})\mathbf{a}(\mathbf{x}) \quad (12.145)$$

MLS is an approximation unless special interpolating weights are used. It is well suited to noisy scattered data and can reproduce polynomials up to the order of its basis. A sufficiently rich and well distributed neighbourhood is required. Near a boundary, all points may lie on one side and the moment matrix $\mathbf{A}$ can become poorly conditioned. Artificial support points or a larger neighbourhood can improve the situation, but they also introduce assumptions.

Kriging

Kriging treats the source values as samples of a spatially correlated random field. The estimate is again a weighted sum:

$$\hat{u}(\mathbf{x}) = \sum_i w_i(\mathbf{x}) u_i \quad (12.146)$$

but the weights are determined from a covariance or variogram model. In ordinary kriging:

$$\begin{bmatrix} \mathbf{C} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ \mu \end{bmatrix} = \begin{bmatrix} \mathbf{c}(\mathbf{x}) \\ 1 \end{bmatrix} \quad (12.147)$$

where:

$$\begin{aligned} C_{ij} &= C(\mathbf{x}_i, \mathbf{x}_j) \\ c_i(\mathbf{x}) &= C(\mathbf{x}_i, \mathbf{x}) \end{aligned} \quad (12.148)$$

The constraint $\mathbf{1}^T \mathbf{w} = 1$ makes the estimate unbiased for an unknown constant mean. Kriging can represent anisotropic correlation, measurement noise and spatial length scales. It also provides an estimate of interpolation uncertainty. These benefits require a justified covariance model and make the method more involved than deterministic mesh-to-mesh interpolation.

12.5.9 Conservative Field Transfer

Pointwise interpolation preserves a field value locally, but it does not normally preserve its integral. Suppose a distributed load p is mapped between two surface meshes. Even if the mapped pressure looks smooth, the total force:

$$\mathbf{F} = \int_{\Gamma} p \mathbf{n} \, d\Gamma \quad (12.149)$$

and the total moment:

$$\mathbf{M} = \int_{\Gamma} (\mathbf{x} - \mathbf{x}_0) \times p \mathbf{n} \, d\Gamma \quad (12.150)$$

may change. This is unacceptable when the mapped field represents a conserved quantity.

L^2 Projection

Let the target field be represented by target shape functions:

$$u_t(\mathbf{x}) = \sum_i N_i^t(\mathbf{x}) u_i^t \quad (12.151)$$

An L^2 projection chooses the target coefficients so that the mapping error is orthogonal to the target approximation space:

$$\int_{\Omega_{overlap}} N_i^t (u_t - u_s) \, d\Omega = 0 \quad (12.152)$$

Substitution gives:

$$\mathbf{M}_t \mathbf{u}_t = \mathbf{C}_{ts} \mathbf{u}_s \quad (12.153)$$

where:

$$\begin{aligned} (M_t)_{ij} &= \int_{\Omega_{overlap}} N_i^t N_j^t \, d\Omega \\ (C_{ts})_{ik} &= \int_{\Omega_{overlap}} N_i^t N_k^s \, d\Omega \end{aligned} \quad (12.154)$$

If the target shape functions form a partition of unity, the constant function belongs to the target space. Summing the projection equations then shows that the scalar integral of the field is preserved. If the coordinate functions also belong to the target space and the same geometry is used for integration, first moments are preserved as well. For vector tractions on differently approximated surfaces, changed normals and geometry still require a separate force and moment check.

The difficult part is not the final linear system. It is the integration over the geometric intersections of source and target elements. In one dimension, the intersections are intervals. In two dimensions, intersecting surface elements form polygons that must be segmented and integrated. In three dimensions, intersecting volumes are still more complicated.

The following one-dimensional example projects a piecewise linear source field onto a different piecewise linear target mesh. Each overlap interval is split at every source and target node, so both fields are polynomial within each integration segment. It uses the Gaussian elimination routine introduced in the RBF example.

```

1 import numpy as np
2
3
4 def Linear_Element_Shape(x: float, x_0: float, x_1: float) -> np.ndarray:
5     length = x_1 - x_0
6     if length <= 0.0:
7         raise ValueError("Element coordinates must be increasing.")
8     xi = (x - x_0) / length
9     return np.array([1.0 - xi, xi])
10
11
12 def Find_Interval(nodes: np.ndarray, x: float) -> int:
13     """Return the element index containing x, including the last end point."""
14     if x < nodes[0] or x > nodes[-1]:
15         raise ValueError("Point lies outside the mesh interval.")
16     index = int(np.searchsorted(nodes, x, side="right") - 1)
17     return min(max(index, 0), nodes.size - 2)
18
19
20 def L2_Project_Linear_1D(source_nodes: np.ndarray,
```

```

21         source_values: np.ndarray,
22         target_nodes: np.ndarray) -> np.ndarray:
23     """Project a linear source field onto a linear target mesh."""
24     source_nodes = np.asarray(source_nodes, dtype=float)
25     source_values = np.asarray(source_values, dtype=float)
26     target_nodes = np.asarray(target_nodes, dtype=float)
27
28     if source_values.shape[0] != source_nodes.size:
29         raise ValueError("Each source node must have one source value.")
30     if np.any(source_nodes[1:] <= source_nodes[:-1]):
31         raise ValueError("Source nodes must be strictly increasing.")
32     if np.any(target_nodes[1:] <= target_nodes[:-1]):
33         raise ValueError("Target nodes must be strictly increasing.")
34     if target_nodes[0] < source_nodes[0] or target_nodes[-1] > source_nodes[-1]:
35         raise ValueError("This example requires the target mesh inside the source
36         ↪ interval.")
37
38     n_target = target_nodes.size
39     mass = np.zeros((n_target, n_target), dtype=float)
40     right_shape = (n_target,) + source_values.shape[1:]
41     right = np.zeros(right_shape, dtype=float)
42
43     # Split the integration domain at every source and target node.
44     break_points = np.unique(np.concatenate((source_nodes, target_nodes)))
45     lower = target_nodes[0]
46     upper = target_nodes[-1]
47     break_points = break_points[(break_points >= lower) & (break_points <=
48     ↪ upper)]
49
50     gauss_coordinate = 1.0 / np.sqrt(3.0)
51     gauss_points = (-gauss_coordinate, gauss_coordinate)
52
53     for a, b in zip(break_points[:-1], break_points[1:]):
54         if b <= a:
55             continue
56
57         middle = 0.5 * (a + b)
58         source_element = Find_Interval(source_nodes, middle)
59         target_element = Find_Interval(target_nodes, middle)
60
61         source_x0 = source_nodes[source_element]

```

```

60     source_x1 = source_nodes[source_element + 1]
61     target_x0 = target_nodes[target_element]
62     target_x1 = target_nodes[target_element + 1]
63
64     for gauss in gauss_points:
65         x = 0.5 * (a + b) + 0.5 * (b - a) * gauss
66         weight = 0.5 * (b - a)
67
68         Ns = Linear_Element_Shape(x, source_x0, source_x1)
69         Nt = Linear_Element_Shape(x, target_x0, target_x1)
70         source_value = np.tensordot(
71             Ns,
72             source_values[source_element:source_element + 2],
73             axes=(0, 0),
74         )
75
76         target_indices = (target_element, target_element + 1)
77         for i_local, i_global in enumerate(target_indices):
78             right[i_global] += weight * Nt[i_local] * source_value
79             for j_local, j_global in enumerate(target_indices):
80                 mass[i_global, j_global] += (
81                     weight * Nt[i_local] * Nt[j_local]
82                 )
83
84     return Gaussian_Elimination(mass, right)
85
86
87 source_nodes = np.array([0.0, 0.4, 1.0, 1.7, 2.0])
88 source_values = np.array([0.0, 1.0, 0.2, 1.4, 1.0])
89 target_nodes = np.array([0.0, 0.7, 1.2, 2.0])
90
91 target_values = L2_Project_Linear_1D(
92     source_nodes,
93     source_values,
94     target_nodes,
95 )
96 print(target_values)

```

The consistent target mass matrix gives the true L^2 projection. A lumped

matrix produces a cheaper local approximation, but changes the projection. When transferring loads, a dual formulation or a mapping of nodal resultants may be used so that the virtual work is preserved.

Mortar Mapping

Mortar methods impose compatibility weakly between nonmatching meshes. Let $\mathbf{u}_s$ and $\mathbf{u}_t$ be source and target traces on an interface and let $\boldsymbol{\lambda}$ be a multiplier field. The weak compatibility condition is:

$$\int_{\Gamma} \boldsymbol{\lambda}^T (\mathbf{u}_t - \mathbf{u}_s) d\Gamma = 0 \quad (12.155)$$

After discretisation this gives coupling matrices built from products of source, target and multiplier shape functions over their overlap. Mortar mappings are used for nonconforming mesh coupling, contact, domain decomposition and fluid-structure interaction.

The multiplier space and numerical integration must be chosen carefully. A poor choice can create rank deficiency or oscillatory transfer. The additional complexity is justified when consistency and conservation across nonmatching meshes are more important than a simple pointwise value.

12.5.10 Mapping Vector and Tensor Quantities

For a vector field expressed in one common Cartesian coordinate system, scalar interpolation weights may be applied component by component:

$$\hat{\mathbf{v}}(\mathbf{x}) = \sum_i w_i(\mathbf{x}) \mathbf{v}_i \quad (12.156)$$

If source and target values use different local bases, the components must first be transformed to a common frame. Let $\mathbf{R}_s$ and $\mathbf{R}_t$ contain the source and target basis vectors. Then:

$$\begin{aligned}\mathbf{v}_{global} &= \mathbf{R}_s \mathbf{v}_{source} \\ \mathbf{v}_{target} &= \mathbf{R}_t^T \mathbf{v}_{global}\end{aligned}\tag{12.157}$$

A second-order tensor transforms as:

$$\mathbf{T}_{target} = \mathbf{R}_t^T \mathbf{R}_s \mathbf{T}_{source} \mathbf{R}_s^T \mathbf{R}_t\tag{12.158}$$

Interpolating tensor components in inconsistent local frames can create a physically meaningless result even when each scalar interpolation is numerically correct.

Rotations require additional care. Rotation vectors are only locally additive. For large rotations, directly averaging Euler angles or rotation matrices does not generally produce a valid or path-independent rotation. Unit quaternions can be interpolated with normalized linear interpolation for nearby rotations or spherical interpolation for two rotations. The sign ambiguity of a quaternion must be resolved before interpolation because $\mathbf{q}$ and $-\mathbf{q}$ represent the same orientation.

12.5.11 Accuracy, Robustness and Validation

A mapping algorithm should be tested with fields whose exact behaviour is known. Useful checks are:

- **Source reproduction:** an interpolating method should return u_i when evaluated at $\mathbf{x}_i$.
- **Partition of unity:** mapping a constant field should return the same constant everywhere in the interpolation domain.
- **Linear consistency:** map an affine field and compare with its exact value at the targets.
- **Conservation:** compare integrated force, moment, mass, heat or flux before and after transfer.

- **Bounds:** check whether a method creates overshoot or values outside physically admissible limits.
- **Extrapolation distance:** report how far every extrapolated target lies from the source domain.
- **Neighbour quality:** detect nearly collinear, coplanar or otherwise insufficient point configurations.
- **Mesh refinement:** repeat the transfer with finer source and target discretisations and check convergence.

For source values u_i^{exact} and mapped values $\hat{u}_i$, common error measures are:

$$\begin{aligned}
 e_{max} &= \max_i |\hat{u}_i - u_i^{exact}| \\
 e_{RMS} &= \sqrt{\frac{1}{n_t} \sum_{i=1}^{n_t} (\hat{u}_i - u_i^{exact})^2} \\
 e_{rel} &= \frac{\sqrt{\sum_i (\hat{u}_i - u_i^{exact})^2}}{\sqrt{\sum_i (u_i^{exact})^2}}
 \end{aligned} \tag{12.159}$$

No single interpolation method is best for every quantity. A practical choice starts from the information available and the property that must be preserved:

- use nearest neighbour for discrete labels,
- use element shape functions when source connectivity is known,
- use IDW, RBF or MLS for scattered continuous data,
- state and limit every extrapolation rule,
- use an L^2 or mortar projection when integral conservation is the primary requirement.

12.6 Modal Assurance Criterion and Similarity Metrics

A comparison of two numerical or experimental models rarely reduces to checking whether individual numbers are equal. Eigenvectors may have different scaling, a reversed sign or a constant complex phase, even when they describe the same physical deformation. Closely spaced modes may also rotate within their common modal subspace, so that two valid bases look different when compared vector by vector.

Similarity measures remove selected ambiguities and reduce the comparison to one or more dimensionless values. The result is useful only when the removed ambiguity is physically irrelevant. A scale-invariant criterion, for example, cannot reveal an amplitude error. Therefore modal correlation should normally be accompanied by natural-frequency differences, scale factors and direct residual measures.

Let ϕ_r denote mode r from a reference model and ψ_s mode s from a second model. Both vectors must first be expressed at the same physical coordinates, in the same order and in the same coordinate systems. A correlation value calculated from mismatched degrees of freedom has no physical meaning, even when the resulting number is close to one.

12.6.1 Complex Inner Product and Normalised Correlation

For real mode shapes, the ordinary dot product is sufficient. Experimental modes, damped-system modes and frequency-response functions may be complex. The corresponding inner product uses the conjugate transpose:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^H \mathbf{y}. \quad (12.160)$$

A positive-definite weighting matrix $\mathbf{W}$ may be introduced:

$$\langle \mathbf{x}, \mathbf{y} \rangle_W = \mathbf{x}^H \mathbf{W} \mathbf{y}. \quad (12.161)$$

With $\mathbf{W} = \mathbf{I}$, every retained coordinate receives the same algebraic weight. A diagonal matrix may represent coordinate importance or quadrature weights. A mass matrix gives the inner product used by structural modes. The weighting matrix must be the same for both vectors and must not create a negative squared norm.

The normalised complex correlation coefficient is:

$$\rho(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^H \mathbf{W} \mathbf{y}}{\sqrt{(\mathbf{x}^H \mathbf{W} \mathbf{x})(\mathbf{y}^H \mathbf{W} \mathbf{y})}}. \quad (12.162)$$

The magnitude satisfies $0 \leq |\rho| \leq 1$. For real vectors it is the cosine of the angle between the two vectors. The argument of ρ gives the constant phase rotation that best aligns the vectors. Multiplying either vector by a non-zero scalar changes this phase but not $|\rho|$.

12.6.2 Modal Assurance Criterion

The **Modal Assurance Criterion** (MAC) is the squared magnitude of the normalised correlation coefficient:

$$\text{MAC}(\phi_r, \psi_s) = \frac{|\phi_r^H \mathbf{W} \psi_s|^2}{(\phi_r^H \mathbf{W} \phi_r)(\psi_s^H \mathbf{W} \psi_s)} = |\rho_{rs}|^2. \quad (12.163)$$

Geometric Interpretation for Real Vectors

For two non-zero real vectors in two or three dimensions, the ordinary dot product has the familiar geometric interpretation

$$\mathbf{x}^T \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos \alpha, \quad (12.164)$$

where $\alpha \in [0, \pi]$ is the angle between the oriented vectors. With the unweighted Euclidean inner product, substitution into Equation 12.163 gives

$$\text{MAC}(\mathbf{x}, \mathbf{y}) = \frac{(\mathbf{x}^T \mathbf{y})^2}{(\mathbf{x}^T \mathbf{x})(\mathbf{y}^T \mathbf{y})} = \cos^2 \alpha. \quad (12.165)$$

The same relation is valid in any finite dimension, but in two and three dimensions it can be visualised directly as the angle between arrows. For example, vectors separated by 60° have

$$\text{MAC} = \cos^2 60^\circ = 0.25. \quad (12.166)$$

Squaring the cosine removes its sign. Parallel vectors ($\alpha = 0$) and antiparallel vectors ($\alpha = \pi$) therefore both have a MAC of one. Likewise, angles α and $\pi - \alpha$ give the same MAC. This is not a defect for mode-shape comparison because the sign of a real eigenvector is arbitrary, but it means that MAC does not preserve the orientation of an ordinary geometric vector.

When the vectors are treated as unoriented lines, the relevant acute angle is

$$\beta = \arccos(\sqrt{\text{MAC}}), \quad 0 \leq \beta \leq \frac{\pi}{2}. \quad (12.167)$$

Thus, $\text{MAC} = 0.81$ corresponds to an unoriented angle $\beta = \arccos(0.9) \approx 25.84^\circ$. The full oriented angle α cannot be recovered from MAC alone; the sign of the normalised dot product is also required.

For a positive-definite weighting matrix $\mathbf{W}$, the same interpretation holds in the weighted geometry:

$$\cos \alpha_W = \frac{\mathbf{x}^T \mathbf{W} \mathbf{y}}{\sqrt{(\mathbf{x}^T \mathbf{W} \mathbf{x})(\mathbf{y}^T \mathbf{W} \mathbf{y})}}, \quad \text{MAC}_W = \cos^2 \alpha_W. \quad (12.168)$$

The angle α_W is generally not the visible Euclidean angle between the two arrows. The weighting changes the geometry by stretching and rotating the coordinate space. A mass-weighted MAC therefore measures the angle induced by the mass matrix.

```

1 import numpy as np
2
3
4 def Angle_And_MAC(vector_1, vector_2, tolerance=1.0e-14):
5     vector_1 = np.asarray(vector_1, dtype=float).reshape(-1)
6     vector_2 = np.asarray(vector_2, dtype=float).reshape(-1)
7
8     if vector_1.size not in (2, 3) or vector_2.size != vector_1.size:
9         raise ValueError("Use two vectors of equal length in 2D or 3D.")
10
11     norm_1 = np.sqrt(np.dot(vector_1, vector_1))
12     norm_2 = np.sqrt(np.dot(vector_2, vector_2))
13
14     if norm_1 <= tolerance or norm_2 <= tolerance:
15         raise ValueError("The angle is undefined for a zero-length vector.")
16
17     cosine = np.dot(vector_1, vector_2) / (norm_1 * norm_2)
18     cosine = min(1.0, max(-1.0, float(cosine)))
19
20     oriented_angle = np.arccos(cosine)
21     mac = cosine**2
22     unoriented_angle = np.arccos(abs(cosine))
23
24     return np.degrees(oriented_angle), mac, np.degrees(unoriented_angle)
25
26
27 x_2d = np.array([1.0, 0.0])

```

```

28 y_2d = np.array([0.5, np.sqrt(3.0) / 2.0])
29 print(Angle_And_MAC(x_2d, y_2d))
30 # (60.0 degrees, 0.25, 60.0 degrees)
31
32 x_3d = np.array([1.0, 0.0, 0.0])
33 y_3d = np.array([-1.0, -1.0, -1.0])
34 print(Angle_And_MAC(x_3d, y_3d))
35 # (125.264 degrees, 1/3, 54.736 degrees)

```

The MAC lies between zero and one. Its limiting values mean:

- $\text{MAC}_{rs} = 1$: the two vectors are linearly dependent,
- $\text{MAC}_{rs} = 0$: the two vectors are orthogonal in the selected inner product.

For a real mode, a sign reversal does not change the MAC:

$$\text{MAC}(\phi, -\phi) = 1. \quad (12.169)$$

For a complex mode, neither does a constant phase rotation:

$$\text{MAC}(\phi, e^{i\theta}\phi) = 1. \quad (12.170)$$

This invariance is desirable because eigenvectors have arbitrary scale and phase. It also explains what MAC does *not* test. A MAC of one does not show that two modes have the same natural frequency, modal mass or physical amplitude. It only shows that the vectors have the same direction in the selected vector space.

There is no universal value that separates a correct mode pair from an incorrect one. The interpretation depends on modal density, the number and placement of observed coordinates, measurement noise and the intended use of the model. A reduced set of coordinates can make physically distinct modes look nearly parallel. A high MAC must therefore be read together with the available spatial information and with the off-diagonal terms of the complete MAC matrix.

```
1 import numpy as np
2
3
4 def Apply_Weight(vector, weight=None):
5     vector = np.asarray(vector)
6
7     if weight is None:
8         return vector
9
10    weight = np.asarray(weight)
11
12    if weight.ndim == 1:
13        if weight.size != vector.shape[0]:
14            raise ValueError("The weight vector has the wrong length.")
15        return weight * vector
16
17    if weight.ndim == 2:
18        if weight.shape != (vector.shape[0], vector.shape[0]):
19            raise ValueError("The weight matrix has the wrong shape.")
20        return weight @ vector
21
22    raise ValueError("Weight must be None, a vector or a square matrix.")
23
24
25 def Weighted_Inner_Product(left, right, weight=None):
26     left = np.asarray(left).reshape(-1)
27     right = np.asarray(right).reshape(-1)
28
29     if left.size != right.size:
30         raise ValueError("The vectors must contain the same coordinates.")
31
32     return np.vdot(left, Apply_Weight(right, weight))
33
34
35 def MAC(reference, comparison, weight=None, tolerance=1.0e-14):
36     reference = np.asarray(reference).reshape(-1)
37     comparison = np.asarray(comparison).reshape(-1)
38
39     cross = Weighted_Inner_Product(reference, comparison, weight)
40     norm_reference = Weighted_Inner_Product(reference, reference, weight).real
```

```

41     norm_comparison = Weighted_Inner_Product(comparison, comparison, weight).real
42
43     if norm_reference <= tolerance or norm_comparison <= tolerance:
44         raise ValueError("MAC is undefined for a zero-length vector.")
45
46     value = abs(cross)**2 / (norm_reference * norm_comparison)
47
48     # Remove only round-off excursions outside the theoretical interval.
49     return float(min(1.0, max(0.0, value.real)))
50
51
52 phi = np.array([1.0, 2.0, -1.0, 0.5])
53 psi = -3.0 * phi
54 chi = np.array([2.0, -1.0, 0.0, 0.0])
55
56 print(MAC(phi, psi)) # 1.0: different scale and sign, same shape
57 print(MAC(phi, chi)) # smaller: a genuinely different vector direction

```

12.6.3 Direction- and Magnitude-Sensitive Vector Measures

MAC is useful when the scale and sign or phase of a vector are arbitrary. It is not a complete comparison of two vectors. In particular,

$$\text{MAC}(\mathbf{x}, \mathbf{x}) = \text{MAC}(\mathbf{x}, -\mathbf{x}) = \text{MAC}(\mathbf{x}, 10\mathbf{x}) = 1. \quad (12.171)$$

The three comparisons have the same vector line, but they do not have the same orientation or magnitude. When these quantities are physically meaningful, MAC should be accompanied by measures that retain them.

Signed Cosine Similarity

For non-zero real vectors, the signed cosine similarity is

$$c(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2} = \cos \alpha. \quad (12.172)$$

Unlike MAC, the signed cosine is not squared. It therefore distinguishes parallel and antiparallel vectors:

$$c(\mathbf{x}, \mathbf{x}) = 1, \quad c(\mathbf{x}, -\mathbf{x}) = -1. \quad (12.173)$$

The measure retains direction but still ignores magnitude. Multiplication of one vector by a positive scalar does not change c . For a negative scalar, only the sign changes. The relation to MAC is

$$\text{MAC}(\mathbf{x}, \mathbf{y}) = c(\mathbf{x}, \mathbf{y})^2. \quad (12.174)$$

Consequently, MAC alone cannot determine whether the signed cosine is $+\sqrt{\text{MAC}}$ or $-\sqrt{\text{MAC}}$.

Norm Ratio and Direct Relative Error

The ratio of the vector magnitudes is

$$r = \frac{\|\mathbf{y}\|_2}{\|\mathbf{x}\|_2}. \quad (12.175)$$

The quantity r describes scale but contains no directional information. A simple magnitude-only error is

$$e_{size} = |r - 1|. \quad (12.176)$$

Direction and magnitude are both retained by the direct relative vector error:

$$e_{rel,x} = \frac{\|\mathbf{y} - \mathbf{x}\|_2}{\|\mathbf{x}\|_2}. \quad (12.177)$$

The subscript x indicates that $\mathbf{x}$ is the reference. Interchanging the vectors generally changes the result. Expanding the squared numerator gives

$$\|\mathbf{y} - \mathbf{x}\|_2^2 = \|\mathbf{x}\|_2^2 + \|\mathbf{y}\|_2^2 - 2\mathbf{x}^T \mathbf{y}, \quad (12.178)$$

$$= \|\mathbf{x}\|_2^2 (1 + r^2 - 2r \cos \alpha). \quad (12.179)$$

Therefore,

$$e_{rel,x}^2 = 1 + r^2 - 2r \cos \alpha = 1 + r^2 - 2rc. \quad (12.180)$$

This equation separates the two sources of disagreement. The norm ratio r contains the scale difference, while the signed cosine c contains the orientation difference. Replacing c by MAC is insufficient because the sign of c has been lost.

For vectors of equal magnitude, $r = 1$, and

$$e_{rel,x} = \sqrt{2 - 2 \cos \alpha} = 2 \sin \left(\frac{\alpha}{2} \right). \quad (12.181)$$

Thus equal and parallel vectors give zero error, equal orthogonal vectors give $\sqrt{2}$, and equal antiparallel vectors give an error of two. MAC gives one for both the first and the last case.

The same definition can use a positive-definite weighting matrix:

$$e_{rel,x,W} = \frac{\|\mathbf{y} - \mathbf{x}\|_W}{\|\mathbf{x}\|_W}, \quad \|\mathbf{z}\|_W = \sqrt{\mathbf{z}^H \mathbf{W} \mathbf{z}}. \quad (12.182)$$

Unlike MAC, the direct complex-vector error is sensitive to a constant phase rotation. This is appropriate when both vectors share a physical phase reference. If their global phase is arbitrary, the phase should first be aligned, for example by the modal scale factor introduced later.

Symmetric Normalised Difference

A reference-independent alternative normalises the difference by the sum of both magnitudes:

$$e_{sym} = \frac{\|\mathbf{x} - \mathbf{y}\|_2}{\|\mathbf{x}\|_2 + \|\mathbf{y}\|_2}. \quad (12.183)$$

The triangle inequality gives

$$0 \leq e_{sym} \leq 1. \quad (12.184)$$

The value is zero only for identical vectors. It reaches one for antiparallel vectors and also when one vector is zero while the other is not. In terms of r and the signed cosine,

$$e_{sym}^2 = \frac{1 + r^2 - 2rc}{(1 + r)^2}. \quad (12.185)$$

A bounded similarity score may be defined as

$$S_{sym} = 1 - e_{sym}. \quad (12.186)$$

This score is convenient for ranking, but the error e_{sym} is usually easier to interpret because it remains a normalised distance. Neither quantity should be called MAC because it tests different properties.

For a unit reference vector, several limiting cases are:

comparison	c	r	MAC	$e_{rel,x}$	e_{sym}
$\mathbf{y} = \mathbf{x}$	1	1	1	0	0
$\mathbf{y} = 2\mathbf{x}$	1	2	1	1	1/3
$\mathbf{y} = -\mathbf{x}$	-1	1	1	2	1
$\mathbf{y} \perp \mathbf{x}, \ \mathbf{y}\ = 1$	0	1	0	$\sqrt{2}$	$1/\sqrt{2}$

The table shows why no single number can simultaneously replace MAC, signed correlation and scale comparison. A useful report may contain the MAC, signed cosine, norm ratio and a direct error. Together they state whether two vectors have the same line, orientation, magnitude and actual coordinates.

For eigenvectors, a direct magnitude comparison is meaningful only after a common normalisation has been imposed. Examples are mass normalisation or unit displacement at the same reference coordinate. Without a common normalisation, the norm ratio merely compares arbitrary eigenvector scaling. For physical displacement vectors, forces or frequency-response functions with a common calibration, the direct errors retain information that MAC intentionally removes.

```

1 import numpy as np
2
3
4 def Euclidean_Norm(vector):
5     vector = np.asarray(vector, dtype=float).reshape(-1)
6     return np.sqrt(np.dot(vector, vector))
7
8
9 def Direction_And_Size_Metrics(reference, comparison,
10                                tolerance=1.0e-14):

```

```

11     reference = np.asarray(reference, dtype=float).reshape(-1)
12     comparison = np.asarray(comparison, dtype=float).reshape(-1)
13
14     if reference.size != comparison.size:
15         raise ValueError("The vectors must have the same length.")
16
17     norm_reference = Euclidean_Norm(reference)
18     norm_comparison = Euclidean_Norm(comparison)
19
20     if norm_reference <= tolerance or norm_comparison <= tolerance:
21         raise ValueError("Direction is undefined for a zero vector.")
22
23     signed_cosine = (
24         np.dot(reference, comparison)
25         / (norm_reference * norm_comparison)
26     )
27     signed_cosine = min(1.0, max(-1.0, float(signed_cosine)))
28
29     difference = comparison - reference
30     norm_difference = Euclidean_Norm(difference)
31     norm_ratio = norm_comparison / norm_reference
32
33     return {
34         "signed_cosine": signed_cosine,
35         "mac": signed_cosine**2,
36         "norm_ratio": norm_ratio,
37         "size_error": abs(norm_ratio - 1.0),
38         "relative_error": norm_difference / norm_reference,
39         "symmetric_error": (
40             norm_difference / (norm_reference + norm_comparison)
41         ),
42     }
43
44
45 x = np.array([1.0, 0.0])
46 comparisons = {
47     "same": np.array([1.0, 0.0]),
48     "twice": np.array([2.0, 0.0]),
49     "opposite": np.array([-1.0, 0.0]),
50     "orthogonal": np.array([0.0, 1.0]),
51 }

```

```

52
53 for name, y in comparisons.items():
54     print(name, Direction_And_Size_Metrics(x, y))

```

12.6.4 AutoMAC and CrossMAC Matrices

A modal model normally contains several vectors. Arranging the modes as columns,

$$\Phi = [\phi_1 \ \phi_2 \ \cdots \ \phi_{n_r}], \quad \Psi = [\psi_1 \ \psi_2 \ \cdots \ \psi_{n_s}], \quad (12.187)$$

allows every pair to be compared. The **CrossMAC** matrix contains:

$$(\mathbf{C}_{MAC})_{rs} = \text{MAC}(\phi_r, \psi_s). \quad (12.188)$$

Rows correspond to reference modes and columns to comparison modes. A clear one-to-one modal correspondence appears as one dominant value in each matched row and column. If two comparison modes both correlate strongly with one reference mode, the causes may include closely spaced modes, an exchanged mode order, insufficient coordinates or an actual modelling difference.

An **AutoMAC** matrix compares a set with itself:

$$(\mathbf{A}_{MAC})_{rs} = \text{MAC}(\phi_r, \phi_s). \quad (12.189)$$

Its diagonal is one. Large off-diagonal terms mean that the retained coordinates do not distinguish the corresponding modes well. Exact modes of a symmetric undamped model are orthogonal with respect to the mass matrix, not necessarily with respect to the unweighted Euclidean product. Consequently, an unweighted AutoMAC may have non-zero off-diagonal terms even for a mathematically correct modal basis.

```
1 import numpy as np
2
3
4 def MAC_Matrix(reference_modes, comparison_modes=None, weight=None):
5     reference_modes = np.asarray(reference_modes)
6
7     if reference_modes.ndim != 2:
8         raise ValueError("Modes must be columns of a two-dimensional array.")
9
10    if comparison_modes is None:
11        comparison_modes = reference_modes
12    else:
13        comparison_modes = np.asarray(comparison_modes)
14
15    if comparison_modes.ndim != 2:
16        raise ValueError("Modes must be columns of a two-dimensional array.")
17
18    if reference_modes.shape[0] != comparison_modes.shape[0]:
19        raise ValueError("Both mode sets must contain the same coordinates.")
20
21    result = np.empty((reference_modes.shape[1], comparison_modes.shape[1]))
22
23    for r in range(reference_modes.shape[1]):
24        for s in range(comparison_modes.shape[1]):
25            result[r, s] = MAC(
26                reference_modes[:, r],
27                comparison_modes[:, s],
28                weight,
29            )
30
31    return result
32
33
34 reference = np.array([
35     [ 1.0,  0.0,  1.0],
36     [ 0.0,  1.0,  1.0],
37     [-1.0,  0.0,  1.0],
38     [ 0.0, -1.0,  1.0],
39 ])
40
```

```

41 # The first two modes are exchanged and slightly perturbed.
42 comparison = np.array([
43     [ 0.05,  1.00,  1.00],
44     [ 1.00,  0.05,  0.95],
45     [-0.05, -1.00,  1.05],
46     [-1.00, -0.05,  1.00],
47 ])
48
49 print(MAC_Matrix(reference, comparison))

```

12.6.5 Weighted MAC and Cross-Orthogonality

When complete finite element mode shapes and a compatible mass matrix are available, the natural structural inner product is obtained with $\mathbf{W} = \mathbf{M}$. The mass-weighted MAC becomes:

$$\text{MAC}_M(\phi_r, \psi_s) = \frac{|\phi_r^H \mathbf{M} \psi_s|^2}{(\phi_r^H \mathbf{M} \phi_r)(\psi_s^H \mathbf{M} \psi_s)}. \quad (12.190)$$

A full mass matrix cannot normally be applied directly to a sparse set of test coordinates. Extracting only the measured rows and columns of $\mathbf{M}$ does not in general preserve the inner product of the omitted coordinates. In that case an identity or carefully defined diagonal weighting is more transparent than a matrix described incorrectly as the physical mass matrix.

Before taking the absolute square, the normalised weighted products form a **cross-orthogonality matrix**:

$$C_{rs} = \frac{\phi_r^H \mathbf{W} \psi_s}{\sqrt{(\phi_r^H \mathbf{W} \phi_r)(\psi_s^H \mathbf{W} \psi_s)}}. \quad (12.191)$$

Thus:

$$\text{MAC}_{rs} = |C_{rs}|^2. \quad (12.192)$$

The MAC discards the sign or phase of C_{rs} , while the cross-orthogonality matrix retains it. If two real mass-normalised modal bases have the same order and orientation, $\mathbf{C}$ is close to the identity. A diagonal entry of -1 means that the shape is identical but its arbitrary sign is reversed.

```

1 import numpy as np
2
3
4 def Normalise_Mode(mode, weight=None, tolerance=1.0e-14):
5     mode = np.asarray(mode)
6     norm_squared = Weighted_Inner_Product(mode, mode, weight).real
7
8     if norm_squared <= tolerance:
9         raise ValueError("A zero mode cannot be normalised.")
10
11     return mode / np.sqrt(norm_squared)
12
13
14 def Cross_Orthogonality(reference_modes, comparison_modes, weight=None):
15     reference_modes = np.asarray(reference_modes)
16     comparison_modes = np.asarray(comparison_modes)
17
18     if reference_modes.shape[0] != comparison_modes.shape[0]:
19         raise ValueError("Both sets must contain the same coordinates.")
20
21     reference_normalised = np.column_stack([
22         Normalise_Mode(reference_modes[:, i], weight)
23         for i in range(reference_modes.shape[1])
24     ])
25     comparison_normalised = np.column_stack([
26         Normalise_Mode(comparison_modes[:, i], weight)
27         for i in range(comparison_modes.shape[1])
28     ])
29
30     weighted_comparison = np.column_stack([
31         Apply_Weight(comparison_normalised[:, i], weight)
32         for i in range(comparison_normalised.shape[1])
33     ])
34

```



```

35     return reference_normalised.conj().T @ weighted_comparison
36
37
38 mass = np.diag([2.0, 1.0, 2.0, 1.0])
39 cross_orthogonality = Cross_Orthogonality(reference, comparison, mass)
40 weighted_mac = abs(cross_orthogonality)**2
41
42 print(cross_orthogonality)
43 print(weighted_mac)

```

12.6.6 Modal Scale Factor and Scaled Residual

MAC intentionally ignores scale. To compare amplitudes, choose a reference mode ϕ and scale the comparison mode ψ by a complex factor a . The factor that minimises the weighted least-squares residual

$$J(a) = \|\phi - a\psi\|_W^2 \quad (12.193)$$

is the **Modal Scale Factor** (MSF):

$$a_{MSF} = \frac{\psi^H \mathbf{W} \phi}{\psi^H \mathbf{W} \psi}. \quad (12.194)$$

The scaled comparison vector is:

$$\hat{\psi} = a_{MSF} \psi. \quad (12.195)$$

For real modes, the sign of a_{MSF} aligns the arbitrary modal sign. For complex modes, its argument aligns the constant phase and its magnitude aligns the scale. The remaining relative error is:

$$e_{mode} = \frac{\|\phi - a_{MSF}\psi\|_W}{\|\phi\|_W}. \quad (12.196)$$

With the same inner product and the optimum complex scale factor, Equations (12.163) and (12.196) are related by:

$$e_{mode} = \sqrt{1 - \text{MAC}(\phi, \psi)}. \quad (12.197)$$

The relation is useful for interpretation: MAC close to one corresponds to a small angular residual. It also means that this particular scaled residual does not provide information independent of MAC. The scale factor itself does.

```

1 import numpy as np
2
3
4 def Modal_Scale_Factor(reference, comparison, weight=None,
5                       tolerance=1.0e-14):
6     denominator = Weighted_Inner_Product(comparison, comparison, weight)
7
8     if denominator.real <= tolerance:
9         raise ValueError("The comparison mode has zero length.")
10
11     numerator = Weighted_Inner_Product(comparison, reference, weight)
12     return numerator / denominator
13
14
15 def Scaled_Modal_Error(reference, comparison, weight=None):
16     reference = np.asarray(reference)
17     comparison = np.asarray(comparison)
18
19     scale = Modal_Scale_Factor(reference, comparison, weight)
20     residual = reference - scale * comparison
21
22     residual_norm = Weighted_Inner_Product(residual, residual, weight).real
23     reference_norm = Weighted_Inner_Product(reference, reference, weight).real
24

```

```

25     return scale, np.sqrt(residual_norm / reference_norm)
26
27
28 reference_mode = np.array([1.0, 2.0, -1.0, 0.5])
29 comparison_mode = 2.0j * reference_mode + np.array([0.00, 0.02, 0.00, -0.01])
30
31 scale, error = Scaled_Modal_Error(reference_mode, comparison_mode)
32 mac = MAC(reference_mode, comparison_mode)
33
34 print(scale)
35 print(error)
36 print(np.sqrt(1.0 - mac))

```

12.6.7 Coordinate Modal Assurance Criterion

MAC indicates whether a pair of complete mode vectors agrees, but it does not show where the disagreement occurs. The **Coordinate Modal Assurance Criterion** (COMAC) reverses the comparison. First, m corresponding mode pairs are selected and scaled consistently. Then the values at one coordinate j are compared over all paired modes:

$$\text{COMAC}_j = \frac{|\sum_{r=1}^m \overline{\phi_{jr}} \psi_{jr}|^2}{\left(\sum_{r=1}^m |\phi_{jr}|^2\right) \left(\sum_{r=1}^m |\psi_{jr}|^2\right)}. \quad (12.198)$$

COMAC is therefore a MAC performed along the modal direction of the two mode shape matrices at a fixed physical coordinate. A value near one means that this coordinate behaves consistently over the selected mode pairs. A low value marks a coordinate that contributes strongly to the disagreement.

COMAC requires more preparation than MAC:

1. the two mode sets must first be paired,
2. arbitrary sign or phase differences must be aligned,
3. the modes should be normalised or otherwise scaled consistently,
4. coordinates with negligible amplitude over all selected modes should not be interpreted as reliable localisation information.

A low COMAC value is not automatically evidence of a local modelling defect. It may also be caused by a sensor orientation error, coordinate mapping error, measurement noise, a poorly paired mode or a node located close to modal lines.

```

1 import numpy as np
2
3
4 def Align_Mode_Set(reference_modes, comparison_modes, weight=None):
5     reference_modes = np.asarray(reference_modes)
6     comparison_modes = np.asarray(comparison_modes)
7
8     if reference_modes.shape != comparison_modes.shape:
9         raise ValueError("COMAC requires already paired mode sets.")
10
11     aligned = comparison_modes.astype(
12         np.result_type(reference_modes, comparison_modes, complex),
13         copy=True,
14     )
15
16     for mode in range(reference_modes.shape[1]):
17         scale = Modal_Scale_Factor(
18             reference_modes[:, mode],
19             aligned[:, mode],
20             weight,
21         )
22         aligned[:, mode] *= scale
23
24     return aligned
25
26

```

```

27 def COMAC(reference_modes, comparison_modes, align=True,
28           tolerance=1.0e-14):
29     reference_modes = np.asarray(reference_modes)
30     comparison_modes = np.asarray(comparison_modes)
31
32     if reference_modes.shape != comparison_modes.shape:
33         raise ValueError("The paired mode matrices must have the same shape.")
34
35     if align:
36         comparison_modes = Align_Mode_Set(reference_modes, comparison_modes)
37
38     numerator = abs(
39         np.sum(np.conj(reference_modes) * comparison_modes, axis=1)
40     )**2
41     denominator = (
42         np.sum(abs(reference_modes)**2, axis=1)
43         * np.sum(abs(comparison_modes)**2, axis=1)
44     )
45
46     result = np.full(reference_modes.shape[0], np.nan, dtype=float)
47     valid = denominator > tolerance
48     result[valid] = numerator[valid] / denominator[valid]
49     return np.clip(result, 0.0, 1.0)
50
51
52 paired_reference = reference[:, :2]
53 paired_comparison = comparison[:, [1, 0]]
54
55 # Introduce a local discrepancy at coordinate 2.
56 paired_comparison = paired_comparison.copy()
57 paired_comparison[2, :] += np.array([0.45, -0.35])
58
59 print(COMAC(paired_reference, paired_comparison))

```

12.6.8 Repeated Modes and Subspace Similarity

Individual vectors are not always the correct objects to compare. Consider a repeated eigenvalue. Any linearly independent rotation of its modal basis spans

the same physical eigenspace. Two solvers may therefore return:

$$\Psi = \Phi \mathbf{R}, \quad (12.199)$$

where $\mathbf{R}$ is an orthogonal or unitary matrix. Individual diagonal MAC values can then be low even though both bases describe exactly the same subspace.

Let $\mathbf{Q}_\Phi$ and $\mathbf{Q}_\Psi$ contain weighted-orthonormal bases of two subspaces with the same dimension p :

$$\mathbf{Q}_\Phi^H \mathbf{W} \mathbf{Q}_\Phi = \mathbf{I}, \quad \mathbf{Q}_\Psi^H \mathbf{W} \mathbf{Q}_\Psi = \mathbf{I}. \quad (12.200)$$

The singular values of $\mathbf{Q}_\Phi^H \mathbf{W} \mathbf{Q}_\Psi$ are the cosines of the principal angles between the subspaces. A simple basis-invariant mean similarity is:

$$S_{sub} = \frac{1}{p} \|\mathbf{Q}_\Phi^H \mathbf{W} \mathbf{Q}_\Psi\|_F^2. \quad (12.201)$$

The value is one when the subspaces are identical and zero when they are orthogonal. Unlike a diagonal MAC average, S_{sub} does not depend on the particular basis chosen inside either subspace. For unequal subspace dimensions, normalising by the smaller dimension answers whether the smaller subspace is contained in the larger one.

```

1 import numpy as np
2
3
4 def Weighted_Gram_Schmidt(vectors, weight=None, tolerance=1.0e-12):
5     vectors = np.asarray(vectors)
6     basis = []
7
```

```

8     for column in range(vectors.shape[1]):
9         vector = vectors[:, column].astype(
10             np.result_type(vectors, complex),
11             copy=True,
12         )
13
14         # Two passes reduce the loss of orthogonality.
15         for _ in range(2):
16             for q in basis:
17                 vector -= q * Weighted_Inner_Product(q, vector, weight)
18
19             norm_squared = Weighted_Inner_Product(vector, vector, weight).real
20             if norm_squared > tolerance**2:
21                 basis.append(vector / np.sqrt(norm_squared))
22
23     if not basis:
24         raise ValueError("The vectors do not span a non-zero subspace.")
25
26     return np.column_stack(basis)
27
28
29 def Subspace_Similarity(first, second, weight=None):
30     q_first = Weighted_Gram_Schmidt(first, weight)
31     q_second = Weighted_Gram_Schmidt(second, weight)
32
33     weighted_second = np.column_stack([
34         Apply_Weight(q_second[:, i], weight)
35         for i in range(q_second.shape[1])
36     ])
37     cross = q_first.conj().T @ weighted_second
38
39     dimension = min(q_first.shape[1], q_second.shape[1])
40     return float(np.sum(abs(cross)**2).real / dimension)
41
42
43 subspace_a = np.array([
44     [1.0, 0.0],
45     [0.0, 1.0],
46     [0.0, 0.0],
47 ])
48

```

```

49 angle = 0.73
50 rotation = np.array([
51     [np.cos(angle), -np.sin(angle)],
52     [np.sin(angle), np.cos(angle)],
53 ])
54 subspace_b = subspace_a @ rotation
55
56 print(MAC_Matrix(subspace_a, subspace_b))
57 print(Subspace_Similarity(subspace_a, subspace_b)) # exactly the same plane

```

12.6.9 Natural-Frequency Agreement and Mode Pairing

MAC compares shapes and is independent of eigenvalues. For a candidate pair, the signed relative natural-frequency difference may be written:

$$e_{f,rs} = \frac{f_s^{comparison} - f_r^{reference}}{f_r^{reference}}. \quad (12.202)$$

The absolute percentage error is $100|e_{f,rs}|$. Frequency agreement alone is not sufficient for pairing because different modes may cross or lie close together. MAC alone is also insufficient because spatially undersampled modes may look similar. A practical pairing procedure considers both.

One possible dimensionless assignment cost is:

$$c_{rs} = w_{MAC} (1 - \text{MAC}_{rs}) + w_f e_{f,rs}^2, \quad (12.203)$$

where the weights state the relative importance assigned by the analyst. This is not a universal physical law; it is a transparent optimisation criterion. The complete set of pairs should be selected as a global assignment problem. Choosing the largest entry independently in every MAC row can assign one comparison mode to several reference modes and may miss the best overall pairing.

After pairing, report at least the mode numbers, both frequencies, relative frequency error, MAC and modal scale factor. For repeated or strongly coupled modes, report subspace similarity in place of forcing an arbitrary one-to-one vector pairing.

12.6.10 Frequency Response Assurance Criterion

Mode-shape correlation requires modal parameter extraction. The measured and calculated frequency-response functions can instead be compared directly. Let $\mathbf{h}_A$ and $\mathbf{h}_B$ contain complex FRF samples from the same frequencies and channels. The **Frequency Response Assurance Criterion** (FRAC) has the same algebraic form as MAC:

$$\text{FRAC}(\mathbf{h}_A, \mathbf{h}_B) = \frac{|\mathbf{h}_A^H \mathbf{W}_f \mathbf{h}_B|^2}{(\mathbf{h}_A^H \mathbf{W}_f \mathbf{h}_A)(\mathbf{h}_B^H \mathbf{W}_f \mathbf{h}_B)}. \quad (12.204)$$

The vectors may be constructed in different ways:

- for one input-output pair, stack the FRF values over a selected frequency range,
- at one frequency, stack all measured response channels,
- for a global comparison, stack several channels and frequency lines into one vector.

The construction must be stated because it determines what a single FRAC value means. A global value can hide a narrow frequency range with poor correlation. A frequency-by-frequency or moving-window FRAC reveals where the disagreement occurs.

FRAC is invariant to one constant complex scale factor over the selected vector. Thus two FRFs that differ only by a constant gain and phase shift have FRAC one. The corresponding complex scale factor is:

$$a_H = \frac{\mathbf{h}_B^H \mathbf{W}_f \mathbf{h}_A}{\mathbf{h}_B^H \mathbf{W}_f \mathbf{h}_B}. \quad (12.205)$$

The unsquared normalised correlation coefficient from Equation (12.162) is also useful for FRFs. Its magnitude is $\sqrt{\text{FRAC}}$, while its argument gives the best constant phase offset between the two selected FRF vectors.

Frequency weights can prevent a densely sampled or very large-amplitude region from dominating the comparison. For non-uniform frequency spacing, quadrature weights represent the integration intervals. Logarithmic amplitude comparison may be useful when anti-resonances and low-amplitude regions are important, but it answers a different question from complex FRAC and should be reported separately.

```

1 import numpy as np
2
3
4 def Complex_Correlation(reference, comparison, weight=None,
5                         tolerance=1.0e-14):
6     reference = np.asarray(reference).reshape(-1)
7     comparison = np.asarray(comparison).reshape(-1)
8
9     cross = Weighted_Inner_Product(reference, comparison, weight)
10    norm_reference = Weighted_Inner_Product(reference, reference, weight).real
11    norm_comparison = Weighted_Inner_Product(comparison, comparison, weight).real
12
13    if norm_reference <= tolerance or norm_comparison <= tolerance:
14        raise ValueError("Correlation is undefined for a zero vector.")
15
16    return cross / np.sqrt(norm_reference * norm_comparison)
17
18
19 def FRAC(reference_frf, comparison_frf, weight=None):
20     correlation = Complex_Correlation(reference_frf, comparison_frf, weight)
21     return float(abs(correlation)**2)
22
23
24 def Moving_FRAC(reference_frf, comparison_frf, half_window):
25     reference_frf = np.asarray(reference_frf).reshape(-1)
26     comparison_frf = np.asarray(comparison_frf).reshape(-1)
27
28     if reference_frf.size != comparison_frf.size:

```

```

29         raise ValueError("The FRFs must use the same frequency lines.")
30
31     result = np.empty(reference_frf.size)
32
33     for centre in range(reference_frf.size):
34         start = max(0, centre - half_window)
35         stop = min(reference_frf.size, centre + half_window + 1)
36         result[centre] = FRAC(
37             reference_frf[start:stop],
38             comparison_frf[start:stop],
39         )
40
41     return result
42
43
44 frequency = np.linspace(0.0, 200.0, 401)
45 reference_frf = 1.0 / (100.0**2 - frequency**2 + 2.0j * 0.02 * 100.0 * frequency)
46 comparison_frf = 1.0 / (103.0**2 - frequency**2 + 2.0j * 0.025 * 103.0 *
47     ↪ frequency)
48
49 correlation = Complex_Correlation(reference_frf, comparison_frf)
50 print(FRAC(reference_frf, comparison_frf))
51 print(abs(correlation), np.angle(correlation))
52 local_frac = Moving_FRAC(reference_frf, comparison_frf, half_window=10)
53 worst = np.argmin(local_frac)
54 print(frequency[worst], local_frac[worst])

```

12.6.11 Direct Error Measures

Correlation criteria measure alignment but intentionally discard some information. Direct errors should therefore accompany them. For complex vectors sampled with non-negative weights w_i , the relative RMS error is:

$$e_{RMS,rel} = \sqrt{\frac{\sum_i w_i |x_i - y_i|^2}{\sum_i w_i |x_i|^2}}. \quad (12.206)$$

Unlike MAC or FRAC, this measure detects amplitude differences. It is not

symmetric: the first vector is the reference. A symmetric normalised error can be defined by dividing by the mean energy of both vectors:

$$e_{sym} = \sqrt{\frac{2 \sum_i w_i |x_i - y_i|^2}{\sum_i w_i |x_i|^2 + \sum_i w_i |y_i|^2}}. \quad (12.207)$$

For FRFs it is often useful to separate magnitude and phase. A logarithmic magnitude difference is:

$$e_{dB,i} = 20 \log_{10} \frac{|y_i|}{|x_i|}, \quad (12.208)$$

while the wrapped phase difference is:

$$\Delta\varphi_i = \text{atan2}(\sin(\varphi_{y,i} - \varphi_{x,i}), \cos(\varphi_{y,i} - \varphi_{x,i})). \quad (12.209)$$

Phase is poorly defined where both magnitudes are close to zero. A phase-error summary should therefore use an amplitude threshold or physically justified weights rather than treating every frequency line equally.

```

1 import numpy as np
2
3
4 def Relative_RMS_Error(reference, comparison, weight=None,
5                       tolerance=1.0e-14):
6     reference = np.asarray(reference).reshape(-1)
7     comparison = np.asarray(comparison).reshape(-1)
8
9     if reference.size != comparison.size:
10         raise ValueError("Both vectors must have the same length.")
11
12     residual = reference - comparison
13     numerator = Weighted_Inner_Product(residual, residual, weight).real
14     denominator = Weighted_Inner_Product(reference, reference, weight).real
15

```

12.6. MODAL ASSURANCE CRITERION AND SIMILARITY METRICS 429

```

16     if denominator <= tolerance:
17         raise ValueError("The reference vector has zero energy.")
18
19     return np.sqrt(numerator / denominator)
20
21
22 def Symmetric_Normalised_Error(first, second, weight=None,
23                                tolerance=1.0e-14):
24     first = np.asarray(first).reshape(-1)
25     second = np.asarray(second).reshape(-1)
26     residual = first - second
27
28     numerator = 2.0 * Weighted_Inner_Product(residual, residual, weight).real
29     denominator = (
30         Weighted_Inner_Product(first, first, weight).real
31         + Weighted_Inner_Product(second, second, weight).real
32     )
33
34     if denominator <= tolerance:
35         raise ValueError("Both vectors have zero energy.")
36
37     return np.sqrt(numerator / denominator)
38
39
40 def Magnitude_And_Phase_Error(reference, comparison, minimum_magnitude=0.0):
41     reference = np.asarray(reference).reshape(-1)
42     comparison = np.asarray(comparison).reshape(-1)
43
44     magnitude_reference = abs(reference)
45     magnitude_comparison = abs(comparison)
46     valid = (
47         (magnitude_reference > minimum_magnitude)
48         & (magnitude_comparison > minimum_magnitude)
49     )
50
51     magnitude_db = np.full(reference.size, np.nan)
52     phase = np.full(reference.size, np.nan)
53
54     magnitude_db[valid] = 20.0 * np.log10(
55         magnitude_comparison[valid] / magnitude_reference[valid]
56     )

```

```

57
58     phase_difference = np.angle(comparison[valid]) - np.angle(reference[valid])
59     phase[valid] = np.arctan2(
60         np.sin(phase_difference),
61         np.cos(phase_difference),
62     )
63
64     return magnitude_db, phase
65
66
67 print(Relative_RMS_Error(reference_frf, comparison_frf))
68 print(Symmetric_Normalised_Error(reference_frf, comparison_frf))

```

12.6.12 Choosing and Reporting Similarity Measures

The metrics answer different questions:

- **MAC** asks whether two mode vectors have the same shape after arbitrary scaling and constant phase are ignored.
- **AutoMAC** asks whether the retained coordinates distinguish the modes within one set.
- **Cross-orthogonality** retains the sign or phase of normalised weighted modal products.
- **MSF** gives the scale and constant phase required to align one vector with another.
- **COMAC** localises coordinates that disagree over an already paired and consistently scaled group of modes.
- **Subspace similarity** compares repeated or closely spaced modal groups without depending on the arbitrary basis inside them.
- **FRAC** applies the same normalised complex correlation idea to frequency-response data.
- **Direct RMS, magnitude and phase errors** retain amplitude and local error information that correlation criteria discard.

A defensible modal comparison should document:

1. the common coordinates and their transformations,
2. omitted and interpolated degrees of freedom,
3. the weighting inner product,
4. the mode-pairing rule,
5. natural-frequency errors,
6. MAC or subspace similarity,
7. modal scale factors and direct residuals,
8. the frequency and coordinate ranges used for every FRF metric.

No scalar criterion can prove that two models are physically equivalent. A similarity number is a compressed diagnostic. Its value is understood only through the data included in the comparison and through the information that the normalisation deliberately removes.

12.7 Fast Fourier Transform (FFT)

Many measured and simulated quantities are functions of time. A displacement, force or acceleration history may be difficult to interpret directly when several oscillations are superposed. The Fourier transform rewrites the same signal as a combination of harmonic functions and thereby shows which frequencies are present.

The **Discrete Fourier Transform** (DFT) is the transform applied to a finite sequence of sampled values. The **Fast Fourier Transform** (FFT) is not a different transform. It is an efficient algorithm for evaluating the DFT. This distinction is important: the sampling, scaling and leakage properties belong to the DFT, while the FFT only reduces the amount of work required to calculate it.

12.7.1 Discrete Fourier Transform

Let a continuous signal $x(t)$ be sampled with a constant sampling interval Δt . A record containing N samples is written as:

$$\begin{aligned} x_n &= x(n\Delta t), & n &= 0, 1, \dots, N-1 \\ f_s &= \frac{1}{\Delta t} \\ T &= N\Delta t \end{aligned} \tag{12.210}$$

where f_s is the sampling frequency and T is the length of the sampled record. The DFT coefficients are:

$$X_k = \sum_{n=0}^{N-1} x_n \exp\left(-i 2\pi \frac{kn}{N}\right), \quad k = 0, 1, \dots, N-1 \tag{12.211}$$

The inverse transform reconstructs the samples:

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \exp\left(+i 2\pi \frac{kn}{N}\right) \tag{12.212}$$

This book uses the common convention in which the forward transform is unscaled and the factor $1/N$ is applied to the inverse transform. Other software may distribute the scaling differently, but the product of the forward and inverse scaling factors must equal $1/N$.

Each coefficient X_k is the projection of the sampled signal onto a complex harmonic basis vector. Its associated frequency is:

$$f_k = \frac{k f_s}{N} = \frac{k}{T} \tag{12.213}$$

and therefore the frequency spacing is:

$$\Delta f = \frac{f_s}{N} = \frac{1}{T} \quad (12.214)$$

A longer record improves the spacing of the frequency bins. Increasing only the sampling frequency while keeping the record length unchanged does not improve Δf ; it merely increases the highest representable frequency and the number of samples.

12.7.2 Frequencies of the DFT Bins

The DFT returns an array of N complex coefficients. The array index k is not itself a frequency; it must be converted using the sampling frequency and the number of samples. For the first part of the array, the relation is:

$$f_k = k\Delta f = \frac{kf_s}{N} \quad (12.215)$$

The DFT output is ordered with the zero-frequency bin first, followed by the positive-frequency bins and then the negative-frequency bins. A signed bin index may be defined as:

$$\tilde{k} = \begin{cases} k, & 0 \leq k \leq \lfloor N/2 \rfloor \\ k - N, & \lfloor N/2 \rfloor < k < N \end{cases} \quad (12.216)$$

and the corresponding signed frequency is:

$$f_k = \tilde{k} \frac{f_s}{N} \quad (12.217)$$

For an even number of samples, the bin $k = N/2$ lies at the Nyquist frequency $f_s/2$. The sampled values cannot distinguish $+f_s/2$ from $-f_s/2$, so either sign may be used when labelling the full spectrum. This book labels that bin as positive. For an odd number of samples, no bin lies exactly at the Nyquist frequency.

For example, let $N = 8$ and $f_s = 800$ Hz. The bin spacing is:

$$\Delta f = \frac{800}{8} = 100 \text{ Hz} \quad (12.218)$$

and the full DFT array corresponds to:

$$[0, 100, 200, 300, 400, -300, -200, -100] \text{ Hz} \quad (12.219)$$

For a real-valued signal, only the non-negative part is normally retained. Its frequency array contains $\lfloor N/2 \rfloor + 1$ bins:

$$f_k = \frac{k f_s}{N}, \quad k = 0, 1, \dots, \lfloor N/2 \rfloor \quad (12.220)$$

The following function constructs either frequency array directly from the sampling information:

```

1 import numpy as np
2
3
4 def DFT_Frequencies(n_samples: int,
5                     sample_rate: float,
6                     one_sided: bool = False) -> np.ndarray:
7     """Return the frequencies associated with DFT array indices."""
8     if n_samples < 1:
9         raise ValueError("n_samples must be positive")
10    if sample_rate <= 0.0:
11        raise ValueError("sample_rate must be positive")
12
13    if one_sided:
14        n_bins = n_samples // 2 + 1
15        return np.arange(n_bins) * sample_rate / n_samples
16
17    k = np.arange(n_samples)
18    signed_k = np.where(k <= n_samples // 2, k, k - n_samples)

```

```

19     return signed_k * sample_rate / n_samples
20
21
22 full_frequency = DFT_Frequencies(8, 800.0)
23 one_sided_frequency = DFT_Frequencies(8, 800.0, one_sided=True)
24
25 print(full_frequency)
26 # [  0.  100.  200.  300.  400. -300. -200. -100.]
27
28 print(one_sided_frequency)
29 # [  0.  100.  200.  300.  400.]

```

When a spectral peak occurs at array index k , its frequency is therefore obtained either from $k f_s / N$ or by indexing the corresponding frequency array. Keeping the coefficient and frequency arrays in the same order avoids mistakes when plotting or selecting spectral components.

The direct evaluation of the DFT computes N sums containing N terms. Its operation count is therefore proportional to:

$$\mathcal{O}(N^2) \quad (12.221)$$

This is acceptable for short sequences and useful as a reference implementation, but it becomes expensive for long measurement records.

12.7.3 Radix-2 Cooley–Tukey Algorithm

The FFT obtains the same coefficients by reusing repeated calculations. For a radix-2 transform, the number of samples is a power of two:

$$N = 2^p \quad (12.222)$$

Introduce the twiddle factor:

$$W_N = \exp\left(-i\frac{2\pi}{N}\right) \quad (12.223)$$

so that the DFT can be written:

$$X_k = \sum_{n=0}^{N-1} x_n W_N^{kn} \quad (12.224)$$

The sequence is separated into samples with even and odd indices:

$$\begin{aligned} X_k &= \sum_{m=0}^{N/2-1} x_{2m} W_N^{2mk} + \sum_{m=0}^{N/2-1} x_{2m+1} W_N^{(2m+1)k} \\ &= \sum_{m=0}^{N/2-1} x_{2m} W_{N/2}^{mk} + W_N^k \sum_{m=0}^{N/2-1} x_{2m+1} W_{N/2}^{mk} \end{aligned} \quad (12.225)$$

The two sums are DFTs of length $N/2$. Denote them by E_k and O_k . The first half of the result is then:

$$X_k = E_k + W_N^k O_k, \quad k = 0, 1, \dots, N/2 - 1 \quad (12.226)$$

Using $W_N^{k+N/2} = -W_N^k$, the second half follows without another pair of transforms:

$$X_{k+N/2} = E_k - W_N^k O_k \quad (12.227)$$

The combination:

$$\begin{aligned} X_k &= E_k + W_N^k O_k \\ X_{k+N/2} &= E_k - W_N^k O_k \end{aligned} \quad (12.228)$$

is commonly called an FFT **butterfly**. The same split is applied recursively until transforms of length one remain. The work satisfies:

$$C(N) = 2C\left(\frac{N}{2}\right) + \mathcal{O}(N) \quad (12.229)$$

and therefore:

$$C(N) = \mathcal{O}(N \log_2 N) \quad (12.230)$$

For example, a direct DFT of $N = 2^{20}$ samples requires on the order of 2^{40} complex operations, whereas a radix-2 FFT requires on the order of $20 \cdot 2^{20}$. The exact operation count depends on the implementation, but the difference in scaling is the reason why frequency-domain processing of long records is practical.

The recursive formulation is particularly clear and is used in the example below. Production FFT libraries generally use iterative, in-place and cache-aware implementations and support lengths that are not powers of two by combining several radices or by using algorithms such as Bluestein's method.

12.7.4 Negative Frequencies and Real Signals

The indices above $N/2$ represent negative frequencies. A convenient signed interpretation is:

$$f_k = \begin{cases} \frac{k f_s}{N}, & 0 \leq k \leq N/2 \\ \frac{(k - N) f_s}{N}, & N/2 < k < N \end{cases} \quad (12.231)$$

For a real-valued time signal, the DFT has conjugate symmetry:

$$X_{N-k} = X_k^* \quad (12.232)$$

The negative-frequency half therefore contains no independent information. A one-sided spectrum keeps the bins from zero to the Nyquist frequency:

$$f_{\text{Nyquist}} = \frac{f_s}{2} \quad (12.233)$$

The zero-frequency coefficient is the sum of the samples. Its amplitude after division by N is the mean value of the signal. The Nyquist bin, present for an even number of samples, is also unpaired. All other positive-frequency bins have matching negative-frequency bins.

12.7.5 Amplitude and Phase Scaling

For a real signal and the transform convention used above, a one-sided peak amplitude spectrum is obtained from:

$$A_k = \frac{2|X_k|}{N}, \quad 0 < k < N/2 \quad (12.234)$$

The factor two collects the equal positive- and negative-frequency contributions into the retained positive-frequency bin. It must not be applied to the direct-current or Nyquist bins:

$$\begin{aligned} A_0 &= \frac{|X_0|}{N} \\ A_{N/2} &= \frac{|X_{N/2}|}{N} \end{aligned} \quad (12.235)$$

The phase is:

$$\varphi_k = \text{atan2}(\text{Im}(X_k), \text{Re}(X_k)) \quad (12.236)$$

Amplitude conventions must always be stated. Peak amplitude, root-mean-square amplitude, power spectrum and power spectral density are different

quantities. For a sinusoidal component, its root-mean-square amplitude is its peak amplitude divided by $\sqrt{2}$. Power spectral density scaling is discussed in the following spectral-density section.

12.7.6 Sampling, Aliasing and Leakage

Sampling cannot distinguish frequencies that differ by an integer multiple of the sampling frequency. A component above the Nyquist frequency is therefore folded into the represented frequency range. This is **aliasing**. It cannot be removed after sampling. The analogue signal must be sufficiently band-limited, usually by an anti-aliasing filter, before it is sampled.

The DFT also treats the finite record as one period of an infinitely repeated signal. If the values at the two ends do not join continuously, the repeated signal contains a discontinuity. Energy from a harmonic component then spreads into neighbouring frequency bins. This is **spectral leakage**.

Leakage is absent for a harmonic whose record contains an integer number of periods, because its frequency lies exactly on a DFT bin:

$$f = k\Delta f \quad (12.237)$$

For general measurement records this condition is rarely satisfied. A window reduces the discontinuity by gradually reducing the samples near the ends of the record. For example, the Hann window is:

$$w_n = \frac{1}{2} \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right) \quad (12.238)$$

and the transformed samples become:

$$x_n^{(w)} = w_n x_n \quad (12.239)$$

A window reduces distant side lobes but broadens the main spectral peak. It also changes the measured amplitude. For deterministic harmonic amplitudes, a simple correction uses the **coherent gain**:

$$G_c = \frac{1}{N} \sum_{n=0}^{N-1} w_n \quad (12.240)$$

so that the amplitude spectrum is divided by G_c . Different windows trade peak width against side-lobe suppression; no window is best for every purpose.

Appending zeros increases the number of displayed frequency samples and makes the plotted spectrum smoother, but it does not improve the physical frequency resolution $1/T$. Only a longer measured record does that. Zero-padding can, however, make interpolation of an isolated spectral peak more convenient.

12.7.7 Python Implementation

The following direct DFT implements the definition literally. It is useful for checking the FFT on short sequences and for showing where the quadratic cost comes from.

```

1 import numpy as np
2
3
4 def DFT(x: np.ndarray) -> np.ndarray:
5     """Evaluate the discrete Fourier transform directly.
6
7     The implementation uses NumPy arrays for storage and complex arithmetic.
8     """
9     x = np.asarray(x, dtype=complex).reshape(-1)
10    n_samples = x.size
11    X = np.zeros(n_samples, dtype=complex)
12
13    for k in range(n_samples):
14        value = 0.0j
15        for n in range(n_samples):
16            angle = -2.0 * np.pi * k * n / n_samples
17            value += x[n] * np.exp(1.0j * angle)
18    X[k] = value

```



```

19
20     return X

```

A recursive radix-2 Cooley–Tukey implementation follows directly from the even–odd derivation. The input length is checked explicitly because silently padding or truncating the data would change the transform being calculated.

```

1  import numpy as np
2
3
4  def FFT_Radix2(x: np.ndarray) -> np.ndarray:
5      """Evaluate a radix-2 FFT.
6
7      Args:
8          x: one-dimensional real or complex sequence. Its length must be a
9              positive power of two.
10
11      Returns:
12          The unscaled forward DFT of x.
13      """
14      x = np.asarray(x, dtype=complex).reshape(-1)
15      n_samples = x.size
16
17      if n_samples == 0:
18          raise ValueError("x must contain at least one sample")
19      if n_samples & (n_samples - 1):
20          raise ValueError("radix-2 FFT requires a power-of-two length")
21      if n_samples == 1:
22          return x.copy()
23
24      even = FFT_Radix2(x[0::2])
25      odd = FFT_Radix2(x[1::2])
26
27      k = np.arange(n_samples // 2)
28      twiddle = np.exp(-2.0j * np.pi * k / n_samples) * odd
29
30      X = np.empty(n_samples, dtype=complex)
31      X[:n_samples // 2] = even + twiddle

```

```

32     X[n_samples // 2:] = even - twiddle
33     return X
34
35
36 def IFFT_Radix2(X: np.ndarray) -> np.ndarray:
37     """Evaluate the inverse transform using conjugation of the forward FFT."""
38     X = np.asarray(X, dtype=complex).reshape(-1)
39     if X.size == 0:
40         raise ValueError("X must contain at least one coefficient")
41
42     return np.conjugate(FFT_Radix2(np.conjugate(X))) / X.size

```

The inverse relation follows from changing the sign in the complex exponential. Conjugation allows the same forward implementation to be reused:

$$\text{IDFT}(\mathbf{X}) = \frac{1}{N} [\text{DFT}(\mathbf{X}^*)]^* \quad (12.241)$$

The following helper forms a one-sided peak-amplitude spectrum of a real signal. It optionally removes the mean and applies a Hann window. The frequency array is constructed from the sample rate and record length using the bin relation derived above.

```

1  import numpy as np
2
3
4  def One_Sided_Spectrum(x: np.ndarray,
5                          sample_rate: float,
6                          remove_mean: bool = True,
7                          hann_window: bool = True):
8      """Return frequency, peak amplitude and phase for a real signal.
9
10     The sequence length must be a power of two because FFT_Radix2 is used.
11     Amplitudes are corrected for the coherent gain of the selected window.
12     """
13     x = np.asarray(x, dtype=float).reshape(-1).copy()
14     n_samples = x.size

```

```

15
16     if n_samples < 2:
17         raise ValueError("at least two samples are required")
18     if n_samples & (n_samples - 1):
19         raise ValueError("the sequence length must be a power of two")
20     if hann_window and n_samples < 4:
21         raise ValueError("the Hann window requires at least four samples")
22     if sample_rate <= 0.0:
23         raise ValueError("sample_rate must be positive")
24
25     if remove_mean:
26         x -= np.sum(x) / n_samples
27
28     if hann_window:
29         n = np.arange(n_samples)
30         window = 0.5 - 0.5 * np.cos(2.0 * np.pi * n / (n_samples - 1))
31     else:
32         window = np.ones(n_samples)
33
34     coherent_gain = np.sum(window) / n_samples
35     X = FFT_Radix2(x * window)
36
37     n_positive = n_samples // 2 + 1
38     X_positive = X[:n_positive]
39     frequency = DFT_Frequencies(
40         n_samples,
41         sample_rate,
42         one_sided=True,
43     )
44
45     amplitude = np.abs(X_positive) / (n_samples * coherent_gain)
46     amplitude[1:-1] *= 2.0
47     phase = np.angle(X_positive)
48
49     return frequency, amplitude, phase

```

For example, a signal containing components at 50 Hz and 120 Hz may be transformed as follows:

```

1 import numpy as np
2
3
4 sample_rate = 1024.0
5 n_samples = 1024
6 time = np.arange(n_samples) / sample_rate
7
8 signal = (3.0 * np.sin(2.0 * np.pi * 50.0 * time)
9           + 0.8 * np.cos(2.0 * np.pi * 120.0 * time))
10
11 frequency, amplitude, phase = One_Sided_Spectrum(
12     signal,
13     sample_rate,
14     remove_mean=True,
15     hann_window=False,
16 )
17
18 # The largest bins occur at 50 Hz and 120 Hz with peak amplitudes 3.0 and 0.8.
19 indices = np.argsort(amplitude)[-2:]
20 for index in indices[np.argsort(frequency[indices])]:
21     print(f"{frequency[index]:7.1f} Hz   {amplitude[index]:7.3f}")

```

Here the record length is one second, so $\Delta f = 1$ Hz, and both harmonics coincide exactly with DFT bins. No window is required for this constructed example. A general measured signal should not be expected to have this property.

12.7.8 Relation to Campbell Diagrams and Order Analysis

A single FFT describes one time record under approximately stationary operating conditions. A Campbell diagram repeats this local spectral calculation over a run-up, coast-down or sequence of operating points and retains rotational speed as a second coordinate. Order analysis then expresses frequency relative to the shaft rotational frequency. The following section develops both representations and explains when ordinary time blocks must be replaced by angular resampling.

12.8 Campbell Diagram and Order Analysis

Rotating machines rarely operate at only one constant speed. During a run-up or coast-down, excitation frequencies linked to the rotation move through the frequency range and may cross structural resonances. A single spectrum cannot show this development because it contains no operating-speed coordinate. Campbell diagrams and order maps retain that additional coordinate.

Two different plots are commonly called a **Campbell diagram**:

1. A *modal Campbell diagram* plots calculated natural frequencies against rotational speed. Gyroscopic systems may show separate forward- and backward-whirl branches.
2. A *response Campbell diagram* plots a measured or simulated spectral amplitude against frequency and rotational speed. It is a field, usually displayed as colour or contours.

Both plots use the same geometrical idea. Structural frequencies form branches $f_j(n_{\text{rpm}})$, while rotational excitations form **order lines**. Their intersections identify speeds at which a rotating excitation can excite a structural mode.

12.8.1 Rotational Frequency and Orders

For a shaft speed n_{rpm} in revolutions per minute, the rotational frequency is:

$$f_{\text{rot}} = \frac{n_{\text{rpm}}}{60}. \quad (12.242)$$

An excitation at o cycles per revolution has frequency:

$$f_o(n_{\text{rpm}}) = o f_{\text{rot}} = o \frac{n_{\text{rpm}}}{60}. \quad (12.243)$$

The dimensionless quantity o is the **order**. The first order, written 1x, repeats once per revolution. The second order repeats twice per revolution. A rotor with z equally spaced blades produces a blade-passing component at order z , and a gear with z teeth produces its mesh frequency at order z relative to that gear shaft.

A constant order is a straight line through the origin in a frequency–speed Campbell diagram. A speed-independent structural resonance is approximately horizontal. If a natural frequency f_n is independent of speed, its intersection with order o occurs at the critical speed:

$$n_{\text{critical}} = \frac{60f_n}{o}. \quad (12.244)$$

For a rotating structure, the natural frequency may itself depend on speed. Centrifugal stiffening, stress softening and gyroscopic terms can curve or split its branch. The critical speed must then satisfy:

$$f_j(n_{\text{critical}}) = o \frac{n_{\text{critical}}}{60}. \quad (12.245)$$

An intersection is a possible resonance condition, not a prediction of response amplitude by itself. The actual response also depends on excitation strength, damping, mode shape, measurement position and the time spent near the crossing.

12.8.2 Response Campbell Diagram from Block Spectra

Let $x(t)$ be a measured response and let $n_{\text{rpm}}(t)$ be the corresponding shaft speed. The signal is divided into overlapping blocks. For block b , a windowed FFT gives a local complex spectrum:

$$X_b(f_k) = \text{DFT} \{w_m x_{b,m}\}. \quad (12.246)$$

Each block is assigned a representative speed, commonly the mean speed in the block:

$$\bar{n}_b = \frac{1}{N_b} \sum_{m=0}^{N_b-1} n_{b,m}. \quad (12.247)$$

The spectra are then arranged into the field:

$$A(\bar{n}_b, f_k) = |X_b(f_k)|. \quad (12.248)$$

Frequency is one axis, speed is the other axis, and amplitude is represented by colour or contours. The complex spectrum may also be retained so that phase and complex order tracks can be extracted later.

The speed slices should be sorted before constructing the plotted field. If several blocks have the same representative speed, their complex spectra may be averaged. Averaging magnitudes alone discards phase and generally gives a different result. A uniformly spaced speed axis can be produced afterwards by interpolating between the sorted slices. This *rebinning* improves the regularity of the displayed field but does not create additional physical resolution.

Run-up and coast-down data should normally be kept separate. Sorting both into one speed axis and averaging duplicate speeds can hide hysteresis, thermal drift, clearance changes or nonlinear jump phenomena that depend on sweep direction.

12.8.3 Frequency and Speed Resolution

For a block with N_b samples and sample rate f_s , the block duration and frequency spacing are:

$$T_b = \frac{N_b}{f_s}, \quad \Delta f = \frac{f_s}{N_b} = \frac{1}{T_b}. \quad (12.249)$$

A longer block improves frequency resolution but includes a larger speed change. If the speed changes from $n_{\min,b}$ to $n_{\max,b}$ inside the block, the speed spread is:

$$\Delta n_b = n_{\max,b} - n_{\min,b}. \quad (12.250)$$

Order o consequently moves through approximately:

$$\Delta f_{o,b} \approx o \frac{\Delta n_b}{60} \quad (12.251)$$

within the block. When $\Delta f_{o,b}$ is comparable to or larger than a frequency bin, the order ridge is smeared even if the FFT itself is calculated correctly. Block length is therefore a compromise between frequency resolution and speed stationarity.

Overlap creates more speed slices and a smoother-looking diagram, but it does not change Δf . Strong overlap also means that neighbouring slices contain much of the same data and are not statistically independent.

The amplitude convention must be stated. The implementations below return a one-sided *peak-amplitude* spectrum corrected for the coherent gain of the Hann window. RMS amplitude would be smaller by $\sqrt{2}$ for a sinusoid. A decibel plot additionally requires a clearly stated reference value.

12.8.4 Python Implementation of a Campbell Field

The following helper calculates the one-sided complex amplitude of one block. It uses the radix-2 FFT introduced in the previous section. Keeping the complex values permits subsequent amplitude, phase and order calculations.

```

1 import numpy as np
2
3
4 def One_Sided_Complex_Amplitude(x: np.ndarray,
5                                 sample_rate: float,
6                                 remove_mean: bool = True,
7                                 hann_window: bool = True):
8     """Return frequency and one-sided complex peak amplitude."""
```



```

9     x = np.asarray(x, dtype=float).reshape(-1).copy()
10    n_samples = x.size
11
12    if n_samples < 2:
13        raise ValueError("at least two samples are required")
14    if n_samples & (n_samples - 1):
15        raise ValueError("the block length must be a power of two")
16    if sample_rate <= 0.0:
17        raise ValueError("sample_rate must be positive")
18
19    if remove_mean:
20        x -= np.sum(x) / n_samples
21
22    if hann_window:
23        index = np.arange(n_samples)
24        window = 0.5 - 0.5 * np.cos(
25            2.0 * np.pi * index / (n_samples - 1)
26        )
27    else:
28        window = np.ones(n_samples)
29
30    coherent_gain = np.sum(window) / n_samples
31    spectrum = FFT_Radix2(x * window)
32
33    n_positive = n_samples // 2 + 1
34    complex_amplitude = (
35        spectrum[:n_positive] / (n_samples * coherent_gain)
36    )
37
38    # Interior positive-frequency bins represent both members of a
39    # conjugate pair. DC and the Nyquist bin occur only once.
40    complex_amplitude[1:-1] *= 2.0
41
42    frequency = DFT_Frequencies(
43        n_samples,
44        sample_rate,
45        one_sided=True,
46    )
47    return frequency, complex_amplitude

```

Before optional rebinning, the slices are sorted and duplicate speed values are averaged. A positive tolerance may be used to combine nearly equal speed values.

```

1  import numpy as np
2
3
4  def Sort_And_Average_Speed_Slices(speed_rpm: np.ndarray,
5                                   field: np.ndarray,
6                                   speed_spread: np.ndarray,
7                                   tolerance_rpm: float = 0.0):
8      """Sort slices by speed and average duplicate or near-duplicate rows."""
9      speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)
10     field = np.asarray(field, dtype=complex)
11     speed_spread = np.asarray(speed_spread, dtype=float).reshape(-1)
12
13     if field.shape[0] != speed_rpm.size:
14         raise ValueError("one field row is required for each speed")
15     if speed_spread.size != speed_rpm.size:
16         raise ValueError("one speed spread is required for each speed")
17     if tolerance_rpm < 0.0:
18         raise ValueError("tolerance_rpm must be non-negative")
19
20     permutation = np.argsort(speed_rpm, kind="stable")
21     speed_sorted = speed_rpm[permutation]
22     field_sorted = field[permutation]
23     spread_sorted = speed_spread[permutation]
24
25     grouped_speed = []
26     grouped_field = []
27     grouped_spread = []
28
29     first = 0
30     while first < speed_sorted.size:
31         last = first + 1
32         while (last < speed_sorted.size
33               and speed_sorted[last] - speed_sorted[first]
34                 <= tolerance_rpm):
35             last += 1

```

```

36         grouped_speed.append(np.mean(speed_sorted[first:last]))
37         grouped_field.append(np.mean(field_sorted[first:last], axis=0))
38         grouped_spread.append(np.max(spread_sorted[first:last]))
39         first = last
40
41
42     return (
43         np.asarray(grouped_speed),
44         np.asarray(grouped_field),
45         np.asarray(grouped_spread),
46     )
47
48
49 def Interpolate_Field_Rows(x: np.ndarray,
50                             field: np.ndarray,
51                             x_new: np.ndarray) -> np.ndarray:
52     """Linearly interpolate every field column along its first axis."""
53     x = np.asarray(x, dtype=float).reshape(-1)
54     field = np.asarray(field)
55     x_new = np.asarray(x_new, dtype=float).reshape(-1)
56
57     if x.size < 2:
58         raise ValueError("at least two source rows are required")
59     if field.shape[0] != x.size:
60         raise ValueError("field.shape[0] must equal x.size")
61     if np.any(np.diff(x) <= 0.0):
62         raise ValueError("x must be strictly increasing")
63     if x_new[0] < x[0] or x_new[-1] > x[-1]:
64         raise ValueError("x_new must remain inside the source interval")
65
66     result = np.empty((x_new.size,) + field.shape[1:], dtype=field.dtype)
67
68     for row, value in enumerate(x_new):
69         right = np.searchsorted(x, value, side="right")
70         if right == 0:
71             result[row] = field[0]
72         elif right == x.size:
73             result[row] = field[-1]
74         else:
75             left = right - 1
76             fraction = (value - x[left]) / (x[right] - x[left])

```

```

77         result[row] = (
78             (1.0 - fraction) * field[left]
79             + fraction * field[right]
80         )
81
82     return result

```

The Campbell construction collects the local spectra, records the speed spread of every block, sorts the slices and optionally rebins them to a uniform speed axis. The argument `resample=N` requests N uniformly spaced speed bins.

```

1  import numpy as np
2
3
4  def Campbell_Diagram(signal: np.ndarray,
5                      speed_rpm: np.ndarray,
6                      sample_rate: float,
7                      block_size: int,
8                      overlap: float = 0.5,
9                      duplicate_tolerance_rpm: float = 0.0,
10                     resample: int | None = None):
11     """Construct a complex response Campbell field."""
12     signal = np.asarray(signal, dtype=float).reshape(-1)
13     speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)
14
15     if signal.size != speed_rpm.size:
16         raise ValueError("signal and speed_rpm must have equal length")
17     if block_size < 2 or block_size > signal.size:
18         raise ValueError("invalid block_size")
19     if block_size & (block_size - 1):
20         raise ValueError("block_size must be a power of two")
21     if not 0.0 <= overlap < 1.0:
22         raise ValueError("overlap must lie in [0, 1)")
23
24     hop_size = max(1, int(round(block_size * (1.0 - overlap))))
25     starts = range(0, signal.size - block_size + 1, hop_size)
26
27     block_speeds = []

```

```

28     speed_spreads = []
29     spectra = []
30     frequency = None
31
32     for start in starts:
33         stop = start + block_size
34         block_speed = speed_rpm[start:stop]
35
36         frequency, complex_amplitude = One_Sided_Complex_Amplitude(
37             signal[start:stop],
38             sample_rate,
39             remove_mean=True,
40             hann_window=True,
41         )
42
43         block_speeds.append(np.mean(block_speed))
44         speed_spreads.append(np.max(block_speed) - np.min(block_speed))
45         spectra.append(complex_amplitude)
46
47     block_speeds, spectra, speed_spreads = (
48         Sort_And_Average_Speed_Slices(
49             np.asarray(block_speeds),
50             np.asarray(spectra),
51             np.asarray(speed_spreads),
52             tolerance_rpm=duplicate_tolerance_rpm,
53         )
54     )
55
56     if resample is not None:
57         if resample < 2:
58             raise ValueError("resample must be at least two")
59         speed_uniform = np.linspace(
60             block_speeds[0],
61             block_speeds[-1],
62             resample,
63         )
64         spectra = Interpolate_Field_Rows(
65             block_speeds,
66             spectra,
67             speed_uniform,
68         )

```

```

69         speed_spreads = Interpolate_Field_Rows(
70             block_speeds,
71             speed_spreads[:, None],
72             speed_uniform,
73         )[:, 0]
74         block_speeds = speed_uniform
75
76     return {
77         "speed_rpm": block_speeds,
78         "frequency_hz": frequency,
79         "complex_amplitude": spectra,
80         "amplitude": np.abs(spectra),
81         "speed_spread_rpm": speed_spreads,
82     }

```

The returned speed-spread array is a useful diagnostic. A highly smeared region can be identified before interpreting broad spectral ridges as damping or as multiple physical components.

12.8.5 Example of a Response Campbell Diagram

The following synthetic run-up contains a first-order component, a second-order component amplified near a 100 Hz structural resonance, a sixth order and a stationary 140 Hz tone. The rotational angle is obtained by trapezoidal integration of the speed.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4
5  def Rotational_Angle(time: np.ndarray,
6                      speed_rpm: np.ndarray) -> np.ndarray:
7      """Integrate rotational speed to angle in radians."""
8      time = np.asarray(time, dtype=float).reshape(-1)
9      speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)
10

```

```

11     if time.size != speed_rpm.size:
12         raise ValueError("time and speed_rpm must have equal length")
13     if time.size < 2 or np.any(np.diff(time) <= 0.0):
14         raise ValueError("time must be strictly increasing")
15
16     angular_speed = 2.0 * np.pi * speed_rpm / 60.0
17     angle = np.zeros(time.size)
18     angle[1:] = np.cumsum(
19         0.5
20         * (angular_speed[1:] + angular_speed[:-1])
21         * np.diff(time)
22     )
23     return angle
24
25
26 sample_rate = 2048.0
27 duration = 8.0
28 time = np.arange(int(sample_rate * duration)) / sample_rate
29 speed_rpm = 600.0 + 3600.0 * time / duration
30 angle = Rotational_Angle(time, speed_rpm)
31
32 second_order_frequency = 2.0 * speed_rpm / 60.0
33 natural_frequency = 100.0
34 damping_ratio = 0.04
35 frequency_ratio = second_order_frequency / natural_frequency
36 resonance = 1.0 / np.sqrt(
37     (1.0 - frequency_ratio**2)**2
38     + (2.0 * damping_ratio * frequency_ratio)**2
39 )
40 resonance /= np.max(resonance)
41
42 signal = (
43     0.45 * np.sin(angle)
44     + 1.20 * resonance * np.sin(2.0 * angle)
45     + 0.20 * np.sin(6.0 * angle)
46     + 0.12 * np.sin(2.0 * np.pi * 140.0 * time)
47 )
48
49 campbell = Campbell_Diagram(
50     signal,
51     speed_rpm,

```

```

52     sample_rate,
53     block_size=1024,
54     overlap=0.75,
55     resample=120,
56 )
57
58 reference = np.max(campbell["amplitude"])
59 level_db = 20.0 * np.log10(
60     np.maximum(campbell["amplitude"] / reference, 1.0e-6)
61 )
62
63 plt.figure()
64 plt.pcolormesh(
65     campbell["speed_rpm"],
66     campbell["frequency_hz"],
67     level_db.T,
68     shading="auto",
69 )
70 for order in (1.0, 2.0, 6.0):
71     plt.plot(
72         campbell["speed_rpm"],
73         order * campbell["speed_rpm"] / 60.0,
74         linewidth=1.0,
75     )
76 plt.xlabel("Rotational speed [rpm]")
77 plt.ylabel("Frequency [Hz]")
78 plt.ylim(0.0, 180.0)
79 plt.colorbar(label="Relative amplitude [dB]")
80 plt.tight_layout()
81 plt.show()

```

In the frequency-speed plane, the three shaft-synchronous components follow straight order lines. The fixed 140 Hz component remains horizontal. The second-order ridge becomes strongest near:

$$n_{\text{critical}} = \frac{60 \cdot 100}{2} = 3000 \text{ rpm.} \quad (12.252)$$

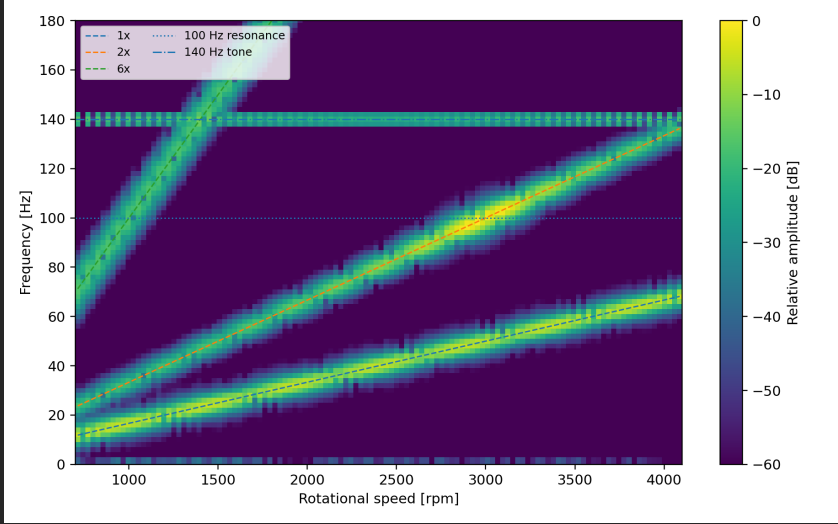


Figure 12.2: Synthetic response Campbell diagram. The first, second and sixth orders rise linearly with speed. The second order is amplified where it crosses the 100 Hz resonance, while the stationary 140 Hz tone remains horizontal.

12.8.6 Waterfall Representation of the Response Campbell Field

The response Campbell field in Equation (12.248) may also be displayed as a **waterfall diagram**. No new transform is performed. Each row of the Campbell field is drawn as an individual spectrum at its corresponding speed:

$$\mathcal{W}_b = \{(f_k, \bar{n}_b, A(\bar{n}_b, f_k)) \mid k = 0, \dots, N_f - 1\}. \quad (12.253)$$

The frequency axis remains horizontal, the spectra are separated along the speed axis, and amplitude is shown by the height of each curve. A waterfall is therefore a three-dimensional perspective view of the same data that a response Campbell diagram displays by colour or contours. A time waterfall can be formed in exactly the same manner by replacing speed with block-centre time.

The two views emphasise different properties. A colour map makes ridge geometry, order lines and weak components easier to compare quantitatively. A waterfall makes the individual spectra and the growth and decay of resonant peaks more intuitive. Perspective, curve overlap and the selected viewing angle can, however, hide weaker components behind stronger curves. Only every few speed slices are therefore commonly drawn.

When amplitudes are converted to decibels, every slice should use the same global reference. Normalising each spectrum separately would make every speed slice appear equally strong and would remove the amplitude increase at a resonance. The chosen decibel floor also affects the visual appearance but not the underlying Campbell field.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 def Plot_Campbell_Waterfall(campbell: dict,
6                             maximum_frequency: float | None = None,
7                             stride: int = 5,
8                             decibel: bool = True,
9                             floor_db: float = -60.0):
10     """Plot selected speed slices of a response Campbell field."""
11     speed_rpm = np.asarray(campbell["speed_rpm"], dtype=float)
12     frequency = np.asarray(campbell["frequency_hz"], dtype=float)
13     amplitude = np.asarray(campbell["amplitude"], dtype=float)
14
15     if amplitude.shape != (speed_rpm.size, frequency.size):
16         raise ValueError("the Campbell field dimensions are inconsistent")
17     if stride < 1:
18         raise ValueError("stride must be positive")
19
20     if maximum_frequency is None:
21         maximum_frequency = frequency[-1]
22     frequency_mask = frequency <= maximum_frequency
23     frequency_plot = frequency[frequency_mask]
24     amplitude_plot = amplitude[:, frequency_mask]
25
26     if decibel:

```

```

27     reference = np.max(amplitude_plot)
28     if reference <= 0.0:
29         raise ValueError("the Campbell field contains no positive amplitude")
30     minimum_ratio = 10.0**(floor_db / 20.0)
31     ordinate = 20.0 * np.log10(
32         np.maximum(amplitude_plot / reference, minimum_ratio)
33     )
34     amplitude_label = "Relative amplitude [dB]"
35 else:
36     ordinate = amplitude_plot
37     amplitude_label = "Peak amplitude"
38
39 figure = plt.figure()
40 axis = figure.add_subplot(111, projection="3d")
41
42 rows = list(range(0, speed_rpm.size, stride))
43 if rows[-1] != speed_rpm.size - 1:
44     rows.append(speed_rpm.size - 1)
45
46 for row in rows:
47     speed_line = np.full(frequency_plot.size, speed_rpm[row])
48     axis.plot(
49         frequency_plot,
50         speed_line,
51         ordinate[row],
52         linewidth=0.8,
53     )
54
55 axis.set_xlabel("Frequency [Hz]")
56 axis.set_ylabel("Rotational speed [rpm]")
57 axis.set_zlabel(amplitude_label)
58 figure.tight_layout()
59 return figure, axis
60
61
62 Plot_Campbell_Waterfall(
63     campbell,
64     maximum_frequency=180.0,
65     stride=6,
66     decibel=True,
67 )

```

```
68 plt.show()
```

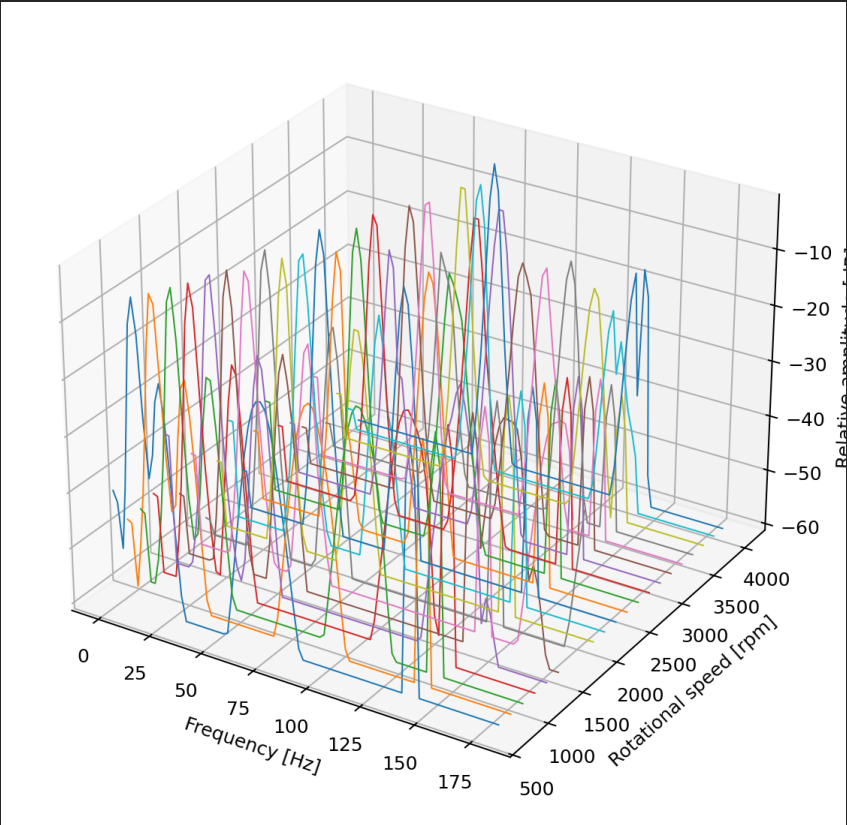


Figure 12.3: Waterfall representation of the synthetic response Campbell field. Each line is one spectrum at one rotational speed. The order-related peaks move in frequency, while the stationary 140 Hz component remains at the same frequency.

The response Campbell colour map in Figure 12.2 and the waterfall in Figure 12.3 contain the same spectral slices. They should not be confused with rainflow cycle counting, which starts from a load or stress history and answers

a fatigue question rather than a frequency-content question.

12.8.7 Order Map Obtained from a Campbell Field

The order corresponding to frequency f at speed n_{rpm} is:

$$o = \frac{f}{f_{\text{rot}}} = \frac{60f}{n_{\text{rpm}}}. \quad (12.254)$$

A Campbell field can therefore be sampled along the frequency curves $f = on_{\text{rpm}}/60$ to form an order map:

$$A_o(n_{\text{rpm}}, o) = A\left(n_{\text{rpm}}, o \frac{n_{\text{rpm}}}{60}\right). \quad (12.255)$$

Because the required frequency usually lies between FFT bins, interpolation is necessary. Interpolating the complex spectrum retains amplitude and phase more consistently than separately interpolating magnitude and wrapped phase.

This conversion changes coordinates but does not remove spectral smearing that already occurred inside the time blocks. It is suitable when speed is nearly constant within each block. Angular resampling, introduced later, is preferable for rapid speed changes.

```

1 import numpy as np
2
3
4 def Interpolate_Complex_Spectrum(frequency: np.ndarray,
5                                   spectrum: np.ndarray,
6                                   target_frequency: np.ndarray) -> np.ndarray:
7     """Interpolate a complex spectrum at requested frequencies."""
8     frequency = np.asarray(frequency, dtype=float).reshape(-1)
9     spectrum = np.asarray(spectrum, dtype=complex).reshape(-1)
10    target_frequency = np.asarray(target_frequency, dtype=float).reshape(-1)
11
```

```

12     if spectrum.size != frequency.size:
13         raise ValueError("spectrum and frequency must have equal length")
14     if np.any(np.diff(frequency) <= 0.0):
15         raise ValueError("frequency must be strictly increasing")
16
17     result = np.full(target_frequency.size, np.nan + 1.0j * np.nan)
18
19     for index, value in enumerate(target_frequency):
20         if value < frequency[0] or value > frequency[-1]:
21             continue
22
23         right = np.searchsorted(frequency, value, side="right")
24         if right == 0:
25             result[index] = spectrum[0]
26         elif right == frequency.size:
27             result[index] = spectrum[-1]
28         else:
29             left = right - 1
30             fraction = (
31                 (value - frequency[left])
32                 / (frequency[right] - frequency[left])
33             )
34             result[index] = (
35                 (1.0 - fraction) * spectrum[left]
36                 + fraction * spectrum[right]
37             )
38
39     return result
40
41
42 def Campbell_To_Order_Map(campbell: dict,
43                           n_order_bins: int = 401,
44                           maximum_order: float | None = None):
45     """Interpolate a complex Campbell field onto a common order axis."""
46     speed_rpm = np.asarray(campbell["speed_rpm"], dtype=float)
47     frequency = np.asarray(campbell["frequency_hz"], dtype=float)
48     field = np.asarray(campbell["complex_amplitude"], dtype=complex)
49
50     if np.any(speed_rpm <= 0.0):
51         raise ValueError("order conversion requires positive speed")
52     if n_order_bins < 2:

```

```

53         raise ValueError("n_order_bins must be at least two")
54
55     common_limit = frequency[-1] / np.max(speed_rpm / 60.0)
56     if maximum_order is None:
57         maximum_order = common_limit
58     if maximum_order <= 0.0 or maximum_order > common_limit:
59         raise ValueError("maximum_order exceeds the common frequency range")
60
61     order = np.linspace(0.0, maximum_order, n_order_bins)
62     order_field = np.empty((speed_rpm.size, order.size), dtype=complex)
63
64     for row, speed in enumerate(speed_rpm):
65         target_frequency = order * speed / 60.0
66         order_field[row] = Interpolate_Complex_Spectrum(
67             frequency,
68             field[row],
69             target_frequency,
70         )
71
72     return {
73         "speed_rpm": speed_rpm,
74         "order": order,
75         "complex_amplitude": order_field,
76         "amplitude": np.abs(order_field),
77     }

```

In an order map, shaft-synchronous excitations become approximately horizontal. A speed-independent structural resonance follows:

$$o_n(n_{\text{rpm}}) = \frac{60f_n}{n_{\text{rpm}}}, \quad (12.256)$$

and therefore curves downward as speed increases. This is the complementary view of the frequency–speed Campbell diagram: orders are horizontal and fixed frequencies are curved.

```

1 import numpy as np
2 import matplotlib.pyplot as plt

```

```
3
4
5 order_map = Campbell_To_Order_Map(
6     campbell,
7     n_order_bins=481,
8     maximum_order=8.0,
9 )
10
11 reference = np.nanmax(order_map["amplitude"])
12 level_db = 20.0 * np.log10(
13     np.maximum(order_map["amplitude"] / reference, 1.0e-6)
14 )
15
16 plt.figure()
17 plt.pcolormesh(
18     order_map["speed_rpm"],
19     order_map["order"],
20     level_db.T,
21     shading="auto",
22 )
23 plt.xlabel("Rotational speed [rpm]")
24 plt.ylabel("Order [-]")
25 plt.ylim(0.0, 8.0)
26 plt.colorbar(label="Relative amplitude [dB]")
27 plt.tight_layout()
28 plt.show()
```

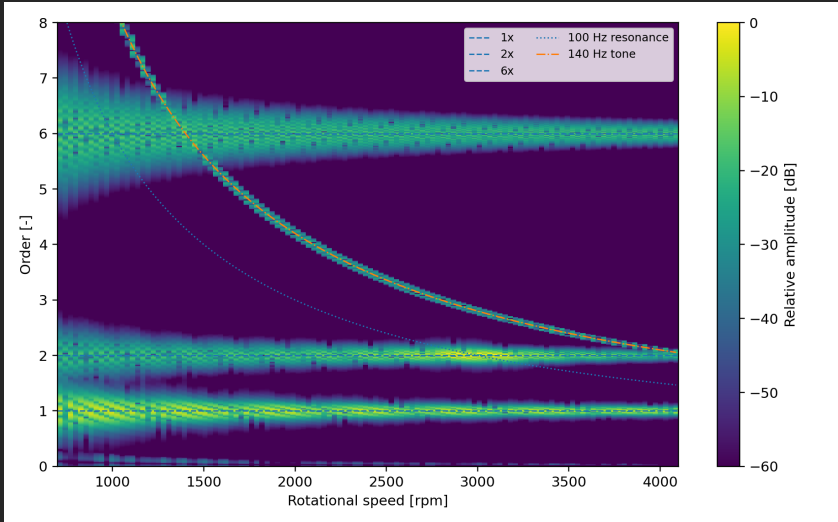


Figure 12.4: Order map obtained from the synthetic Campbell field. The first, second and sixth orders are horizontal. The stationary 140 Hz component curves towards lower order as speed increases.

12.8.8 Extraction of Individual Order Tracks

Often the full field is less useful than the amplitude and phase of selected orders. For each speed slice, the complex spectrum is interpolated at the order frequency from Equation (12.243). The resulting complex track contains both quantities:

$$a_o(n) = |X_o(n)|, \quad \varphi_o(n) = \arg X_o(n). \quad (12.257)$$

```

1 import numpy as np
2
3
4 def Extract_Order_Tracks(campbell: dict,
5                           orders: np.ndarray) -> dict:
6     """Extract complex order tracks from a Campbell field."""

```

```

7     speed_rpm = np.asarray(campbell["speed_rpm"], dtype=float)
8     frequency = np.asarray(campbell["frequency_hz"], dtype=float)
9     field = np.asarray(campbell["complex_amplitude"], dtype=complex)
10    orders = np.asarray(orders, dtype=float).reshape(-1)
11
12    tracks = np.empty((speed_rpm.size, orders.size), dtype=complex)
13
14    for row, speed in enumerate(speed_rpm):
15        target_frequency = orders * speed / 60.0
16        tracks[row] = Interpolate_Complex_Spectrum(
17            frequency,
18            field[row],
19            target_frequency,
20        )
21
22    return {
23        "speed_rpm": speed_rpm,
24        "order": orders,
25        "complex_amplitude": tracks,
26        "amplitude": np.abs(tracks),
27        "phase": np.angle(tracks),
28    }
29
30
31 tracks = Extract_Order_Tracks(
32     campbell,
33     orders=np.array([1.0, 2.0, 6.0]),
34 )

```

Phase is meaningful only relative to a consistent rotational reference. Speed obtained from a slowly varying analogue channel may be sufficient for amplitude tracking but not for absolute order phase. A once-per-revolution tachometer or encoder reference is required when phase relative to the shaft is important.

12.8.9 Angular Resampling for Order Tracking

During a rapid speed sweep, equal time increments do not correspond to equal shaft-angle increments. A time-domain sinusoid of constant order is therefore

a chirp. A block FFT assumes approximately stationary frequency and smears the component if its frequency changes appreciably inside the block.

Order tracking removes this motion by changing the independent variable from time to rotational angle. The angle is obtained from the rotational speed:

$$\theta(t) = \theta_0 + \int_{t_0}^t 2\pi \frac{n_{\text{rpm}}(\tau)}{60} d\tau. \quad (12.258)$$

The response is then interpolated at uniformly spaced angles:

$$\theta_m = \theta_0 + m\Delta\theta, \quad x_m^{(\theta)} = x(t(\theta_m)). \quad (12.259)$$

A component at order o becomes an ordinary sinusoid in angle:

$$x^{(\theta)}(\theta) = A \cos(o\theta + \varphi). \quad (12.260)$$

The following implementation performs trapezoidal angle integration and linear interpolation onto a uniform angular grid. It requires monotonically increasing angle. Data containing a reversal or a prolonged zero-speed interval should be split into monotonic rotational segments.

```

1 import numpy as np
2
3
4 def Linear_Interpolate_Monotone(x: np.ndarray,
5                                 y: np.ndarray,
6                                 x_new: np.ndarray) -> np.ndarray:
7     """Linear interpolation for a strictly increasing source coordinate."""
8     x = np.asarray(x, dtype=float).reshape(-1)
9     y = np.asarray(y).reshape(-1)
10    x_new = np.asarray(x_new, dtype=float).reshape(-1)
11
12    if y.size != x.size:
13        raise ValueError("x and y must have equal length")

```

```

14     if np.any(np.diff(x) <= 0.0):
15         raise ValueError("x must be strictly increasing")
16     if x_new[0] < x[0] or x_new[-1] > x[-1]:
17         raise ValueError("x_new must remain inside the source interval")
18
19     result = np.empty(x_new.size, dtype=np.result_type(y, float))
20
21     for index, value in enumerate(x_new):
22         right = np.searchsorted(x, value, side="right")
23         if right == 0:
24             result[index] = y[0]
25         elif right == x.size:
26             result[index] = y[-1]
27         else:
28             left = right - 1
29             fraction = (value - x[left]) / (x[right] - x[left])
30             result[index] = (
31                 (1.0 - fraction) * y[left]
32                 + fraction * y[right]
33             )
34
35     return result
36
37
38 def Angular_Resample(time: np.ndarray,
39                     signal: np.ndarray,
40                     speed_rpm: np.ndarray,
41                     samples_per_revolution: int):
42     """Resample a time signal at uniform shaft-angle increments."""
43     time = np.asarray(time, dtype=float).reshape(-1)
44     signal = np.asarray(signal, dtype=float).reshape(-1)
45     speed_rpm = np.asarray(speed_rpm, dtype=float).reshape(-1)
46
47     if signal.size != time.size or speed_rpm.size != time.size:
48         raise ValueError("time, signal and speed_rpm must have equal length")
49     if samples_per_revolution < 2:
50         raise ValueError("samples_per_revolution must be at least two")
51
52     angle = Rotational_Angle(time, speed_rpm)
53     if np.any(np.diff(angle) <= 0.0):
54         raise ValueError("rotational angle must be strictly increasing")

```

```

55
56     angle_step = 2.0 * np.pi / samples_per_revolution
57     first_index = int(np.ceil(angle[0] / angle_step))
58     last_index = int(np.floor(angle[-1] / angle_step))
59     angle_uniform = (
60         np.arange(first_index, last_index + 1) * angle_step
61     )
62
63     signal_angle = Linear_Interpolate_Monotone(
64         angle,
65         signal,
66         angle_uniform,
67     )
68     speed_angle = Linear_Interpolate_Monotone(
69         angle,
70         speed_rpm,
71         angle_uniform,
72     )
73
74     return angle_uniform, signal_angle, speed_angle

```

If the angular signal has M samples per revolution and a block spans R revolutions, then:

$$N_b = MR, \quad \Delta o = \frac{1}{R}, \quad o_{\text{Nyquist}} = \frac{M}{2}. \quad (12.261)$$

Thus the number of revolutions per block controls order resolution, while the number of samples per revolution controls the highest representable order. This is the angular-domain counterpart of time-record duration and sample rate.

```

1  import numpy as np
2
3
4  def Angular_Order_Map(time: np.ndarray,
5                        signal: np.ndarray,
6                        speed_rpm: np.ndarray,

```

```

7             samples_per_revolution: int,
8             revolutions_per_block: int,
9             overlap: float = 0.5,
10            duplicate_tolerance_rpm: float = 0.0,
11            resample: int | None = None):
12    """Construct an order map after uniform-angle resampling."""
13    _, signal_angle, speed_angle = Angular_Resample(
14        time,
15        signal,
16        speed_rpm,
17        samples_per_revolution,
18    )
19
20    block_size = samples_per_revolution * revolutions_per_block
21    if block_size < 2 or block_size & (block_size - 1):
22        raise ValueError(
23            "samples_per_revolution * revolutions_per_block "
24            "must be a power of two"
25        )
26    if block_size > signal_angle.size:
27        raise ValueError("the angular record is shorter than one block")
28    if not 0.0 <= overlap < 1.0:
29        raise ValueError("overlap must lie in [0, 1)")
30
31    hop_size = max(1, int(round(block_size * (1.0 - overlap))))
32    starts = range(0, signal_angle.size - block_size + 1, hop_size)
33
34    index = np.arange(block_size)
35    window = 0.5 - 0.5 * np.cos(
36        2.0 * np.pi * index / (block_size - 1)
37    )
38    coherent_gain = np.sum(window) / block_size
39
40    order = (
41        np.arange(block_size // 2 + 1)
42        / revolutions_per_block
43    )
44    block_speeds = []
45    speed_spreads = []
46    spectra = []
47

```

```

48     for start in starts:
49         stop = start + block_size
50         block = signal_angle[start:stop].copy()
51         block -= np.mean(block)
52
53         spectrum = FFT_Radix2(block * window)
54         complex_amplitude = (
55             spectrum[:block_size // 2 + 1]
56             / (block_size * coherent_gain)
57         )
58         complex_amplitude[1:-1] *= 2.0
59
60         local_speed = speed_angle[start:stop]
61         block_speeds.append(np.mean(local_speed))
62         speed_spreads.append(np.max(local_speed) - np.min(local_speed))
63         spectra.append(complex_amplitude)
64
65     block_speeds, spectra, speed_spreads = (
66         Sort_And_Average_Speed_Slices(
67             np.asarray(block_speeds),
68             np.asarray(spectra),
69             np.asarray(speed_spreads),
70             tolerance_rpm=duplicate_tolerance_rpm,
71         )
72     )
73
74     if resample is not None:
75         if resample < 2:
76             raise ValueError("resample must be at least two")
77         speed_uniform = np.linspace(
78             block_speeds[0],
79             block_speeds[-1],
80             resample,
81         )
82         spectra = Interpolate_Field_Rows(
83             block_speeds,
84             spectra,
85             speed_uniform,
86         )
87         speed_spreads = Interpolate_Field_Rows(
88             block_speeds,

```

```

89         speed_spreads[:, None],
90         speed_uniform,
91     )[:, 0]
92     block_speeds = speed_uniform
93
94     return {
95         "speed_rpm": block_speeds,
96         "order": order,
97         "complex_amplitude": spectra,
98         "amplitude": np.abs(spectra),
99         "speed_spread_rpm": speed_spreads,
100     }

```

For the synthetic run-up, choosing 64 samples per revolution and 16 revolutions per block gives:

$$N_b = 1024, \quad \Delta o = 0.0625, \quad o_{\text{Nyquist}} = 32. \quad (12.262)$$

The first, second and sixth orders then remain at fixed order positions despite the changing speed.

```

1 order_map_angular = Angular_Order_Map(
2     time,
3     signal,
4     speed_rpm,
5     samples_per_revolution=64,
6     revolutions_per_block=16,
7     overlap=0.75,
8     resample=120,
9 )

```

12.8.10 Tachometer Processing and Practical Limitations

Angular resampling is only as accurate as the estimated angle. Integrating a noisy speed signal accumulates phase error. A once-per-revolution tachometer

provides discrete angular reference events and prevents long-term drift. An encoder with several pulses per revolution provides more local information, but missing or false pulses must be detected before interpolation.

A practical procedure is:

1. detect and validate tachometer edges;
2. assign a known angular increment to consecutive edges;
3. interpolate angle monotonically between the edges;
4. resample the response at uniform angular increments;
5. apply an angular-domain window and FFT;
6. associate each block with its mean speed and speed spread.

The original time sampling must be high enough for the largest frequency present at the highest speed. Angular resampling cannot recover a component that was already aliased in the time-domain acquisition. Conversely, at low speed a fixed number of angular samples per revolution may correspond to a low effective time sample rate. Anti-alias filtering and the selected speed range must therefore be considered together.

If two shafts rotate at different speeds, the stated order always refers to one chosen reference shaft. A component that is synchronous with another shaft may appear as a non-integer or speed-dependent order. Gear ratios, slip and variable transmission ratios should be applied explicitly rather than assuming that every ridge belongs to the reference shaft.

12.8.11 Interpretation and Validation

A Campbell or order map should be checked against the underlying time and speed data. Useful validation steps include:

1. Verify the FFT frequency bins, amplitude convention and window gain on a harmonic of known amplitude.

- 2. Plot the speed spread of every block. Broad ridges in blocks with large Δn_b may be processing smear rather than physical damping.
- 3. Overlay expected order lines and known structural frequencies. A ridge should follow the appropriate geometry in both frequency and order views.
- 4. Extract selected complex order tracks and compare them with direct narrow-band or synchronous calculations.
- 5. Process run-up and coast-down separately before deciding whether they can be combined.
- 6. Confirm that speed sorting, duplicate averaging and optional uniform speed rebinning do not alter integrated amplitudes unexpectedly.

The frequency–speed Campbell diagram and the speed–order map are not competing representations. They emphasize different structures. A constant order is easy to identify in the order map, while a speed-independent resonance is easy to identify in the frequency Campbell diagram. Interpreting both views together is usually more reliable than relying on either one alone.

12.9 Rainflow Cycle Counting

A **rainflow analysis** is not another representation of a Campbell field. Campbell and waterfall plots describe how spectral content varies with speed or time. Rainflow counting operates directly on a scalar load, strain or stress history and reduces it to fatigue cycles.

Method	Coordinates or out-put	Main purpose
Response Campbell	speed, frequency and amplitude	rotating-vibration diagnosis
Waterfall	stacked spectra versus speed or time	perspective view of spectra
Rainflow counting	range, mean and cycle count	fatigue-cycle description

The conventional rainflow result does not retain frequency, phase or the exact time ordering of the cycles. If cycle counts must also be associated with speed, temperature or another operating state, the record must be divided or each counted cycle must be linked back to the relevant interval before this information is discarded.

12.9.1 Ranges, Means and Half Cycles

Let $s(t)$ be a scalar stress history. Consecutive equal values and samples that are not local extrema are first removed. The remaining reversal sequence is:

$$r_0, r_1, \dots, r_{N_r-1}. \quad (12.263)$$

A counted excursion between reversal values r_a and r_b is described by its range, amplitude and mean:

$$\Delta s = |r_b - r_a|, \quad s_a = \frac{\Delta s}{2}, \quad s_m = \frac{r_a + r_b}{2}. \quad (12.264)$$

A closed interior cycle has count $c = 1$. An unclosed excursion at the beginning or end of a finite record has count $c = 1/2$. Two identical half cycles therefore contribute the same total count as one full cycle. Whether residual half cycles should be retained, repeated to close a periodic record or treated by another record-specific convention must be stated.

Rainflow counting is often explained by imagining rain flowing down a pagoda roof formed by the rotated stress history. Numerically, a stack of reversals is more direct. When a new reversal arrives, the two most recent ranges are compared:

$$R_{\text{old}} = |r_{j-1} - r_{j-2}|, \quad R_{\text{new}} = |r_j - r_{j-1}|. \quad (12.265)$$

If $R_{\text{new}} < R_{\text{old}}$, the older excursion remains open. If $R_{\text{new}} \geq R_{\text{old}}$, the older excursion is closed and counted. An excursion touching the beginning of the current stack is counted as a half cycle; an enclosed excursion is counted as a full cycle. After all reversals have been processed, the adjacent residual excursions are counted as half cycles.

12.9.2 Python Implementation of Rainflow Counting

The following implementation returns one row per counted cycle. The columns are cycle range, cycle mean and count. It deliberately retains half cycles rather than silently forcing the record to be periodic.

```

1 import numpy as np
2
3
4 def Reversal_Points(signal: np.ndarray) -> np.ndarray:
5     """Return endpoints and local extrema after removing plateaus."""
6     values = np.asarray(signal, dtype=float).reshape(-1)
7     if values.size < 2:
8         raise ValueError("at least two samples are required")
9
10    compact = [values[0]]
11    for value in values[1:]:
12        if value != compact[-1]:
13            compact.append(value)
14
15    compact = np.asarray(compact)
16    if compact.size == 1:
17        return compact.copy()
18
19    reversals = [compact[0]]
20    for index in range(1, compact.size - 1):
21        incoming = compact[index] - compact[index - 1]
22        outgoing = compact[index + 1] - compact[index]
23        if incoming * outgoing < 0.0:
24            reversals.append(compact[index])
25    reversals.append(compact[-1])
26    return np.asarray(reversals)
27
28
29 def Rainflow_Cycles(signal: np.ndarray) -> np.ndarray:
30     """Return rows [range, mean, count] using a reversal stack."""
31     reversals = Reversal_Points(signal)
32     stack = []
33     cycles = []

```

```

34
35     for value in reversals:
36         stack.append(float(value))
37
38         while len(stack) >= 3:
39             old_range = abs(stack[-2] - stack[-3])
40             new_range = abs(stack[-1] - stack[-2])
41
42             if new_range < old_range:
43                 break
44
45             cycle_mean = 0.5 * (stack[-3] + stack[-2])
46
47             if len(stack) == 3:
48                 if old_range > 0.0:
49                     cycles.append([old_range, cycle_mean, 0.5])
50                     stack.pop(0)
51             else:
52                 if old_range > 0.0:
53                     cycles.append([old_range, cycle_mean, 1.0])
54                     last = stack.pop()
55                     stack.pop()
56                     stack.pop()
57                     stack.append(last)
58
59     for first, second in zip(stack[:-1], stack[1:]):
60         cycle_range = abs(second - first)
61         if cycle_range > 0.0:
62             cycles.append([
63                 cycle_range,
64                 0.5 * (first + second),
65                 0.5,
66             ])
67
68     if not cycles:
69         return np.empty((0, 3), dtype=float)
70     return np.asarray(cycles, dtype=float)
71
72
73 triangular_history = np.array([0.0, 10.0, 0.0])
74 print(Rainflow_Cycles(triangular_history))

```

75 # Two boundary half cycles of range 10; their total count is one.

A constant or monotonic record contains no closed interior cycles. Its total change appears only as a residual half cycle. This is appropriate because a single ramp has not completed a load reversal.

12.9.3 Rainflow Matrix

Individual cycles are commonly collected into a two-dimensional histogram. Let ρ_i be cycle-range bin edges and μ_j mean-value bin edges. The rainflow-matrix entry is:

$$N_{ij} = \sum_c c_c \mathbf{1}(\rho_i \leq \Delta s_c < \rho_{i+1}) \mathbf{1}(\mu_j \leq s_{m,c} < \mu_{j+1}), \quad (12.266)$$

where c_c is 1 for a full cycle and 1/2 for a half cycle. Summing the matrix gives the total number of equivalent full cycles, including the fractional contribution of residual excursions.

```

1 import numpy as np
2
3
4 def Rainflow_Matrix(cycles: np.ndarray,
5                     range_edges: np.ndarray,
6                     mean_edges: np.ndarray) -> np.ndarray:
7     """Bin rows [range, mean, count] into a rainflow matrix."""
8     cycles = np.asarray(cycles, dtype=float)
9     range_edges = np.asarray(range_edges, dtype=float).reshape(-1)
10    mean_edges = np.asarray(mean_edges, dtype=float).reshape(-1)
11
12    if cycles.ndim != 2 or cycles.shape[1] != 3:
13        raise ValueError("cycles must contain [range, mean, count] rows")
14    if np.any(np.diff(range_edges) <= 0.0):
15        raise ValueError("range_edges must be strictly increasing")

```

```

16     if np.any(np.diff(mean_edges) <= 0.0):
17         raise ValueError("mean_edges must be strictly increasing")
18
19     matrix = np.zeros((range_edges.size - 1, mean_edges.size - 1))
20
21     for cycle_range, cycle_mean, count in cycles:
22         range_bin = np.searchsorted(
23             range_edges,
24             cycle_range,
25             side="right",
26         ) - 1
27         mean_bin = np.searchsorted(
28             mean_edges,
29             cycle_mean,
30             side="right",
31         ) - 1
32
33         if cycle_range == range_edges[-1]:
34             range_bin = matrix.shape[0] - 1
35         if cycle_mean == mean_edges[-1]:
36             mean_bin = matrix.shape[1] - 1
37
38         if (0 <= range_bin < matrix.shape[0]
39             and 0 <= mean_bin < matrix.shape[1]):
40             matrix[range_bin, mean_bin] += count
41
42     return matrix

```

The example below contains oscillations with changing amplitude and mean value. Its time history and resulting rainflow matrix are shown together. The matrix contains cycle counts, not spectral amplitude.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 time_rf = np.linspace(0.0, 20.0, 4001)
6 envelope = 65.0 + 20.0 * np.sin(2.0 * np.pi * 0.08 * time_rf)

```

```

7 mean_stress = 25.0 * np.sin(2.0 * np.pi * 0.03 * time_rf)
8 stress = (
9     mean_stress
10    + envelope * np.sin(2.0 * np.pi * 1.4 * time_rf)
11    + 18.0 * np.sin(2.0 * np.pi * 0.37 * time_rf)
12 )
13
14 cycles = Rainflow_Cycles(stress)
15 range_edges = np.linspace(0.0, 1.001 * np.max(cycles[:, 0]), 17)
16 mean_edges = np.linspace(
17     np.min(cycles[:, 1]) - 1.0e-9,
18     np.max(cycles[:, 1]) + 1.0e-9,
19     17,
20 )
21 counts = Rainflow_Matrix(cycles, range_edges, mean_edges)
22
23 figure, axes = plt.subplots(2, 1)
24 axes[0].plot(time_rf, stress)
25 axes[0].set_xlabel("Time [s]")
26 axes[0].set_ylabel("Stress")
27
28 mesh = axes[1].pcolormesh(
29     mean_edges,
30     range_edges,
31     counts,
32     shading="flat",
33 )
34 axes[1].set_xlabel("Cycle mean")
35 axes[1].set_ylabel("Cycle range")
36 figure.colorbar(mesh, ax=axes[1], label="Cycle count")
37 figure.tight_layout()
38 plt.show()

```

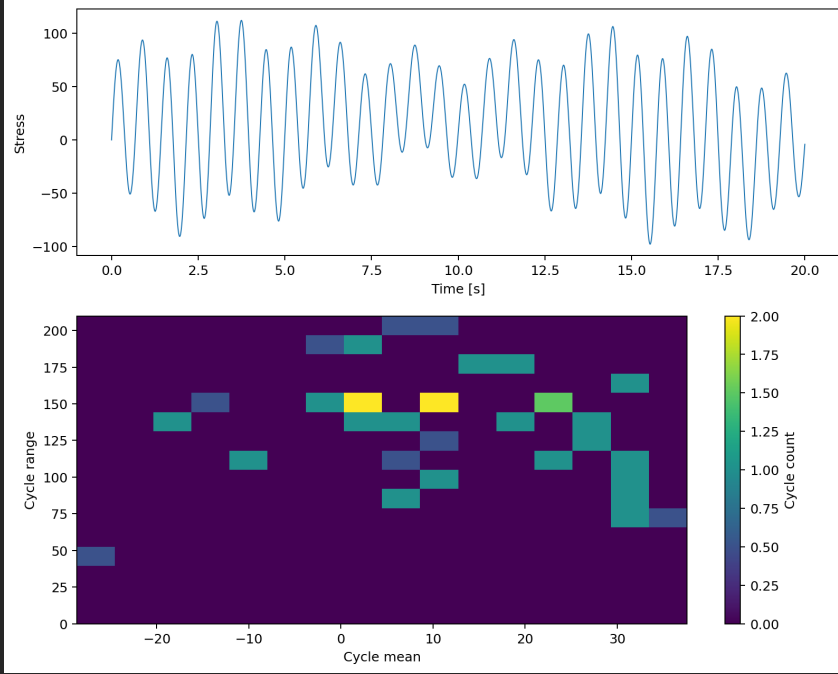


Figure 12.5: Variable stress history and its rainflow matrix. Each matrix cell accumulates full and half cycles having a similar range and mean value. The result contains no frequency or rotational-speed coordinate.

12.9.4 Relation to Fatigue Damage

Rainflow counting supplies the cycle ranges, means and counts required by a fatigue model; it does not calculate damage by itself. For a stress-life curve that assigns an allowable life N_c to each corrected cycle amplitude, linear Palmgren–Miner accumulation is:

$$D = \sum_c \frac{c_c}{N_c}. \quad (12.267)$$

Before looking up N_c , a mean-stress correction may be required because two

cycles with the same range but different means need not cause the same damage. Material data, surface condition, stress concentration, multiaxial state and the chosen fatigue model remain separate modelling inputs. The rainflow matrix is therefore an economical description of the load history, not a complete fatigue assessment.

12.10 Power Spectral Density, Cross Spectral Density and Coherence

12.11 Sorting

12.12 Reverse Cuthill-McKee Algorithm for rearranging Sparse matrix

12.13 Topology Optimization

12.14 Generalised Eigenvalue Problem

Chapter 13

Miscellaneous

13.1 Bolt

13.1.1 Bolt pretension

The relation between the **tightening moment** and bolt **pretension force** can be characterised by the following equation:

$$M_A = F_V \left[\frac{d_2}{2} \tan(\varphi^\circ + \rho') + \mu_K \frac{d_{K,R}}{2} \right] \quad (13.1)$$

with:

$$d_{K,R} = \frac{d_k + d_i}{2} \quad (13.2)$$

$$\varphi^\circ = \arctan \left(\frac{P}{d_2 * \pi} \right) \quad (13.3)$$

$$\rho' = \arctan \left(\frac{\mu_G}{\cos \frac{\beta}{2}} \right) \quad (13.4)$$

at standard thread angle of $\beta = 60^\circ$:

$$\rho' = \arctan (\mu_G * 1.155) \quad (13.5)$$

where:

M_A ... Tightening Moment [Nmm]

F_V ... Pretension Force [N]

d_2 ... Bolt thread midiameter ((core + outer diameter) / 2) [mm]

ρ' ... Bolt thread friction angle [$^\circ$]

φ° ... Bolt thread angle of rising (pitch) [$^\circ$]

P ... Bolt thread pitch [mm]

μ_G ... Thread friction coefficient [-]

μ_K ... Friction coefficient of bolt head [-]

β ... Thread angle (angle between 'teeth') [$^\circ$]

$d_{K,R}$... Middle diameter of friction surface under bolt head [mm]

d_K ... Outer diameter of surface under bolt head [mm]

d_i ... Diameter of the hole in the material [mm]